LambLisp

Lisp for Real-Time Control

01 Red Fox Alpha



Copyright 2026 Frobenius Norm LLC

1 Getting Started With Real-time *LambLisp* **3**
1.1 License . . . 3
1.2 Prerequisites . . . 3
1.3 Quick Start: The Package You Received . . . 5
1.4 Quick Start: Try LambLisp on Linux . . . 8
1.5 Quick Start: Try LambLisp on Windows . . . 8
1.6 Running LambLisp in a Container . . . 10
1.7 Quick Start: Try LambLisp in Your Browser . . . 14
1.8 Using the REPL . . . 14
1.9 Deploy to ESP32 . . . 15
1.10 What Happens at Startup . . . 20
1.11 What's in the package? . . . 20

2 *Lisp* in a nutshell **23**

3 Why Lisp? **27**

4 Why *LambLisp*? **29**
4.1 Real-time control . . . 29
4.2 Multi-platform . . . 29
4.3 Widely recognized *Scheme* language specification . . . 29
4.4 Reuse of existing C++/Arduino libraries . . . 30
4.5 Adaptive real-time garbage collection . . . 30
4.6 Open API for new *Lisp* primitives . . . 30
4.7 Incremental over-the-air updates without reboot . . . 30
4.8 First-class hierarchical dictionaries . . . 31
4.9 Incremental, just-in-time compilation . . . 31
4.10 Lexical scoping . . . 31
4.11 Tail recursion and tail-calls . . . 32
4.12 Procedures are first-class data types . . . 33
4.13 Runtime collaboration with C++ code . . . 33
4.14 Key feature summary . . . 34

5 LambLisp Architecture Internals **35**
5.1 Background . . . 35
5.2 The *dictionary* type . . . 36
5.3 Hierarchical dictionaries . . . 37
5.4 Execution environment is a *hierarchical dictionary* . . . 37
5.5 Objects are wrappers around *hierarchical dictionaries* . . . 39

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.6 Interpreter & compiler organization . . . 40
5.7 LambLisp Virtual Machine . . . 44
5.8 Memory Management . . . 47
5.9 Real-time Guarantees . . . 55
5.10 Scalability . . . 59
5.11 Language Internals and C++ API . . . 66
5.12 C++ Integration . . . 73

6 GC Best Practices 83
6.1 The gc-frame . . . 84
6.2 Who protects what . . . 85
6.3 Order the calls so values are consumed . . . 86
6.4 Root what you hold . . . 87
6.5 Procedures that take their arguments as an array . . . 88
6.6 Procedures that can throw . . . 88
6.7 What it looks like when you get it wrong . . . 89
6.8 Checklist . . . 89

7 LambLisp Extensions 91
7.1 Language Extensions . . . 91
7.2 Dictionary (Environment) Operations . . . 92
7.3 Structured Data . . . 93
7.4 Arbitrary-Precision Integers (Bignums) . . . 97
7.5 Cryptography . . . 97
7.6 Bytecode Virtual Machine . . . 98
7.7 Native Code Generator (NCG) . . . 98
7.8 Diagnostics and Development Tools . . . 99
7.9 Arduino / Embedded I/O . . . 100
7.10 I2C (Wire) . . . 102
7.11 WiFi . . . 102
7.12 UDP Datagrams . . . 105
7.13 Network Time (SNTP) . . . 106
7.14 Over-the-Air Updates . . . 106
7.15 Runtime and GC Tuning . . . 112
7.16 LLIP — LambLisp Interaction Protocol . . . 117
7.17 LCD1602 Display . . . 135
7.18 Ultrasonic Sonar (HC-SR04) . . . 136
7.19 PCA9685 PWM Controller . . . 136

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.20 WS2812 Addressable LED Strip (NeoPixel) . . . 136
7.21 SPI Bus . . . 137
7.22 OneWire (1-Wire Bus) . . . 138
7.23 Freenove 4WD Car Kit — Peripheral Summary . . . 138
7.24 Camera and On-Board Display (esp32-s3-eye) . . . 139
7.25 Bundled Scheme Libraries . . . 141
7.26 Behaviors: Cooperative Scheduling . . . 145
7.27 ESP32 System . . . 146
7.28 LambLisp Modbus Interface . . . 148
7.29 LambLisp PROFIBUS and PROFINET Interface . . . 160
7.30 LambLisp BACnet/IP Interface . . . 172
7.31 Bundled Demos . . . 177

8 Worked Examples 181
8.1 Worked Example: Autonomous Floor Mapping with the Freenove 4WD Car . . . 182

9 LambLisp Compatibility Matrix 197
9.1 Legend . . . 197
9.2 Coverage Summary . . . 197
9.3 Detailed Compatibility . . . 199
9.4 *See:* Real-time Guarantees · Runtime and GC Tuning . . . 216

10 LambLisp Frequently Asked Questions 217
10.1 Getting Started . . . 217
10.2 Concepts . . . 218
10.3 Language and Conformance . . . 220
10.4 Integration and Operation . . . 222

11 Other Scheme Implementations 225

12 Topic Documentation 227
12.1 Hardware & Platform Builtins (xmop3) . . . 227

13 Class Documentation 295
13.1 autobuf_t< _T_ > Class Template Reference . . . 295
13.2 Cell Class Reference . . . 297
13.3 Cell::CellFeatures Struct Reference . . . 334
13.4 Lamb Class Reference . . . 334
13.5 LambStdioClass Class Reference . . . 375
13.6 LL_File Class Reference . . . 376

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

13.7 LL_File_System Class Reference . . . 377
13.8 LL_Panel Struct Reference . . . 377
13.9 LL_String Class Reference . . . 379
13.10 LLArduinoBackend Struct Reference . . . 381
13.11 LLArduinoBufPrint Class Reference . . . 381
13.12 LLArduinoSerial Class Reference . . . 383
13.13 LLGpiodLine Struct Reference . . . 384
13.14 LLGpiodPin Struct Reference . . . 384
13.15 Print Class Reference . . . 385

14 Acknowledgements **387**

15 Glossary **389**

16 List of Figures **397**

17 Index **399**



Figure 1 Eponymous Lamb

LambLisp by Copyright 2026 Frobenius Norm LLC.

LambLisp is a **real-time** implementation of the **Scheme** dialect of the **LISP** programming language. *Scheme* is governed by standards known as *Scheme R5RS* and *Scheme R7RS*. LambLisp is designed primarily for *Scheme R5RS*, with some features from *Scheme R7RS* and **Common Lisp** that aid in development of **real-time embedded control systems**.

LambLisp provides a robust and scalable Hardware Abstraction Layer (HAL). The LambLisp HAL adds many features focused on embedded controls, and enables convenient interoperability with existing device drivers or other C/C++ libraries.

LambLisp runs across a three-tier platform stack:

- **ESP32** / **ESP32-S3** (32-bit Xtensa LX7) — real-time embedded boards running physical hardware. The Espressif ESP32-S3-DevKitC-1 is the primary test board. The Freenove 4WD Car Kit (classic ESP32) is an autonomous vehicle demo exercising I2C, PWM, analog/digital I/O, and real-time task queues.
- **Linux x86-64** — edge server or development workstation tier. Full LambLisp with CUDA bindings for desktop GPU development. Acts as a supervisory controller for one or more embedded boards via the LLIP protocol.
- **Linux AArch64 (ARM64)** / **Nvidia Jetson Orin Nano** — edge AI tier. Full LambLisp with CUDA bindings for on-board neural inference and control. Demonstrated on the Nvidia Jetson Orin Nano development kit.

All three tiers communicate peer-to-peer via **LLIP** (LambLisp Interaction Protocol), enabling a scalable supervisory Lisp application that spans embedded sensors, local GPU inference, and cloud compute within a single unified language environment.

Try it in one line, with nothing installed:

```
docker run --rm -it ghcr.io/lamblisp/lamblisp:runtime
```

That boots straight to the `LL=>` prompt. `:runtime` is a **multi-architecture index**, so `docker pull` selects `linux/amd64` or `linux/arm64` to match your host – Apple Silicon, a Raspberry Pi, a Jetson or an ARM cloud instance all get a native image rather than emulation, which matters here because an emulated timing figure is a meaningless one. The per-architecture tags `:runtime-amd64` and `:runtime-arm64` remain available if you need to pin one explicitly. See Running in a Container.

Where to get *LambLisp*:

Linux	the `LambLispRT-v01-linux_*` packages. Requires glibc 2.34 or later – Debian 12, Ubuntu 22.04 LTS, RHEL 9 or newer
Windows	a native `LambLisp.exe` – `LambLispRT-v01-windows_x86_64`
Container	`ghcr.io/lamblisp/lamblisp:runtime` – no host glibc or CPU requirement at all
ESP32 and other boards	the per-board `LambLispRT-v01-esp32*` packages
Browser	open `demo/index.html` from any package – no download, no toolchain

For more information:

- Read the LambLisp manual online at `https://wawhite3.github.io/LambLispRT-v01-linux_x86_64/html/index.html`
- Find the LambLisp manual in PDF at `https://wawhite3.github.io/LambLispRT-v01-linux_x86_64/LambLisp.pdf`
- Download the full LambLisp repositories from `https://github.com/wawhite3/?tab=repositories&q=LambLispRT`
- Visit the developer `Frobenius Norm` at `https://frobeniusnorm.com`.

Note

The code repository is renewed frequently, with history deleted. Always check back to see if a new repo is available, not just an update to the previous repo.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 1

Getting Started With Real-time *LambLisp*

1.1 License

LambLisp is copyright 2026 by Frobenius Norm LLC, a Massachusetts corporation.

LambLisp is a commercial product and requires a license when used to generate revenue, to promote other products or services, or for other commercial activity. Licensing provides *LambLisp* customization, hardware application development assistance, post-deployment support, and a warranty.

LambLisp may be used for non-commercial purposes, but comes with no support and no warranty of any kind.

Warning

> *LambLisp* is in **ALPHA** state. Do not deploy it on any application where life or property may be endangered.

1.2 Prerequisites

What you need depends on what you are doing, and two of the three routes need no toolchain at all.

I want to...	I need	start at
run *LambLisp* and write Scheme	a Linux host with **glibc 2.34 or later** (Debian 12, Ubuntu 22.04 LTS, RHEL 9 or newer)	Quick Start: Linux
the same, on Windows	64-bit Windows 10 or later	Quick Start: Windows
the same, without installing anything	any host with a working Docker	Running in a Container

I want to...	I need	start at
build anything from the package I downloaded	a Linux machine and *platformio*	Quick Start: The Package You Received
build and flash ESP32 firmware	a Linux machine, *platformio*, and the rest of this section	Deploy to ESP32

Only the last row needs a development toolchain. If you are evaluating the language, or your host is older than the glibc floor above, the container is the shortest path and installs nothing.

For embedded development you also need:

- Linux development machine (x86_64 or aarch64).
- Familiarity with embedded cross-compilation and basic Linux shell use.
- *platformio* (`pio`) — the build system used for all *LambLisp* targets.

To install *platformio*:

```
curl -fsSL https://raw.githubusercontent.com/platformio/platformio-core-installer/master/get-platformio.py | python3
ln -s ~/.platformio/penv/bin/pio ~/.local/bin/pio
```

Clone the repository (replace `vxx` with the current version number):

```
git clone https://github.com/wawhite3/LambLispRT-vxx ~/src/LambLispRT-vxx
cd ~/src/LambLispRT-vxx
```

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

1.3 Quick Start: The Package You Received

Every *LambLisp* package is **locked to one target**, and this section is the whole path from the download to a `LL=>` prompt. A short standalone copy of it ships in each package as `QUICKSTART.md`; this is the long form.

1.3.1 What is in the package

`BUILD-INFO`, in the package root, names the target and the exact commit, build date, toolchain and feature flags that produced the binaries. Quote those lines in any support request — they answer "which source is this?" in one place, which nothing else in the package can do.

```
head -8 BUILD-INFO
```

Packages come in two shapes:

If the directory has...	it is	and you can
`platformio.ini`, `src/`, `lib/`, `ll_pio/`, `liblamblisp.a`	a **development package** — Linux, or an embedded board	run the prebuilt binary *and* rebuild from source
only `LambLisp.exe` or `LambLisp.bin` beside `data/`, `scm/`, `demo/`, `html/`	a **runtime package** — Windows, and other binary-only targets	run it; build on a Linux development package

The full contents are listed under What's in the package?.

1.3.2 Build

With *platformio* installed as described under Prerequisites:

```
pio run
```

There is no `-e` flag to get right. The `platformio.ini` in a package is generated for that one target and pins it with `default_envs`, so the bare command is the whole command — here and in every command below. The environment table in Supported Platforms and Boards describes the full development tree; a package carries the one row that applies to it.

Note

> **Do not run `pio pkg install --platform espressif32` first.** The `platform =` line in the package's `platformio.ini` is pinned to a specific release URL, and `pio run` installs exactly that. The `pkg install` command above fetches a *different*, older platform alongside it — the official `platform-espressif32`, which is still pinned to Arduino 2.0.17 and an ESP-IDF that went end-of-life in 2024. Nothing will appear to go wrong at that moment.

What the first build does, so it does not look like a hang. It downloads a compiler toolchain and framework — on the order of a gigabyte, several minutes on a good link, with long silent stretches. It puts them in `~/.platformio-idf55`, **not** the usual `~/.platformio`, because the package's `platformio.ini` sets `core_dir` there deliberately: the build needs a specific ESP-IDF, and an isolated core directory keeps it from colliding with any other *platformio* project already on the machine. Later builds are incremental and quick.

Output lands in `.pio/build/<target>/`.

1.3.3 Flash an embedded board

Connect the board and run **both**:

```
pio run --target upload         # the firmware
pio run --target uploadfs       # the data/ filesystem image
```

Warning

> `uploadfs` **is not optional, and skipping it does not look like a missing step.** The firmware loads `setup.scm` and the standard library off the board's LittleFS filesystem at boot, and the package's `data/` directory *is* that filesystem image. Flash the firmware alone and the board comes up at a `LL=>` prompt with almost nothing defined in it — no error, no complaint about a missing file, just a language with its library gone. Either order works. Re-run `uploadfs` on its own whenever a file under `data/` changes.

platformio finds the serial port itself. With more than one board attached, name it:

```
pio run --target upload --upload-port /dev/ttyUSB0
```

Resolve the port at run time; do not bake a device node into a script. Serial ports are enumerated dynamically and move between boots and between machines.

Note

> On the **ESP32-S3-DevKitC-1**, use the `UART` port, not the `USB` port. The board has two: `UART` carries upload and the serial REPL, `USB` is the onboard JTAG debugger. Flashing over the wrong one fails in a way that reads like a broken board.

Building inside a container has one hard limit that is worth knowing before wiring anything up — see Flashing from a container.

1.3.4 Talk to the board

```
pio device monitor
```

115200 baud, already set in the package's `platformio.ini`. Any terminal emulator works too (`screen`, `minicom`) on the board's `/dev/ttyUSB*` or `/dev/ttyACM*`.

For anything past a REPL, use the **LLIP** client shipped in the package as `tools/llip`. It speaks *LambLisp*'s own interaction protocol over the same serial line, or over TCP:

```
tools/llip --serial /dev/ttyUSB0 repl             # line REPL
tools/llip --serial /dev/ttyUSB0 eval "(+ 1 2)"   # evaluate one form
tools/llip --serial /dev/ttyUSB0 send app.scm     # push a file to the board
tools/llip --serial /dev/ttyUSB0 recv log.txt     # pull a file back
tools/llip --help                                 # everything else
```

`send` is the fast path during development: push a changed `.scm` and reload it, with no reflash. The protocol itself is documented under LLIP.

1.3.5 `data/` and `scm/` are not the same thing

`data/`	the **filesystem image** for this one target — the subset of the library the board boots with. This is what `uploadfs` writes and what `setup.scm` finds at startup.
`scm/`	the **complete** *LambLisp* Scheme library source, shipped whole with every package regardless of target. Load from it at run time, or copy a file into `data/` and re-run `uploadfs` to boot with it.

An application goes in `data/setup.scm` — see What Happens at Startup.

1.3.6 Making it yours

- **Feature set** — edit `build_flags` in `platformio.ini`. The `-DLL_*` flags select the drivers and numeric types compiled in (`LL_I2C`, `LL_WIFI`, `LL_MODBUS`, `LL_BIGNUM`, ...). Dropping what an application does not use buys flash and RAM. `BUILD-INFO` records the set the package shipped with.
- **Your own C++ primitives** — `src/` carries `main.cpp` and the device drivers, `lib/` the vendored Arduino libraries. `liblamblisp.a` is the VM itself, linked by the package's `platformio.ini` with `-L$PROJECT_DIR -llamblisp`. Adding a *LambLisp* builtin that calls existing C/C++ code is described under Open API for new ∗Lisp∗ primitives.
- **Do not unpin the platform.** The `platform =` release URL is the only in-package record of which ESP-IDF this was built and measured against, and the memory constants in the source were measured on that layout.

1.3.7 When it does not work

What you see	What it is
`LL=>` comes up but almost nothing is defined	the filesystem was never flashed — run `pio run --target uploadfs`; or, on a host build, *LambLisp* was started from outside the package directory
The first build sits silent for minutes	it is downloading the toolchain into `~/.platformio-idf55`. Let it finish.
`upload` hangs or cannot open the port	the wrong USB port on a DevKitC-1 (use `UART`); a port the user has no permission on (join the owning `dialout`/`tty` group); or a container on macOS or Windows, which cannot see USB at all
Windows reports an unrecognised publisher	`LambLisp.exe` is not code-signed — *More info*, then *Run anyway*
Files missing on the board after a partition change	a flash-layout change presents as **failed reads**, not as an allocation error. Re-check `board_build.partitions`.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

1.4 Quick Start: Try LambLisp on Linux

The Linux build is the fastest way to explore *LambLisp* — no hardware required.

```
pio run -e linux_x86_64
```

The executable is placed in `.pio/build/linux_x86_64/program`. Run it from the repo root so it can find the `data/` directory, which contains `setup.scm` and the standard library files:

```
cd ~/src/LambLispRT-vxx
.pio/build/linux_x86_64/program
```

You will see startup messages followed by the *LambLisp* REPL prompt:

```
LL=>
```

Try a few expressions:

```
LL=> (+ 1 2)
3
LL=> (define (greet name) (string-append "Hello, " name "!"))
LL=> (greet "world")
"Hello, world!"
LL=> (load "data/PCA9685.scm" 1)
```

Press `Ctrl-C` to exit.

1.5 Quick Start: Try LambLisp on Windows

LambLisp runs natively on 64-bit Windows. It is the same language and the same conformance result as the Linux build – what differs is the host interface, described at the end of this chapter.

1.5.1 What you need

64-bit Windows 10 or later. Nothing else: the download is self-contained, and there is no installer, no runtime to add, and no registry change.

1.5.2 Get it

Download `LambLispRT-v01-windows_x86_64.zip` and unpack it anywhere – your desktop is fine. You will get:

```
LambLispRT-v01-windows_x86_64\
    LambLisp.exe          the interpreter
    data\                 setup.scm and the standard library it loads at boot
    scm\                  the full Scheme library source
    LambLisp.pdf          this manual
    html\                 the same manual as a web page
    demo\                 the in-browser demo -- open demo\index.html
```

The first run will show a SmartScreen warning. `LambLisp.exe` is not code-signed, so Windows reports an unrecognised publisher. Choose *More info* then *Run anyway*. We would rather tell you this than have it surprise you: an unsigned binary is a statement about our certificate, not about the file you downloaded.

1.5.3 Run it

Open a Command Prompt or PowerShell **in the unpacked folder** and run:

```
LambLisp.exe
```

Working directory matters: `LambLisp.exe` loads `data\setup.scm` at startup by relative path, so starting it from elsewhere gives `cannot open input file: setup.scm`. See *Running from any directory* below.

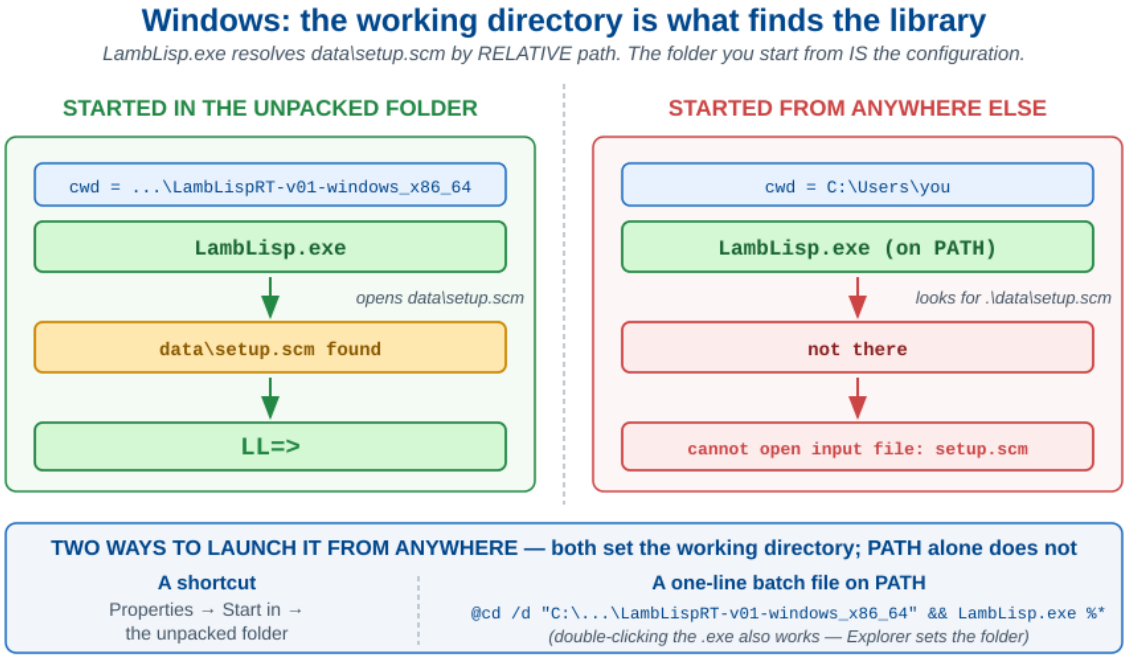


Figure 1.1 How LambLisp.exe finds its library on Windows

You will see startup messages followed by the prompt:

```
LL=>
```

Try a few expressions:

```
LL=> (+ 1 2)
3
LL=> (* 111111111 111111111)
12345678987654321
LL=> (define (greet name) (string-append "Hello, " name "!"))
LL=> (greet "world")
"Hello, world!"
```

The second one is worth a moment: that product exceeds 64 bits, and *LambLisp* answers it exactly because the numeric tower promotes to arbitrary-precision integers automatically.

Press `Ctrl-C` to exit.

1.5.4 Running from any directory

`LambLisp.exe` finds its library relative to the current directory, so adding the folder to `PATH` alone is not enough – running it from elsewhere will fail to find `setup.scm`. Two ways to launch it from anywhere:

A shortcut. Right-click `LambLisp.exe`, *Create shortcut*, then open the shortcut's *Properties* and set **Start in** to the unpacked folder. Double-clicking the shortcut then works from wherever it lives, including the Start Menu or the taskbar.

A one-line batch file. Save this as `lamblisp.cmd` somewhere on your `PATH`, with the path edited to match where you unpacked:

```
@cd /d "C:\Users\you\LambLispRT-v01-windows_x86_64" && LambLisp.exe %*
```

Then `lamblisp` works from any prompt. (Double-clicking `LambLisp.exe` in Explorer also works as-is, because Explorer sets the working directory to the containing folder.)

1.5.5 What is the same as Linux, and what is not

The language is identical. The same conformance suite runs on both with the same result – **1355 pass, 0 fail, 15 exception** – and the 15 are the cases needing full re-entrant `call/cc`, listed under *Real-time Guarantees*. Your `.scm` files move between platforms unchanged.

Differences are confined to the host interface:

	Windows	Linux
GPIO procedures (`pinMode`, `digitalWrite`, `digitalRead`, ...)	present, in-memory loopback	same loopback, or real pins where the hardware is
I^2C and hardware serial	not present	present where the hardware is
TCP sockets, `LLIP` remote debugging	not present	present
ESP32 cross-build and flashing	not from Windows	full

The GPIO row is worth reading carefully, because "present" does not mean "connected to anything". The 26 `commonio` procedures are registered and callable, and `digitalRead` returns whatever `digitalWrite` last wrote to that pin – a loopback, so code written against them runs unmodified on the desktop before it ever reaches a board. That is the same behaviour any host build has, Linux included; what Linux adds is a real backend when real hardware is present.

For embedded development – building firmware and flashing a board – use the Linux build. The Windows build is for writing and running *Scheme*: language work, algorithm work, and running the test suites.

1.6 Running LambLisp in a Container

A published container image runs *LambLisp* with nothing installed on your machine, and with no dependency on your host's glibc, CPU vintage or distribution.

1.6.1 Try the language

```
docker run --rm -it ghcr.io/lamblisp/lamblisp:runtime
```

That drops you at `LL=>` with the standard library loaded. `--rm` removes the container when you exit; nothing persists on your machine.

To work on your own files, mount a directory and load by its path inside the container:

```
docker run --rm -it -v "$PWD:/work" ghcr.io/lamblisp/lamblisp:runtime
LL=> (load "/work/my-program.scm" 1)
```

Writing works too, so output lands back on your host:

```
LL=> (let ((p (open-output-file "/work/results.txt")))
       (write-string "..." p)
       (close-port p))
```

Do not add `-w /work`**.** *LambLisp* loads its standard library from `data/` relative to the working directory, which is why the image sets one and starts there. Overriding it stops the container before *LambLisp* runs at all, with a `no such file or directory` from the runtime rather than anything that mentions Scheme. Mount wherever you like and use absolute paths inside the container; that is the pattern above, and it is the one to use.

1.6.2 What is in the image, and what is not

`ghcr.io/lamblisp/lamblisp:runtime` carries the prebuilt host runtime, the complete Scheme library, this manual, and the browser demo. It carries **no toolchain** – no compiler, no PlatformIO, no LaTeX. That is deliberate: the binaries are already built and the documentation is already generated, so a toolchain would add gigabytes to produce files that are in the image already.

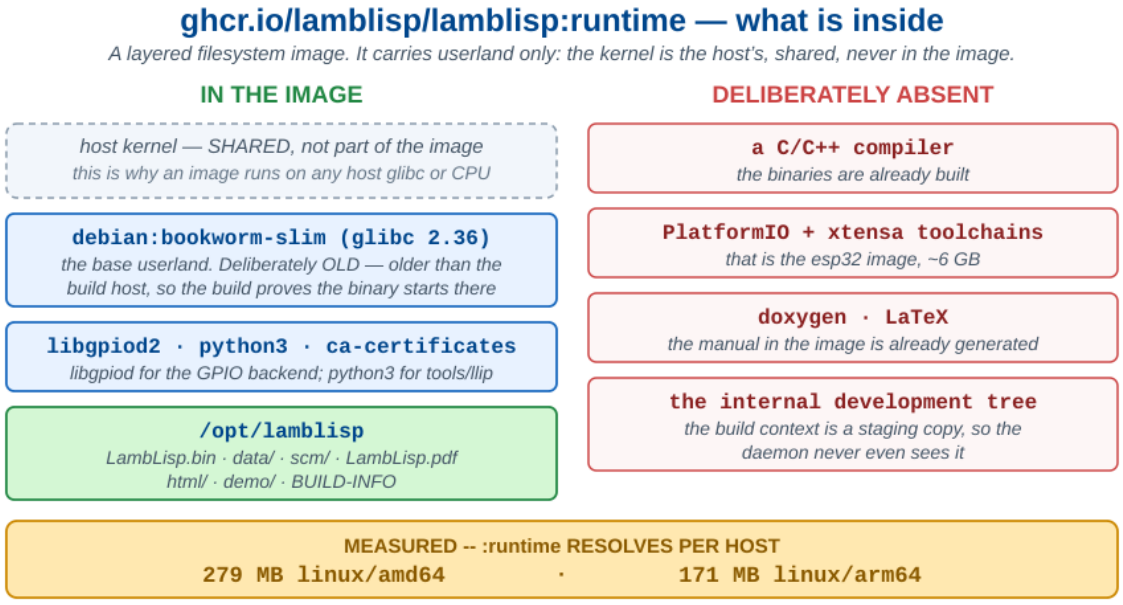


Figure 1.2 What is inside the runtime image

`:runtime` is a multi-architecture index, so `docker pull` resolves it to your host's architecture. The per-architecture tags are published too, for when you need to pin one – a CI job building for a different host, or a registry mirror that does not carry indexes:

host architecture	resolved automatically from	explicit tag	download
linux/amd64	ghcr.io/lamblisp/lamblisp:runtime	:runtime-amd64	279 MB
linux/arm64	ghcr.io/lamblisp/lamblisp:runtime	:runtime-arm64	171 MB

Those sizes are measured, not estimated, and they differ enough to be worth stating separately. `:full` is **amd64 only** – it carries cross-compilers, so an arm64 variant would be a different build rather than a re-tag.

Pulling the amd64 tag on an arm64 host does not fail loudly – Docker runs it under emulation, which is slow and makes any timing you measure inside it meaningless. Check what you actually have with `docker inspect --format '{{.Architecture}}' ghcr.io/lamblisp/lamblisp:runtime`.

Building and flashing ESP32 firmware is not in this image. For that, use the Linux package and a local PlatformIO install; see *Deploy to ESP32*, and in particular *Flashing from a container* if you intend to build inside a container at all – USB passthrough does not work on every host.

1.6.3 The documentation is already inside

```
/opt/lamblisp/LambLisp.pdf      this manual
/opt/lamblisp/html/index.html   the same manual as a web page
/opt/lamblisp/demo/index.html   the in-browser demo
```

1.6.4 Which release is in the image

Every image records the packages it was built from:

```
docker run --rm ghcr.io/lamblisp/lamblisp:runtime cat /opt/lamblisp/BUILD-INFO
```

The same information is on the image itself, without running it:

```
docker image inspect --format '{{index .Config.Labels "org.opencontainers.image.revision"}}' \
    ghcr.io/lamblisp/lamblisp:runtime
```

The commit recorded there matches the published release repository, so an image and a checkout of the same release are the same code.

1.6.5 Building your own *LambLisp*: the `full` image

`ghcr.io/lamblisp/lamblisp:runtime` lets you *run* the language. A second image lets you *build* it – your own C++ primitives, your own Scheme library, linked against the prebuilt runtime library – with no toolchain installed on your machine:

```
docker pull ghcr.io/lamblisp/lamblisp:full
```

It is much larger than the runtime image (about 5.8 GB against 273 MB) because it carries every published release together with the cross-compilers for all of them.

What is inside. Twelve release packages under `/opt/lamblisp`, one per target – the ESP32-S3 boards, the ESP32-C5, the Freenove WROVER and 4WD kits, Linux x86-64 (plain, CUDA and HIP), Windows, and the loader. Each is the complete package you would otherwise download: `src/` minus the internals, the full

scm/ library, lib/, liblamblisp.a, a prebuilt LambLisp.bin, the manual and the HTML reference. PlatformIO and every toolchain are preinstalled, with PLATFORMIO_CORE_DIR and PATH already set, so nothing needs configuring.

Just run it. The image's default command starts the host runtime, so this drops you at a prompt exactly like the runtime image:

```
docker run --rm -it ghcr.io/lamblisp/lamblisp:full
```

Build a customised runtime. This is what the image is for. Each package's platformio.ini is locked to its own target, so pio run needs no -e:

```
docker run --rm -it ghcr.io/lamblisp/lamblisp:full bash
cd /opt/lamblisp/LambLispRT-v01-linux_x86_64
# add a primitive under src/, or edit the Scheme library under scm/
pio run
```

A host build of an unmodified package takes about half a minute. Cross-building for a board is the same command in that board's directory:

```
cd /opt/lamblisp/LambLispRT-v01-esp32s3-n8r2 && pio run
```

Keep your work. A container filesystem disappears when the container exits, so copy a package out to a mounted directory before editing it:

```
docker run --rm -it -v "$PWD/work:/work" ghcr.io/lamblisp/lamblisp:full bash
cp -a /opt/lamblisp/LambLispRT-v01-linux_x86_64 /work/mylamb
cd /work/mylamb && pio run
```

On Windows and macOS. The image is a Linux amd64 container, which is the ordinary case for Docker Desktop – on Windows it runs under the WSL2 backend, on Intel macOS natively. Two differences are worth knowing before you copy the commands above:

- **Path syntax.** The -v "$PWD/work:/work" form is shell syntax. In PowerShell write -v ${PWD}/work:/work, and in cmd.exe -v cd%/work:/work. Keep the container side (/work) exactly as shown.
- **On Apple Silicon or Windows-on-ARM**, an amd64 image runs under emulation. Builds will work and will be slow; do not take timing measurements from an emulated container.

--device passthrough for flashing does not work on Docker Desktop at all – the daemon runs inside a VM that cannot see your host's USB ports. On Windows you would need usbipd-win to attach the device into WSL2 first. **Build in the container, flash from the host** is the simpler arrangement and the one we recommend.

Building is not flashing. The image builds firmware; it cannot reach a USB device unless you pass one in explicitly (--device /dev/ttyACM0) and the container user has permission for it. Flashing from a container has a further limitation worth reading before you rely on it – see Flashing from a container.

Verifying your download. docker pull prints Status: Downloaded newer image whether or not anything was actually transferred – if the layers are already in your local store, the command returns in seconds having fetched nothing. That is normally what you want, but it means the status line alone does not prove a download happened. The honest check is the digest:

```
docker inspect --format '{{index .RepoDigests 0}}' ghcr.io/lamblisp/lamblisp:full
```

and, for which release you have:

```
docker run --rm ghcr.io/lamblisp/lamblisp:full \
    cat /opt/lamblisp/LambLispRT-v01-linux_x86_64/BUILD-INFO
```

1.6.6 A note on host compatibility

The image runs on any host with a working Docker. The host's own glibc version and CPU generation do not matter, because everything the binary needs is inside the image – which makes the container the most predictable way to run *LambLisp* on an older enterprise distribution.

The bare Linux packages have their own floor: they require **glibc 2.34 or later**, which covers Debian 12, Ubuntu 22.04 LTS, RHEL 9 and anything newer. If your host is older than that, the container is the supported route rather than a convenience.

1.7 Quick Start: Try LambLisp in Your Browser

`Open the live REPL` — no install, no account, nothing to download. It is the same interpreter that runs on an ESP32-S3, compiled to WebAssembly and running in your tab. Type `(define (fact n) (if (= n 0) 1 (* n (fact (- n 1)))))`, then `(fact 30)`.

It is a *language* demonstration and reports no timings, deliberately. LambLisp's real-time claim is a bounded garbage-collector pause on a microcontroller; a browser tab — with its own just-in-time compiler, its own collector, and its own background-tab throttling between you and the hardware — cannot demonstrate that either way. The pause figures that *are* claimed are measured on the boards, and appear in the product brief.

1.8 Using the REPL

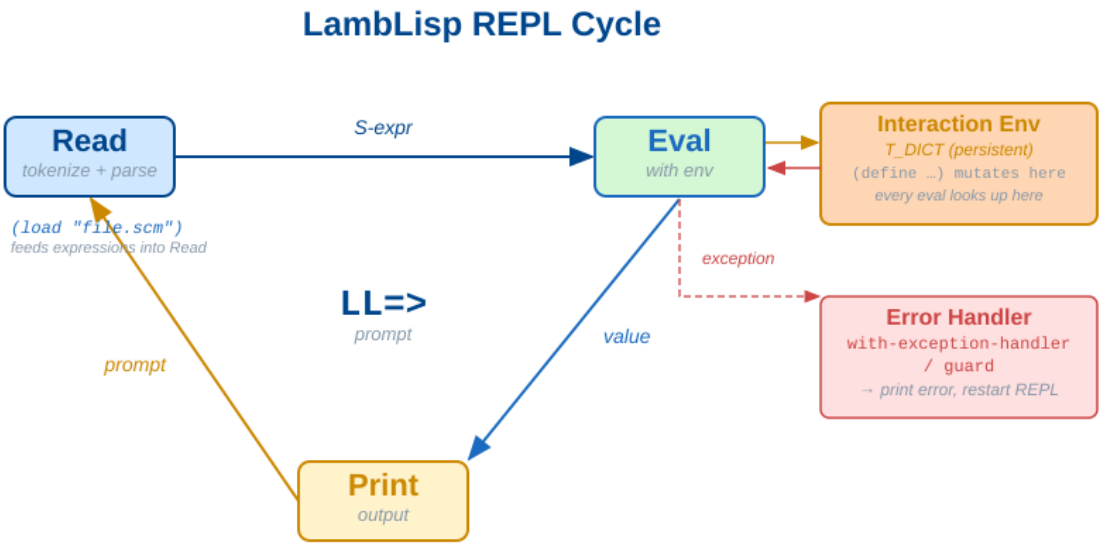


Figure 1.3 LambLisp REPL Cycle

The `LL=>` prompt accepts any Scheme expression. Input is line-buffered — the REPL evaluates each line as it arrives.

Loading files:

On Linux, the working directory is the repo root, so prefix paths with `data/`:

```
LL=> (load "data/PCA9685.scm" 1)   ; verbose load
LL=> (load "data/PCA9685.scm" 0)   ; quiet load
```

On ESP32, the working directory is `data/` on SPIFFS, so use the bare filename:

```
LL=> (load "PCA9685.scm" 1)
```

Error messages:

When an error occurs, *LambLisp* prints a diagnostic and returns to the REPL — the session is not lost. The GC and VM state remain valid; you can continue evaluating expressions immediately.

Exiting:

- Linux: press `Ctrl-C`.
- ESP32: reset the board or power-cycle.

1.9 Deploy to ESP32

1.9.1 Supported Platforms and Boards

LambLisp ships with pre-configured PlatformIO environments for all supported hardware. The table below lists every environment in `platformio.ini`.

Environment	Architecture	Flash / PSRAM	Notes
`esp32-s3-devkitc-1`	ESP32-S3	4 MB / 8 MB	Primary test board; onboard JTAG debugger
`esp32s3-n8r2`	ESP32-S3	8 MB / 2 MB	Higher flash, less PSRAM
`Freenove-4WD-Car-Kit-ESP32`	ESP32 (classic)	4 MB / 8 MB	Robotics demo platform
`linux_x86_64`	Linux x86-64	—	Full-featured POSIX build, no CUDA
`linux_x86_64_cuda`	Linux x86-64	—	POSIX build with CUDA support
`linux_x86_64_profile`	Linux x86-64	—	gprof profiling build (`-O1 -pg`)
`linux_aarch64`	Linux AArch64	—	Cross-compile for Jetson / ARM servers
`linux_aarch64_cuda`	Linux AArch64	—	Jetson native build with CUDA
`linux_x86_64_hip`	Linux x86-64	—	POSIX build with ROCm/HIP support

Environment	Architecture	Flash / PSRAM	Notes
`windows_x86_64`	Windows x86-64	—	Native `LambLisp.exe`, cross-built with MinGW-w64. Language build only — no sockets, no raw terminal, no native code generator
`wasm32_browser`	WebAssembly	—	The in-browser demo shipped as `demo/` in every package
`wasm32_wasi`	WebAssembly	—	WASI build, used by the conformance tier

ESP32 Hardware

- **ESP32-S3-DevKitC-1** — the primary test board. It has two USB ports: the first (`UART`) is used for firmware upload and the serial REPL; the second (`USB`) connects to the onboard JTAG debugger. Connect both to your development computer. The JTAG port works with VS Code, codelite, and other debugger GUIs.
- **ESP32-S3 N8R2** — a compact ESP32-S3 module with 8 MB flash and 2 MB PSRAM. Suited to applications that need more code storage but can work with a smaller heap.
- **Freenove 4WD Car Kit (ESP32)** — an alternative that exercises LambLisp's full control capabilities: digital & analog I/O, I2C motor control, strip LEDs, timers, and task queues. This board uses the classic ESP32 (not S3) and has no onboard JTAG debugger.

Note

It is useful to keep both an ESP32-DevKit and your target hardware available. When a difficult bug appears, the DevKit's JTAG debugger can save hours. Arrange pin definitions and error handling so the application runs safely on the DevKit even when target hardware is absent.

Linux / Jetson Builds

LambLisp runs natively on Linux without any microcontroller hardware. The Linux builds share the same Scheme interpreter, GC, and standard library as the ESP32 builds; only the hardware-specific modules (Arduino GPIO, I2C, WS2812, etc.) are omitted.

To build and run the Linux x86-64 version:

```
pio run -e linux_x86_64
.pio/build/linux_x86_64/program
```

For the CUDA-enabled build (requires an NVIDIA GPU and CUDA toolkit):

```
pio run -e linux_x86_64_cuda
.pio/build/linux_x86_64_cuda/program
```

For profiling with gprof:

```
pio run -e linux_x86_64_profile
.pio/build/linux_x86_64_profile/program
gprof .pio/build/linux_x86_64_profile/program gmon.out | less
```

The AArch64 environments cross-compile on Linux x86-64 using the AArch64 GCC toolchain, or build natively on the Jetson Orin Nano itself:

```
pio run -e linux_aarch64_cuda    # on Jetson: builds and runs natively
```

CUDA Integration

CUDA support is compiled in on the `linux_x86_64_cuda` and `linux_aarch64_cuda` environments. The C++ layer registers thin bindings to the CUDA runtime API, keeping their **native names** so they map one-to-one onto the familiar `cuda*` calls. At startup, `setup.scm` automatically loads `Cuda.scm`, which adds device-introspection and demonstration helpers on top of those bindings.

Runtime bindings (registered by the CUDA build):

Procedure	Description
`cudaGetDeviceCount`	Number of CUDA-capable devices visible to the process.
`cudaGetDeviceProperties`	Device properties (name, total memory, compute capability, ...) for a device index.
`cudaMalloc` / `cudaFree`	Allocate / free device (GPU) memory.
`cudaMallocHost`	Allocate page-locked (pinned) host memory.
`cudaMemcpy`	Copy between host and device; direction constants `cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`, `cudaMemcpyDeviceToDevice`.
`cudaMemset`	Fill device memory with a byte value.
`cudaStreamCreate` / `cudaStreamSynchronize`	Create a stream / block until its queued work completes.
`cudaDeviceSynchronize`	Block until all device work completes.
`cudakernel-vectorAdd`	Built-in element-wise vector-add kernel (example / benchmark).
`cudaLaunch-kernel1`	Launch a registered device kernel.

Helpers from `Cuda.scm`:

Procedure	Description
`have-Cuda`	`#t` when the CUDA bindings are present in the current build.
`cuda-printprops` / `cuda-printprops-brief`	Print full / one-line device properties.
`cudaStart`	Report device count and properties at startup.
`cudaTest1` / `cudaTest2`	Self-test / demo kernels (vector-add round-trip).

An AMD **HIP/ROCm** backend (`linux_x86_64_hip`) mirrors the same binding names, so code written against the `cuda*` API runs unchanged on AMD GPUs.

`Cuda.scm` is always loaded by `setup.scm` on CUDA builds. On non-CUDA builds the file is absent and the load is skipped silently. The `linux_x86_64_cuda` environment is convenient for developing and testing CUDA-dependent code on a desktop GPU before deploying to the Jetson.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

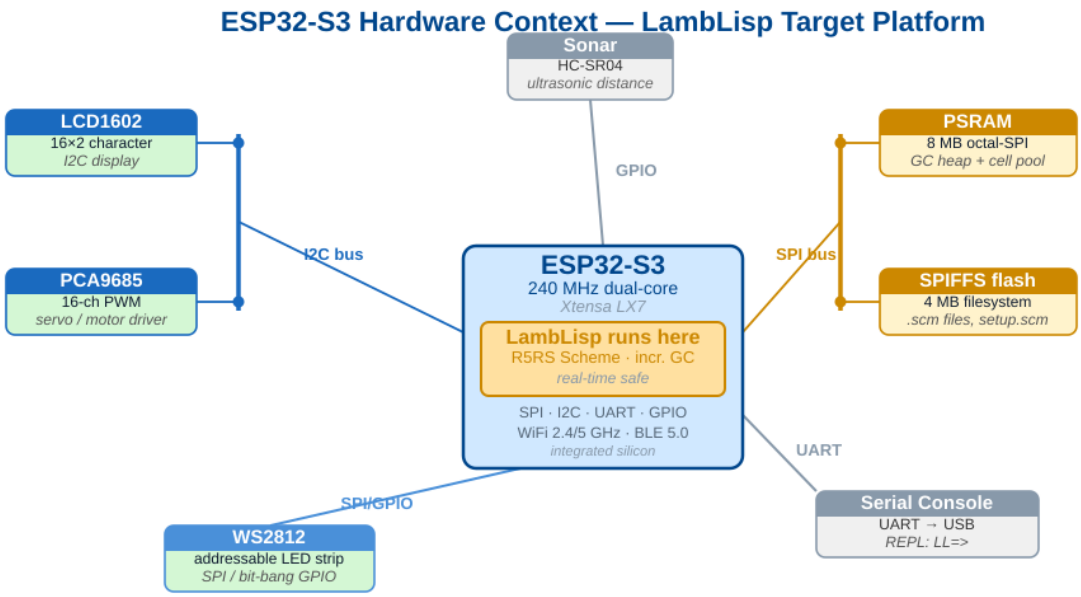


Figure 1.4 ESP32-S3 Hardware Context

1.9.2 Build and Flash

From a package, there is no environment to name — see Quick Start: The Package You Received, which is the shorter path and the one a customer wants:

```
pio run                       # build
pio run --target uploadfs     # the data/ filesystem image -- NOT optional
pio run --target upload       # the firmware
```

From the full development tree, where `platformio.ini` carries every environment in the table above, name the one to build:

```
pio run -e esp32-s3-devkitc-1
pio run -e esp32-s3-devkitc-1 --target uploadfs
pio run -e esp32-s3-devkitc-1 --target upload
```

For the Freenove 4WD Car Kit, substitute `Freenove-4WD-Car-Kit-ESP32` for the environment name.

Note

There is no separate platform-install step, and the one that looks obvious is wrong. `platformio.ini` pins `platform =` to a specific release URL and `pio run` installs exactly that on first use. `pio pkg install --platform espressif32` fetches a *different*, older platform — the official `platform-espressif32`, still pinned to Arduino 2.0.17 and an ESP-IDF that went end-of-life in 2024 — and nothing appears to go wrong when it does.

1.9.3 Flashing from a container

If you build inside a Docker container, **check this before you wire anything up**, because the symptom otherwise looks like broken hardware:

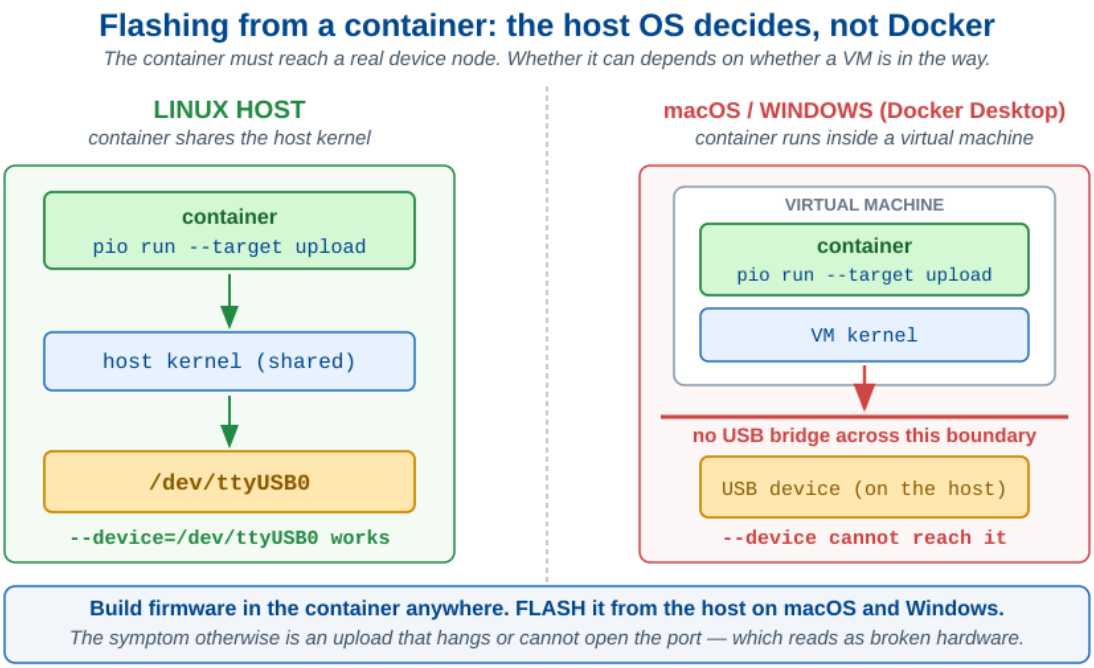


Figure 1.5 Why USB passthrough depends on the host OS

host	build firmware	flash a board
Linux	yes	yes – pass the port in, e.g. `--device=/dev/ttyUSB0` or `--device=/dev/ttyACM0`, and make sure the container user is in the owning tty group
macOS, Windows (Docker Desktop)	yes	**no**

On macOS and Windows, Docker Desktop runs your container as a guest inside a virtual machine, and **there is no USB bridge into that VM**. No Docker flag fixes this; the device simply is not there. The container can compile firmware perfectly and `--target upload` will then hang or report that it cannot open the port.

On those hosts, build in the container if you like, but **flash from the host** with a host-side `esptool.py`, pointing it at the `.bin` the container produced. That is the arrangement we recommend and the only one we test: *build in the container, flash from the host*.

On Windows, the host can supply what Docker cannot. The missing piece is a USB bridge into the VM, and `usbipd-win` is one – it attaches a host USB device into WSL2, after which a container on the WSL2 backend sees it as an ordinary `/dev/ttyACM*` and `--device` works:

```
winget install usbipd               # once, in an elevated PowerShell
usbipd list                         # find the board's BUSID, e.g. 2-4
usbipd bind   --busid 2-4           # once per device
usbipd attach --wsl --busid 2-4     # each time you plug it in
```

Then, from WSL2, pass the port in the usual way (`--device=/dev/ttyACM0`). **This path is not part of the supported route and we do not test it** – it is recorded here because the sentence above is true of Docker and not of the host, and a reader who concludes "impossible" would be giving up one step too early. On macOS there is no equivalent: flash from the host.

Do not hardcode a device node in an image or a script. Serial ports are enumerated dynamically and move between boots and between machines; resolve the port at run time and pass it in.

1.9.4 Connect to the REPL

After flashing, open a serial monitor at 115200 baud:

```
pio device monitor -e esp32-s3-devkitc-1
```

Or use any terminal emulator (`screen`, `minicom`, etc.) on the appropriate `/dev/ttyUSB*` or `/dev/ttyACM*` device. You will see the startup log followed by the `LL=>` prompt.

1.10 What Happens at Startup

When *LambLisp* starts, `main.cpp` (provided in the repo) does the following:

1. Creates a `Lamb` virtual machine instance.
2. Installs all native C++ operators (hardware I/O, math, etc.).
3. Loads `setup.scm` from the `data/` directory.
4. Calls the Scheme procedure `loop` defined in `setup.scm` on every iteration of the Arduino `loop()` cycle.

The provided `setup.scm` loads the standard *LambLisp* library files and defines a `loop` procedure suited to the demo hardware. For a new application, edit `setup.scm` (or the files it loads) to define the behavior you need. The minimum `setup.scm` for a custom application is:

```
(define (loop)
  ;; your control code here
  )
```

The file name `setup.scm` is fixed. The procedure name `loop` follows the Arduino convention but is not enforced by the VM; any number of *LambLisp* procedures can be called from C++ `loop()` at different points.

1.11 What's in the package?

LambLisp is delivered as a finished executable, but more importantly, as a set of library components that allow scalability by adding more native functions to *LambLisp*. There is a separate package for each supported architecture.

The package root contains these files and directories. A runtime package carries the documentation and runtime rows only; a development package carries all of them.

File or Dir	Description
`QUICKSTART.md`	Run it, build it, flash it — the short path, and the place to start
`README.md`	A mini-README, with links to the manual online
`BUILD-INFO`	Provenance: which commit, toolchain, build date and feature flags produced these binaries

File or Dir	Description
`LambLisp.bin`	Pre-built application binary — a host executable, or ESP32 firmware, depending on the target
`liblamblisp.a`	The *LambLisp* VM as a static library, linked by `platformio.ini` with `-L$PROJECT_DIR -llamblisp`
`LambLisp.pdf`	This manual, in PDF form
`html/`	This manual, in HTML form — open `html/index.html`
`LambLisp_ProductBrief.pdf`	Specifications, embedded benchmarks and measured GC-pause figures
`LambLisp_Announcement.pdf`	Product announcement
`demo/`	The in-browser demo — open `demo/index.html`, no toolchain needed
`src/`	C++ main program (`main.cpp`), platform layer, and device driver source
`lib/`	Vendored Arduino libraries, and selected source examples for extensibility
`data/`	The filesystem image for **this** target: `setup.scm`, the standard library it loads, features and I/O interfaces. `uploadfs` writes this to the board.
`scm/`	The **complete** *LambLisp* Scheme library source, shipped whole with every target — see `data/` and `scm/`
`tools/llip`	The LLIP command-line client: REPL, remote eval and file transfer over serial or TCP — see LLIP
`platformio.ini`	The build configuration, generated and **locked to this package's one target**
`ll_pio/`	Board, variant and partition definitions referenced by `platformio.ini`
`llloader/`	OTA factory-partition recovery loader, on OTA-capable targets — see OTA

There are also architecture-specific copies of the generically named `LambLisp.bin` and `liblamblisp.a`, which may assist those deploying multiple architectures.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 2

Lisp in a nutshell

Elements of *Lisp*
Spaces are separators between expressions.
If an expression starts with `(`, then it is a **list** and requires a matching `)`.
Otherwise, the next group of characters, up to a space, is a **symbol**.
In both cases the expression is called a **symbolic expression**, or **S-expression**.
A *list* contains 0 or more S-expressions between its parenthesis, as in `() (foo) (1 (2 three) 4)`.
There is an **environment** that associates symbols with values.
There is a small foundational set of **procedures** that accept arguments and return new S-expressions.
An **evaluator** continuously reads *S-expressions*, evaluates them, and returns the result.
If the evaluator encounters a list, the first item in the list must be a *procedure* and it will be **applied** to the arguments provided.

Note

To **apply** a procedure to a list of arguments, the *formal parameters* of the *procedure* are paired with the actual arguments provided in the application. Then those pairs are added as bindings to the environment of the *lambda expression*. Finally, the body of the *procedure* is evaluated in that augmented environment.

In the *Lisp* world, any item not a list (or more accurately a *pair*) is called an **atom**. Numbers, strings, and vectors all are *atoms*. Perhaps surprisingly, the foundational *Lisp* does not require these, as they can be implemented using purely symbolic programming. There is an important distinction here between the *mathematical foundations of Lisp*, the *programming language called Lisp*, and *implementations of Lisp*.

Many areas of mathematics have only thrived after a notation was introduced to facilitate reasoning about it. Think of *sine*, *cosine*, the *square root* symbol, integral and differential calculus, matrix algebra, etc.

Likewise, *Lisp* established a small set of symbols and their behavior when encountered in a program. In the days of pen and paper, Greek letters would have been used, as they were in the **lambda calculus** from which *Lisp* inherits its intellectual grounding and the **lambda** operator. In the teletype era, only the typewriter symbols were available.

Copyright 2026 Frobenius Norm LLC™ All specifications subject to change.

Lisp was created to perform *symbolic reasoning*. *Lisp's* elements of reasoning about symbolic expressions:

Lisp symbolic expressions	Description of operation
symbols	A sequence of non-spaces, such as `x`, `2day`, and `next<>thursday`. Non-space characters may appear anywhere in a *symbol*.
`#f` and `#t`	`#f` represents **false**, while `#t` represents **true**. Anything not *false* is considered *true*.
`()` or `NIL` or `nil`	A singleton that is an *atom* and a *list*, but not a *pair*. Used as a sentinel to terminate lists.
lists	A set of parenthesis enclosing 0 or more *symbols* or *lists*, as in `(1-element-list)` and `(nested (nested () list) lists)`
environment	An association between symbols and values. Often implemented as a list of pairs: `((x 12) (2day 42))`.
`if`	Execute test and choose from alternate code paths.
`eq?`	Accepts 2 arguments and returns **false** if they are not the same S-expression.
`define`	Update or install new `(symbol value)` pair in the current environment.
`set!`	Update existing `(symbol value)` pair in the current environment.
`lambda`	Creates a **procedure**; i.e., a set of formal parameters, a body of *Lisp* code, and the environment in which it was defined.
`eval`	S-expression **evaluation**; if a list, the *procedure* at the front of the list is **applied** to the remaining arguments.
`apply`	Execute the body of a *lambda*, in its original environment enhanced with the arguments to `apply` paired to the procedure's formal parameters.
`quote`	Prevent evaluation of something that would normally be evaluated.
`macro`	A procedure that accepts its arguments unevaluated, i.e., as source code, and returns new source code to be executed in its place.

Some terms should be defined in the *Lisp* context; for example *operators*, *functions*, *procedures*, *macros* are related concepts, but not identical.

Important axiomatic behaviors
The word **operator** is used for any S-expression found in the first position of a list at evaluation time.
An *operator* must resolve to a *procedure* before it can be *applied*, i.e., the operator position may be occupied by a symbol or other expression.
The word **function** is ambiguous and may refer to a *lambda* expression or a *procedure*.
A **lambda expression** defines a function's parameters and body, such as `(lambda (p1 p2 p3) (do-something p1 p2 p3))`
The first argument of a *lambda expression* is the list of *formal parameters*, and the remaining arguments (the *body*) are S-expressions to be evaluated at runtime.
A **procedure** is a *pair* consisting of a *lambda expression* and an *environment* in which the procedure was created.
A **macro** is a procedure whose arguments are not evaluated before the code body is applied; afterwards the result of applying the code body is further evaluated.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Note

When a *lambda expression* is executed, it returns a *procedure*. A *procedure* records the formal arguments and the body of the *lambda expression*, and also the *environment* in which the *lambda expression* was evaluated.

This by itself is an incredibly powerful set of mathematics, but note that it has no numbers or strings. That is because, from the point of view of symbolic expressions, numbers and strings are just symbols. It is easy to write a function to add 2 digits when the digits are just `0` and `1`. From this point of view numbers are not fundamental or axiomatic. Instead they are a result of agreement on the meaning of symbols and their combination. The same can be said of strings. When the problem is presented in terms of symbols and their relationships, programs become amenable to proofs of correctness by familiar methods such as substitution and induction.

Adding numbers, strings, and vectors to *Lisp's* capabilities provides computing facilities for real-world problem types. Adding these types means adding representations for them, and adding operators that can manipulate them. *Lisp* follows the usual rules for representing numbers and strings, and so below we present a list of typical symbolic expressions:

```
"This is a string atom, below is a number atom"
1
"Below is a list of number atoms"
(1 2 3)
"Below is a list of mixed types, atoms and a nested list"
(42 "another string" (nested list))
"Below is an arithmetic list expression; it must be evaluated to produce the result 15"
(+ 1 2 (* 3 4))
"Below is a symbol atom; when evaluated the result is the symbol's value in the current environment"
sym
```

Numbers, strings, and symbols are *atoms*. When a number or string is encountered by the evaluator it is not further evaluated, or put another way, it evaluates to itself. When a symbol is encountered by the evaluator, the symbol is located in the current environment, and the evaluator returns the value associated with the symbol.

Note that the expression `(1 2 3)` is a valid S-expression (a list), but the evaluator will reject it because the first element of the list (`1`) is not a procedure. The evaluator must be able to *apply* the first element, which therefore must be a *procedure*.

Note

An important feature of *Lisp* is the equivalence of code and data. In some sense, everything is data, until it is submitted to the evaluator function `eval`, at which point it is executed as code.

For example, the job of the *Lisp reader* is to accept *Lisp* input from an implementation-defined *port*, and submit that input to the evaluator. Until it gets to the evaluator, it's just data.

Lisp was developed by discovery, rather than predefinition, and each successful experiment led to another language "standard feature". Eventually this led to overstuffing, as witnessed by Guy Steele's 1000-page tome *Common Lisp*. With a specification that size, many partially compatible subsets emerged, losing some of the *Common* in *Common Lisp*.

In reaction to the size of Common Lisp, an effort was made to extract the most useful and essential behaviors of Lisp into a simpler package, which became *Scheme*. Just as effectively as Guy Steele had recorded *Common Lisp* for posterity, he was also a key figure in factoring out the fundamental behaviors exhibited by a Lisp machine. Those basic behaviors, in combination, could be used to produce all the other more complex behavior. The result is the set of functions found in *Scheme*.

The *Scheme* variant of *Lisp* has many features familiar to programmers in other languages, such as *lexical scoping* and *dynamic typing*, as well as advanced features such as tail recursion, first-class procedures, and continuation-passing style.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 3

Why Lisp?

LISP is the original language of artificial intelligence, and is the second-oldest programming language still in use (after Fortran). It was designed for *symbolic reasoning* rather than computation. From the theoretical base beginning about 1960, *LISP* quickly grew into a practical language with several well-supported variants. *LISP* and LISP programs were always uppercase; the 8-bit byte had not yet been standardized and text was often packed into 6-bit chunks with no room for lowercase.

Today, approaching the mid-21st century, *Lisp* is active in many variants: SKILL from Cadence Design, Auto↩ Desk Lisp, ChezScheme from Cisco, and others. Like *LambLisp*, these are often tailored to solve problems in a specific domain, and descend from either *Scheme* or the earlier *Common Lisp*. *LambLisp* is optimized for intelligent real-time control of physical processes.

The *LambLisp* bibliography contains a mini-history of automated symbolic reasoning since the Renaissance. It contains work beginning with Isaac Newton and Bertrand Russell, through to Alonzo Church, John Mc↩ Carthy, and Edsger Dijkstra, as well as many less famous and more recent authors. In the 21st century development of automated symbolic reasoning continues, with recent publications on already well-trodden topics such as real-time garbage collection and macro expansion. New approaches to artificial intelligence have created renewed interest in systems (like *Lisp* and its descendents) that offer provable correctness and traceable results.

Any papers cited in the *LambLisp* documentation can be found in the bibliography at
`https://github.com/wawhite3/LambLisp-Bibliography`

Additionally, these books provide excellent exposure to many of the issues encountered when developing a *Lisp* programming system:

- Structure and Interpretation of Computer Programs (Abelson and Sussman). Known as SICP, it was a standard Computer Science textbook for many years.
- Lisp in Small Pieces (Queinnec). Another excellent textbook, which drew lessons from SICP and examines more topics and more detail.
- The Art of Computer Programming (Knuth). Written in the pre-C era, TAOCP is an encyclopedia of algorithms that remain foundational to modern computer science.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Lisp invented, aggregated, or evolved all the important techniques used in programming languages today, including interactive computing, just-in-time compilation, on-the-fly code updates, macro expansion, polymorphism, object systems, lexical and dynamic scoping, tail recursion, garbage collection, and exception handling.

Lisp offers tremendous scalability. At the *micro* end, *Lisp* makes an ideal *assembly language*, with a simple prefix notation that can transform directly into machine instructions. At the *macro* end, new functions can be defined and combined dynamically in a way not possible in C++ or Python. These new functions can be the results of substantial analysis performed elsewhere (call it **AI**), putting it at a scale billions of times larger than the assembly language application.

Lisp is **interactive** and *Lisp* programs can be modified on-the-fly, while they are running. Programs can evolve. In other languages, predefined parameters may be tuned during operation, but the code does not change. In *Lisp*, the code may continually evolve, and provides *Lisp* with the power of induction and of continuous improvement.

The concept of interactive computing applies not only to interaction with people. **Machine-to-machine interaction** is an inherent behavior of the interactive nature of *Lisp*. In *LambLisp*-based control systems new algorithms can be developed off-system, and then downloaded without interrupting the controlled process. Today, those off-system resources can be **AI assets** in the cloud, and *LambLisp* can be the target of **AI-generated code**.

An important part of the attraction of *Lisp* is the simple syntax, based on just a few rules that are compact and fast on any processor, even in 1960. This led to a variety of *Lisp* dialects; two of the most important are *Common Lisp* and *Scheme*. *Common Lisp* is in the "large Lisp" category, although there are many partially-compatible subset implementations.

Scheme is in the "medium-size Lisp" category, and was a purposeful attempt to distill the fundamental behaviors of *Common Lisp* into a simpler set.

In the lifetime of *Lisp*, many processors have come and gone. Today ARM-based processors have scaled to be competitive in many control applications. There is a wide variety of boards and modules available, and plenty of inexpensive parts running on the commonly available I2C bus. Today's 32-bit and 64-bit microprocessors and edge SoCs present an ideal platform for *Lisp* real-time control.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 4

Why *LambLisp*?

There are some features that make *LambLisp* special in the *Lisp* menagerie.

4.1 Real-time control

LambLisp provides bounded loop times, sub-microsecond interrupt response, and non-blocking asynchronous scheduling through eight real-time guarantees spanning GC latency, loop control, ISR paths, and deferred tasks.

See: Real-time Guarantees · Garbage Collection · Memory and GC Tuning

4.2 Multi-platform

LambLisp is a multi-platform library: the same application Scheme code runs unchanged on ESP32, Jetson, and Linux. The platform-specific layer is thin — Arduino-style I/O on ESP32, POSIX I/O on Linux, CUDA bindings on Jetson. All language semantics, the GC, and the REPL behave identically. Develop interactively on Linux; deploy to ESP32 by copying the `.scm` file.

See: Multi-Platform Support

4.3 Widely recognized *Scheme* language specification

LambLisp implements the **Scheme R5RS** core in full. The one deliberate departure is that `call/cc` is **escape-only**: it supports non-local exit, but not re-entrant continuations that resume an extent after it has been left. `dynamic-wind` is implemented and runs its `after` thunk on both the normal exit and the error unwind. All other R5RS features are supported. See *Why only escape-only `call/cc`?* for the reasoning and the two limits. **R7RS-small is complete apart from re-entrant continuations.** The library system — `define-library` and `import`, including the `only`, `except`, `prefix` and `rename` forms — is implemented, so the sole remaining gap is the escape-only `call/cc` described above. Of the 1370 R7RS conformance tests, **1360 pass and 0 fail**; every one of the 10 exceptions is that single re-entrant case. See the Compatibility Matrix. *Scheme* is widely taught and there is plenty of information online. There are also many desktop versions of *Scheme* available for study.

See: Compatibility Matrix

4.4 Reuse of existing C++/Arduino libraries

LambLisp can incorporate existing C/C++ libraries unchanged; existing Arduino drivers require no modification. Six integration patterns — stateless procedures, stateful GC-managed objects, device facades, syntax forms, proprietary distribution, and data marshaling — cover the full range of embedding needs.

See: C++ Integration

4.5 Adaptive real-time garbage collection

Lisp behavior depends on a continuous supply of memory, which must be reclaimed when no longer in use, and then later reissued. The reclamation process is called **garbage collection** or just **GC**.

Until the 1990s, *Lisp* could not be applied to real-time control applications due to unpredictable lapses in control during *GC*. While *GC* was occurring, the controlled application was left unsupervised. This problem was solved by Taichi Yuasa, based on earlier work by Edsger Dijkstra. Yuasa provided a provably correct solution that divided *GC* into increments, and provided formulas for the key metrics in a correct implementation.

In *LambLisp* real-time guarantees are provided by Yuasa's *GC* formula. *LambLisp* continuously calculates the *GC* start threshold (Yuasa's M metric) and allocates additional memory when required (Yuasa's N metric).

LambLisp realizes additional performance gains with *GC* idle-time look-ahead. By setting a value for *idle threshold*, idle time can be applied to garbage collection; later during peak loads GC passes can be omitted altogether. This effectively pushes some workload out of the peak periods and into idle periods, reducing peak loop time.

These idle-time collections will be larger than the normal GC quantum. A small additional benefit is therefore obtained because GC loop overhead is spread over a larger collection than the normal GC quantum.

See: Garbage Collection · Memory and GC Tuning

4.6 Open API for new *Lisp* primitives

Any C++ function with the three-argument `mop3` signature becomes a native Scheme procedure with no FFI overhead — the same mechanism used to implement all LambLisp built-ins.

See: C++ Integration

4.7 Incremental over-the-air updates without reboot

LambLisp supports incremental hot-reload without a reboot — new Scheme code takes effect immediately in the running control loop, with no flash erase and no control gap.

See: Code Lifecycle: Incremental Hot-Reload · Hot-Reload Without Reboot

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

4.8 First-class hierarchical dictionaries

A part of the success of Python has been its pervasive use of hash tables. Hash tables are also used extensively in *LambLisp*, supporting the representation of environments, dictionaries, and objects. *LambLisp* dictionaries are *hierarchical*, providing a parent/child relationship between hash tables. The internal execution environment of *LambLisp* is a dictionary.

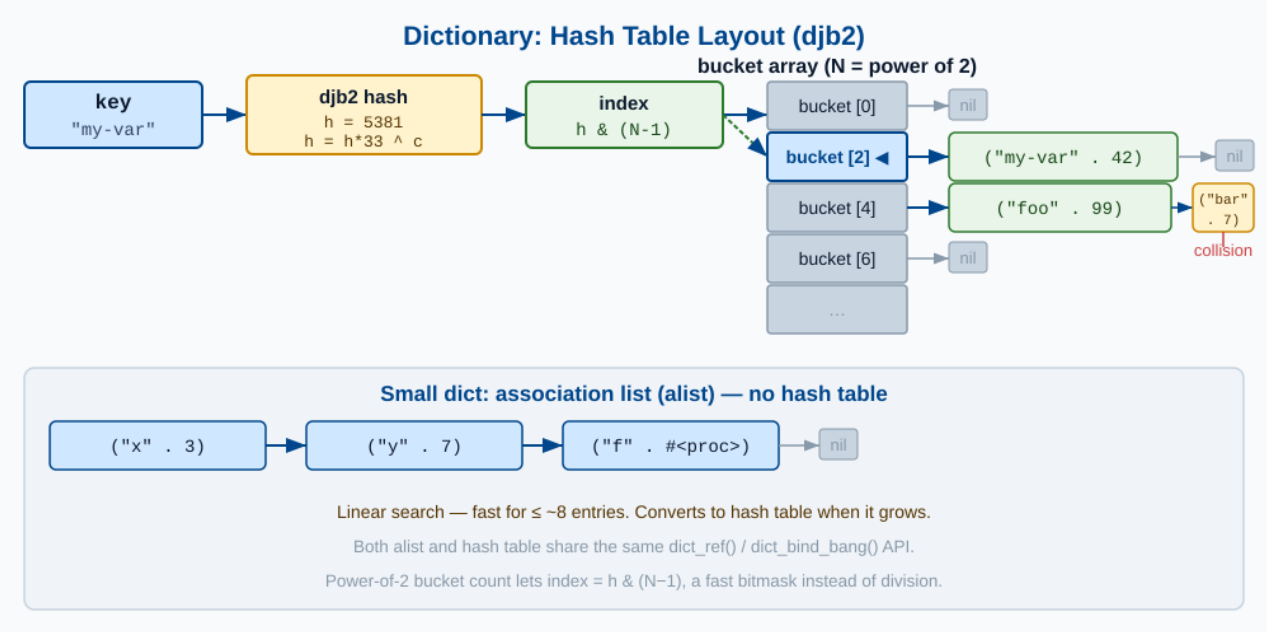


Figure 4.1 Dictionary: Hash Table Layout (djb2)

See: The dictionary type · Hierarchical dictionaries · Execution environment

4.9 Incremental, just-in-time compilation

Much of *LambLisp* behavior is implemented in the *runtime system*, a collection of compiled routines that perform the basic operations defined in the *Scheme* specification. The process of compiling begins with "expanding" those symbols whose values are functions. In this case, "expanding" means to replace the symbol with its value (a *procedure*) wherever appropriate. This can be done case-by-case; when a *LambLisp* program is fully transformed this way, execution becomes a series of linked C++ function calls. The downside to this is that debug information is lost.

See: Scalable Execution Pipeline

4.10 Lexical scoping

Lexical scoping is what most programmers are already familiar with. C/C++ and Python programmers will discuss *local variables* and *global variables*; these are typical in lexical scoping. Some *Lisps* (but not *Scheme* or *LambLisp*) have *dynamic environments* and *special variables*, which do not follow the familiar lexical convention. In *Scheme* and *LambLisp* there is only *lexical scoping*.

See: Execution environment is a hierarchical dictionary

4.11 Tail recursion and tail-calls

Tail calls allow many problems to be solved recursively without a growing stack, allowing compact, inductive code in place of iteration.

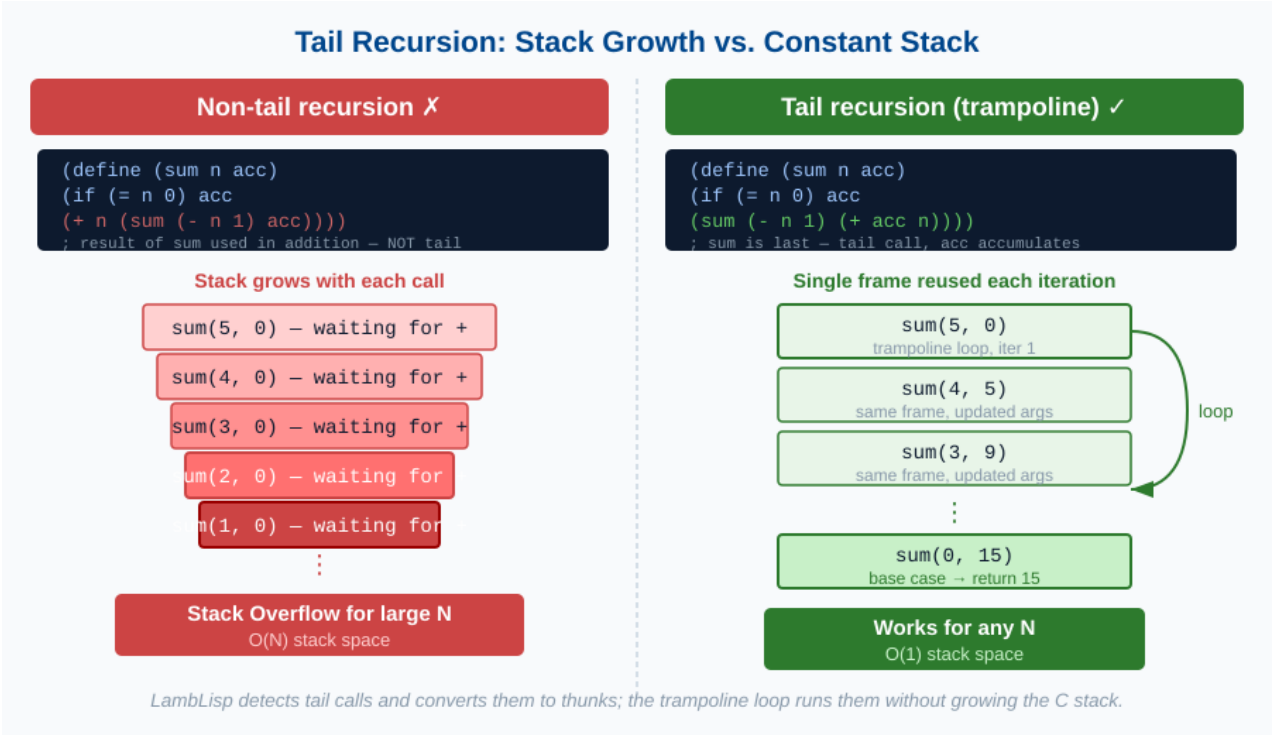


Figure 4.2 Tail Recursion: Stack Growth vs. Constant Stack

Continuation-passing style is a programming idiom on which the next step in a computation is always an extra argument to every procedure. That "next step" is called a *continuation*. When a procedure has determined its result, instead of returning, it tail-calls the continuation. This is an elegant way to solve many problems by induction.

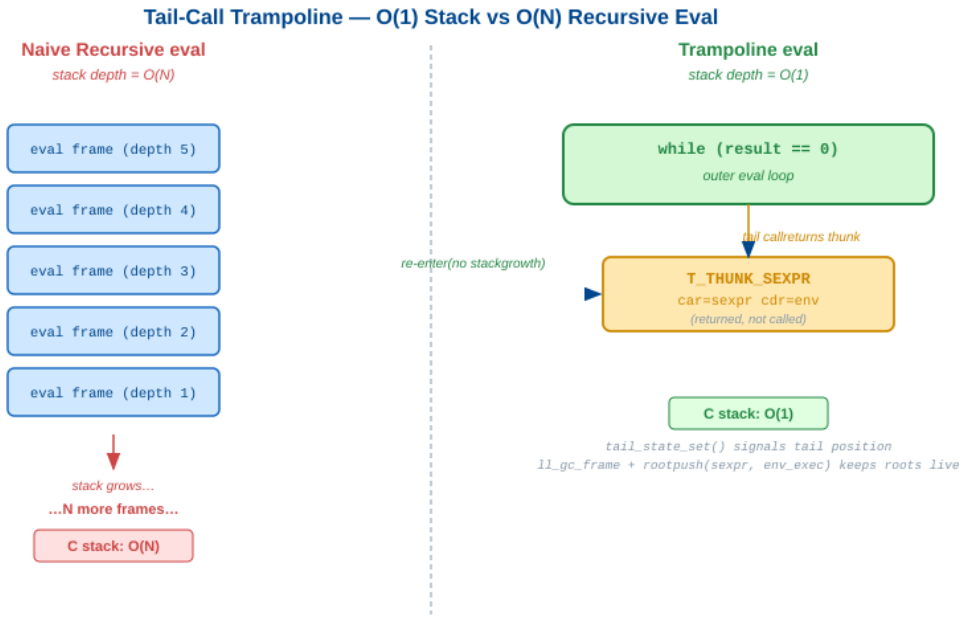


Figure 4.3 Trampoline: O(1) Tail Calls

See: Interpreter & compiler organization

4.12 Procedures are first-class data types

Procedures are created as the result of evaluating a **lambda expression**. They get created, stored, and destroyed, just as any other value. They can be passed as arguments to other procedures. In LambLisp the **nlambda** and **macro** expressions are specialized *procedures*.

See: lambda, nlambda, and macro

4.13 Runtime collaboration with C++ code

C++ objects with persistent state can be placed under LambLisp GC control and passed as first-class Scheme values; they are automatically deleted when they become unreachable.

See: C++ Integration

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

4.14 Key feature summary

LambLisp Features
R5RS core fully implemented; `call/cc` is escape-only by design and `dynamic-wind` is supported. R7RS-small complete apart from re-entrant continuations — 1360 of 1370 conformance tests pass, 0 fail.
Additional features from Scheme R7RS and Common Lisp specifications.
Runs on 32-bit ESP32/ESP32-S3, 64-bit x86-64, and 64-bit AArch64 (Jetson).
Full Linux builds — essential edge and supervisory tier, not a port.
High-level implementation of language primitives for compact high performance.
Bytecode compiler and native code generator (JIT) for x86-64, AArch64, and Xtensa LX7.
Arduino-compatible interfaces for digital & analog I/O, I2C, WiFi, etc.
Easy addition of manufacturer-provided C/C++ hardware drivers.
Easy integration of application-specific C++ functions.
Leverage existing Arduino skill set, gcc-based tool chain, C runtime.
Adaptive incremental garbage collector for fast, uniform loop times.
Selectable run-time packages (math, ethernet etc) to control program size.
Virtual memory capability from remote or local storage.
Incremental code updates over-the-air without pausing control or rebooting.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 5

LambLisp Architecture Internals

Terms introduced in this chapter — cell, sexpr, dict, env, GC phase, thunk, mop3, NCG — are defined in the Glossary at the end of this document.

5.1 Background

5.1.1 Brief history of Lisp and Scheme

During the 1960s, a great many of the desired and desirable behaviors of programming languages were still being worked out. While Fortran was used heavily for numeric computation, *lexical scoping* (which C programmers experience as "local variables") arose in the Algol language, and later "message passing" systems (primarily *SmallTalk*) resulted in the "tail recursion" technique and interactive computing.

LISP provided a significant abstraction of the computation process. By operating primarily on addresses rather than values, *Lisp* provided a method for creating new and arbitrary data structures. A great deal of research and discussion was expended on functions and their abstract properties (fexprs, property lists, nlambdas and the rest). With these tools and techniques, *Lisp* provided a facility for reasoning, not just calculating.

The experimental nature of *Lisp* led to a large number of overlapping solutions to overlapping problems. This led to the development of **Common Lisp**, a 1000-page compendium of *Lisp* functions that had proven useful.

At the same time, there was a realization that "more and bigger" *Lisp* was not necessarily a good thing, producing many incompatible partial implementations. An alternative mindset emerged: a small set of language primitives would be easier to standardize, provided that set was sufficient to construct the richer functions found in *Common Lisp*.

These and other ideas — especially *tail recursion* and *continuation-passing style* — came together in *Scheme*, designed in the 1970s to integrate the theoretical advances and practical understanding accumulated over the previous 15 years.

Scheme went through several iterations and the usual technical-committee process, resulting in two main specifications today: R5RS and R7RS. There has always been tension between *Scheme* minimalists, who prefer a small set of primitives, and utilitarians, who prefer a language useful for real-world problems. This led to the R7RS specification being divided into R7RS-small and R7RS-large, with R7RS-small already being significantly larger than R5RS.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.2 The *dictionary* type

In computer science, a *dictionary* is a data structure and set of operations for managing `(key value)` pairs.

LambLisp function	Dictionary operations
dict-set!	install or replace a `(key value)` pair
dict-ref?	return a `(key value)` pair based on its key
dict-ref	return a value based on its key

The name *dictionary* applies to the features and functions performed, and not to any particular implementation. A sequential search or a database query are both useful implementations of a dictionary for different purposes, with different performance expectations.

LambLisp uses several advanced techniques to ensure high performance dictionaries in embedded control.

Large or long-lived dictionaries are implemented as *hash tables*. When a dictionary is large or long-lived, great performance gains can be obtained by distributing the `(key value)` pairs throughout an array. To perform a lookup, compute a *hash value* for the key, and use that as an array index. The `(key value)` pair, if present, will be in the list of items at that index. The hash function, used to create a hash value from a `key`, produces a fixed-length number from the contents of the key. The output of the hash function must be the same whenever the same key is presented as input. The hash values produced should be widely distributed among the hash table indexes.

Small dictionaries are implemented as association lists, or **alists**. When a dictionary is small or short-lived, the one-time cost of creating a hash table is unjustified.

In hashed dictionaries, at each element of the hashed array there is a (very short) *alist*, so *alist lookup* is a foundational operation and should be as fast as possible.

LambLisp optimizes symbol lookup by computing hash values once when a symbol is created, and storing the hash value with the symbol. The most common symbol operation (test for equality of symbols) reduces to a fast pointer comparison when using a dictionary.

The dictionary type described above is a *flat dictionary*. That is, there is just one vector, and a `(key value)` pair is either present or not.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.3 Hierarchical dictionaries

LambLisp implements *hierarchical dictionaries*. A LambLisp *hierarchical dictionary* consists of a list of *frames*, where each frame is a *flat dictionary*. Hereafter, any reference to *dictionaries* should be understood to refer to LambLisp *hierarchical dictionaries*. In cases where a *flat dictionary* is required, it will be called out specifically.

The fundamental operation in a dictionary is **lookup**. A *lookup* must be done to support every other dictionary operation: create, modify, delete etc. The *lookup* operation consists of providing a key K, and querying the dictionary for a `(key value)` pair having a key matching the key K provided.

When performing a dictionary lookup, each frame is checked in turn for a matching key. If a key is not present in the top frame of the dictionary, it may be present in succeeding (parent) frames. The *dictionary* is a first-class type in LambLisp, available to the Lisp application.

The dictionary type is a fairly general purpose data structure. Keys and values may be any type. Each frame may be an alist or vector, with the choice made on frame size and expected lifetime. Symbol keys are computed once and stored with the symbol, speeding up dictionary operations where the keys are symbols (as in environments and commonly as in object member identifiers).

The primary method of creating dictionaries is with the function `alist->dict`, which traverses an association list and returns a dictionary. The `(key value)` pairs of the alist are distributed into a hash table, which becomes a frame in the dictionary. It is also possible to specify a parent dictionary when creating a dictionary, or NIL if there is no parent (as in a *flat dictionary*). Because dictionaries are lists (of frames), they can be appended or examined with *car* and *cdr*, just as with other lists.

In addition to being available in the *Lisp* application, *hierarchical dictionaries* are central to several other aspects of LambLisp.

5.4 Execution environment is a *hierarchical dictionary*

The execution environment is a hierarchical dictionary. In the *Scheme* specification, there are 2 environments mentioned, the *syntactic environment* and the *interaction environment*. The syntactic environment contains all those definitions that are found in the *Scheme* specification. The other is called the *interaction environment*. This is where application definitions are stored.

In LambLisp, the execution environment (which is a dictionary) is initialized with 2 frames. The base frame consists of the *syntactic environment* described in the *Scheme* spec, and the second (top) frame is the *interaction environment* also described there. Top-level definitions are installed in the interaction environment.

When a function is called, its formal parameters are paired with the associated runtime values, and that set of pairs is added as a new frame on top of the current execution environment. This naturally and elegantly implements lexical scoping.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

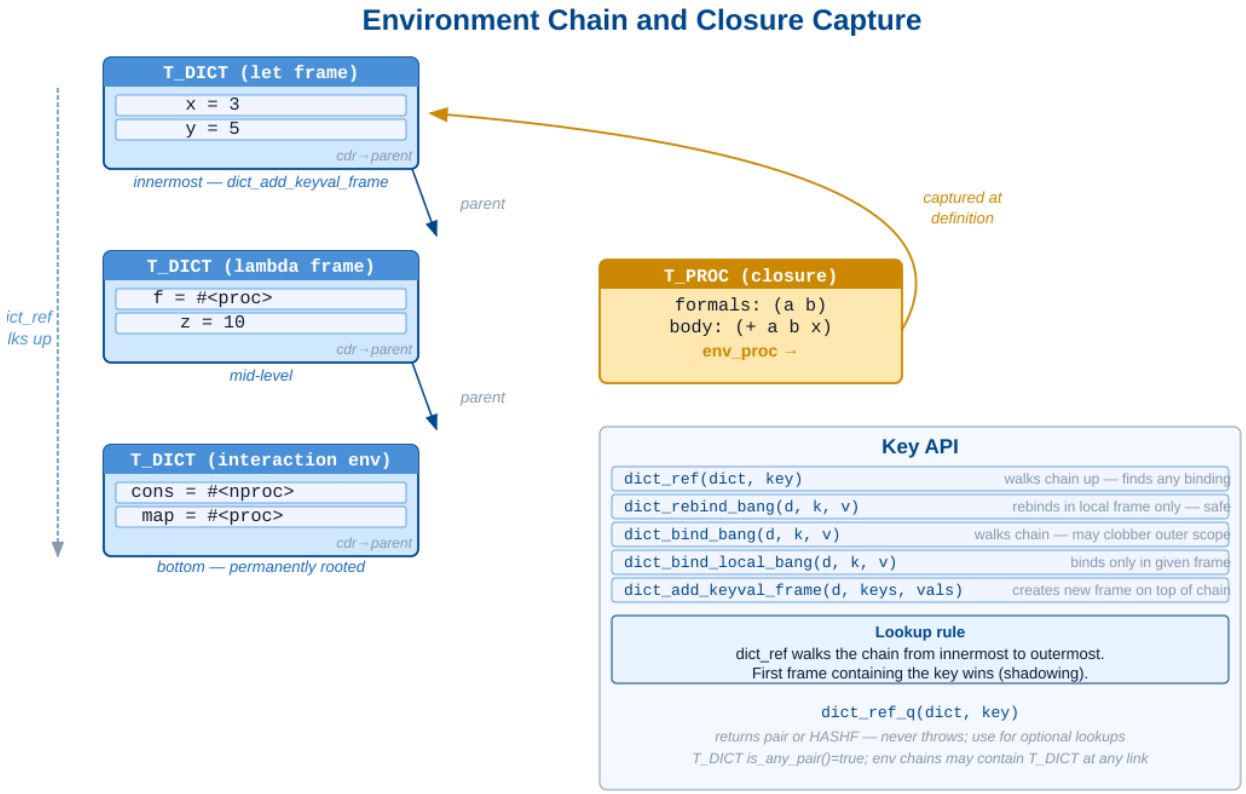


Figure 5.1 Environment Chain and Closure Capture

5.5 Objects are wrappers around *hierarchical dictionaries*

Hierarchical dictionaries also form the basis for LambLisp's Object System (LOBS). Just as the dictionary presents an efficient model for the execution environment, it also provides an excellent base for an object-based approach.

In the object view each frame in a dictionary represents an instance of a class, while the parent frames in the dictionary represent the parent object instance. This leads naturally to inheritance behavior, including multiple inheritance.

In the LambLisp Object System, there is no need to separate class definition from class instantiation. An association list is used to create a new dictionary, and the same alist can be used to produce new instances. In effect, the alist *is* the class definition.

This leads to some interesting freedoms as compared with the C++ or Python approach to class definition and instantiation.

LOBS features
Objects are created from an association list and (optionally) parent objects.
Objects inherit from their parent objects.
Multiple objects may inherit from the same parent object.
Objects may initialize from the same alist, but have different parent objects.
Objects may initialize from different alists, but may share a common parent object.
Objects may have new fields added/deleted any time, as any other dictionary.

To use LOBS as a C++-style object system, do the following:

1. Initialize all objects of the same type from the same association list.
2. Create and initialize new parent objects for each new child object.

To round out the object model, LambLisp provides `dict->obj`. You have correctly perceived that this will convert a dictionary into an "object". This provides a minimal object-style accessor for a dictionary, such that `(some-object 'foo)` will return the value of the field `foo` in the object `some-object`. and `(some-object 'foo 42)` will set the corresponding field on `some-object`.

The simple *set/get* object model suffices for many cases. It is possible to add more complex behaviors by assigning procedures to dictionary values, and/or by enclosing dictionaries in a wrapper procedure.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.6 Interpreter & compiler organization

Language implementations depend on *primitives*; these are the language elements that are assumed to exist when a programmer is programming in that language. For a high-level language, the *if-then-else* sequence is a common language primitive.

Low-level languages, such as assembly language, also have a set of primitives. Each primitive does less work when used, but by coding directly in assembly language optimization can be implemented that cannot at the higher level, and the result can be many times faster than a higher-level language. At some level, even a compiled high-level language is still interpreting a lower-level machine.

Many available Lisp implementations are based on underlying *abstract machines*. Two of the most important such abstract machines are the **SECD machine** (Landin, 1964) and the **CEK machine** (Felleisen and Friedman, 1987). *TinyScheme*, for example, uses an *SECD machine* as its execution target.

LambLisp implements interpretation of Lisp at a high level; each of the functions in the *Scheme R5RS/R7RS* specifications is implemented in C++. They all share the same C++ signature:

```
Sexpr_t any_function(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

The `lamb` parameter is the Lamb virtual machine instance which is to execute the operation. The parameter `sexpr` is the *symbolic expression* to be operated upon by the function, and `env_exec` is the environment in which to execute the function. The return type is also a *symbolic expression*. Note that `env_exec` is known to be a dictionary, and that its keys are all symbols.

This organization has several implications.

First, the language primitives are coded directly in C++, rather than interpreted at run time through a lower-level bytecode system. This makes compilation of the language primitives easy; simply replace the symbol with its value at read time, using a macro facility.

Second, the *Scheme* specification describes only a small set of primitives, with much of the language being "derived expressions". These derived expressions are not language primitives, but instead they either get expanded once at read time, or interpreted at run time. Their performance depends entirely on the expanded versions.

In the highest-performing cases, macros expand into machine code, utilizing the CPU registers directly to carry out *Lisp* operations. That high performance comes with costs. In an embedded system, that type of compiler would be included onboard, taking up space even though used only occasionally. Alternatively, an offboard compiler could be used. That makes some features difficult to implement, such as on-the-fly incremental code updates.

The derived expressions in the *Scheme* specification form a "compiler" that transforms *Scheme* source code into expanded code that relies on relatively few low-level primitives. Some of those lower-level primitives, such as *if*, *define*, and *set!*, are fundamental to the application of *lambda calculus* to computing, while others (such as mathematical functions) are necessary to solve practical problems.

The simple compiler has a disadvantage: the code it generates is simpler to execute than the source, but not simple enough to be fast unless it is designed to produce assembly-level native code. For example, an expansion of `cond` results in many sequential `if` functions to be executed. In an optimizing compiler, a series of low-level test-and-jump sequences will be coded inline, one for each `if`. This will be very fast.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Short of that, executing `cond` requires a pass through the main loop for each test clause. A system that expanded `cond`, but required sequential execution of `if` (in the *Lisp* context), would be relatively slower.

LambLisp has implemented nearly all the *Scheme* language elements as primitives, written in C++. This reduces the number of steps to be interpreted between source code intake, transformation into an *abstract syntax tree* (AST), and its execution in C++. In the design of *LambLisp*, a `cond` in *Lisp* source code corresponds directly to a C++ function `cond(...)`. Encountering `cond` in an expression does not result in a chain of small bits of `if` execution. Instead, the C++ `cond` is called, either after its symbol is resolved in interpreted mode, or called directly through a pointer in compiled mode. All the clauses of the `cond` are executed in a loop at C++ speed, and not in a loop at the *Lisp*-language level or in a bytecode interpreter.

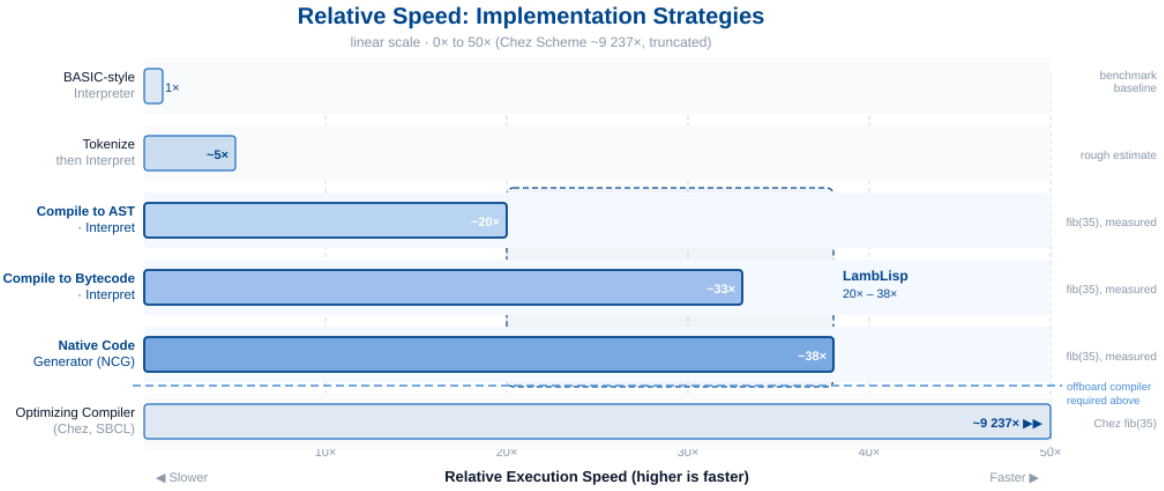


Figure 5.2 Performance Characteristics of Several Implementation Strategies

The chart above shows where each strategy lands on a log scale of relative execution speed. *LambLisp*'s measured results confirm that bytecode and NCG tiers outperform the AST interpreter by 2–100× depending on the benchmark: loop-intensive code (iota, sieve) benefits most; recursive code (fib, tak) shows only a modest gain because each AST eval step invokes a full C++ primitive and performs more useful work per dispatch than a bytecode fetch-decode cycle.

The resource chart below shows the cost side of the same picture.

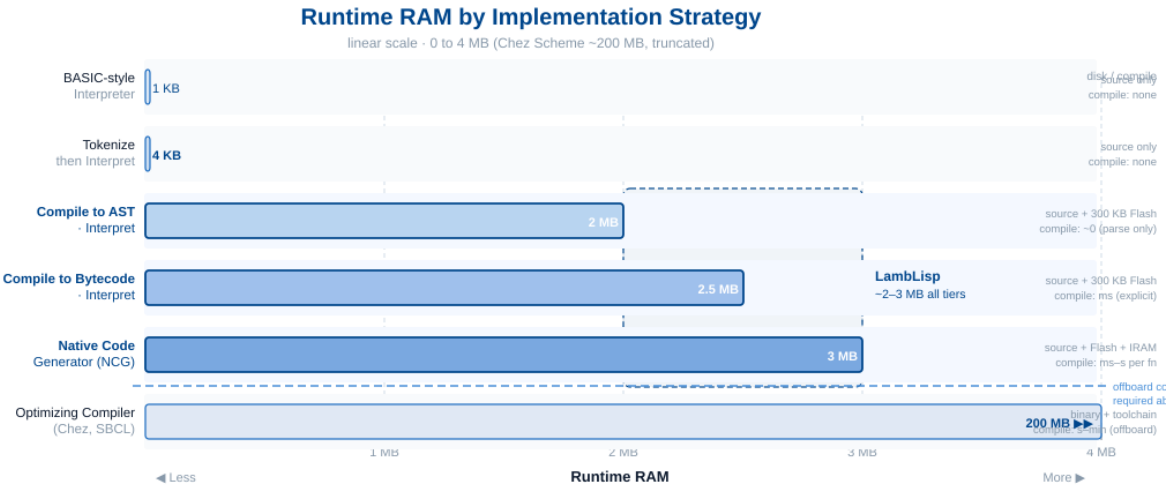


Figure 5.3 Runtime RAM by Implementation Strategy

The AST, Bytecode, and NCG tiers of *LambLisp* cluster tightly at ∼2–3 MB — the same GC heap fits all three. The dashed line marks the boundary where an offboard compiler becomes required; above it, the runtime image alone exceeds the total addressable PSRAM of an ESP32-S3. Chez Scheme's runtime stands as far to the right on the resource axis as it does on the performance axis, confirming that the ∼250× speed advantage comes with a ∼100× RAM cost — a trade-off that does not exist for embedded targets.

Why the compiled tier does not run on embedded hardware

The optimizing compilers — Chez Scheme, SBCL, Chicken, Bigloo, Racket — deliver their performance by making assumptions that are false on an ESP32:

Requirement	Desktop	ESP32 / LambLisp
External toolchain	Required (compiler + linker)	None — source loaded on device
RAM	10 MB – 1 GB	∼2 MB PSRAM
Operating system	Full POSIX (mmap, fork, exec)	FreeRTOS (no VM, no fork)
Writable+executable memory	Available	Constrained; IRAM alias required
Garbage collector pauses	Tolerated (ms to tens of ms)	Must not exceed loop period (10 ms)
Hot reload without reboot	Not supported	Core requirement

Compilation step. Chez and SBCL compile Scheme to native code on the development machine, producing a binary that requires linking against a full runtime. The compiler itself requires hundreds of megabytes. Chicken compiles Scheme to C and then to native code via GCC — the device never sees source code. *LambLisp* reads `.scm` files directly from SPIFFS at runtime; no offboard tool is ever involved.

Memory. The Chez runtime image alone exceeds the entire addressable PSRAM of an ESP32-S3. LambLisp fits in approximately 300 KB of Flash plus a configurable heap, making it a Scheme runtime designed for the target, not a port of a desktop system.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Real-time GC. Desktop collectors are designed for large heaps where a multi-millisecond stop-the-world pause is acceptable. A 10 ms GC pause violates a 10 ms control loop deadline. LambLisp's Yuasa incremental collector bounds the pause to a small constant per allocation, regardless of heap size.

Hot reload. Compiled systems require edit → compile → link → flash → reboot — tens of seconds of downtime during which the controlled process is uncontrolled. LambLisp can receive new code over LLIP and execute `(load "/new-behavior.scm")` inside the running loop.

The performance gap between LambLisp and an optimizing compiler (∼250× for recursive code) is the price of eliminating all these dependencies. For applications that need that last 250×, the NCG tier is the path forward.

The two execution paths available in *LambLisp* — direct AST interpretation and optional bytecode compilation — are shown in detail in the pipeline diagram below. Both paths share the same GC heap, environment chain, and interaction environment; the bytecode compiler is an optional acceleration step triggered explicitly by the application.

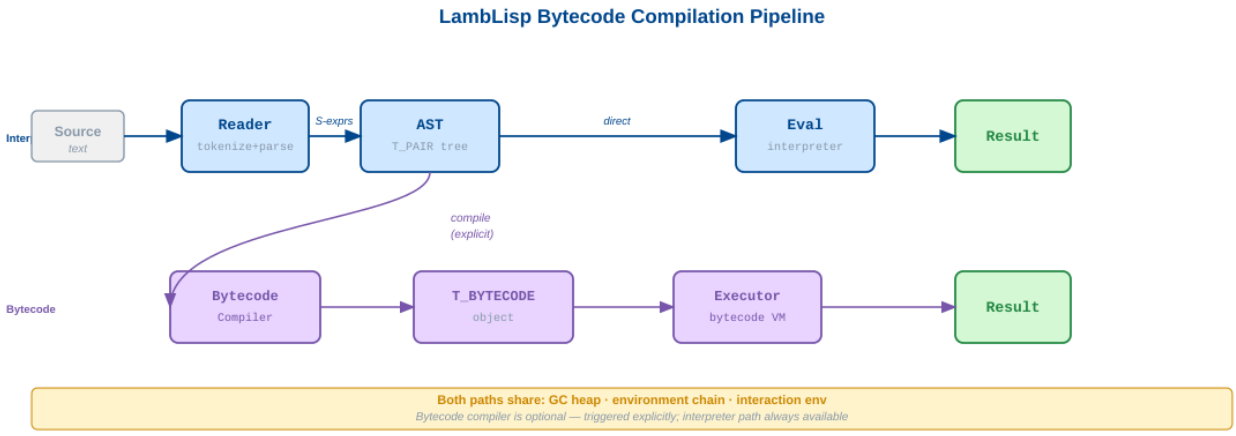


Figure 5.4 Bytecode Compilation Pipeline

The compilation process begins by reading the source code and creating an *Abstract Syntax Tree* (AST). The AST is an internal representation of the program, with each element of the source code being an element of the AST.

To represent the AST, *LambLisp* uses a "direct-connect" technique, in which high-level Lisp operations (implemented in C++) are executed through pointers directly embedded into Lisp objects. These objects correspond directly to high-level features in the language specification (define, lambda, etc). All of the LambLisp language primitives are implemented this way, and it is possible to add additional LambLisp functions implemented in C++. They are treated as any other first-class function.

5.7 LambLisp Virtual Machine

5.7.1 Control Applications as Virtual Machines.

It will help us to think of the running application as layers of virtual machines. In fact, there is a stack of virtual machines, starting at the bottom with a single transistor, and finishing at the top with whatever runtime inputs the system is designed to respond to.

Those inputs themselves arrive from another, external virtual machine, which has its own internal organization and its own set of inputs, computation, and outputs.

It is also useful to identify the boundary between the *hardware virtual machines* and *software virtual machines*. The illustration below shows a few layers representing the virtual machines involved.

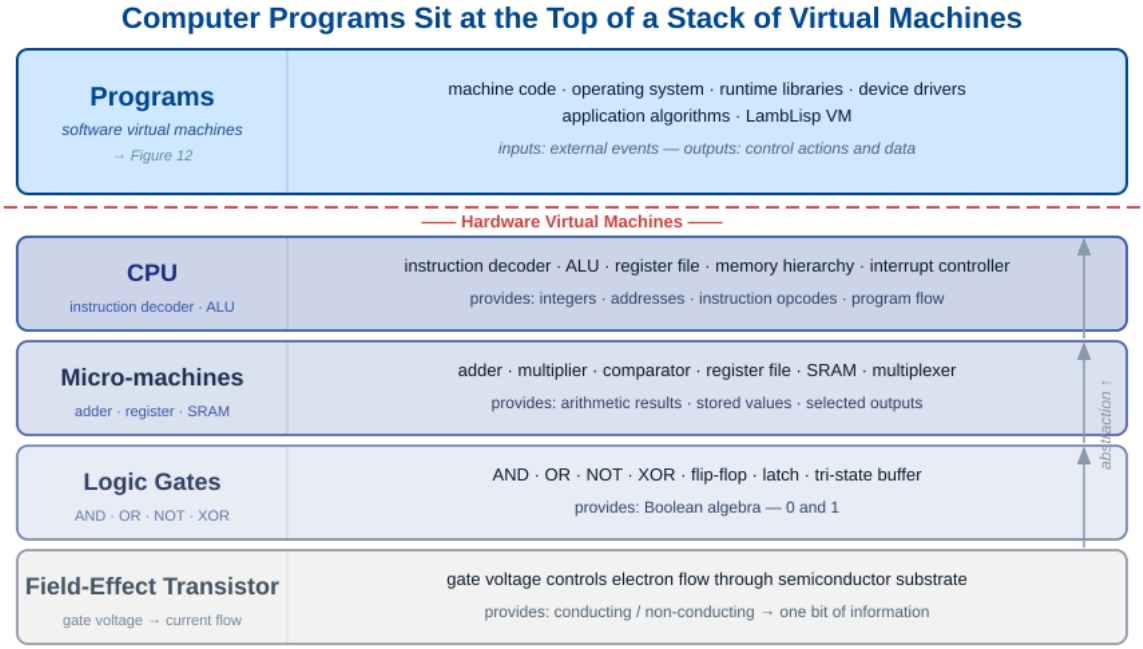


Figure 5.5 A CPU shown as a stack of Virtual Machines

The item labeled "Programs" above is its own virtual machine, consisting of multiple software layers. The layers below that are typically considered "hardware".

At the bottom is the very lowest virtual machine - a field effect transistor. These are operated such that the gate either allows or blocks electron flow through the substrate. When we ascribe meaning to the flow (either a 1 or a 0), the physical flows become meaningful symbols that people can use to communicate.

Transistors are assembled into several types of micro-machines that, when used together, become what we recognize as a "CPU". Symbols of additional complexity are created at every level: numbers, sums and products of numbers, comparison of numbers, interpreting numbers as instructions, etc.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

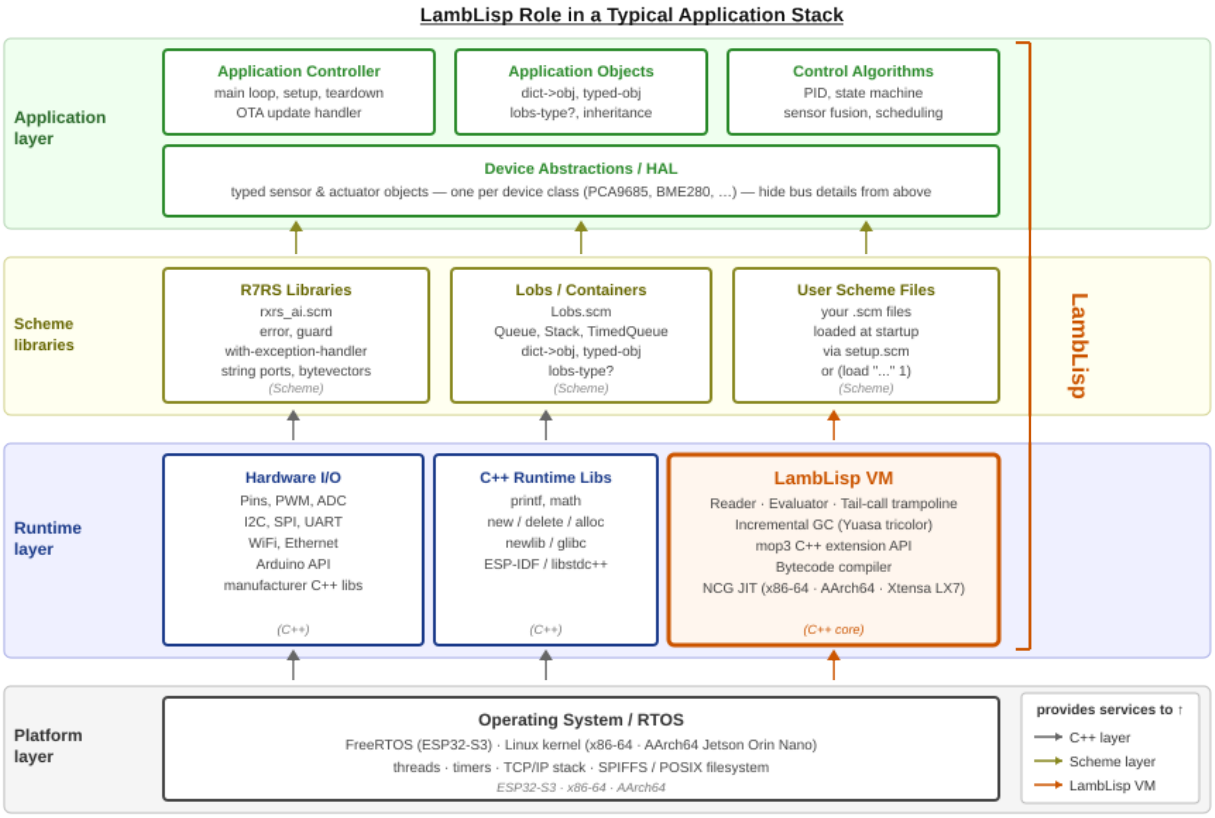


Figure 5.6 Typical Control Application Organization

The illustration above shows a typical control application breakdown. In this view, the code entities form a set with a *partial ordering* relationship, so any of the code entities in the diagram can rely on any of the other entities at an equal or lower level. They can also rely on any entity to the left.

In many loop-based control applications, there are a also few exceptional dependencies, whereby an entity may call another to its right or above. These often occur at system startup. For example, if there is one entity TIME that is responsible for knowing the time, it should not fail under any circumstances, even of not initialized and is unsure of the time. It must provide an answer of some kind.

Another example is when several different types of measurements are required to meet thresholds before the next process step can proceed. These measurements are often averaged over time, and a simultaneous value is required for all them to ensure process synchronization. Each software entity may have some work to do at synchronization time. Each may query the application control layer to obtain information about the larger environment it is operating in, such as whether the current loop is the synchronization loop, and adapt its behavior as needed.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.7.2 LambLisp Block Diagram

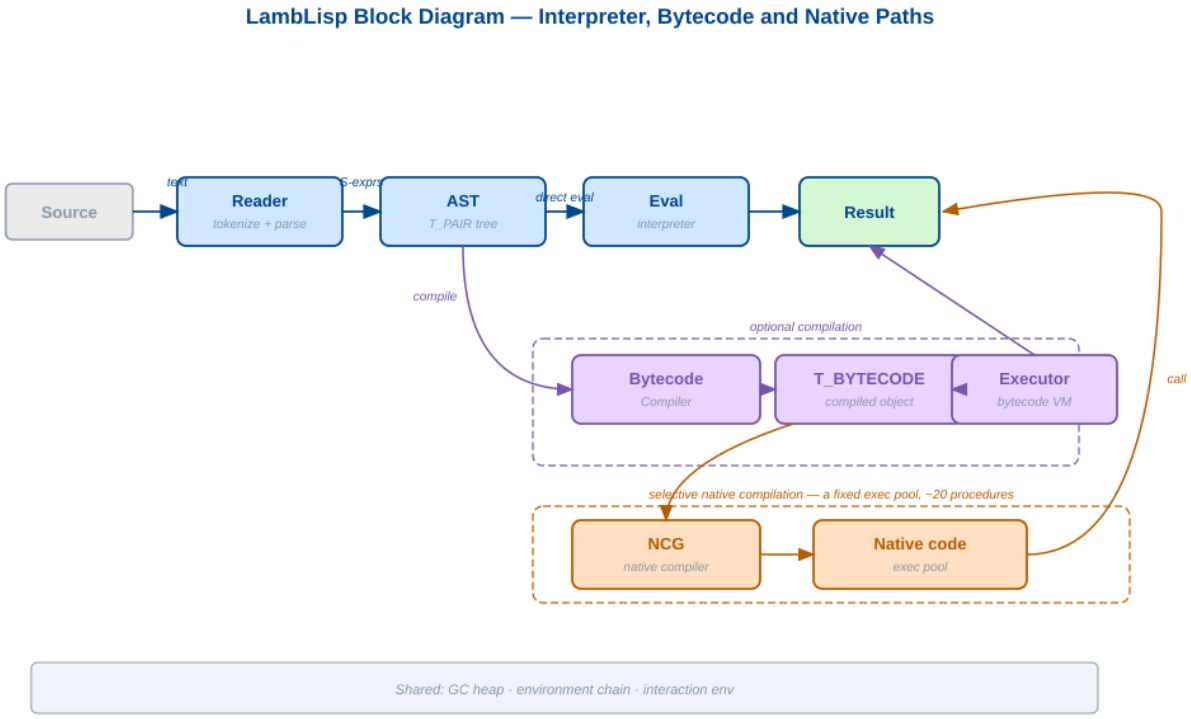


Figure 5.7 LambLisp Block Diagram

When source code is read in, the *reader* front end of the interpreter tokenizes the text, parses the resulting code, and constructs an *abstract syntax tree (AST)*. The AST is the executable version of the code, an ordered collection containing all the parts and their relationships. Because the resulting executable is a series of direct links, the AST can be traversed rapidly without a lookup table, and because the tree end nodes are high-level functions (rather than low-level bytecodes), the result is a fast interpreter.

Once the interpreter is available, and correctly executes the AST, the macro facility of *Lisp* is used to transform high-level nodes in the AST into a collection of lower-level nodes that perform the same computation. In a simple *Lisp*, consisting only of *if, eq? define, lambda, set! and cons*, every correct program will be factored into those primitives.

When macros are expanded at *read time*, then the effect is one of incremental compilation. After each macro is expanded into a series of pointers to native C++ code, the result is a compiled program consisting of a tree of native code pointers to be traversed. In addition to allowing for faster execution, this tree-structured architecture is what allows *Lisp* programs to be modified on-the-fly.

One clear example of the performance advantage is in the *Scheme cond* function, which acts as a sequential if-then-elseif. Using the minimal set of primitives, *cond* would expand into a series of *if* functions, each of which would need to be processed by the *Lisp* eval loop. In *LambLisp*, *cond* is a primitive implemented in C++, and does not require its own set of passes through the *eval loop* of *Lisp*. This provides improved performance.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.8 Memory Management

5.8.1 Overview

Memory reuse was recognized as a challenge right from the early days of programmable computers. Most computational results are intermediate; once calculated, they are quickly combined with other results and no longer required. *Garbage collection* is the pejorative term for activities related to the identification and reclamation of unused memory. Every programming language must have a strategy for reusing the memory space that no longer holds useful results.

There are 4 main ways to allocate memory: 1) static allocation, 2) stack allocation, 3) heap allocation, and 4) "manually".

Static allocation is done once before the program starts or immediately after. The memory remains available for use by the application for the life of the program. If this memory is to be used for multiple purposes, the processes involved in reuse must be done "manually" by the running application.

Stack allocation is what C programmers experience as "local variables". When a function is called, a new block is allocated from the top of the execution stack. These blocks are interwoven with the return address of the calling functions. When a function returns, the local storage block is popped off and the calling function's context is now at top of stack. In embedded control systems, the available stack space is usually small, often just 4k or 8k bytes.

The memory not allocated statically or to the stack is referred to as the "heap". C and C++ programmers must manage this space manually, using the alloc()/free() or new/delete functions, and employ bespoke tracking mechanisms to distinguish used space from unused. In C++ the bespoke aspect can be somewhat mitigated by requiring exclusive use of constructors/destructors for memory allocation, but still it is a problem for which the programmer must provide a solution.

Many other languages allocate large blocks once from the heap, and automatically manage the interior allocation of those blocks according to the requirements of the language. Among these languages are Python, Java, JavaScript, Lua, and of course Lisp.

5.8.2 Cell Memory Model and List Structures

At the lowest level, LambLisp depends on a *cell* structure in 3 consecutive words of computer memory. There are several assumptions about the computer words:

- each word consists of multiple bytes, and the bytes are individually addressable
- each word is capable of holding an address
- each word is capable of holding a signed integer of useful size
- 2 consecutive words of memory may be used to hold a floating-point value.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Of the 3 words in a Cell, the first word is called the *tag* and contains information about the cell, such as the cell type and its garbage collection state. If the tag indicates that the cell is a pair type, then the Lisp *car* and *cdr* are represented in the following 2 words of the cell. If the tag indicates an atom (such as a symbol, integer, string etc), then the remaining parts of the Cell are used to encode the details of the atom.

Nearly all *Lisp* implementations use this *tagged* arrangement, sometimes with variations such as putting the tag, *car*, and *cdr* in separate parallel arrays, or storing some of the tag bits in unused bits of the *car* and *cdr* fields. These variations are generally not useful on today's microprocessors for 2 reasons: 1) Data structure fields are aligned on word boundaries by the C++ compiler. Defeating this feature results in reduced performance. 2) Separate, parallel arrays will reduce the efficiency of the hardware processor's onboard cache.

Instead of a barrier, these features provide a new opportunity for embedded real-time control with *LambLisp*.

First, *LambLisp* devotes an entire byte for type codes. At present only 5 of the 8 type bits are used. It has proven beneficial to performance to add new types that correspond to the objects handled by the *Lamb↩ Lisp* interpreter. For example, the tail recursion implementation results in many *thunks*, which are code expressions paired with an environment to run them in. As it turns out, there are 2 kinds of thunks: a single S-expression, or a list of S-expressions. Once freed from the imperative to reduce the number of types, adding just a few types provides immense leverage in the *LambLisp* Virtual Machine. Also, using the entire byte means no masking operation is required, speeding up the very common type-testing operation.

Second, *LambLisp* devotes the second byte to Cell state information. Garbage collection relies on a state-based assessment of each Cell, and cells have 5 GC states, requiring 3 bits. That leaves 5 bits unused, as well as 3 additional unused values for the GC state. These are available for debugging purposes.

Third, in word 0 of each Cell only bytes 0 and 1 are required for the tag. In LambLisp, the extra bytes in the tag word (word 0 of each Cell) are used to store *immediate* types, such as short strings or bytevectors. Using these types for short strings or byte vectors eliminates the need for heap operations. These can be especially useful for small-packet interactions with onboard devices via SPI or I2C.

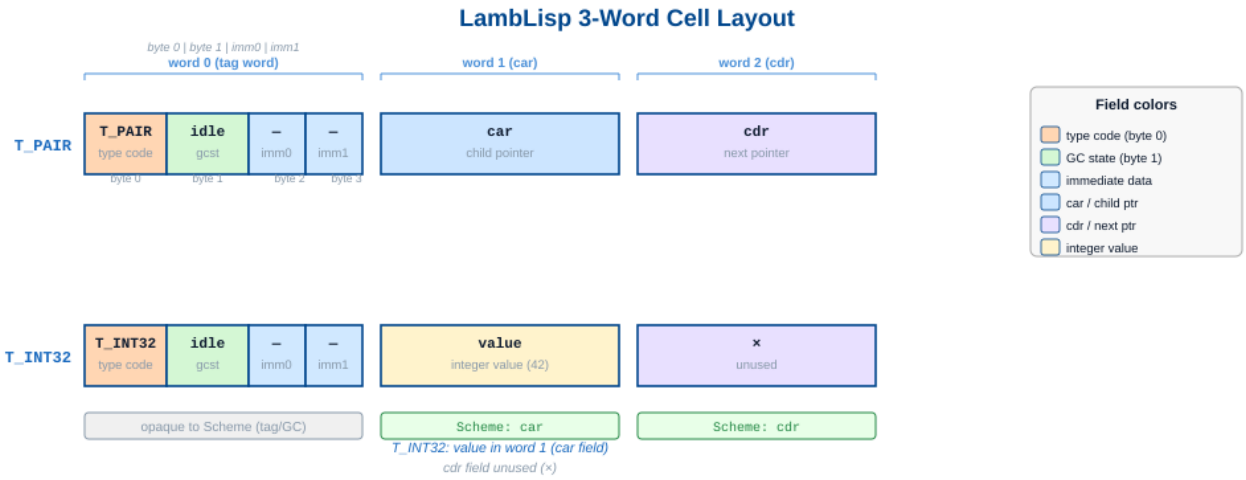


Figure 5.8 LambLisp VM Cell Structure

At the *Lisp* level, only the *car* and *cdr* are visible; the tag word is not directly available. However, the *Lisp* language provides *predicates* for identifying the type of an S-expression, either a pair (using the *pair?* predicate), or a type of atom (using predicates *integer?*, *real?*, etc). This pattern is encouraged in applications programs, where new data types may be created and predicates provide confirmation of an object's type.

The following illustrates an example of a list structure constructed from the cells described above:

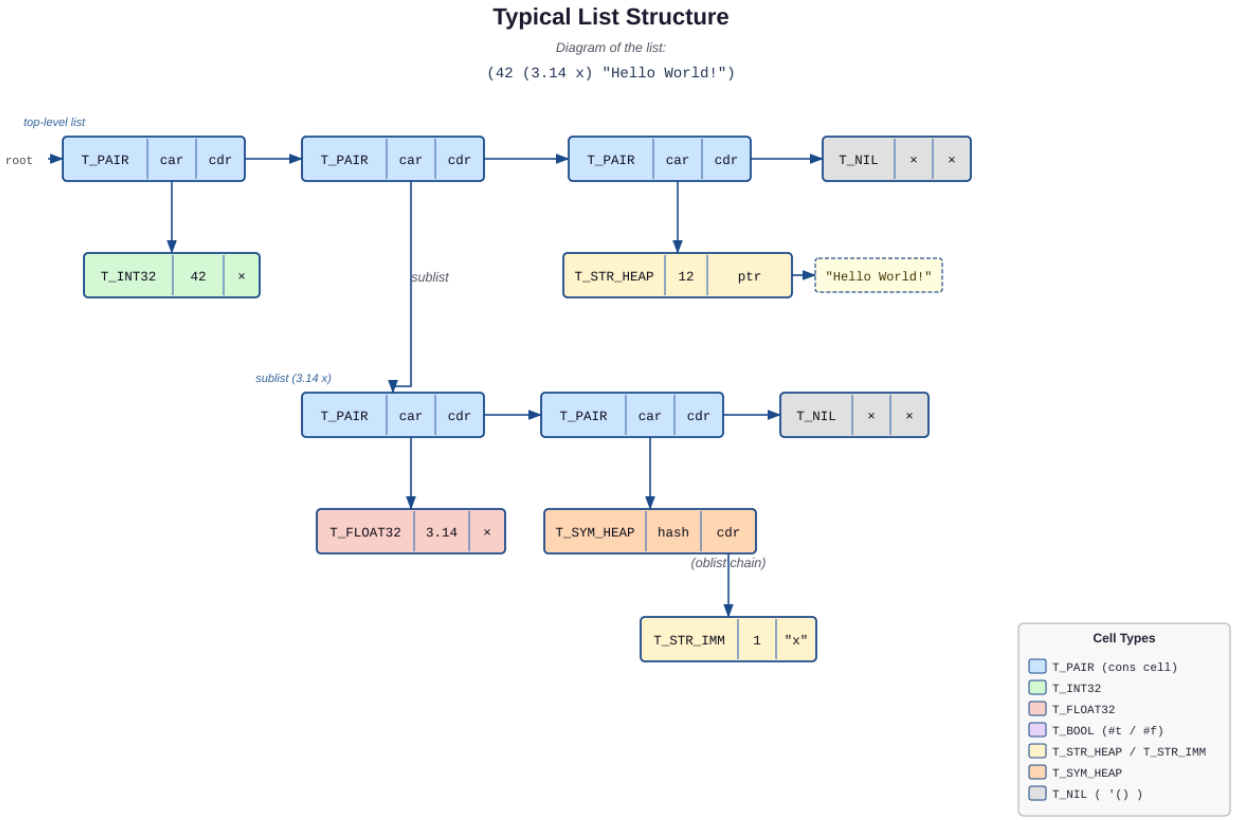


Figure 5.9 Typical List Structure

5.8.3 Cell Slots and What the Collector Traverses

The cell-type reference annotates each type with the pair of values its two slots hold, written as `(car cdr)`. That notation is worth reading carefully, because it is what tells you whether a slot holds a reference the collector follows or a value stored in place:

Notation	The slot holds
`Sexpr_t`	A reference to another cell. The collector follows it.
A named scalar type — `LL_int32`, `Charst_t`, `Bytest_t`	That value directly, with no cell of its own.
`wide`	Half of an eight-byte value that occupies **both** slots where a machine word is four bytes.
`reserved`	Nothing; the slot is unused.
`slot-typed`	Either of the first two, decided per slot at run time and recorded in the cell's flags.

The `slot-typed` case is the interesting one, and an ordinary pair is slot-typed too: `cons` packs a small integer or float straight into the slot rather than allocating a cell for it, and `car` and `cdr` build the number back when you read it. This is invisible from Scheme. It matters in C++ only if you reach past those accessors into the raw words, in which case ask `car_is_ptr()` or `car_is_embedded()` rather than assuming — and see GC Best Practices, because an accessor that has to build a number back allocates a cell to do it.

Two different questions, two different predicates. *Is this a pair?* and *does the collector look inside this?* are not the same question in LambLisp, and the C++ API keeps them apart:

- `is_any_pair()` asks what **Scheme** sees as a pair — what `car` and `cdr` will accept.
- `is_slot_typed()` asks what the **garbage collector** traverses.

Exact rationals and complex numbers answer *no* to the first and *yes* to the second. A rational is built from a numerator and a denominator, a complex number from a real and an imaginary part, and the collector traces those parts — but your program sees one number, so `car` and `cdr` reject them exactly as they reject any other number:

```
(pair? 1/2)    ;=> #f
(car 1/2)      ;=> error: 1/2 is not a pair
```

That is the answer a full Scheme such as Chez gives. If you write a C++ procedure that stores a cell reference into a slot, `is_slot_typed()` is the predicate that tells you whether that reference will be traced.

5.8.4 Garbage Collection

The literature on garbage collection is broad and deep, with several important behavioral features identified among the various possible algorithms, primarily:

Garbage collector distinguishing features
copying vs. not copying
compacting vs. not compacting
support for circular structures
need for auxiliary memory during GC
need for computational pause during GC
average throughput vs. peak pause time
concurrent access to shared memory

In a real-time control system, the discriminating factor for implementing a garbage collector is that it can perform its function incrementally. Time spent on GC is time spent not actively monitoring the controlled process. LambLisp implements a tunable control parameter (the *GC time quantum*), that limits the time spent on each GC increment, ensuring that the cell allocator will not become starved of free cells.

LambLisp's memory reuse strategy relies on these key papers:

Key GC Papers	
Knuth 1963	In "The Art of Programming", Algorithm 2.3.5b describes the fundamental GC stack-based marking strategy
Dijkstra 1976	Describes the "tricolor abstraction" and proves the correctness of incremental garbage collection
Yuasa 1990	Provides a method for determining optimum memory size and free reserve during incremental GC

LambLisp implements the stack-based idle-mark-sweep garbage collection cycle described by Knuth, the tricolor abstraction described by Dijkstra, and actively adapts GC performance based on analysis of the kind described by Yuasa.

Yuasa developed 2 parameters, M (GC start threshold), and N (minimum required cell population). The garbage collector begins in the *idle* state. *Marking* begins when the free cell reserve falls below the GC start threshold, and proceeds incrementally. When marking is complete, the *sweep* phase is begun and also proceeds incrementally. Each increment is less than or equal to the configured GC time quantum. Once sweeping has finished, the GC state is *idle* again until the next M threshold is reached.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

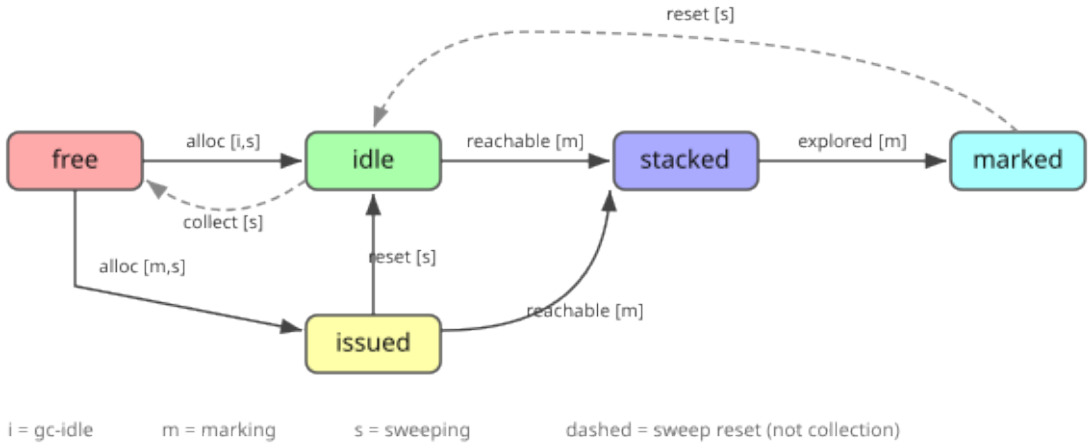


Figure 5.10 LambLisp VM Cell Life Cycle

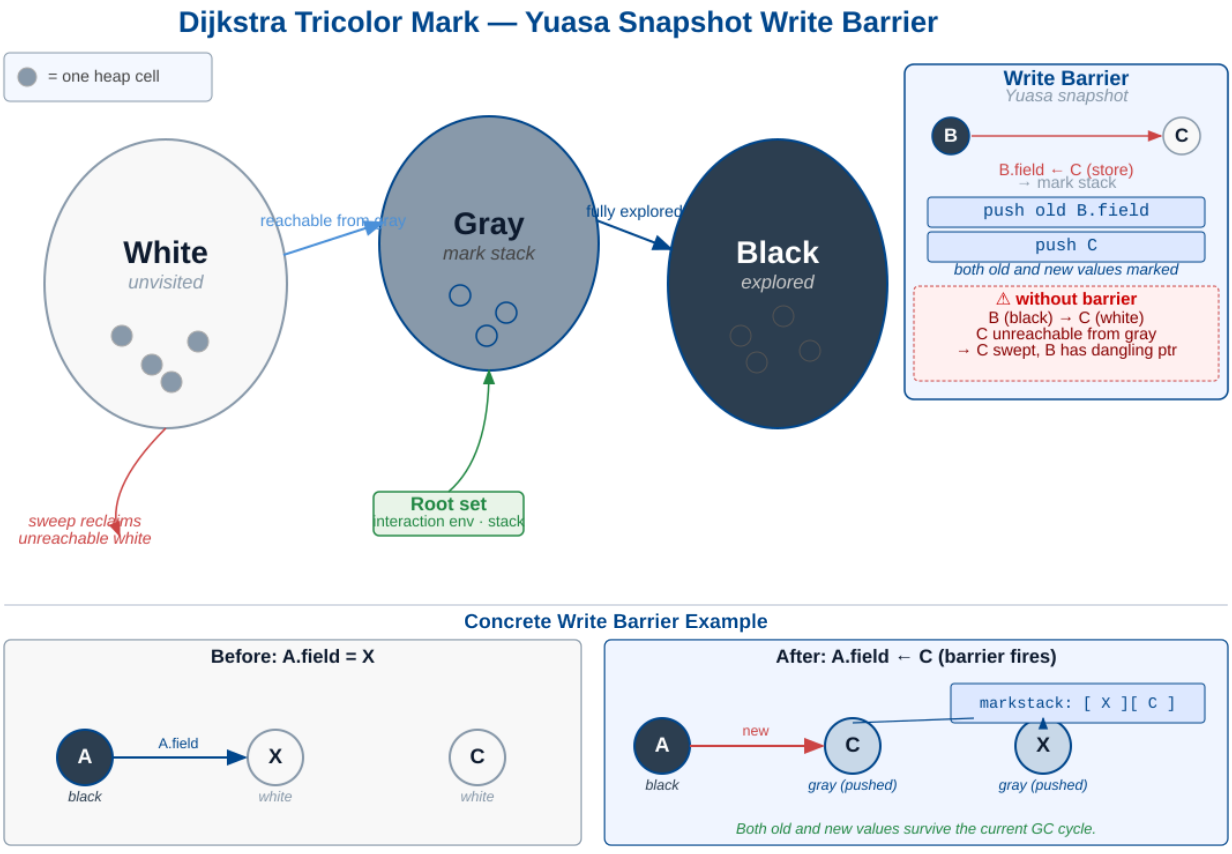


Figure 5.11 Tricolor Marking and Yuasa Write Barrier

The timeline below is a slice: 200 microseconds, showing how a quantum of marking or sweeping sits between allocations. The figure after it zooms out to a whole control-loop period, which is where the real-time claim actually lives — GC quanta are inserted at safe points between evaluations, so the loop never stops

to collect. Spare time at the end of a frame, measured against `loop-deadline-ms` rather than the idler's soft `loop-target-ms`, earns the collector a bonus quantum.

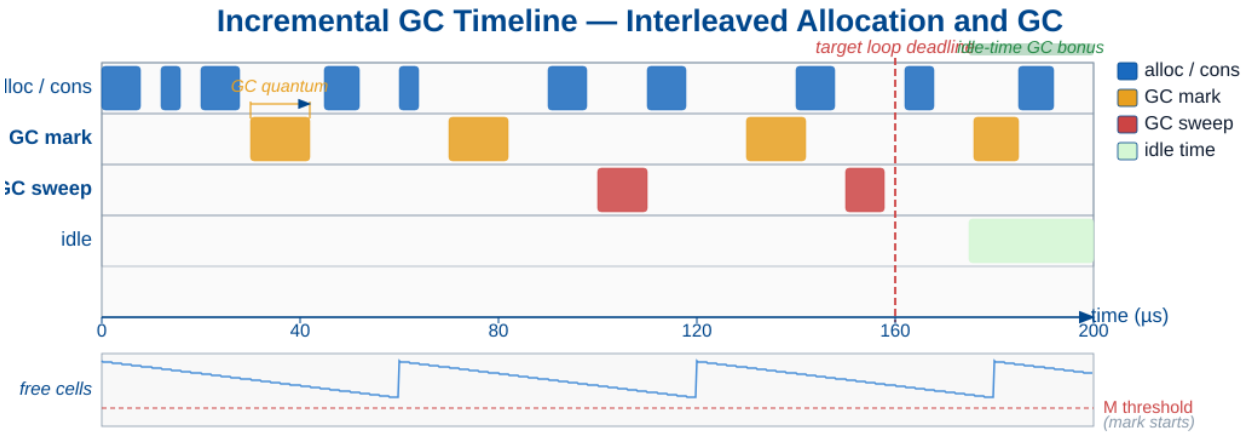


Figure 5.12 Incremental GC Timeline

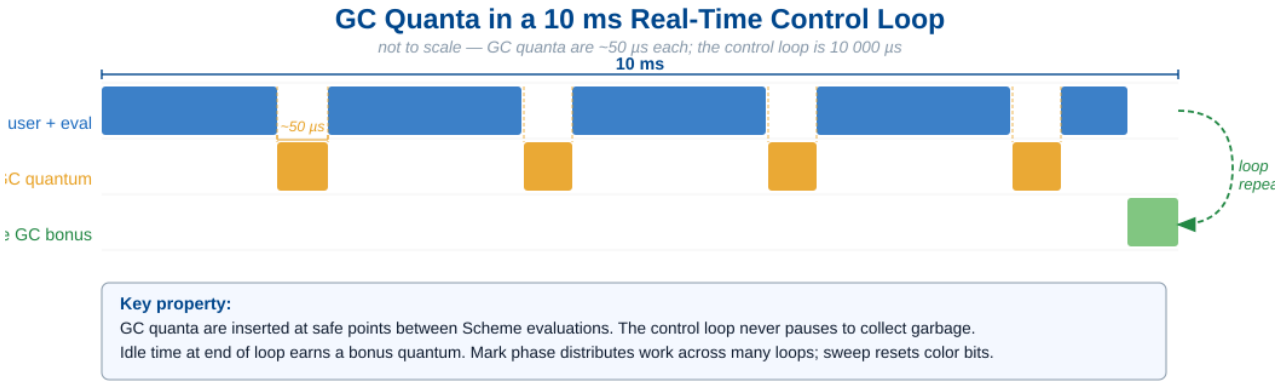


Figure 5.13 GC quanta inside a real-time control loop

The GC strategy followed by LambLisp is as follows:

- Each cell produced must be matched by an incremental amount of GC, and on the average, that GC must free up at least one cell per cons.
- LambLisp performs an incremental GC pass at `cons` time.
- LambLisp measures the time required for mark and sweep operations.
- LambLisp provides a tunable parameter, the *GC time quantum*, which is the number of microseconds allocated to garbage collection at `cons` time.
- The time quantum is converted into mark and sweep quanta, which are the number of cells that can be marked/swept in the configured time quantum. The numeric quanta have a minimum value of 2.

- The Yuasa parameters are recomputed at the end of every sweep. The parameter M (minimum free cell reserve) is continuously adjusted as required, and the parameter N (minimum required number of cells) is also checked; if N is greater than the available memory then more memory is requested from the system heap.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.9 Real-time Guarantees

A real-time system guarantees that outputs are updated within a bounded time after inputs change. LambLisp uses loop-based control: the application's `loop` procedure runs repeatedly, reading sensors and driving actuators each cycle. Deadlines are *hard* (missing one is a failure) or *soft* (the system degrades gracefully); LambLisp is soft real-time at the loop level but achieves hard latency in individual ISR operations.

Group	Dimension	Guarantee	Mechanism
GC	Incremental GC	No stop-the-world pause	Yuasa tricolor mark/sweep
GC	GC time quantum	Bounded time per cons	Quantum calibrated internally against measured mark/sweep rates
GC	Idle-time GC	GC absorbs loop slack	`gc-idle-strategy`: `deadline` (spare to `loop-deadline-ms`) or `jitter` (spare vs recent mean)
GC	Adaptive sizing	GC never starves allocator	Yuasa M/N analysis per cycle
Loop	Frame ceiling	Sets the idle-GC budget under the `deadline` strategy	`loop-deadline-ms` variable
Loop	Hard vs soft distinction	Design guidance	Factoring hard deadlines out of the loop
Interrupt	ISR latency	200–400 ns GPIO response	`IRAM_ATTR` + direct register writes
Async	Deferred task queue	Non-blocking callbacks	Time-triggered thunks, no `sleep`

5.9.1 GC Latency — No Stop-the-World Pauses

The central GC guarantee is that no single allocation ever triggers an arbitrarily long pause. LambLisp's garbage collector is **incremental**: each `cons` operation performs a small, bounded quantum of mark or sweep work before returning. The pause per allocation equals the GC time quantum — a tunable parameter — not the total collection time.

The algorithm is Yuasa's incremental tricolor mark/sweep. Three colors track GC state: *white* (not yet reached), *grey* (reached, children not yet scanned), *black* (fully scanned). The Yuasa write barrier — applied at every mutation during the mark phase — ensures that no live object can be missed and no object is freed while still reachable.

The **GC time quantum** bounds how many cells are marked or swept at each cons. It is calibrated internally, as a cell count derived from measured mark and sweep rates, and is **not a Scheme-settable parameter** – earlier documentation named a `gc-quantum` variable, which does not exist. What an application *can* control is how much collection happens between allocations, by choosing an idle-GC strategy (below).

Idle-time GC pushes additional collection work into loop slack, and there are **two strategies**, selected with `gc-idle-strategy`. They regulate different things and neither replaces the other:

gc-idle-strategy	budget given to the collector	what it regulates
deadline	loop-deadline-ms minus the time this iteration used	Frames become equal in length against a **fixed** reference, and the leftover is real headroom – so the collector is fed steadily even when the loop is perfectly regular.
jitter	the mean of recent iterations, minus this one – donated only when positive	Lengthens short iterations toward the running mean and leaves long ones alone, so iteration times bunch together. Needs **no declared period**, which is the right choice when the workload has no meaningful one.
auto (default)	deadline if loop-deadline-ms is set and positive, else jitter	–
off	none	Diagnostic use.

The trade-off is worth stating plainly, because it decides which one you want. jitter regulates the control period without you having to declare one – but because it measures against an average of its own recent history, a perfectly steady loop is never below its own average and so donates **nothing** to the collector, however much headroom it has. Collection then happens on the allocation path instead, where the runtime may have to complete a cycle before allocation continues. deadline does not have that property, because the reference does not move.

If your loop has a period, declare its ceiling in loop-deadline-ms **and leave** gc-idle-strategy **at** auto**.** If it does not, jitter is what you want and is what auto will pick.

Adaptive sizing implements Yuasa's M/N analysis after every sweep. M (the minimum free reserve) and N (the minimum cell population) are recomputed from the current allocation rate. If N exceeds the available pool, additional cell blocks are requested from the system heap up to the configured maximum. The GC never runs dry; the allocator never stalls.

See: Garbage Collection · Memory and GC Tuning

5.9.2 Loop Time — Bounded Control Cycle

Two variables describe the intended shape of an iteration, and they are not interchangeable:

variable	meaning
loop-target-ms	the **soft target** – where the Scheme idler aims to land the frame
loop-deadline-ms	the **frame ceiling** – how long an iteration may take. The idle-GC donation gates on this one.

Under the deadline strategy LambLisp measures elapsed time at loop-end and gives the collector whatever remains before loop-deadline-ms. It is runtime-settable: set! it to retune a live system.

It is a budget, not an enforced period. Nothing pads a short iteration out to it and nothing interrupts one that overruns; the runtime does not currently detect or report a missed deadline. Setting it says how much of each iteration you intend to leave for the collector.

Setting `loop-target-ms` correctly requires measuring the worst-case loop duration under load — peak sensor reads, maximum GC pressure, and any blocking I/O — and adding a margin. LambLisp exposes loop timing statistics accessible from the REPL or over LLIP from a supervisory host.

Hard vs soft factoring: LambLisp is soft real-time at the Scheme level. For operations with a hard deadline — a pulse that must be generated within 1 µs of a trigger — the correct pattern is to handle the deadline in C++ (an `IRAM_ATTR` ISR) and communicate the result to the Scheme loop via a shared flag. The Scheme loop reads the flag every iteration and acts on it; the hard deadline is met in C++, the policy response is handled in Scheme.

See: Memory and GC Tuning

5.9.3 Interrupt Latency — Sub-microsecond ISR

On ESP32, code placed in on-chip SRAM with `IRAM_ATTR` executes from fast SRAM rather than slower PSRAM. Combining IRAM placement with direct hardware register writes (bypassing the Arduino `digitalWrite` abstraction) yields GPIO response latencies of **200–400 ns** at 240 MHz.

```
void IRAM_ATTR fast_isr() {
  GPIO.out_w1ts.val = (1UL << LED_PIN);  // ~200 ns from interrupt assertion
}
```

The ISR sets a flag or writes a hardware register. It does not call any LambLisp API — the VM is not reentrant from an ISR context. The Scheme loop polls the shared flag each iteration and responds; response latency is bounded by the loop time, not the ISR latency.

```
volatile bool g_trigger = false;

void IRAM_ATTR edge_isr() { g_trigger = true; }

// C++ mop3 called from the Scheme loop each iteration:
Sexpr_t fn_trigger_check(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec) {
  if (!g_trigger) return HASHF;
  g_trigger = false;
  return HASHT;
}
```

```
;;; In the control loop - polled every iteration, bounded by loop-target-ms:
(define (loop)
  (when (trigger-check)
    (handle-trigger))
  (loop))
```

5.9.4 Asynchronous Work — Deferred Task Queue

Blocking the Scheme loop with `(sleep ms)` wastes CPU and violates loop-time guarantees. The **deferred task queue** replaces `sleep` with time-triggered thunks that the loop services without blocking.

LambLisp maintains a time-ordered queue of `(deadline . thunk)` pairs. Each loop iteration dequeues and calls all thunks whose deadline has elapsed. Between calls the loop continues normally; no polling or busy-waiting occurs.

```
;;; Schedule a thunk 500 ms from now - the loop keeps running in between:
(task-after 500 (lambda () (set! led-state (not led-state))))

;;; Blink indefinitely - each invocation reschedules itself:
```

```
(define (blink-loop)
  (set! led-state (not led-state))
  (digitalWrite LED_PIN (if led-state HIGH LOW))
  (task-after 250 blink-loop))

(blink-loop)
```

The TimedQueue container in `data/Lobs.scm` implements this pattern at the Scheme level. Motor durations, LED animations, and buzzer tones are expressed as timed thunks rather than blocking `delay` calls, keeping the control loop responsive.

See: Memory and GC Tuning

5.10 Scalability

LambLisp is scalable across eight dimensions, organized into three groups. *Reach* dimensions determine where code runs and how nodes connect. *Speed* dimensions determine how fast it runs. *Footprint* dimensions determine how much memory and flash it consumes. Each dimension can be tuned independently; they compose.

Group	Dimension	From	To	Mechanism
Reach	Platform tier	ESP32: 240 MHz, 2 MB PSRAM	Linux x86-64 / Jetson: 8 GB RAM	Same source, same GC, same REPL
Reach	Network reach	Single node	Multi-node LLIP mesh	LLIP TCP/TLS: eval + filesystem
Reach	AI / machine-to-machine	Fixed compiled algorithm	Runtime AI-generated Scheme	`llip-eval` + `load` without reboot
Speed	Execution tier	AST interpreter	NCG native: 2–10× over BC, up to 742× over interpreter on ESP32	`procedure->bytecode`, `ncg-compile`
Speed	C++ extension	Pure Scheme	Any C++ library at native speed	mop3 interface
Footprint	Feature footprint	Full R5RS/R7RS binary	Stripped minimal build	Build-time module selection
Footprint	Memory / GC	4k cells	All available PSRAM	GC pool + Yuasa adaptive tuning
Footprint	Code lifecycle	Full-image OTA (reboot)	Incremental hot-reload (no reboot)	`(load)` in running loop via LLIP

5.10.1 Group A — Reach

Multi-Platform Support

LambLisp is a multi-platform library. Application Scheme code written for one target runs on all three without modification. The same `.scm` source file, the same garbage collector, and the same `LL=>` REPL are present whether the host is an ESP32-S3 at 240 MHz with 2 MB of PSRAM, a Jetson Orin Nano at 1.5 GHz with 8 GB of RAM, or an x86-64 Linux workstation. Platform selection is a build-time flag in `platformio.ini`; no source changes are needed to move between targets.

What is portable — identical on all three targets:

Category	Examples
Core language	Closures, tail calls, `define`, `let`, `letrec`, quasiquote
Macro system	`syntax-rules`, `macro`, `nlambda`, `macroexpand`
Standard library	R5RS + R7RS-small: lists, strings, vectors, chars, numbers

Category	Examples
Exception system	`with-exception-handler`, `raise`, `guard`, `error`
Compilation	`procedure->bytecode`, `ncg-compile` (all three NCG backends)
GC	Yuasa incremental tricolor, same parameters
REPL + LLIP	`LL=>` prompt, `(load)`, remote eval server
Source files	`.scm` files loaded with `(load "file.scm")`

What is platform-specific — the thin HAL:

Category	ESP32	Linux / Jetson
Digital I/O	`digitalRead`, `digitalWrite`, `analogRead` — real pins	same procedures, present. On a Jetson (`linux_aarch64`) they drive **real pins** through libgpiod; on other hosts they are an in-memory loopback — `digitalRead` returns what `digitalWrite` last wrote
I2C / SPI / WiFi	`Wire.begin`, `SPI.begin`, `WiFi.begin`, etc. — real peripherals	same procedures, present but inert — the calls succeed and drive nothing
GPU / CUDA	not available	`cudaMalloc`, inference bindings
Loop driver	`(loop)` called by Arduino `loop()`	top-level runs directly

Platform detection — the string `lamb-board` identifies the current target at runtime. Three boolean predicates derived from it are defined in `setup.scm`:

```
linux?   ;;; #t on linux_x86_64 and linux_aarch64
esp32?   ;;; #t on all ESP32 / ESP32-S3 board variants
jetson?  ;;; #t on linux_aarch64 (Jetson Orin Nano)
```

Use them for the small amount of platform-specific initialization that belongs in application code:

```
;;; Portable core - runs identically everywhere:
(define (read-temperature) (i2c-read-byte sensor-addr))

;;; Platform-specific startup - thin and explicit:
(when esp32?
  (Wire.begin SDA-PIN SCL-PIN)
  (Wire.setClock 400000))

(when linux?
  (define (i2c-read-byte addr) (linux-i2c-read "/dev/i2c-1" addr)))
```

Develop on Linux, deploy to ESP32 — the recommended workflow:

1. Write and test application logic interactively at the `LL=>` REPL on Linux. The full language, GC, and standard library are available.
2. Wrap hardware calls in `(when esp32? ...)` guards or stub them for Linux.
3. Copy the `.scm` files to the ESP32 flash filesystem via LLIP; evaluate `(load)`. No recompilation, no reflashing of application code.

See: Deploy to ESP32 · LLIP

Network Reach

A single LambLisp node on an ESP32 can be reached by any other LambLisp node — a Jetson, a Linux workstation, or another ESP32 — using **LLIP** (LambLisp Interaction Protocol), a line-oriented S-expression protocol over TCP.

LLIP exposes two capabilities over a single connection: remote filesystem operations (upload, download, directory listing) and remote evaluation. The `llip-eval` call sends any Scheme expression to the remote node and returns its result as a Scheme value. A supervisory controller can query live sensor state, reconfigure parameters, and deploy new behavior without interrupting the controlled process.

Multiple ESP32 boards can be addressed from a single Jetson or Linux host, forming a mesh where the supervisory node coordinates behavior across the fleet. All nodes speak the same language; no translation layer is needed.

```
;; From a Jetson supervisory node, reach two ESP32 boards:
(define left-wheel  (llip-connect-tcp "10.0.0.11" 5555 "secret-token"))
(define right-wheel (llip-connect-tcp "10.0.0.12" 5555 "secret-token"))

(llip-eval left-wheel  '(Motor.speed))        ; query live RPM
(llip-eval right-wheel '(set! pid-kp 0.85))   ; tune PID gain live
```

See: LLIP — LambLisp Interaction Protocol

AI and Machine-to-Machine

Because LambLisp code is data — S-expressions that can be constructed, inspected, and transmitted as strings — an AI system or supervisory process can generate new Scheme code and deploy it to a running embedded controller without reboot or recompilation.

The workflow: an off-system AI analyzes telemetry (sensor traces, timing logs, error rates), generates a revised algorithm as a Scheme expression or `.scm` file, uploads it over LLIP, and evaluates `(load "/new-policy.scm")` in the running loop. The new definition replaces the old one in the interaction environment immediately; the GC reclaims the old code. No flash erase, no reboot, no control gap.

This is the embedded embodiment of Lisp's historical strength: interactive, live, self-modifying programs. The "off-system resource" is no longer a human typing at a REPL — it is an AI agent operating at machine speed.

```
;; Supervisory AI node - generate and deploy a new control policy:
(define board (llip-connect-tcp "10.0.0.11" 5555 "secret-token"))

(llip-write board "/policy.scm" (ai-generate-policy telemetry))  ; upload
(llip-eval  board '(load "/policy.scm"))                          ; hot-load
(llip-eval  board '(policy-version))                             ; → "2026-04-28-r3"
```

5.10.2 Group B — Speed

Execution Tiers: AST → Bytecode → Native Code

LambLisp evaluates Scheme directly from the parsed S-expression (AST) by default, with no mandatory compilation step. This gives fast startup and interactive REPL response without any warm-up phase.

For procedures that run frequently, two optional compilation stages are available:

- **Bytecode** (`procedure->bytecode`, `compile-environment!`): compiles a procedure or an entire environment to a compact bytecode representation. The bytecode executor is a tight dispatch loop that eliminates the overhead of tree-walking the AST on every call. Measured speedup on the trampoline VM: 1.7–3.3×.
- **Native Code Generator** (`ncg-compile`, `ncg-compile-environment!`): compiles bytecode further to native machine instructions for x86-64, AArch64, and Xtensa LX6/LX7 (ESP32/ESP32-S3). The NCG eliminates the dispatch loop and produces code that runs directly on the CPU with no interpreter overhead. On ESP32 LX6 at 240 MHz (2026-05-20), measured speedup over bytecode: tak 7×, ackermann 6×, tree 7×, fib-rec 10×, qsort 5×; over interpreter: tak 324×, sieve 341×, fib-memo 742×, qsort 219×. NCG→NCG recursive calls bypass argument-list allocation entirely, which explains the large gains on recursion-heavy benchmarks. Several inline fast paths accelerate the most common operations: `vector-ref`/`vector-set!` skip the shim for `T_SVEC_HEAP` + `T_INT32` index; NCG→NCG call dispatch bypasses the three-level shim chain; `cons` grabs from the free list without calling `gc_pass`; 9 primops (`not`, `eq?`, `zero?`, `positive?`, `negative?`, `set-car!`, `set-cdr!`, `integer?`, `boolean?`) and composed accessors (`caar`, `cadr`, `caddr`, `cddr`, etc.) are expanded inline by the bytecode compiler to single-opcode instructions.

The three levels are complementary. Interactive development stays at the AST level; time-critical routines can be promoted to bytecode or native code with a single function call. NCG is available and recommended on all three targets (ESP32, Jetson, Linux).

C++ Extension

LambLisp has a clean, simple interface to C++ code. All the LambLisp native functions have the same signature. As part of the control application, additional native features can be added and will run at full C++ speed. These C++ additions may be useful in several circumstances:

- Implement a low-level hardware abstraction and make it available to Lisp.
- Provide a Lisp interface to a specialized computation library.
- Improve performance at key algorithmic bottlenecks.

For example, LambLisp implements the *digitalWrite* function this way:

```
Sexpr_t mop3_cio_digitalWrite(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
{
  Sexpr_t sx_pin = lamb.car(sexpr);         //Get the pin # in S-expression form
  Sexpr_t val    = lamb.cadr(sexpr);        //Get the value in S-expression form
  LL_int32 pin   = sx_pin->mustbe_int32();  //throw error if an INT not provided

  digitalWrite(pin, val != HASHF);          //in Scheme, any value "not false" is "true"
  return val;                               //return whatever value the pin was set to
}
```

To use this C++ function from within LambLisp to set pin 38 HIGH and then LOW:

```
(digitalWrite 38 #t)
(digitalWrite 38 #f)
```

See: C++ Integration

5.10.3 Group C — Footprint

Feature Footprint

At build time, LambLisp allows fine-grained control of the system functions that are included in the application image.

For example, LambLisp supports all the math functions described in the *Scheme R5RS/R7RS* specifications, and by default all are included in the build. Functions that are not needed can be commented out in the source code and will not be linked into the control application.

This same approach can be used for other groups of function, such as ports and strings. These groups of functions are completely defined in the spec to provide a full solution in their respective domains, but in practice each application uses only a subset of the available capability.

As another example, few embedded control applications will need the full set of case-independent string operators, and they are easy to add in Lisp later if needed. These functions are candidates for implementation in Lisp and loading at runtime, rather than including them in every binary application image.

Memory and GC Tuning

LambLisp has an adaptable real-time garbage collection implementation that can be scaled in several ways. At every *cons* operation, the GC has an opportunity to run, and may mark or sweep cells at that time.

GC parameter	Function
GC time quantum	Amount of time to spend on GC for each cons
Cell block size	# cells per block
Max cell blocks	maximum # of blocks to allocate
GC % threshold	Increase memory until threshold reached (or max)

The GC time quantum determines how many mark or sweep operations to do during each increment. A smaller number results in shorter GC interruptions, but lower total throughput.

The cell block size is not critical, but the minimum system setup needs 4k cells, and so 8k is a natural default for this parameter, allowing space for a useful *Lisp* application.

The maximum number of cell blocks can be chosen to tune the amount of time spent in GC. The amount of time spent marking depends on the number of cells used by the *LambLisp* program, and not the total number of cells available. Conversely, when sweeping, the entire cell population must be swept.

With more cells in the population, more cells are swept and reclaimed with each GC mark/sweep cycle, because the fixed cost of marking the used cells is amortized over the entire cell population. More memory results in less time spent in garbage collection, but with diminishing returns as more memory is added with constant program size.

LambLisp implements several adaptive tuning mechanisms in the GC implementation.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

First, *LambLisp* provides an incremental garbage collector, based on the work of Taichi Yuasa. This eliminates extended pauses for garbage collection. Yuasa began with a stack-based, mark & sweep, stop-the-world garbage collector, and transformed it into an incremental adaptation. In the incremental version, each time a new memory cell is issued, some quantum of garbage collection must also be done. The requirement is to keep the reclamation of memory ahead of the issuance, so for each memory cell issued, at least 1 must be reclaimed.

Yuasa determined several parameters required to optimize incremental garbage collection.

- The size of the *Lisp* program.
- How many marks or sweeps to perform at each new memory issuance.
- The minimal amount of free memory before GC begins.
- The size of memory required to support the ongoing issuance rate.

The amount of memory and the amount of free cell reserve is adjusted dynamically to preserve realtime behavior, based on a Yuasa analysis after every marking phase.

Another principal concern is the amount of CPU time devoted to garbage collection. In LambLisp, the percentage of time used in GC is one factor used to determine whether to expand the cell population. If the percentage time in GC is above the configured threshold, then an additional cell block is added. As long as the threshold is exceeded, new blocks will be added up to the configured maximum.

The Yuasa analysis determines how many cells total are required to maintain realtime production, and how many must be kept in free reserve to issue while GC collects some new free cells. The Yuasa parameters are recalculated during every garbage collection.

To allow for maximum throughput while maintaining low GC pause time, *LambLisp* provides a *target loop time* parameter. When each loop has finished, but before returning to the C++ main() that called it, the *LambLisp* virtual machine will check if the target loop time has been reached yet. If not, then there are potentially some idle tasks that could be done, that might perhaps usefully fill the time.

The most important of these is garbage collection. If there is loop time left before the target time limit, and if the garbage collection process is active (marking or sweeping) and not idle, *LambLisp* will calculate a new, temporary GC time quantum based on the time remaining, and run a GC incremental pass with that time quantum. This tends to put much of the GC effort into "idle time", so the GC subsystem moves more quickly into its own *idle* phase, where no GC is done at all until the Yuasa M parameter is breached.

Interestingly, the printing of messages on *Serial* (the Arduino serial port, or equivalent on other platforms) also takes time, despite the UARTs and software buffering involved. Two messages printed during the same loop might easily run the loop time up to 10 milliseconds, regardless of what the loop time is without the messages. *LambLisp* provides a means to make the printing occur only if idle time is available. This is useful for informational but noncritical messages. This feature has the added advantage of not computing the message contents if time is not available; it is not just skipping printing, but the entire message evaluation.

The *idle time* feature is available to *LambLisp* applications, as well as in the underlying C++ interface.

See: Garbage Collection

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Code Lifecycle: Incremental Hot-Reload

In C++-based systems, half of the persistent storage must be devoted to over-the-air updates. An entire new program image is downloaded before the existing image is marked obsolete. After that, the storage for the old image is still required for future OTA updates, but otherwise is not used again. A reboot is required to start the new image.

With *LambLisp*, updates are incremental. New Scheme definitions are loaded into the running interaction environment; the new code takes effect immediately and the GC reclaims the old definitions. No flash erase, no reboot, no control gap.

LambLisp can receive new code over LLIP and execute `(load "/new-behavior.scm")` inside the running loop — the controlled process keeps running throughout. Because programs are ASTs and ASTs are data structures, startup code used once can be purged from memory and reloaded on demand, algorithms can be chosen and downloaded at runtime, and new behaviors can be introduced in the field without interrupting the controlled process.

```
;; Deploy a new behavior to a running board - no reboot:
(llip-write board "/new-behavior.scm" new-code)
(llip-eval  board '(load "/new-behavior.scm"))
```

See: Execution environment · LLIP — LambLisp Interaction Protocol

5.11 Language Internals and C++ API

5.11.1 lambda, nlambda, and macro

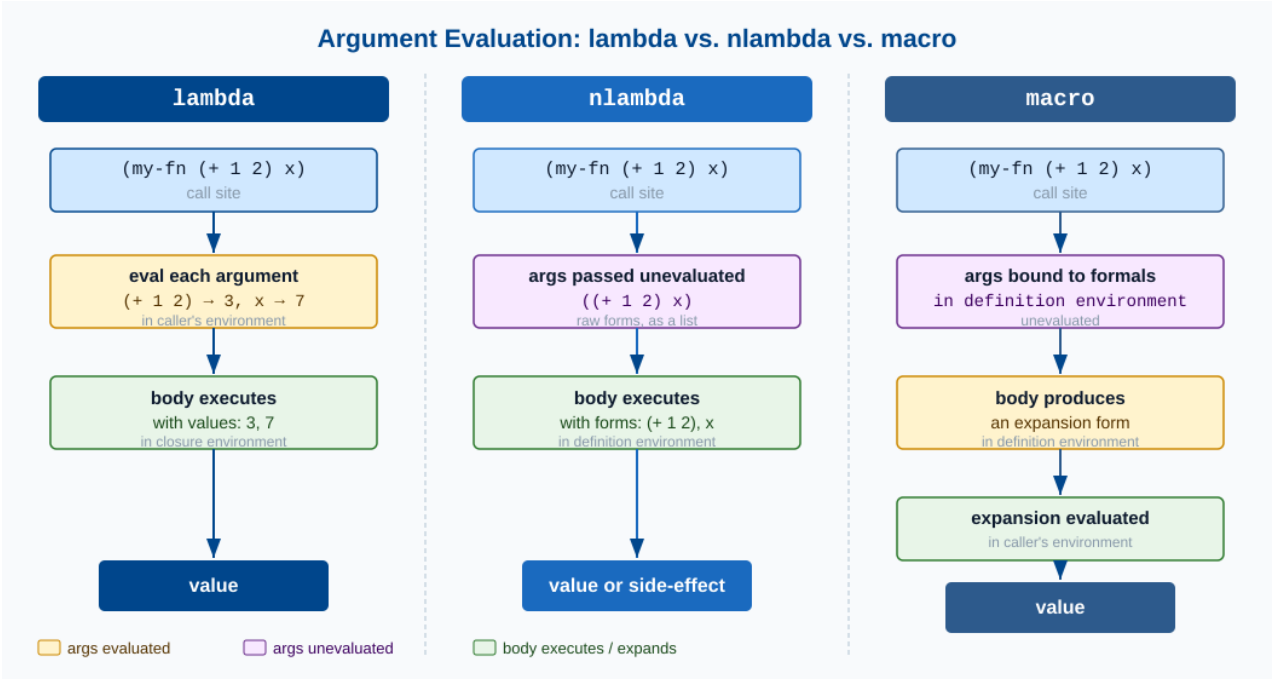


Figure 5.14 Argument Evaluation: lambda vs. nlambda vs. macro

From the beginning, *Lisp* functions have been divided into **special functions** and other functions, which have no special designation. They are *special* primarily in that they do not interact with their parameters in the same way as non-special functions. The main differences between these *special functions* and the non-special variety are these:

- Normal lambda functions have their arguments evaluated before the function is applied.
- Special functions receive some or all of their arguments unevaluated.
- Special functions may also be special in many other ways that are function-specific.

While the notion of "function-specific features of special functions" may seem vague, it is a result of the evolutionary nature of early *Lisp*. It was the first computer programming language that understood itself, and that meta-understanding required and produced a lot of exploration.

The topic of *special functions* became inextricably bound up with the topic of **macros**. In all the many *Lisp* implementations over the years, macros have the largest variety of implementations of any other feature in the language.

A major shortcoming of *Scheme R5RS* is the inclusion of `syntax-rules` as the sole language element for a macro capability. Prior to *R5RS*, `syntax-rules` was optional. It is not required to implement the rest of *Scheme*, when the simpler **nlambda** construct will do. There is no lower-level macro feature in the language

itself that could support `syntax-rules`, even though there are several to choose from in previous *Lisps*. To use `syntax-rules` for creating macros first requires an implementation of another (non-Lisp) pattern-matching language. Therefore, every *Scheme* implementation needs to invent or borrow some other lower-level macro facility to support implementation of `syntax-rules`. This puts the cart before the horse, and goes against the original *Scheme* minimalist values.

As a result, there is no common best practice on the implementation of `syntax-rules`. And it gets worse: `syntax-rules` has now been superseded by `syntax-case`, which has the same drawbacks, while presenting new backward-compatibility challenges.

In *Common Lisp*, a macro system is described that has its own set of quirks, such as not being able to use *apply* with a macro.

lambda

A **lambda expression** takes a formal parameter list and a code body and returns a **procedure** – a first-class value that encapsulates three things: the parameter list, the body, and the **environment at the point of creation**. That captured environment is called a *closure*.

In *LambLisp* a procedure is represented as a pair: one part holds the lambda expression, the other holds the environment dict that was current when `lambda` was evaluated. When the procedure is later applied, a new frame is pushed on top of the captured environment, the arguments are bound into that frame, and the body executes.

Because the environment is captured **by reference**, not by value, any top-level definitions added after the closure was created are visible to it – they exist in the same dict chain.

```
(define (make-adder n)
  (lambda (x) (+ x n)))   ;;; captures n from make-adder's frame

(define add5 (make-adder 5))
(add5 3)   ;;=> 8
```

When applying a lambda procedure, every argument is **evaluated** before the procedure body runs.

nlambda

In *LambLisp*, the foundation for implementing *special forms* is **nlambda**, a concept inherited from **InterLisp**. The *nlambda* operator returns a value of type **nprocedure**. An *nprocedure* has the same internal structure as a lambda procedure – a body and a captured environment – but it differs in one critical way:

Note

> The key distinction between `lambda` and `nlambda` is that **nprocedure arguments are not evaluated**. The unevaluated argument expressions are bound directly to the formal parameters. This allows the procedure body to receive and manipulate source code as data.

nlambda supports two formal parameter styles. The standard list form binds each argument expression to its corresponding parameter:

```
;;; list form: each argument expression bound to one parameter
(define show-args (nlambda (a b) (list a b)))
(show-args (+ 1 1) foo)   ;;=> ((+ 1 1) foo)   -- no evaluation
```

The **bare-symbol** form binds the *entire* argument list to a single name. This is the pattern used to implement Scheme syntax forms; in *LambLisp*, primitives like `when`, `cond`, and `let` are implemented as C++ `llop` functions with the same unevaluated-argument contract:

```
;;; bare-symbol form: entire argument list bound to one name
(define show-all (nlambda args args))
(show-all x (+ 1 2) 'hello)   ;;=> (x (+ 1 2) (quote hello))
```

The *nlambda* feature, combined with quasiquote, is sufficient to implement all of *Scheme's* syntax forms. It is also the foundation on which the *LambLisp* implementation of `syntax-rules` is built.

macro

LambLisp provides the **macro** operator, which returns a **macro transformer**. Like *nlambda*, a macro transformer receives its arguments unevaluated. The critical difference is what happens to the return value:

- An **nlambda** body computes a value and returns it directly to the caller.
- A **macro** body computes a value (*the expansion*) and that expansion is then **evaluated**, with the result returned to the caller.

Applying the *macro transformer* to arguments is called *expanding* the macro; evaluating the expansion is called *executing* it. A typical macro body uses quasiquote to construct the expansion:

```
(define my-when
  (macro (test . body)             ;;; test and body received unevaluated
    `(if ,test (begin ,@body))))   ;;; expansion is an if-expression

(my-when (> x 0)
  (display x)
  (newline))
;;; expands to: (if (> x 0) (begin (display x) (newline)))
;;; then that if-expression is evaluated
```

Read-time vs. evaluation-time expansion. The *LambLisp* reader calls `macroexpand` as it builds each S-expression. When it encounters a symbol in operator position that is already bound to a macro, the expansion is performed immediately and the *expanded form* replaces the original in the input structure. In this case the macro runs once, at load time, and only the expansion is stored.

If a macro is defined *after* the code that uses it has already been read, the evaluator encounters the macro binding at run time and expands on each call. This produces correct results but at a performance cost. The practical rule is: **define macros before loading the files that use them**.

In *LambLisp* macros are first-class values. They can be stored in variables, passed as arguments, and are garbage-collected like any other pair.

Hygiene

The `nlambda` and `macro` forms described above are **unhygienic**: a name introduced in the macro expansion may accidentally capture a variable from the caller's lexical scope, or be captured by one. For example, a macro that introduces a helper binding named `tmp` will conflict if the caller also has a `tmp` in scope.

LambLisp provides `syntax-rules` (R5RS/R7RS) as a **hygienic** alternative. `syntax-rules` macros are pattern-based and use `gensym` internally to rename introduced bindings, ensuring that macro-introduced names never collide with names at the call site. Use `nlambda`/`macro` when hygiene is not required (syntax forms, DSLs, performance-critical paths); use `syntax-rules` when writing portable, reusable macro libraries.

Note

lambda, **nlambda**, and **macro** all create subtypes of procedures, represented uniformly as pairs of (formals . body) and a captured environment. The table below summarizes the differences:

Form	Arguments	Return value	Hygienic
`lambda`	evaluated	value	n/↩ a
`nlambda`	unevaluated	value	no
`macro`	unevaluated	eval(expansion)	no
`syntax-rules`	unevaluated	eval(expansion)	yes

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.11.2 Hash table and hashing details

The earlier sections describe the dictionary and hierarchical dictionary types. This section covers implementation details relevant to performance and to using non-symbol keys.

Hash table sizing and collisions

When multiple keys hash to the same index it is called a *collision*. Collisions are common but not usually a problem because the alist chains at each index are short. Hash tables are typically sized at 1–3× the expected number of entries, leaving many empty slots and keeping chains short.

Hash table size is rounded up to the next power of 2. This allows the C++ bitwise-and (`&`) operator to extract the index from the full hash value, which is significantly faster than the modulo (`%`) operator.

The oblist

Symbol hashes are computed from the symbol's text when the symbol is first created, and stored with the symbol record in a hash table called *oblist*. The *oblist* is a hash table used as a **set** — it is not a dictionary. Its purpose is to map each symbol's text to a unique address; once interned, two symbols are equal if and only if they share the same *oblist* address. This reduces every symbol-equality test to a single pointer comparison.

Objects and non-symbol keys

Dictionary keys need not be symbols. The hashing rules for non-symbol types are:

- The system hash function is djb2.
- Symbol hashes are precomputed once at creation time.
- Hashes for immutable atoms (integers, reals, etc.) are computed from the atom's value.
- All other hashes use the S-expression address — effectively applying the *eq?* predicate for key matching.

Unlike Python, which requires dictionary keys to be immutable types (tuples rather than lists), LambLisp hashes mutable objects by address, so any type may be used as a key. Dictionaries themselves may be keys or values in other dictionaries.

Because objects are dictionaries and dictionaries are lists of frames, child objects can be created from parent objects on the fly using *append* — no predeclared class hierarchy is required.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.11.3 Lightweight object system

LambLisp's dynamic nature — first-class closures, mutable hierarchical dicts, and runtime environment manipulation — makes several patterns that are awkward in C++ clean and natural in embedded Lisp code.

`dict->obj` converts any dict into a read/write closure accessor:

```
(define (dict->obj dict-in)
  (let* ((dict-out (dict-add-alist-frame dict-in '((__dict__) (__self__))))
         (dict-rdwr (lambda (key . args)
                      (if (null? args)
                          (dict-ref dict-out key)
                          (dict-rebind! dict-out key (car args))))))
    (dict-rebind! dict-out '__dict__ dict-out)
    (dict-rebind! dict-out '__self__ dict-rdwr)
    dict-rdwr))
```

- `(obj 'field)` — read a field.
- `(obj 'field val)` — write a field.
- `(obj '__dict__)` — obtain the underlying dict for direct dict operations.
- `(obj '__self__)` — obtain the closure itself as a first-class value.
- Prototype inheritance: pass a parent dict as `dict-in`; the new frame sits on top and shadowed keys in the parent remain accessible through the chain.

Performance: LambLisp dicts are hash tables, so `dict-ref` is O(1). The cost is one hash lookup vs one array index for `define-record-type` — negligible in practice.

To add a type tag, the recommended pattern is:

```
(define (typed-obj type dict-in)
  (let ((obj (dict->obj dict-in)))
    (obj '__type__ type)
    obj))

(define (lobs-type? obj type)
  (and (procedure? obj)
       (equal? (obj '__type__) type)))
```

Per-type predicate: `(define (MyRecord? x) (lobs-type? x 'MyRecord))`

The object system and containers described here are implemented in `data/Lobs.scm`.

5.11.4 Containers

`data/Lobs.scm` provides three closure-based containers:

Queue — FIFO backed by a list pair (head/tail pointers).

- `(Queue)` — create a new empty queue.
- `(q item ...)` — enqueue one or more items; returns the updated queue list.
- `(q)` — dequeue the front item; returns `#f` if empty.

Enqueue and dequeue are both O(1) amortized.

Stack — LIFO backed by a list.

- `(Stack)` — create a new empty stack.
- `(s item ...)` — push one or more items; returns the updated stack list.
- `(s)` — pop the top item; returns `#f` if empty.

Push and pop are O(1).

TimedQueue — scheduled task queue for motors, LEDs, buzzer, and other timed actuators. Each entry is `(duration . thunk)`. The main loop calls `(tq)` on every tick; entries whose deadline has elapsed are dequeued and their thunk is called.

- `(TimedQueue)` — create a new timed queue.
- `(tq ms thunk)` — schedule `thunk` to run after `ms` milliseconds.
- `(tq)` — service the queue; runs at most one expired entry per call.

The timed queue is the core scheduling mechanism for the Freenove 4WD Car demo: motor durations, LED sequences, and buzzer tones are all expressed as timed thunks rather than blocking `delay` calls, keeping the control loop responsive.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

5.12 C++ Integration

LambLisp is designed so that C++ and Scheme are peers, not separate layers. Any C++ function with the three-argument `mop3` signature becomes a native Scheme procedure with no foreign-function overhead — the same mechanism used to implement all LambLisp built-ins. Existing Arduino libraries, hardware drivers, and C++ classes require no modification. Six dimensions of C++ integration are available; they compose freely.

Group	Dimension	From	To	Mechanism
Binding	Stateless procedure	C++ function	Scheme proc	`mk_Mop3_procst_t` + `dict_bind_bang`
Binding	Stateful object	C++ class instance	GC-managed Scheme value	`mk_cppobj` + deleter
Binding	Syntax form	C++ function	Scheme nlambda	`mk_Mop3_nprocst_t`
Patterns	Device patterns	Arduino driver unchanged	Callable from Scheme	Singleton / I2C / large-driver recipes
Distribution	Proprietary distribution	C++ source private	Binary `.o` linked to LambLisp	mop3 installer linked at build time
Data	Data marshaling	Scheme value	C++ scalar or buffer	`as_int32`, `coerce_float64`, `any_bvec_get_info`

5.12.1 Virtual Machine Lifecycle

A host program creates one virtual machine, initializes it, and uses it for as long as it runs:

```
Lamb *lamb = new Lamb;
lamb->setup();      // build the cell pool, install the built-ins, load the Scheme library
...                 // bind your own procedures, evaluate, run a control loop
delete lamb;        // the real teardown
```

`setup()` is what makes the machine usable: before it runs there is no cell pool, no interaction environment and no Scheme library.

`end()` **is not a teardown, despite the name.** It resets the root stack and forces one complete collection, so every cell that is no longer reachable is finalized and returned to the free list — but it releases nothing back to the system. The cell pool, the expansion blocks and the heap payloads all stay exactly where they are. Use it when you want the machine quiesced and the collector caught up, for instance before taking a memory measurement.

The destructor is the real teardown. `delete lamb` finalizes every cell, frees the blocks and frees the memory manager. On an embedded target this is rarely wanted, because the machine lives for the life of the board; a host program that creates and discards machines must use it, since `end()` alone will grow the process without bound.

5.12.2 Stateless Procedures

`#include "LambLisp.h"` is the only header an extension needs. Everything in this section — the procedure signature, the constructors, the marshaling accessors and the garbage-collection macros — is declared there, and that header ships in every package, so an extension written against it rebuilds from source wherever the package is installed.

All of the LambLisp language primitives conform to the same function signature:

```
Sexpr_t f(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec);
```

Where:

Parameter	Description
Sexpr_t	Return value is a symbolic expression type, which is a pointer to a LambLisp memory cell.
lamb	An instance of a Lamb virtual machine; in C++ produced by `Lamb lamb = new Lamb;`
sexpr	The symbolic expression to be evaluated
env_exec	The environment in which the evaluation should take place.

To make the C++ function available in LambLisp, it must be bound into the execution environment. Because the environment is a hierarchical dictionary, the binding can occur in any frame of the dictionary. To bind function `cpp_func` to the LambLisp function `Lamb-func`, use the Lamb `dict_bind_bang` function.

A few things to note about the `dict_bind_bang` function:

- Like most other functions in Scheme-based Lisps, a *bang* or exclamation point ! is used to highlight functions that cause mutations to existing data.
- `dict_bind_bang` takes 4 arguments, 2 of which are environments (i.e., dictionaries). One is the execution environment. All LambLisp native C++ functions require this argument. The other is the target environment (which is a dictionary) that is to be modified.
- The full signature is:
  ```
  Sexpr_t dict_bind_bang(Sexpr_t dict, Sexpr_t key, Sexpr_t val, Sexpr_t env_exec);
  ```

Sample code for adding a C++ function to LambLisp:

```
//Define your native function somewhere, with the necessary signature.
Sexpr_t my_native_operator(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec);

Sexpr_t sym  = lamb.mk_symbol("Lamb-func", env_exec);  //Create a symbol in Lisp.
Sexpr_t proc = OBJ_UNDEF;
mop3_gc_frame({                                        //Frame: the root is released at its end.
    rootstack.push(sym);                               //Protect sym across the next allocation.
    proc = lamb.mk_Mop3_procst_t(my_native_operator, env_exec);
});

//Bind the symbol to the function in the top-level interaction environment.
lamb.dict_bind_bang(lamb.r5_interaction_environment(), sym, proc, env_exec);
```

As a concrete example, the ESP32-S3 devkit has an onboard LED accessible via the C++ `neopixel` function. Its native signature is:

```
void neopixel(int pin, int red, int green, int blue);
```

The *mop3* wrapper and binding look like this:

```
Sexpr_t mop3_neopixelWrite(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
{
    int pin   = lamb.car(sexpr)->as_int32();
    int red   = lamb.cadr(sexpr)->as_int32();
    int green = lamb.caddr(sexpr)->as_int32();
    int blue  = lamb.cadddr(sexpr)->as_int32();
    neopixel(pin, red, green, blue);
    return OBJ_VOID;
}

Lamb *lamb = new Lamb;
lamb->setup();

Sexpr_t proc = OBJ_UNDEF;
lambp_gc_frame({                                          // `lambp_‘ = the Lamb held by POINTER
    lambp_gcroot_Sexpr_t(sym, lamb->mk_symbol("neopixel", env_exec));   // declare and root in one
    proc = lamb->mk_Mop3_procst_t(mop3_neopixel, env_exec);             // sym survives this
    lamb->dict_bind_bang(env_target, sym, proc, env_exec);
});                                                       // the root is released here
```

Then from *Lisp*:

```
(neopixel 38 128 0 0)  ; pin 38, half red, no green, no blue
```

Note

Garbage collection can happen whenever a new cell is created. A value held only in a C++ local is not visible to the collector. Root it inside a gc-frame, which releases every root at its end with no pops to write; see GC Best Practices.

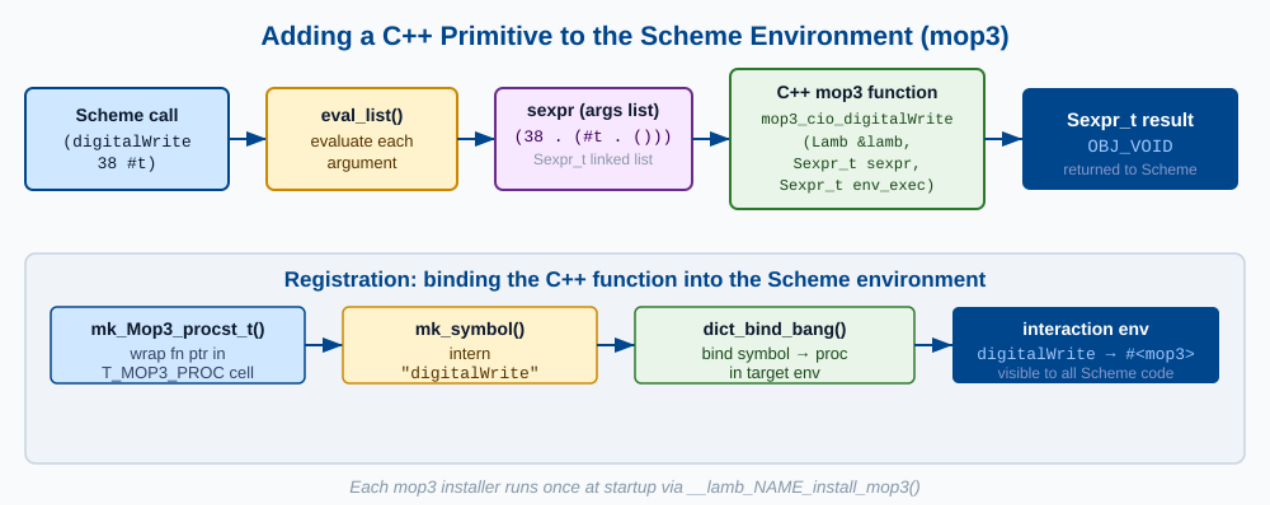


Figure 5.15 Adding a C++ Primitive to the Scheme Environment (mop3)

5.12.3 Garbage Collection in C++ Procedures

A C++ procedure that builds LambLisp values must keep them visible to the garbage collector while it works. The rules, and the gc-frame that implements them, have a chapter of their own: GC Best Practices. Read it before you write your first procedure.

5.12.4 Stateful Objects

The neopixel example above wraps a *stateless* C function — each call is independent. When a C++ object carries persistent state (member variables, dynamically allocated buffers, open handles), use `mk_cppobj()` to place it under LambLisp GC control. The GC owns the wrapper cell. When the cell becomes unreachable, the GC automatically calls the registered *deleter*, which frees the C++ object. No manual `free()` or `delete` is needed from Scheme.

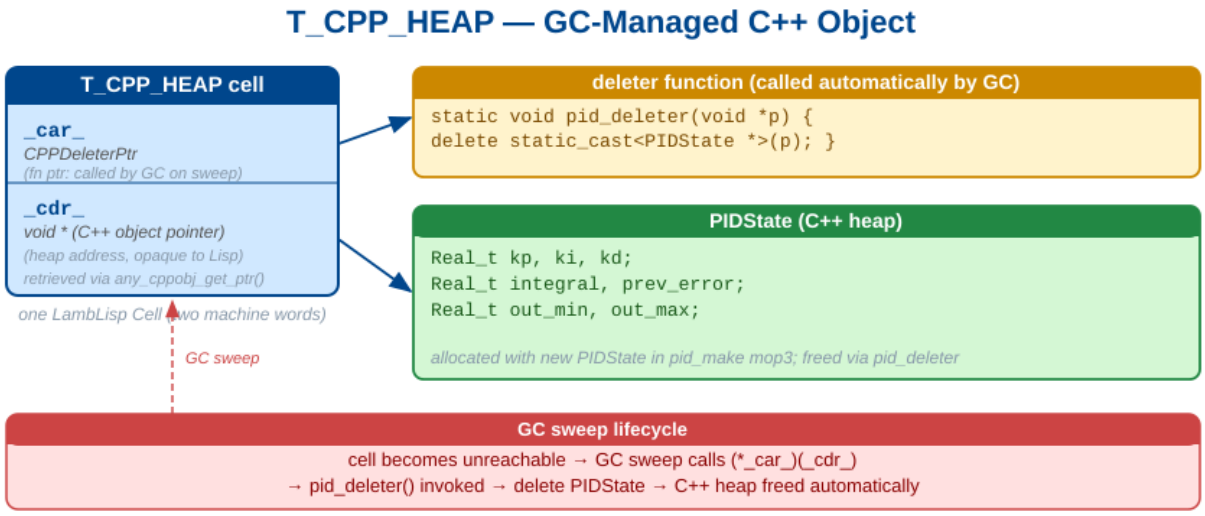


Figure 5.16 T_CPP_HEAP cell layout and GC lifecycle

API:

```
// Wrap a heap-allocated C++ object.  deleter is called by the GC on sweep.
Sexpr_t mk_cppobj(void *obj, CPPDeleterPtr deleter, Sexpr_t env_exec);

// Retrieve the pointer inside a T_CPP_HEAP cell.
void *  any_cppobj_get_ptr();

// typedef for the deleter signature
typedef void CPPDeleter(void *cpp_obj);
```

Example — PID controller:

A PID controller requires state (`integral`, `prev_error`) that persists across calls. Wrapping it in a `T_CPP_HEAP` cell makes each controller a first-class Scheme value: multiple controllers can coexist, be stored in lists, and are collected when no longer referenced.

```
struct PIDState {
  LL_float64 kp, ki, kd;
  LL_float64 integral, prev_error;
  LL_float64 out_min, out_max;
};

static void pid_deleter(void *p) { delete static_cast<PIDState *>(p); }

Sexpr_t PID_mop3_make(Lamb &lamb, Sexpr_t args, Sexpr_t env_exec)
{
  ME("::PID_mop3_make()");
  ll_try {
    auto *pid       = new PIDState;
    pid->kp         = lamb.car(args)->coerce_float64();
    pid->ki         = lamb.cadr(args)->coerce_float64();
    pid->kd         = lamb.caddr(args)->coerce_float64();
```

```
    pid->out_min    = lamb.cadddr(args)->coerce_float64();
    pid->out_max    = lamb.car(lamb.cddddr(args))->coerce_float64();
    pid->integral   = 0;
    pid->prev_error = 0;
    return lamb.mk_cppobj(pid, pid_deleter, env_exec);
  }
  ll_catch();
}

Sexpr_t PID_mop3_update(Lamb &lamb, Sexpr_t args, Sexpr_t env_exec)
{
  ME("::PID_mop3_update()");
  ll_try {
    auto *pid   = static_cast<PIDState *>(lamb.car(args)->any_cppobj_get_ptr());
    LL_float64 sp   = lamb.cadr(args)->coerce_float64();
    LL_float64 meas = lamb.caddr(args)->coerce_float64();
    LL_float64 dt   = lamb.cadddr(args)->coerce_float64();
    LL_float64 err  = sp - meas;
    pid->integral   += err * dt;
    LL_float64 deriv     = (err - pid->prev_error) / dt;
    pid->prev_error  = err;
    LL_float64 out = pid->kp * err + pid->ki * pid->integral + pid->kd * deriv;
    if (out < pid->out_min) out = pid->out_min;
    if (out > pid->out_max) out = pid->out_max;
    return lamb.mk_float64(out, env_exec);
  }
  ll_catch();
}

Sexpr_t PID_mop3_reset(Lamb &lamb, Sexpr_t args, Sexpr_t env_exec)
{
  ME("::PID_mop3_reset()");
  ll_try {
    auto *pid       = static_cast<PIDState *>(lamb.car(args)->any_cppobj_get_ptr());
    pid->integral   = 0;
    pid->prev_error = 0;
    return OBJ_VOID;
  }
  ll_catch();
}
```

From Scheme:

```
;;; Create a velocity PID: kp=1.5 ki=0.02 kd=0.05 out=[-100,100]
(define motor-pid (pid-make 1.5 0.02 0.05 -100.0 100.0))

;;; Control loop - called every dt seconds
(define (control-step setpoint encoder-reading dt)
  (pid-update! motor-pid setpoint encoder-reading dt))

;;; motor-pid is a normal Scheme value - store in a dict, pass to procedures, etc.
;;; When motor-pid goes out of scope, the GC calls pid_deleter automatically.
```

5.12.5 Device Patterns

LambLisp provides several examples demonstrating how to interface with a variety of devices. Here we review the different approaches used for different types of devices.

Low level input/output, such as digitalRead(), analogRead() and similar functions are represented directly in LambLisp, identical to their Arduino counterparts, if one discounts the shift in parenthesis. The Sonar example illustrates the techniques required.

The Sonar example also shows that some things are best done in C++; in this case busy-waiting for a pin to change state and measure its time in state with microsecond resolution. Although the system is blocked waiting for the pin, other approaches fail on the ESP32-Arduino. Using interrupts avoids blocking, but the interrupt latency is large and variable, leading to a useless measurement. Use of threads (pthreads) with a dedicated wait thread leads to the same problem.

Interestingly, the Sonar measurement falls in the same range as the loop time, and that is the source of the requirement for busy-waiting. If the loop time was much longer, the interrupt jitter would not matter. If the loop time was much shorter, we would loop and poll instead of waiting.

The Sonar example also exhibits the *singleton* pattern. There is only 1 Sonar, and it has C++ operations, so it makes sense to bundle it into a C++ class and to create a single instance of that class. The interface then becomes very thin, just a matter of marshalling the parameters in the *mop3* interfaces for each function required in Lisp.

I2C and SPI are common board-level communication protocols used to control embedded devices. SPI is point-to-point, while I2C is a command/response protocol with one command device on the bus at any time, and multiple response devices available to receive commands and report data.

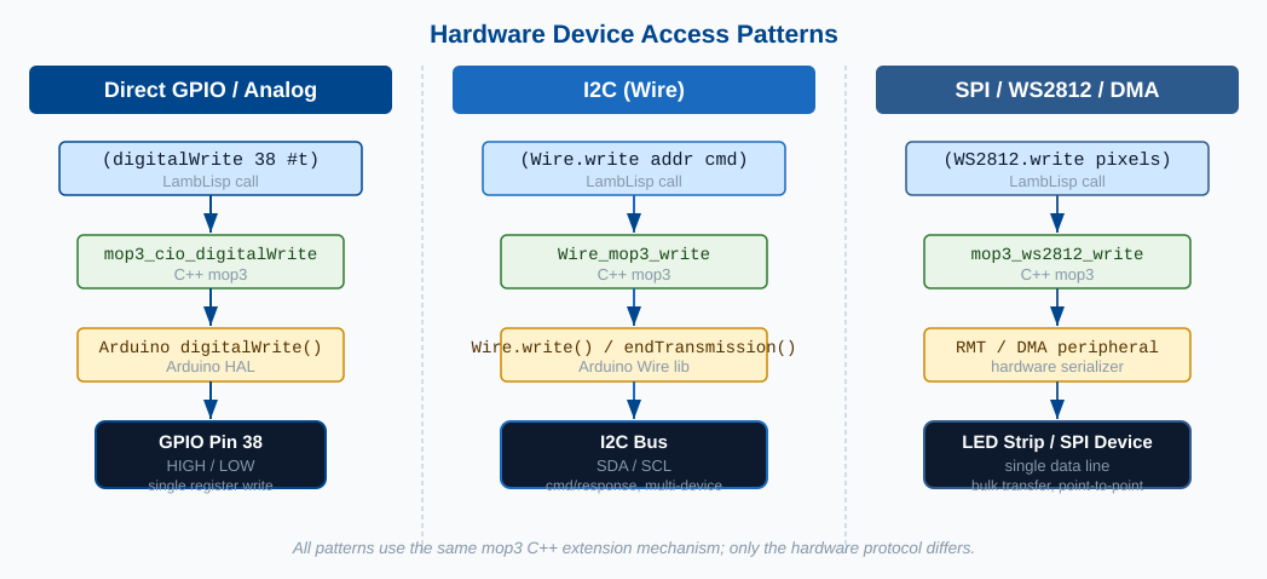


Figure 5.17 Hardware Device Access Patterns

I2C and SPI are both implemented and included in LambLisp. The SPI bindings (`SPI.begin`, `SPI.begin↩Transaction` / `SPI.endTransaction`, `SPI.transfer`, `SPI.transfer-bytevector!`, `SPI.write-bytevector`, `SPI.end`) mirror the Arduino SPI API, following the same principles as the I2C (`Wire.*`) bindings.

Interfaces to I2C devices fall into 3 sizes: small, medium, and large.

A simple device like the PCF8574 can be operated directly from Lisp over I2C with minimal overhead; this device accepts only I2C write and read commands, which are available directly in Lisp. See the file PCF8574 for the minimal example using I2C low-level operations.

Devices like Wire (aka I2C) and WiFi are in the medium category. These libraries depend on predefined *singletons* of a predefined class. The LambLisp implementation is similar to Sonar, but a bit simpler because the Wire/WiFi class and instances are predefined. On the other hand, both Wire and WiFi have a lot more methods and parameters to deal with.

In the case of Wire and WiFi, C++ does a bit of work, checking and returning high-level errors rather than returning raw error codes to Lisp. Mostly though, the interface layer is concerned with marshalling parameters between C++ and Lisp.

The PCA9685 is a 16 channel PWN controller, originally advertised as an LED controller, but in fact it can be used anywhere a controllable square wave is needed. As it turns out, a variety of motors and linear actuators can be controlled this way.

The PCA9685 is simple in principle, but it allows multiple devices on the bus, and allows each of them to respond to a secondary address, which is programmable and may be shared between devices. This allows operating them in groups with a single command.

That is relevant because it makes the driver code much larger, more than most people would want to rework and take ownership. The PCA9685 LambLisp example shows how to incorporate a large driver without needing to understand it in detail or modify it, by interfacing only with the behaviors required for your application.

5.12.6 Syntax Forms

A `T_MOP3_PROC` receives its arguments already evaluated. When you need a C++ function that receives its arguments *unevaluated* — to implement a special form, a short-circuit operator, or a macro-like construct — register it as a `T_MOP3_NPROC` by setting `syntax=true` in the installer table.

The `sexpr` parameter then contains the raw, unevaluated argument list exactly as written at the call site. The C++ function is responsible for evaluating whatever it needs via `lamb.eval()`.

```
// An nlambda that evaluates only its second argument if condition is true:
Sexpr_t fn_when(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec) {
  Sexpr_t cond = lamb.eval(lamb.car(sexpr), env_exec);
  if (cond != HASHF)
    return lamb.eval(lamb.cadr(sexpr), env_exec);
  return OBJ_VOID;
}

// In the installer table:
{ fn_when, "when", true }   // true → T_MOP3_NPROC
```

5.12.7 Proprietary Distribution

Because the mop3 binding mechanism is pure C++, a library vendor can distribute compiled object files (`.o` or a static `.a`) without revealing source code. The installer function — `__lamb_NAME_install_mop3` — is the only symbol the linker needs; the customer adds one call to it at startup and the full C++ API is available as native Scheme procedures.

The Scheme source files that *use* the library (`.scm`) are human-readable and can be shared freely; the C++ implementation remains private. This is the recommended pattern for commercial sensor and actuator drivers distributed as LambLisp extensions.

```
// Vendor delivers: libmysensor.a + mysensor_install.h
// Customer's main.cpp:
#include "mysensor_install.h"

void setup() {
  Lamb *lamb = new Lamb;
  lamb->setup();
  Sexpr_t env = lamb->r5_interaction_environment();
  __lamb_mysensor_install_mop3(*lamb, env, env);  // one call exposes all APIs
}
```

```
; Customer's control.scm - no C++ visible:
(sensor-init 0x48)
(define (read-temperature) (sensor-read-celsius 0x48))
```

5.12.8 Data Marshaling

Every value crossing the Scheme/C++ boundary is a `Sexpr_t` cell pointer. The marshaling API converts between cells and C++ native types.

Direction	Scheme type	C++ call	C++ type
Scheme → C++	integer	`p->as_int32()`	`LL_int32`
Scheme → C++	integer	`p->as_int64()`	`LL_int64`
Scheme → C++	real	`p->coerce_float64()`	`LL_float64`
Scheme → C++	bytevector	`p->any_bvec_get_info(len, ptr)`	`size_t, ByteVec_t`
Scheme → C++	string	`p->str()`	`const char*`
C++ → Scheme	integer	`lamb.mk_integer(v, env)`	`int64_t`
C++ → Scheme	real	`lamb.mk_float64(v, env)`	`double` (`mk_float32` for a `float`)
C++ → Scheme	bytevector	`lamb.mk_bytevector(n, env)`	`size_t`
C++ → Scheme	string	`lamb.mk_string(len, s, env)`	`LL_int32, const char*`
C++ → Scheme	boolean	`HASHT` / `HASHF`	—
C++ → Scheme	void	`OBJ_VOID`	—

`mk_string` **has three forms, and one of them is a** `printf`**.** The form whose second argument is a `const char *` takes a **format string**, not content:

```
Sexpr_t mk_string(Sexpr_t env_exec, const char *fmt, ...);      // a FORMAT -- printf semantics
Sexpr_t mk_string(LL_int32 k, Charst_t src, Sexpr_t env_exec);  // k bytes of content
Sexpr_t mk_string(String s, Sexpr_t env_exec);                  // a String's content
```

Never pass runtime data as `fmt`. `lamb.mk_string(env_exec, buf)` compiles, reads like "make a string out of `buf`", and is a `printf` call with no arguments behind it. A `s` in the data then dereferences an argument that was never passed and a `d` prints whatever happened to be on the stack, so the result comes back longer than the input and carrying memory that is none of your business. For content, use the length-carrying form, which copies `k` bytes and cares nothing for zero bytes or percent signs; for a NUL-terminated runtime string such as a filename, pass `"s"` as the format and the string as an argument:

```
lamb.mk_string(env_exec, "%s", path);           // correct
lamb.mk_string(strlen(path), path, env_exec);   // also correct
```

Note that `-Wformat` on its own does not warn about a non-literal format; the diagnostic that does is `-Wformat-security`.

Errors: throw a C++ exception inside a mop3 and LambLisp's `ll_catch()` intercepts it, converts it to a Scheme error object, and propagates it through `with-exception-handler` / `guard` as a normal R7RS error. No special C++ error-handling code is needed.

```
Sexpr_t fn_safe_divide(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec) {
  ll_try {
    int32_t a = lamb.car(sexpr)->as_int32();
    int32_t b = lamb.cadr(sexpr)->as_int32();
    if (b == 0) throw std::runtime_error("division by zero");
    return lamb.mk_integer(a / b, env_exec);
  }
  ll_catch();
}
```

5.12.9 Allocating Bulk Data

Strings, bytevectors, vectors, bignum digits and compiled bytecode bodies are too large to live inside a cell, so each is a cell plus a **payload**: a block of bytes taken from the same memory the cell pool is built from (see Sizing the cell pool). If your procedure needs such a block of its own, allocate it with

```
void *p = lamb.payload_alloc(nbytes, "my-driver", env_exec);
```

rather than with `new` or `malloc`, because `payload_alloc` carries a defined out-of-memory policy and the raw allocators do not.

The policy, in order. It tries the allocation. If that fails it runs the collector to completion and retries **once** – payload memory is reclaimed by the sweep, which frees the payloads of dead cells, so exhaustion is very often transient. If it still fails it throws a normal LambLisp error.

It never returns `NULL`**,** and that is the whole point. The error is an ordinary Scheme error: `guard` catches it, so an application can shed load and carry on, and a REPL survives to report it. The alternative – a `NULL` written through by a caller that did not check – is a memory fault at an address, which on an embedded target means a reboot, and during startup a boot loop. A real-time node must not fail that way, and must not fail silently.

The second argument is a short name for your subsystem. It appears in the error text and in the log, so make it something you would want to read in a field report.

Chapter 6

GC Best Practices

This chapter is for anyone writing C++ that touches LambLisp values: a mop3 procedure, a device driver, or integration code that calls into a running `Lamb`. Scheme code needs none of it; the collector sees everything a Scheme program can reach.

It is the one part of extending LambLisp that has no analogue in ordinary C++, and getting it wrong produces a fault that appears somewhere else entirely, in code you did not touch. Read it once before you write your first procedure; it is shorter than the bug hunt.

The rule, in one sentence: *every allocating call can run a slice of the collector, and a `Sexpr_t` sitting in a C++ local is not a root.*

`cons`, `eval`, `mk_integer`, `mk_string`, `dict_bind_bang` and every other constructor may run a collection quantum before they return. The collector traces the **root stack**, not your C stack. A value that is not reachable from a root at that moment is collected, its cell goes back on the free list, and the next allocation hands the same cell to someone else. Your procedure then keeps working with a cell that now belongs to another value.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

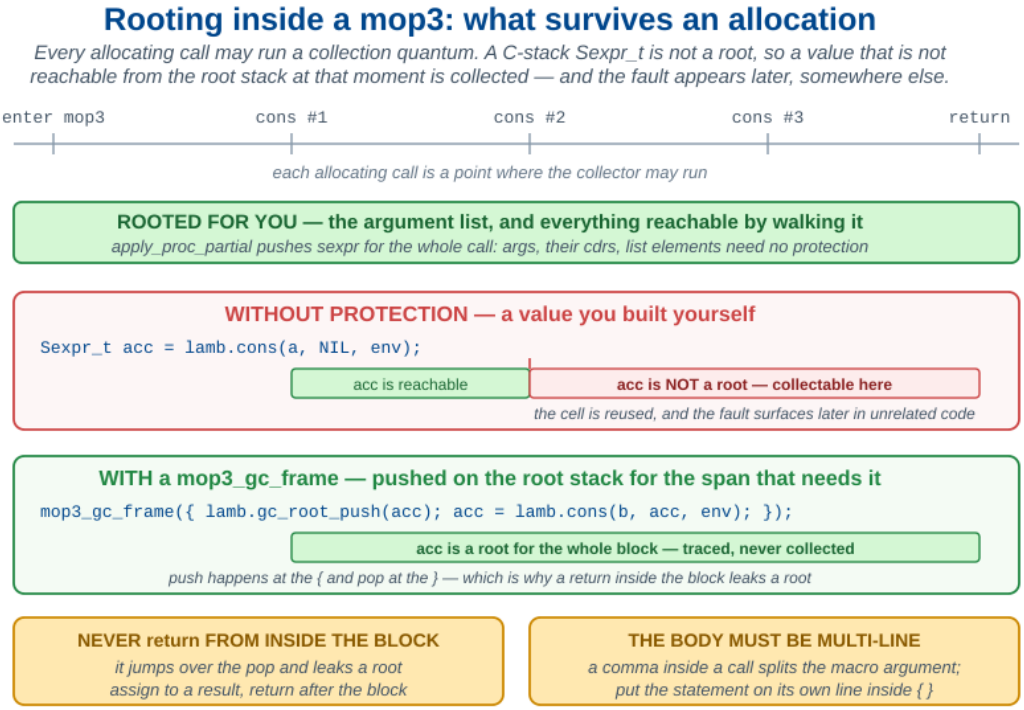


Figure 6.1 Rooting inside a mop3 call

6.1 The gc-frame

Every root you push belongs inside a gc-frame. The frame is the one mechanism LambLisp uses to protect values held in C++, and it is used the same way everywhere in the runtime:

```
Sexpr_t res = NIL;
mop3_gc_frame({
  mop3_gcroot_Sexpr_t(key, lamb.mk_symbol("speed", env_exec));  // declared AND rooted
  Sexpr_t val = lamb.mk_integer(42, env_exec);                  // key survives this
  rootstack.push(val);                                          // as many roots as you need
  res = lamb.cons(key, val, env_exec);
});                                                             // every root released here
return res;
```

A frame records the depth of the root stack when the body starts and restores that depth when the body ends. Inside, you push whatever you need to protect, wherever the code needs it: in a loop, under a condition, after a reassignment. When the frame closes, everything pushed inside it is released in one step.

6.1.1 Why the frame matters

The obvious alternative is to pair every push with a pop. That approach fails in a way that is expensive to find:

- **A skipped pop corrupts someone else's roots.** Add an early exit, an extra branch or a second push and forget the matching pop, and the stack is left one entry too deep. Your caller's own pop then removes *your* entry instead of *its* own, and one of its values becomes collectable with nothing in its code to show why.

- **Counting is a claim that must be re-checked at every edit.** A push inside a conditional needs a pop on exactly the paths where the push happened. Every later change has to preserve that pairing, and nothing checks it.

A frame removes the count. Nothing is paired, so there is nothing to mismatch. The worst that a mistake inside a frame can do is leave the stack *deeper* than it should be, and the next enclosing frame chops that back. In other words, a frame turns a corruption into a slight delay in collection.

Frames also nest freely, cost two integer stores, and work identically in mop3 procedures, in integration code and inside the runtime itself, so there is one idiom to learn and one to read.

6.1.2 Which name to use

The three forms differ only in how your code reaches the interpreter, not in what they do:

you hold	frame	declare-and-root	push
`Lamb &lamb` (a mop3 procedure)	`mop3_gc_frame`	`mop3_gcroot_Sexpr_t`	`rootstack.push`
`Lamb *lamb` (integration code)	`lambp_gc_frame`	`lambp_gcroot_Sexpr_t`	`rootstack.push`

The push is spelled the same either way, because `rootstack` is a global rather than a member of the machine. (A third family, `ll_*`, is internal to the interpreter's own members.)

`mop3_gcroot_Sexpr_t(name, value)` is exactly `Sexpr_t name = value;` followed by a push of it. Prefer it wherever you would otherwise write a declaration immediately followed by a push. A bare `rootstack.push(x)` is right when `x` already exists — a loop accumulator declared outside the loop, or a variable you have just reassigned.

A push roots a value, not a variable. It protects the cell `x` held at the moment of the push. Reassign `x` and the new value is unprotected until you push again. Inside a frame the extra push costs nothing and needs no pop.

6.2 Who protects what

You do not have to root everything you touch. Three rules decide whose job a value is:

what you have	who protects it
your argument list and `env_exec`	**the caller** — they are rooted for the whole call
a value returned by `car`, `cdr` or one of their compositions	**the accessor** — it is rooted when it is created

what you have	who protects it
a value you pass as an argument to a constructor or evaluator	**the callee** — it protects its arguments while it runs
a value a constructor or evaluator **returns**, held across a later allocation	**you**

The first three cover most of a typical procedure. The fourth is where the work is.

Your arguments. `apply_proc_partial` keeps your argument list rooted for the entire call, and the `env_exec` you were given is handed to every collection as a root. That is also why every constructor takes an `env_exec`: the collection your allocation may run uses it as a root, so the environment and everything reachable from it survives. **Pass the** `env_exec` **your procedure was called with.** Do not pass `NIL` to save a parameter, and do not invent one.

Accessor results. A pair slot can hold a small integer or float directly instead of a pointer, so `car` and `cdr` sometimes build a fresh cell to hand the number back. That cell is rooted at the moment the accessor creates it, so you may hold it across later allocations without doing anything.

Even so, **if you want a C value, do not make the cell at all.** The slot readers `car_mustbe_int32`, `cdr_coerce_float32` and their family perform the same type checks and raise the same errors, but allocate nothing:

```
LL_int32 n = lamb.car_mustbe_int32(args);   // no cell is built, so there is nothing to collect
```

They are also faster, and on a microcontroller that matters as much as the safety.

Callee arguments. The core allocating calls (`cons`, `tcons`, `eval`, `eval_list`, `apply_proc_partial`, `append2`, `mk_symbol`, `mk_procedure`) protect their own arguments for the duration of the call. A value you hand straight to one of them needs no root of yours for that call. If you are unsure whether some other call protects an argument, root the argument: a redundant root costs one push.

Returned values. A constructor protects its arguments, *not its result*. A cell you get back from `cons`, `eval`, `mk_string` or any other allocating call is reachable from nothing until you store it somewhere rooted. If another allocation happens before that, root it.

6.3 Order the calls so values are consumed

The cheapest root is the one you never need. Because a callee protects its arguments, a value that is passed **directly into the next call** is safe without a push. So build inner-to-outer, and let each result be consumed as an argument of the call that follows:

```
// The inner cons result is an argument of the outer cons, which protects it.
res = lamb.cons(key, lamb.cons(a, b, env_exec), env_exec);
```

Compare the same work written as separate statements:

```
Sexpr_t inner = lamb.cons(a, b, env_exec);   // unrooted ...
Sexpr_t k2    = lamb.mk_symbol("k", env_exec); // ... and this allocation can collect it
res = lamb.cons(k2, inner, env_exec);
```

The second version needs `inner` rooted. The first needs nothing. Prefer the shape where the protection is visible from the code itself: a reader can see the inner call sitting inside the outer call's argument list, and no later edit can silently separate the two.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

6.4 Root what you hold

What remains after the rules above is a short list: values **you build** and keep across a later allocation that does not take them.

- an accumulator you grow with `cons` in a loop
- a reversed or copied list you are assembling
- an intermediate result held while you compute the next one

The test is mechanical: *between the line that creates this value and the line that stores it into something already rooted, is there any call that can allocate?* If yes, root it.

```
// The accumulator is built here, so it is yours to protect.
Sexpr_t acc = NIL;
for (Sexpr_t l = args; l != NIL; l = lamb.cdr(l)) {
  mop3_gc_frame({
    rootstack.push(acc);                // acc is held across car(l), which may allocate
    acc = lamb.cons(lamb.car(l), acc, env_exec);
  });
}
```

6.4.1 A loop that roots gets a frame per iteration

A frame bounds what it releases, not what its body pushes. A loop *inside* a frame that roots a value each time round grows the root stack with the iteration count, and the stack is finite — 65,536 entries on an ESP32 — so a long enough list exhausts it. Put the frame around the loop *body*, as above, and the depth stays flat however many iterations run:

```
mop3_gc_frame({                         // one frame for the procedure
  for (Sexpr_t p = list; p != NIL; p = lamb.cdr(p)) {
    mop3_gc_frame({                     // and one per iteration
      Sexpr_t item = lamb.car(p);
      res = lamb.cons(item, res, env_exec);
      rootstack.push(res);              // reassigned, so rooted again
    });
  }
});
```

`continue` belongs to this rule: taken inside a frame, it skips that iteration's restore, so a frame whose body ends in a `continue` never lowers the depth. Give the loop body its own frame.

6.4.2 Leaving a frame early

A `return`, `break` or `goto` out of a frame body is acceptable. It skips the restore, so the depth is left above its entry value, and the next enclosing frame's restore chops it back. The cost is that a few values are collected later than they could have been, once per call. Do not restructure working control flow to hoist a `return` out of a frame; that rewrite is more likely to introduce a defect than the `return` was. Writing the result to a variable and returning after the frame is still the tidier shape where it is natural, as in the examples above.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

6.4.3 What may go in a frame body

The body is otherwise unrestricted. The frame macros are variadic, so commas in the body — inside a call like `lamb.cons(a, b, env_exec)` or at the top level like `Sexpr_t p = a, q = b;` — are fine, on one line or many. `#if` / `#endif` inside a body work as usual.

One construct does *not* work: a `#define` or `#undef` written inside the body never takes effect for that body, because the braced block is a macro argument and a definition made in an argument is not in force for the argument's own tokens. Every use of such a macro in the same body is then reported as an undeclared identifier, naming a macro defined a few lines above it. Put the definition before the enclosing block.

6.5 Procedures that take their arguments as an array

A procedure registered with `mk_Mop3_procst_argv_t` is handed its already-evaluated arguments as a C array — `Sexpr_t *argv`, `int nargs` — instead of as a Scheme list. **That array is not on the root stack**, so the guarantee for your argument list does not cover it.

Read every `argv[i]` you need into a C value **before your first allocation**:

```
Sexpr_t my_add_argv(Lamb &lamb, Sexpr_t *argv, int nargs, Sexpr_t env_exec)
{
  LL_int32 a = argv[0]->mustbe_int32();        // read what you need FIRST ...
  LL_int32 b = argv[1]->mustbe_int32();
  return lamb.mk_integer(a + b, env_exec);     // ... and allocate only after
}
```

If you need the argument *cells* rather than their values, root them before you allocate anything:

```
mop3_gc_frame({
  for (int i = 0; i < nargs; i++) rootstack.push(argv[i]);
  res = build_something(lamb, argv, nargs, env_exec);
});
```

The ordinary three-argument form has no such constraint and is the right default; register an array variant only where the call rate justifies one.

6.6 Procedures that can throw

A throw leaves a frame without running its restore. In a mop3 procedure, wrap the body in `mop3_try` / `mop3_catch`: `mop3_try` records the root-stack depth, and `mop3_catch` restores it before the error propagates, so every root pushed inside — at any frame depth, from any throw site, including ones added later — is released.

```
Sexpr_t my_proc(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
{
  ME("my_proc()");
  Sexpr_t res = NIL;
  mop3_try {
    mop3_gc_frame({
      mop3_gcroot_Sexpr_t(acc, lamb.cons(lamb.car(sexpr), NIL, env_exec));
      if (lamb.cdr(sexpr) == NIL) throw lamb.mk_error(env_exec, "%s needs two arguments", me);
      res = lamb.cons(lamb.cadr(sexpr), acc, env_exec);
    });
  }
  mop3_catch();                            // depth restored here, then the error is rethrown
  return res;
}
```

Do not write a `gc_root_pop()` in front of each `throw`; that is correct only until someone adds the next throw site. `mop3_catch(code)` runs `code` after the restore and before the rethrow, if you have other cleanup to do.

Code that holds the `Lamb` by pointer and cannot use `mop3_try` uses the same idea written out: save `rootstack.depth()` before the `try`, and call `lamb->gc_root_setdepth(saved)` in the `catch`. Restoring a depth you saved is the only correct way to unwind. There is deliberately no call that empties the whole root stack, because it would also drop entries your code never pushed: your caller's, and the interaction environment's permanent root.

6.7 What it looks like when you get it wrong

The symptom is almost never at the fault. A collected cell is reused, the corrupted value is used later, and the error names something unrelated — commonly a type error on a form that is obviously well-formed, a list that has acquired a `nil` in the middle of it, or a hard crash with no Scheme error at all. The last is the most common: your type check has already happened, so nothing re-examines the cell after it changes type, and `guard` cannot catch what was never raised.

The failure also depends on where the collector happened to be in its cycle, so the same input fails some fraction of the time and moves with anything that changes allocation timing.

If you are seeing that pattern, look for a value returned by an allocating call and held across a second allocation without a root, before you suspect the collector.

6.8 Checklist

1. Put every root inside a gc-frame. Never pair pushes with pops by hand.
2. Want a C number? Use `car_mustbe_int32` and family — no cell, nothing to protect.
3. Pass results straight into the next call where you can; the callee protects its arguments.
4. Root a value returned by an allocating call if you hold it across another allocation.
5. Push again after reassigning a rooted variable.
6. Give a loop that roots its own frame around the body.
7. Wrap a mop3 that can throw in `mop3_try` / `mop3_catch`.
8. Pass the `env_exec` you were given to every constructor.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 7

LambLisp Extensions

The features in this section are present in LambLisp but are outside the R5RS and R7RS-small specifications. They fall into three broad groups: language and control extensions to the Scheme core, typed numeric storage types for embedded numerical work, and hardware and I/O interfaces specific to the supported platforms (ESP32, Jetson, Linux). The C++ API for adding new native procedures is described separately in the *Language Internals and C++ API* section.

7.1 Language Extensions

Feature	Description
`nlambda`	Defines a procedure that receives its arguments *unevaluated* as a list; the InterLisp mechanism underlying all LambLisp syntax forms.
`macro`	First-class macro value: a transformer procedure applied at eval-time; the expansion is then evaluated. Distinct from `syntax-rules` hygienic macros.
`macro?`	Returns `#t` if its argument is a `macro` value.
`nproc?` / `nprocedure?`	Returns `#t` if its argument is an `nlambda` procedure.
`llop-proc?` / `llop-nproc?`	Returns `#t` for a C++ native procedure or C++ nlambda respectively.
`macroexpand`	Fully expands a macro call form, applying all transformations recursively.
`macroexpand1`	Expands a macro call form exactly one level.
`atom?`	`(atom? x)` — true if `x` is not a pair; the complement of `pair?`.
`ifnot`	`(ifnot test then)` — shorthand for `(if (not test) then)`.
`while`	`(while test body ...)` — loop that executes body while test is true; returns unspecified.
`until`	`(until test body ...)` — loop that executes body until test becomes true; returns unspecified.
`reverse!`	Destructive in-place list reversal; reuses the existing pairs.

Feature	Description
`vector->alist` / `alist->vector`	Convert between association lists and vectors.
`current-environment`	Returns the current lexical environment as a first-class dictionary object.
`add1` / `sub1`	Increment or decrement a number by one.
`pi`	The floating-point value of pi.
`gensym`	Returns a fresh uninterned symbol guaranteed unique across all calls.
`sizeof`	`(sizeof type-keyword)` — returns the C++ `sizeof` of a named type; useful for FFI layout calculations.

Platform detection

Feature	Description
`lamb-board`	String identifying the current build target; set at compile time from the PlatformIO environment name (e.g., `"linux_x86_64"`, `"esp32-s3-devkitc-1"`).
`linux?`	`#t` on all Linux targets (x86-64 and AArch64 / Jetson).
`esp32?`	`#t` on all ESP32 and ESP32-S3 board variants.
`jetson?`	`#t` on `linux_aarch64` (Jetson Orin Nano).

7.2 Dictionary (Environment) Operations

Feature	Description
`dict-ref`	Looks up a key in a hierarchical dictionary, searching parent frames.
`dict-ref?`	Like `dict-ref` but returns `#f` instead of signalling an error when the key is not found.
`dict-bind!`	Binds a new key–value pair in the innermost frame of a dictionary, walking the chain if the key already exists.
`dict-rebind!`	Updates an existing binding in a dictionary without adding a new frame entry.
`dict-bind-alist!`	Binds all key–value pairs from an association list into a dictionary frame.
`dict-rebind-alist!`	Updates multiple existing bindings from an association list.
`dict-add-empty-frame`	Appends a new empty hash-table frame on top of a dictionary chain.
`dict-add-keyval-frame`	Appends a new frame pre-populated with given keys and values.
`dict-add-alist-frame`	Appends a new frame pre-populated from an association list.
`dict-append!`	Appends one dictionary chain to another, linking their frames.
`dict-keys`	Returns a list of all keys visible in a dictionary (all frames).
`dict-values`	Returns a list of all values visible in a dictionary (all frames).
`dict->alist`	Converts a dictionary frame to an association list of `(key . value)` pairs.
`dict->2list`	Returns two parallel lists `(keys values)` for all bindings in a dictionary.

Feature	Description
`alist->dict`	Converts an association list to a single-frame dictionary.
`dict-analyze`	Prints diagnostic statistics about a dictionary (load factor, chain depth, collisions).
`list-analyze`	Prints length and structural statistics about a list.

7.3 Structured Data

LambLisp bundles three lightweight facilities for moving structured data in and out of an embedded target — JSON documents, packed binary frames, and typed numeric arrays — so a device can talk to cloud APIs, wire protocols, and numerical code without external libraries.

7.3.1 JSON

`json-read` parses a JSON string into LambLisp values; `json-write` serialises them back. JSON objects map to association lists with string keys, arrays to lists, `true`/`false` to `#t`/`#f`, and `null` to the symbol `null`; numbers become exact integers or inexact reals as written.

```
(json-read "{\"pin\":4,\"on\":true}")   ;; => (("pin" . 4) ("on" . #t))
(json-write (list 1 2.5 #f 'null))      ;; => "[1,2.5,false,null]"
```

7.3.2 CSV

The `csv` feature (`features/csv.scm`, auto-loaded where shipped) reads and writes comma-separated values with RFC 4180 quoting. A quoted field (one wrapped in double quotes) may contain the delimiter, a newline, or a quote written as `""`. The reader works one character at a time, so a quoted field that spans several physical lines is read correctly. Field values are strings unless you supply a coercion function.

For reading, `csv-read` returns a list of records, where each record is a list of field strings. `csv-read->dicts` treats the first record as a header row and returns a list of dicts keyed by those field names. This is the equivalent of Python's `csv.DictReader`.

Procedure	Result
`(csv-read port [opts])`	list of records (each a list of field strings)
`(csv-read->dicts port [opts])`	list of dicts; the header row keys the rest
`(csv-string->rows str [opts])` / `(csv-string->dicts str [opts])`	same, from a string
`(read-csv path [opts])` / `(read-csv->dicts path [opts])`	same, from a file path
`(csv-parse-line str [opts])`	parse one record string → list of fields

```
(dict-ref (car (read-csv->dicts "sensors.csv")) "temperature")   ; => "21.5"
(csv-string->rows "a,\"x,y\",c\n")         ; => (("a" "x,y" "c"))   ; quoted comma preserved
```

For writing, the same data goes the other way (records or dicts to CSV text). A field is quoted if it contains the delimiter, a quote, or a newline, and any embedded quotes are doubled.

Procedure	Effect
`(rows->csv-string rows [opts])` / `(dicts->csv-string dicts [opts])`	render to a string
`(csv-write port rows [opts])` / `(csv-write-dicts port dicts [opts])`	write to a port

```
(rows->csv-string '(("id" "note") ("1" "has, comma")))
;; => "id,note\r\n1,\"has, comma\"\r\n"
```

The `opts` argument is an alist; all keys are optional:

Key	Default	Meaning
`delimiter`	`#\,`	field separator; e.g. `(integer->char 59)` for `;`, `(integer->char 9)` for tab
`quote`	double-quote (U+0022)	quote character
`trim`	`#f`	strip surrounding whitespace from **unquoted** fields (quoted fields kept verbatim)
`coerce`	identity	`(lambda (col value-string) -> value)` applied per cell in the `->dicts` readers
`columns`	first dict's keys	explicit column order/header for `csv-write-dicts`
`quote-all`	`#f`	writer quotes every field, not just those that need it
`newline`	`"\r\n"`	record terminator the writer emits (`"\n"` for unix)

```
;; coerce two numeric columns to numbers while reading:
(read-csv->dicts "sensors.csv"
  (list (cons 'coerce
              (lambda (col v)
                (if (member col '("temperature" "humidity")) (string->number v) v)))))
```

A few behavior notes. Blank lines are skipped. A data row with too few columns fills the missing ones with `""`; a row with too many drops the extras. Both cases are logged. For any rows of strings, reading back the output of `csv-write` reproduces the original rows. One case is not yet handled: a leading UTF-8 byte-order mark is not stripped automatically, so remove a BOM upstream if your source emits one.

7.3.3 Binary framing (struct)

`struct-pack` / `struct-unpack` (with `struct-pack-into!` for in-place fills and `struct-calcsize` for sizing) convert between LambLisp numbers and packed binary buffers for Modbus registers, sensor maps, and MQTT payloads. Layout is driven by a Python-`struct`-style format string with an explicit endianness prefix, so results never depend on host byte order.

```
(struct-unpack ">HhB" (struct-pack ">HhB" 40001 -120 7))  ;; => (40001 -120 7)
```

7.3.4 Typed numeric arrays (`array-*`)

Typed arrays hold homogeneous, unboxed numeric data. The elements are stored as raw machine numbers, contiguously, in C row-major order. A vector is just the rank-1 case: the same `array-*` operations work on vectors and on higher-rank arrays. The element types are `int32`, `int64`, `float32`, and `float64`. There are also two aliases that follow the host: `int` is the native integer (`int64` on 64-bit hosts, `int32` on ESP32) and

`real` is always `float32`. This set of operations replaces the older `tvec`, `intvector`, `realvector`, and `ndarray` systems.

These are distinct from R7RS `vector` (which holds arbitrary Scheme values) and from R7RS `bytevector` (which holds `u8` bytes).

There are two kinds of constructor. `make-<T>-vector` takes a single length `N` and builds a rank-1 array; passing a shape list to it is an error. `make-<T>-array` takes a shape (a list of dimensions, or the dimensions as separate arguments) and builds an N-dimensional array. Both take an optional `fill` argument, which defaults to zero.

```
(define v (make-float32-vector 4 1.5))        ;; length-4 float32 vector, filled 1.5
(array-ref v 2)                                ;; => 1.5
(define m (make-float32-array (list 2 3)))     ;; 2x3 float32 matrix, zero-filled
(array-set! m 1 2 12.0)
(array-ref (array-transpose m) 2 1)            ;; => 12.0
```

Constructors

Feature	Description
`(make-int32-vector N [fill])` ... `make-float64-vector`	Rank-1 array of the given element type and length `N`.
`(make-int-vector N [fill])`	Rank-1 native-integer array (`int64` on 64-bit hosts, `int32` on ESP32).
`(make-real-vector N [fill])`	Rank-1 `float32` array.
`(make-int32-array shape [fill])` ... `make-float64-array`	N-D array; `shape` is a list of dims or spread dims.
`(make-int-array shape [fill])` / `(make-real-array shape [fill])`	N-D native-integer / `float32` array.

Operations (work on any rank)

Feature	Description
`(array-ref a i j ...)`	Reads one element; element type inferred from the array.
`(array-set! a i j ... v)`	Writes one element.
`array-type`	Element-type tag (compare against `array-type-int32` / `-int64` / `-float32` / `-float64`).
`array-rank`	Number of dimensions.
`(array-dim a k)`	Extent of dimension `k` (0-based).
`array-size`	Total element count.
`array-length`	Length of a rank-1 array.
`array-shape`	Shape as a Scheme list of integers.
`array-data`	The underlying raw data bytevector (row-major).
`(array-reshape a shape)`	Fresh array with a new shape over a copy of the data (element count must match).
`array-transpose`	Fresh array with reversed axes (for a 2-D array, the matrix transpose).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Feature	Description
`(array-slice a dim start end)`	Fresh sub-array over `[start, end)` along dimension `dim`.
`array-copy`	Fresh, independent copy.
`(array-fill! a v)`	Sets every element to `v`.
`array-sum`	Sum of all elements (integer for int arrays, real for float arrays).
`array-randomize01!`	Fills a `float32` array with uniform random values in [0, 1).
`array-randomize11!`	Fills a `float32` array with uniform random values in [-1, 1).
`(array-dot a b)`	Inner product of two rank-1 arrays of equal length and element type.
`array?`	`#t` if its argument is a typed array.
`vector->sparsevec`	Converts a Scheme `vector` to a list of `(index . value)` pairs for non-zero entries.
`sparsevec->vector`	Reconstructs a dense Scheme `vector` from a sparse-vector representation.

Array math (linear algebra)

Linear-algebra and element-wise operations over typed arrays of any rank (a rank-1 array is a vector). Each operation dispatches on the array's element type, and reports an error if the shapes or element types do not match. The layout is row-major (C order), meaning the last index varies fastest in memory, so loops run fastest when the last index is the innermost one. The built-in operations already do this; for example, the CPU path of `array-matmul!` uses i-k-j loop order. See CUDA Integration for moving an array's backing store (`array-data`) to and from GPU device memory.

Feature	Description
`(array-matmul! C A B)`	Matrix product $C = A \cdot B$ for 2-D arrays, written in place. `A` is `m×k`, `B` is `k×n`, `C` must be preallocated `m×n`; all three share one element type. On CUDA builds the `float32` / `float64` cases delegate to cuBLAS `sgemm` / `dgemm` (using the $(A \cdot B)^T = B^T \cdot A^T$ swap so no host-side transpose is needed); otherwise a row-major CPU loop runs. Integer element types always use the CPU loop.
`(array-scale! a s)`	Multiplies every element of `a` by scalar `s`, in place.
`(array-add! a b)`	Element-wise `a[i] += b[i]`, in place (same shape and element type).
`(array-mul! a b)`	Element-wise `a[i] *= b[i]`, in place (same shape and element type).
`(array-copy! dst src)`	Copies `src` into `dst`, in place (same shape and element type). Distinct from the allocating `array-copy`.
`array-contiguous?`	Always `#t`. The `(shape . data)` representation is always contiguous (C row-major); strided zero-copy views are not part of the model. The procedure is kept for API completeness.

```
(define A (make-float32-array (list 2 3)))
(define B (make-float32-array (list 3 2)))
(define C (make-float32-array (list 2 2)))
;; ... fill A and B ...
(array-matmul! C A B)          ;; C = A·B, cuBLAS-accelerated on CUDA builds
(array-scale! C 0.5)           ;; halve every element in place
```

7.4 Arbitrary-Precision Integers (Bignums)

LambLisp integers are exact and have no fixed size limit. They start as machine words and grow to arbitrary precision as needed, so the same code works for a small loop counter and for a 2048-bit cryptographic key. The cryptography library is built on these integers.

A bignum is stored as a 32-bit header (bit 31 is the sign, bits 30:0 are the limb count) followed by N 32-bit limbs (`LL_int32` words). Arithmetic runs in native 32-bit words, the register width of the ESP32-S3 and the other 32-bit MCUs LambLisp runs on, so a 2048-bit modular exponentiation behaves the same on an ESP32-S3 as on a 64-bit Linux host.

LambLisp does not pack small integers into spare pointer bits (by low-bit tagging or NaN-boxing), as some Schemes do. Each integer is its own typed cell holding its own value, and pointers keep their full width. An integer takes one of three types by size: `T_INT32`, `T_INT64`, or `T_BIGNUM`. Arithmetic promotes a value to the next wider type as it grows and narrows it again when it shrinks. An exact integer is never silently converted to an inexact float.

The ordinary operators (`+`, `-`, `*`, `quotient`, and so on) already promote through these three types, so most code never calls a `bignum-*` procedure directly. The procedures below exist for when you want to build or inspect a bignum explicitly.

Bignum primitive	Description
`bignum-add` / `-sub` / `-mul` / `-div` / `-mod` / `-gcd`	Core arithmetic on integers far larger than a machine word.
`bignum-shift` / `bignum-cmp`	Bit shift (×/÷ powers of two) and three-way compare.
`to-bignum`	Coerce any exact integer to a `T_BIGNUM` cell.
`bignum-to-string` / `bignum-from-string`	Decimal/radix text ⇄ bignum (`number->string` / `string->number` handle bignums too).
`bignum?`	`#t` if its argument is a `T_BIGNUM` cell.

7.5 Cryptography

LambLisp includes a small cryptography library for embedded key exchange and message authentication. It is built on the arbitrary-precision integers described above; the same bignum code runs a 2048-bit modular exponentiation on an ESP32-S3 and on a Linux host.

The low-level building blocks are hashing (`sha256` over a bytevector), a cryptographic random source (`ll-random-bytes` and `ll-random-bignum`, which read the platform CSPRNG: `esp_fill_random` on ESP32, `/dev/urandom` on Linux), and modular exponentiation (`modular-exponent`). Diffie-Hellman and RSA are both built from `modular-exponent`.

The bignum, hashing, RNG, and `modular-exponent` operations are C++, compiled in when `LL_BIGNUM` is enabled. The higher-level protocols are written in Scheme. `rsa.scm` provides RSA (encrypt, decrypt, sign, verify), HMAC-SHA256, HKDF-SHA256, bytevector XOR, and bignum/bytevector conversion. `dh_groups.scm` supplies the RFC 3526 2048-bit MODP group (`dh-group-14-prime`, `dh-group-14-generator`). Both files load at startup (from `setup.scm`) on targets whose filesystem image includes them: `linux_x86_64`, `linux_aarch64`, `esp32-s3-devkitc-1`, `esp32s3-n8r2`, and the Freenove 4WD car kit.

Area	Provided by	Highlights
Hashing & randomness	C++ primitives	`sha256` (bytevector → 32-byte digest), `ll-random-bytes`, `ll-random-bignum`
Modular arithmetic & Diffie-Hellman	C++ + `dh_groups.scm`	`modular-exponent`, `dh-group-14-prime`, `dh-group-14-generator`
RSA, HMAC & HKDF	`rsa.scm`	`rsa-encrypt/decrypt/sign/verify`, `hmac-sha256`, `hkdf-sha256`, `bytevector-xor`

A Diffie-Hellman exchange over the built-in 2048-bit group. Both sides compute the same shared secret without transmitting a private key:

```
(define p dh-group-14-prime)
(define g dh-group-14-generator)
(define a (diffie-hellman-keygen p g))   ; Alice: (public . private)
(define b (diffie-hellman-keygen p g))   ; Bob:   (public . private)
;; each side combines its own private key with the peer's public key
(define s-alice (diffie-hellman-shared (car b) (cdr a) p))
(define s-bob   (diffie-hellman-shared (car a) (cdr b) p))
(equal? s-alice s-bob)                    ; => #t  (shared secret agrees)
```

See the source comments in `ll_vm_mop3_bignum.cpp`, `rsa.scm`, and `dh_groups.scm` for the exact argument order and semantics of each procedure.

7.6 Bytecode Virtual Machine

Feature	Description
`procedure->bytecode`	Compiles a `lambda` or `nlambda` to a bytecode object (`T_BYTECODE`); returns the original procedure if the form is not compilable.
`bytecode?`	Returns `#t` if its argument is a bytecode-compiled procedure.
`bytecode-disasm`	Returns a list describing the bytecode instruction sequence of a compiled procedure; intended for debugging.
`compile-environment!`	Walks an environment dictionary and compiles every `lambda`/`nlambda` binding to bytecode in place.

7.7 Native Code Generator (NCG)

Feature	Description
`ncg-compile`	JIT-compiles a bytecode procedure to native machine code for the current platform (x86-64, AArch64, or Xtensa LX7).
`ncg-compiled?`	Returns `#t` if a bytecode procedure has been JIT-compiled to native code.
`ncg-disasm`	Logs the native code address and size of an NCG-compiled procedure.

Feature	Description
`ncg-compile-environment!`	Walks an environment dictionary and NCG-compiles every bytecode binding in place; returns the count compiled.

Native code does not live on the GC heap. It lives in a small, fixed pool of internal RAM that is allocated once at boot and **never reclaimed** — `ncg_pool_used` only ever increases, and a procedure's native code is abandoned rather than freed when its cell is swept. On an ESP32-S3 that pool is 32 KB, and a native procedure averages about 1,659 bytes, so a board gets roughly twenty natively-compiled procedures for the life of the boot.

That is why `ncg-compile-environment!` is the wrong tool on a microcontroller: it spends the whole budget in whatever order the environment hash-walk happens to reach, which on a measured run bought `WS2812.setAll`, `Buzzer.click` and `Sonar.start` — none of them on a hot path. Compile the procedures that matter, by name, with `ncg-compile-transitive!`.

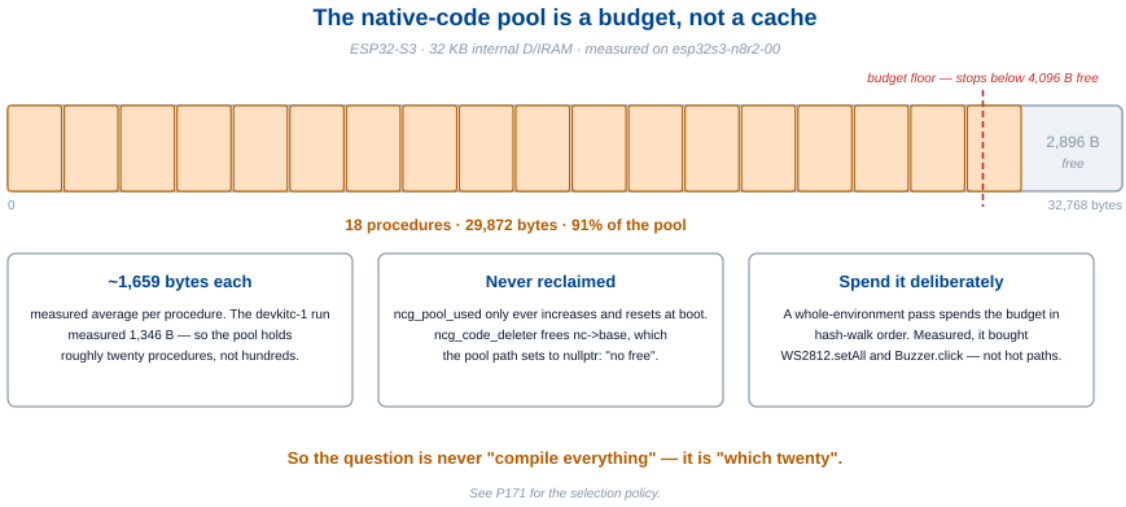


Figure 7.1 The native-code pool is a fixed budget

7.8 Diagnostics and Development Tools

Feature	Description
`lamb-debug`	Sets or queries the LambLisp diagnostic logging level.
`lamb-debug?`	Returns `#t` if diagnostic logging is currently enabled.
`lamb-breakpoint`	Triggers a debugger breakpoint or diagnostic halt; behaviour is platform-dependent.
`lamb-expand`	Expands a form using the current macro environment and prints the result; development aid.
`syslog`	Writes a message to the platform system log (serial console on ESP32, `syslog` on Linux).

Feature	Description
`lamb-oblist`	Returns the interned symbol table (oblist) as a dictionary; useful for inspecting all known symbols.

7.9 Arduino / Embedded I/O

Feature	Description
`digitalRead` / `digitalWrite`	Read or write a digital GPIO pin.
`pinMode`	Configure a GPIO pin as input, output, or input with pull-up.
`analogRead` / `analogWrite`	Read a 12-bit ADC value or write a PWM duty cycle.
`analogReadResolution` / `analogWriteResolution`	Configure ADC/PWM bit resolution.
`delay_ms` / `delay_us`	Busy-wait for a number of milliseconds or microseconds.
`millis` / `micros`	Return elapsed time since boot in milliseconds or microseconds.
`pulseIn` / `pulseInLong`	Measure the duration of a pulse on a GPIO pin.
`shiftIn` / `shiftOut`	Bit-bang serial transfer on arbitrary GPIO pins.
`tone` / `noTone`	Generate or stop a square-wave audio tone on a GPIO pin.
`attachInterrupt` / `detachInterrupt`	Register or remove an ISR callback for a GPIO pin edge.
`digitalPinToInterrupt`	Convert a GPIO pin number to the corresponding interrupt number.
`interrupts` / `noInterrupts`	Enable or disable all hardware interrupts.
`bit` / `bitRead` / `bitSet` / `bitClear` / `bitWrite`	Bit-manipulation helpers operating on integer words.
`highByte` / `lowByte`	Extract the high or low byte of a 16-bit integer.
`HIGH` / `LOW` / `INPUT` / `INPUT_PULLUP` / `OUTPUT`	Arduino GPIO level and pin-mode constants.

7.9.1 Where these run, and what actually drives a pin

The table above is **one API with two implementations**, chosen at run time. On an ESP32 it is Espressif's Arduino core. Everywhere else – a Jetson, a Raspberry Pi, a developer laptop – it is LambLisp's own implementation, and *that* one picks its backend by probing the machine:

backend	reported as	where	what a pin write does
ESP32 Arduino core	–	ESP32 targets	drives the pin

backend	reported as	where	what a pin write does
`libgpiod`	`libgpiod`	Linux SBC with a usable `gpiochip` **and** a pin map for the board	**drives a real header pin**
null (simulated)	`null`	everywhere else, including a laptop	stores the level in memory and reads it back

```
(llarduino-backend)   ==>  libgpiod        ; on a Jetson Orin Nano
(llarduino-backend)   ==>  null            ; on a developer workstation
```

Always ask, rather than assume. The same `linux_aarch64` binary runs on a Jetson with a 40-pin header and on a machine whose `gpiochip` lines are not header pins at all, so the backend is a property of the *machine*, not of the build. The selection happens once, on first use, and falls back to null if **either** a usable chip or a pin map for this board is missing – because a pin number with no map does not identify a line, and driving one anyway would be arbitrary. On hardware, arbitrary means a motor.

The null backend announces itself as `null` for exactly this reason: a simulated backend is honest only while nobody can mistake it for hardware.

7.9.2 Real pins on a Jetson or Raspberry Pi (libgpiod)

Included in the `linux_aarch64` package. `(digitalWrite 18 1)` from Scheme drives a physical header pin.

Pin numbers are BCM numbers – the Raspberry Pi convention – not header positions and not Tegra line names. The Orin Nano's 40-pin header is physically Pi-compatible, so the mapping exists and is applied for you: BCM 18 is header pin 12, which the Tegra SoC calls `PH.07`. Passing a header position (12) instead of a BCM number (18) is legal input and will drive **a different pin**.

```
(pinMode 18 OUTPUT)
(digitalWrite 18 HIGH)          ; BCM 18 == header pin 12 == PH.07 on an Orin Nano
(display (digitalRead 17))
```

What is real on this backend, and what is not. Only *digital* I/O reaches the hardware:

call	on libgpiod
`pinMode` `digitalWrite` `digitalRead`	**real** – drives and reads the line
`analogRead`	returns `-1`; the header has no ADC
`analogWrite` `tone` `noTone`	accepted and do nothing – there is no PWM peripheral behind them
`millis` `micros` `delay_ms` `delay_us`	real (shared timing, not backend-specific)

That is not a gap to be worked around in Scheme. A Jetson's 40-pin header genuinely has no ADC and no hardware PWM in this path, so a servo or an analogue sensor needs an external device – the PCA9685 PWM controller and an I2C ADC are the usual answers, and both are supported.

Requirements on the machine. `libgpiod` **v1** (v1.6.3 is what Jetson L4T and Raspberry Pi OS carry; v2 is source-incompatible and is not supported by this backend), and permission to open the `gpiochip` device – membership of the `gpio` group, or root. Without permission the probe fails and you silently get the **null** backend, so if a pin will not move, check `(llarduino-backend)` first: `null` there means a permissions or mapping problem, not a wiring one.

Boards with a pin map. The Jetson Orin Nano map is generated from NVIDIA's own `jetson-gpio` data rather than transcribed by hand. A board with no map falls back to null; adding one is a table entry, not new logic.

7.10 I2C (Wire)

Feature	Description
`Wire.begin`	Initialize the I2C bus, optionally as a peripheral at a given address.
`Wire.beginTransmission` / `Wire.endTransmission`	Start or end an I2C write transaction to a given device address.
`Wire.requestFrom`	Request bytes from an I2C peripheral.
`Wire.write`	Queue bytes for transmission in the current I2C transaction.
`Wire.read` / `Wire.available`	Read a byte or check how many bytes are available from a peripheral.
`Wire.setClock` / `Wire.setPins` / `Wire.setTimeout`	Configure bus speed, GPIO pin assignment, or transaction timeout.

7.11 WiFi

Feature	Description
`WiFi.begin`	Connect to a WiFi network with an SSID and password.
`WiFi.disconnect`	Disconnect from the current WiFi network.
`WiFi.status`	Return the current WiFi connection status as a two-element list `(code string)`.
`WiFi.localIP` / `WiFi.gatewayIP` / `WiFi.subnetMask`	Query the assigned IP addresses.
`WiFi.macAddress`	Return the device's MAC address as a string.
`WiFi.SSID` / `WiFi.RSSI` / `WiFi.BSSID`	Query the connected network name, signal strength, or access-point MAC.
`WiFi.scanNetworks`	Perform an active scan and return the number of visible networks.
`WiFi.config` / `WiFi.setDNS`	Set a static IP configuration or DNS server address.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Feature	Description
`open-tcp-client-port` / `open-tls-client-port`	Open a TCP or TLS socket to a remote host; returns a Scheme port.
`open-tcp-server-port` / `open-tls-server-port`	Listen on a local port; returns a server port to hand to `server-accept`. The TLS form also takes a PEM certificate and private key: `(open-tls-server-port port cert-pem key-pem)`.

7.11.1 WiFi Station — Advanced

Beyond the basic connect/query verbs, the station API exposes reconnection control, blocking connect waits, and detailed network-info accessors. IP-valued accessors return dotted-decimal strings; MAC/BSSID accessors return colon-separated hex strings.

Procedure	Description
`WiFi.reconnect`	Reconnect to the last access point; returns `#t` on success.
`WiFi.waitForConnectResult`	Block until connected or timeout, then return the `wl_status_t` code. Optional timeout in ms (default `60000`).
`WiFi.isConnected`	Return `#t` if currently associated with an access point.
`WiFi.setAutoReconnect`	Enable or disable automatic reconnection. Optional boolean arg; defaults to enable (`#t`).
`WiFi.broadcastIP`	Return the station's broadcast address.
`WiFi.networkID`	Return the network base (subnet) address.
`WiFi.dnsIP`	Return a configured DNS server address. Optional index selects primary (`0`, default) or secondary (`1`).
`WiFi.subnetCIDR`	Return the subnet prefix length as an integer (CIDR notation, e.g. `24`).
`WiFi.psk`	Return the WPA/WPA2 pre-shared key as a string.
`WiFi.encryptionType`	Return the encryption type of the connected network as an integer. Optional interface index (default `0`).
`WiFi.mode`	Set the radio mode: `0`=off, `1`=STA, `2`=AP, `3`=AP+STA. Returns `#t` on success.
`WiFi.getHostname` / `WiFi.setHostname`	Query or set the station mDNS hostname. `setHostname` returns `#t` on success.
`WiFi.setSleep`	Set power-save mode. Boolean (`#t`/`#f`) toggles modem sleep, or an integer selects `0`=none, `1`=modem, `2`=light. Returns `#t` on success.
`WiFi.channel`	Return the current WiFi channel number.
`WiFi.hostByName`	Resolve a hostname to an IP-address string; returns `#f` on failure.

```
;; Connect, then block (up to 15 s) for the result instead of polling status.
(WiFi.begin "my-ssid" "my-password")
(WiFi.setAutoReconnect #t)            ; auto-reconnect on drop
```

```
(let ((code (WiFi.waitForConnectResult 15000)))
  (if (WiFi.isConnected)
      (begin
        (display (list 'ip     (WiFi.localIP)
                       'cidr   (WiFi.subnetCIDR)
                       'dns    (WiFi.dnsIP)
                       'chan   (WiFi.channel)))
        (newline)
        (WiFi.hostByName "example.com"))   ; => "93.184.216.34" or #f
      (display (list 'connect-failed code))))
```

7.11.2 WiFi Access Point (softAP)

The ESP32 can run its own access point instead of (or alongside) joining one, letting other devices connect directly to the board. This is how a companion computer — for example a Jetson Orin Nano — can reach the board over a private, self-contained network with no external router (the "Jetson control over SoftAP" arrangement).

Procedure	Description
`WiFi.softAP`	Start the soft access point. Args: `ssid`, optional `password`, `channel` (default `1`), `hidden` (`0`/`1`, default `0`), `max-connections` (default `4`). Returns `#t` on success.
`WiFi.softAPConfig`	Set the soft-AP IP configuration. Args: `ip`, `gateway`, `subnet` (dotted-decimal strings). Returns `#t` on success.
`WiFi.softAPdisconnect`	Shut down the soft-AP. Optional boolean also powers the radio off. Returns `#t` on success.
`WiFi.softAPgetStationNum`	Return the number of stations currently associated with the soft-AP.
`WiFi.softAPIP`	Return the soft-AP's own IP address.
`WiFi.softAPBroadcastIP`	Return the soft-AP broadcast address.
`WiFi.softAPSSID`	Return the soft-AP SSID as a string.
`WiFi.softAPmacAddress`	Return the soft-AP MAC address as a colon-separated hex string.
`WiFi.softAPgetHostname` / `WiFi.softAPsetHostname`	Query or set the soft-AP mDNS hostname. `set` returns `#t` on success.

```
;; Bring up a private access point for a directly-attached companion computer.
(WiFi.mode 2)                                ; AP-only mode
(WiFi.softAPConfig "192.168.5.1"             ; our AP address
                   "192.168.5.1"             ; gateway (self)
                   "255.255.255.0")          ; /24 subnet
(WiFi.softAP "lamb-net" "s3cr3t-key" 6 0 4)  ; ch 6, visible, up to 4 clients

(display (list 'ap-ip   (WiFi.softAPIP)      ; => "192.168.5.1"
               'ap-ssid (WiFi.softAPSSID)    ; => "lamb-net"
               'ap-mac  (WiFi.softAPmacAddress)))
(newline)

;; Poll how many clients (e.g. the Jetson) have joined.
(display (list 'stations (WiFi.softAPgetStationNum)))
(newline)
```

7.12 UDP Datagrams

TCP and TLS give you a stream; some protocols need a datagram. `open-udp-port` binds a UDP socket and presents it as an ordinary port, so `close-port` closes it and the collector reclaims it if you drop the reference.

It is deliberately **not** an `input-port?`, and `read-u8` on it will not work. A datagram is not a byte stream: reading one as a stream loses the message boundary, and the boundary is usually the part that matters. Use `udp-recv`, which returns one datagram at a time.

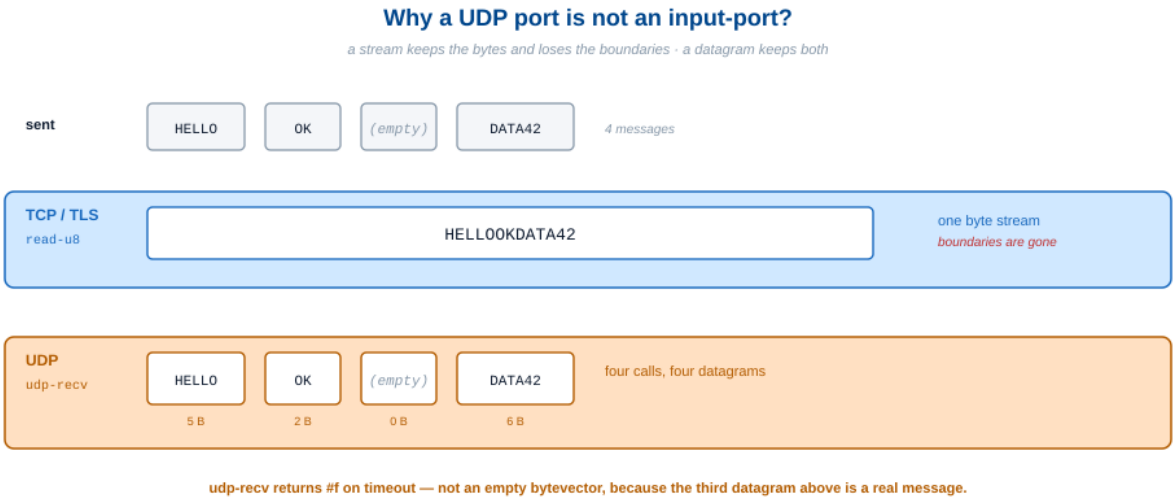


Figure 7.2 A stream keeps the bytes and loses the boundaries

The empty message in that figure is the case worth dwelling on. Sent over TCP it contributes no bytes at all and disappears without trace; received over UDP it is a real datagram of length zero. That is why `udp-recv` answers `#f` for a timeout rather than an empty bytevector — the two are different events, and a receiver that cannot tell them apart will either treat silence as a message or spin forever waiting for one it already had.

Procedure	Returns
`(open-udp-port)` / `(open-udp-port local-port)`	a port bound to `local-port`, or to an ephemeral port when omitted or `0`
`(udp-send port host port-number bytevector)`	`#t` if the datagram was handed to the stack, `#f` otherwise
`(udp-recv port max-bytes timeout-ms)`	a bytevector of the bytes actually received, or `#f` on timeout

Omit the local port unless you need a fixed one. Passing `0`, or nothing, asks the network stack for an ephemeral source port. That is what a client wants: a fixed source port is a fingerprint, and two copies of the same program on one network will collide on it.

`udp-recv` **returns** `#f` **on timeout, not an empty bytevector.** A zero-length datagram is a legitimate thing to receive, and a protocol may use one as a signal. Conflating "nothing arrived" with "something empty arrived"

is how a poll loop spins forever, so the two are distinct values. Check with `(if (udp-recv ...) ...)`, not by testing the length.

The returned bytevector is the **actual** length received, never `max-bytes` padded with zeros — so you can parse it directly without carrying the length alongside it.

```
(define sock (open-udp-port))                    ; ephemeral source port
(udp-send sock "192.168.1.50" 9000 (string->utf8 "ping"))

(let ((reply (udp-recv sock 512 2000)))          ; wait up to 2 s for up to 512 bytes
  (if reply
      (display (utf8->string reply))
      (display "no reply")))
(close-port sock)
```

`max-bytes` must be between 1 and 65535; anything outside that raises an error rather than truncating silently.

7.13 Network Time (SNTP)

An ESP32 has no battery-backed clock, so it boots believing it is 1970. That matters beyond timestamps: TLS certificate validity is checked against the clock, and a board that thinks it is 1970 will reject every certificate it is shown. `nettime` is the time source that fixes it, and it is in `scm/features/nettime.scm` — load it explicitly, it is not part of the startup chain.

Procedure	Returns
`(nettime-fetch)`	Unix seconds from the first server that answers plausibly, or `#f`
`(nettime-fetch-from address)`	Unix seconds from that one server, or `#f`
`(nettime-sync!)`	installs the fetched time as the system clock

```
(load "nettime.scm")
(let ((now (nettime-fetch)))
  (if now
      (begin (display "unix seconds: ") (display now) (newline))
      (display "no time server answered\n")))
```

This is SNTP (RFC 4330), not NTP, and the difference is not pedantry. SNTP is the single-shot subset: one request, one reply, use the answer. It does not discipline the clock, keep a frequency estimate, filter across polls, or reject a false ticker by voting. For getting a board's clock accurate enough that certificate checking works, that is exactly right. If the time *itself* is your measurement — timestamping samples you will later correlate against another instrument — it is not, and you want a real NTP client or a hardware clock.

`nettime-fetch` logs **which** server answered. That is deliberate: a timestamp does not record where it came from, and "which clock did this board believe" is the first question asked when a certificate check disagrees with a log line.

7.14 Over-the-Air Updates

LambLisp can hot-reload Scheme code without a reboot; new code takes effect immediately in the running control loop. The over-the-air update procedures cover the other case: replacing the VM binary itself, which does require a reboot.

The headline for anyone deploying a fleet: a running VM can configure the network, arm an install and reboot into the loader, so a field update needs no cable and no visit. The rest of this section is what that sentence costs and what it guarantees.

7.14.1 The loader is a separate, trusted application

The device carries a small, separate `factory` application — the *llloader* — alongside the LambLisp VM. A running VM cannot rewrite its own flash image, so an update is a handshake across a reboot: the VM records a request in the NVS namespace `ota-p125` and reboots into `factory`, which does the fetch-and-flash and then hands control to the newly installed VM.

The loader is deliberately small and deliberately unlike the VM. It is a plain ESP-IDF application with no Arduino core and no Scheme in it at all, because it is the component that has to work when everything above it has failed. **It is not in the normal boot path.** The boot selector points straight at the VM partition, and the loader runs only as a fallback: on a first-ever boot with that partition empty, after the platform's native rollback flipped the selector back to `factory`, or when the VM deliberately asked for it. Its single job is to get a *valid* VM image into the VM partition and hand control to it.

Why one spare slot rather than two. The usual A/B scheme keeps two full-size application slots so an update can be written to the inactive one, which costs 2N of flash for an N-byte application and is painful on a 4 MB part. Here the VM is updated **in place** — N bytes — and the loader is a fixed overhead `k` that does not grow with the application. The total is `N + k`, not `2N`, and the difference is flash a product can spend on its filesystem instead. Rolling back is not "boot the other slot" but "fetch a known-good image again", which is what the recovery request below does.

7.14.2 Partition layout

An OTA-capable target uses a three-tier table:

Tier	Partition	Holds
1	`factory`	the llloader — the trusted recovery root
2	`ota_0`	the LambLisp VM, the image that normally runs
3	`storage`	the LittleFS filesystem your `.scm` files live on

plus the system `nvs` and `otadata` slots. `otadata` is what selects `factory` versus `ota_0` at boot, so a single-slot OTA layout still requires it.

A `*_noota` partition table has no `factory` slot, and a build using one therefore has no OTA at all — the application occupies the whole application region and there is nowhere for a recovery root to live. The procedures below still exist in such a build and still compile, but `ota-reboot-to-loader` finds no factory partition and returns `#f`. If you intend to update in the field, confirm the target's table before anything else.

7.14.3 The Scheme API

Each procedure returns `#t` on success and `#f` on failure, except where noted below; on the Linux host build they are safe no-ops that return `#f` (or `#t` for `ota-boot-ok!`, which is vacuously true off-device), so application code that calls them builds and runs unchanged on a workstation.

Procedure	Description
`ota-request-install!`	Request an INSTALL on the next boot: the loader fetches and writes a new VM image from the configured URL. Returns `#t` if the request was recorded.
`ota-request-recover!`	Request a RECOVER on the next boot: the loader re-fetches a known-good image (recovery from a bad install).
`ota-request-console!`	Request a CONSOLE fallback on the next boot: the loader drops to an operator BLE/UART console instead of updating.
`ota-set-network! ssid pass url`	Store the Wi-Fi SSID, password, and image URL the loader will use for the network fetch. All three arguments are strings.
`ota-reboot-to-loader`	Point the OTA boot selector at the `factory` loader and restart immediately. **Does not return** on the device.
`ota-boot-ok!`	Confirm the currently running image is healthy: cancel the native rollback and clear the pending request plus the loader's install-retry counter. Always returns `#t`.
`ota-record-death! reason`	Persist `reason` and bump the unit's death count; **returns the new count**, or `#f` if the reason was refused or this is a host build. `reason` is a symbol or string of 1–24 characters from `a-z`, `0-9`, `-` and `_`.
`ota-clear-death!`	Erase the death record and reset the count to zero. Returns `#t`.
`ota-death-record`	The persisted record as `(reason . count)`, or `#f` if this unit has not declared itself dead.

`ota-set-network!` writes three of the loader's configuration keys. The remaining two — the fetch authorization token and the golden-image URL — are not settable from Scheme and are provisioned out of band; see the key table below.

Recording why a unit went down

The death record is separate from the update machinery and answers a different question. `pending_update` says *where should this unit boot next*; the death count says *has this unit stopped working*. A unit that safes its outputs and declares itself unfit calls `ota-record-death!` with a short reason, gets the running count back, and decides from that count whether to retry a normal boot or hand itself to the loader's operator console — early deaths are plausibly transient and are worth retrying; a unit that keeps dying should stop taking itself in and out of service.

A reason that does not match the character rule is **refused, not substituted**. A default reason would be the runtime's opinion about a failure it knows nothing about, recorded as though the application had said it.

`ota-boot-ok!` deliberately does **not** clear the death count. It runs early in a boot and means *this image starts*; starting is not health. Clearing the streak there would reset it before the failure that increments it, so a unit dying every cycle would flap forever and never reach any threshold. Clear it with `ota-clear-death!` when the application has evidence it is **working** — a completed work cycle, a passed self-test.

7.14.4 The loader wire contract (NVS)

The VM and the `factory` loader are separate applications that never run at the same time, so they communicate through a small record in NVS (non-volatile storage). The procedures above are wrappers over this record; it is documented here so you can inspect or drive an update from either side.

- **Namespace:** `ota-p125`
- **Keys:**

Key	Written by	Meaning
`pending_update`	`ota-request-install!` / `-recover!` / `-console!`	`0` = none, `1` = install, `2` = recover, `3` = console. The loader reads it on the next boot to decide what to do.
`wifi_ssid` / `wifi_pass`	`ota-set-network!`	Credentials the loader brings Wi-Fi up with for the network fetch.
`img_url`	`ota-set-network!`	The URL the loader fetches a new VM image from.
`img_url_gold`	provisioning, out of band	The URL of the pinned *known-good* image. A recovery request prefers this one and falls back to `img_url` if it is unset.
`img_token`	provisioning, out of band	Sent as the HTTP `Authorization` header on the image fetch, so an image server can refuse unknown units.
`install_tries`	the loader	Its install-retry counter. Cleared by `ota-boot-ok!`, and by the loader itself once it sees a valid image in `ota_0`.
`death_reason` / `death_count`	`ota-record-death!`	The last declared reason and how many deaths in a row; cleared by `ota-clear-death!`.

Nothing in the loader clears `pending_update` — that is the VM's job, and it matters. A request survives until a booted VM calls `ota-boot-ok!`, so a unit that asks for the operator console and then fails to boot keeps arriving at the console across reboots instead of falling back into the same failing image. The one exception is the board-button exit described below.

`ota-boot-ok!` clears `pending_update` and `install_tries` together. Clearing that pair is what tells the loader the new image booted successfully.

7.14.5 Where the loader gets an image: the source ladder

When the loader needs an image it tries four sources in order, cheapest and most local first, and takes the first one that can supply an image *now*:

Rung	Source	Needs
1	**aux-storage** — a golden or pending image already on the `storage` partition	the image to have been staged there beforehand
2	**network** — WiFi station plus an HTTP fetch from the configured image server	`wifi_ssid` and an image URL in NVS, and reachable infrastructure
3	**console** — an operator pushes an image over UART	a person with a cable
3b	**console over BLE** — the wireless, no-USB variant of rung 3	a person with a phone or laptop in range

A recovery request biases rung 1 toward the pinned known-good image rather than any pending one, and makes rung 2 prefer `img_url_gold`.

Rung 2 is the only unattended recovery path, and that is the sentence to design around. Rung 1 needs an image someone already put on the device. Rungs 3 and 3b both need a human — with a cable or within Bluetooth range. A fleet that recovers itself without anyone travelling to it has exactly one way to do so.

On the network rung the loader fetches `<url>.sha256` as a sidecar before the image itself and compares the digest against what it actually wrote. A mismatch refuses the image. The sidecar is best-effort: if it is absent the image is still written, logged as unverified, and still subject to the platform's own image-header validation — so publish the sidecar, because its absence degrades silently into a weaker check rather than into a failure you would notice. `img_token`, when set, is sent as the `Authorization` header on both requests.

Two build-time levers shrink the loader, and each of them removes a recovery path rather than a convenience:

Flag	Removes	Measured saving
`-DLOADER_NO_WIFI=1`	rung 2 and the whole WiFi stack	322,779 B
`-DLOADER_NO_BLE=1`	rung 3b and the whole Bluetooth stack	161,164 B

WiFi is the larger saving and the larger loss: without it there is no unattended recovery at all. BLE is the smaller saving, but it is the only rung that serves a deployed unit with no staged image, no network **and** no cable — one an operator cannot physically reach with a wire. Both are product decisions about what your recovery contract promises, not build-tuning knobs.

7.14.6 The status lamp

The loader can drive a board's onboard RGB LED as a status display. This is opt-in: define `LOADER_LED_GPIO` with the board's RGB pin and the code compiles in; leave it undefined and nothing changes. It is worth enabling on anything that will be recovered at a bench, because when the loader is running the unit has already failed, and the only other channel is a UART that one process at a time can hold — and attaching a terminal to a USB-serial bridge resets the board, destroying the state you were trying to look at.

Colour	State
dim white	up, deciding what to do
green	about to hand off to a good application
blue	writing an image — **do not unplug**
amber	escalated to recovery
cyan, pulsed	waiting for an operator to push an image
red	out of attempts, or the application declared itself dead

Pass the pin your board actually uses. A wrong pin gives a dark LED, which reads as "the status display does not work" rather than "the pin is wrong"; and because the LED latches its last colour indefinitely, a single lit colour proves nothing — write several in sequence and watch for the change.

7.14.7 The boot-confirm / rollback model

If an update rebooted into a broken VM, there would be no way to recover it remotely. To prevent that, the update flow requires the new image to confirm itself after it boots:

- After the loader installs a new image and reboots into it, the image starts in a *pending-verify* state. If it never confirms itself, the platform's native rollback points the boot selector back at `factory` on the next boot — which hands the unit to the loader, where the source ladder runs again. With a single VM slot there is no "other copy" to boot; recovery means **fetching a known-good image again**, which is what `img_url_gold` is for.
- The new VM confirms itself by calling `ota-boot-ok!` early in a good boot. That call cancels the rollback and clears both `pending_update` and the loader's `install_tries` retry counter.
- Call `ota-boot-ok!` only after you are satisfied the new image is actually working, for example once your control loop has come up cleanly, rather than at the very start of the program. Calling it too early would defeat the rollback.

There are two independent guards here and they catch different failures. The platform's native rollback catches an image that *boots but never confirms itself*. `install_tries` catches an image that *fails to install, repeatedly*. Without the second one a bad image would be re-fetched and re-written forever — install, fail, fall back to factory, install again — and the unit would be indistinguishable from a brick while working very hard. The loader increments the counter on entering an install; after three attempts it stops re-installing that image and escalates to a recovery fetch, which prefers the pinned golden image, and then to the operator console. A boot that finds a valid image in `ota_0` clears the counter, and so does `ota-boot-ok!`.

The operator console is the terminal state, and it is not a one-way door. A unit that exhausts every rung arrives at the console and stays there across reboots, because nothing in the loader clears `pending_update` and the VM is not running to clear it. That is correct — it is the trusted root holding a unit it cannot vouch for — and "staying" means it keeps re-offering the image ladder in bounded passes, so anyone who later attaches a UART or a BLE link can push an image, which is installed through the same writer and the same digest check as a normal install.

For a person with no tools at all, the loader samples the board button: hold it for three seconds and the pending request is cleared and the unit restarts for one normal boot. The hold deliberately does **not** clear the death count — that would grant an amnesty nobody asked for, and after a hold a unit that declares itself dead again must latch again immediately. The button also arms only after it has been *seen released*, so a board whose button pin is strapped or shorted low cannot read as held-from-boot and reboot the unit forever.

7.14.8 Example

A field update, driven entirely from the running VM. Every step below can be issued over an LLIP connection, which is what makes a cable unnecessary:

```
;; 1. Tell the loader where to fetch the new image.  Publish <url>.sha256 beside it.
(ota-set-network! "my-ssid" "my-passphrase"
                  "https://example.com/firmware/LambLisp.bin")

;; 2. Mark the next boot as an INSTALL.
(ota-request-install!)      ; => #t once the request is recorded

;; 3. Reboot into the factory loader.  DOES NOT RETURN.
(ota-reboot-to-loader)
```

Make sure anything that must be safe is safe before step 3: the call does not return, so whatever state your actuators are in when you make it is the state they are in while the unit reboots and reflashes.

The freshly installed VM then confirms itself, once it has evidence it is working:

```
;; ... startup and self-checks have completed successfully ...
(ota-boot-ok!)   ;; cancel rollback, clear the pending request + retry counter
```

If the new VM crashes or hangs before reaching `ota-boot-ok!`, the next boot lands in the loader instead of in the broken image, and the source ladder runs again — so a failed install costs a recovery cycle rather than the device.

Requesting recovery explicitly, when the running VM can tell that it is the problem:

```
(ota-request-recover!)      ;; next boot: re-fetch the pinned known-good image
(ota-reboot-to-loader)
```

And asking for the operator console, when the unit should stop trying and wait for a person:

```
(ota-request-console!)
(ota-reboot-to-loader)
```

A VM that comes back up after a declared death can say why:

```
(let ((rec (ota-death-record)))
  (when rec
    (display "last death: ") (display (car rec))
    (display " x") (display (cdr rec)) (newline)))
```

7.15 Runtime and GC Tuning

The adaptive real-time garbage collector tunes itself continuously from a Yuasa analysis, but LambLisp also exposes its timing, system, and GC/memory knobs directly to Scheme. These procedures let an application measure loop time, query free memory, and — where the automatic policy is not what you want — read and override the GC's tuning parameters. This section names the actual procedures behind the memory-tuning prose above.

Most procedures live in the `Platform.` namespace. A few of the most common carry bare aliases as well (`micros`, `millis`, `delay-ms`, `gc!`). By convention a name ending in `!` is a **setter** (it takes an argument and returns an unspecified value); the same name without `!` is the matching **getter** (it takes no argument and returns the current value).

7.15.1 Timing and system

Procedure	Description
`Platform.micros` / `micros`	Microseconds since boot, as an integer.
`Platform.millis` / `millis`	Milliseconds since boot, as an integer.
`delay-ms n`	Busy-wait for `n` milliseconds using the platform delay.
`Platform.loop-elapsed-us`	Microseconds elapsed since the start of the current loop iteration.
`Platform.loop-elapsed-ms`	Milliseconds elapsed since the start of the current loop iteration.
`Platform.reboot`	Reboot the device (no-op on POSIX).
`lamblisp-exit`	Flush and exit the process (POSIX) or reboot (embedded).
`Platform.repl-quiet! flag`	Enable (`flag` non-`#f`) or disable line-mode quiet console: suppress async log output and per-char prompt redisplay. Used by the LLIP tunnel.
`Platform.random-integer`	A cryptographically random integer.
`Platform.random-real`	A cryptographically random float in `[0, 1)`.
`Platform.random-bytevector`	`(Platform.random-bytevector n)` → a bytevector of `n` cryptographically random bytes.
`Platform.free-heap`	Bytes of free heap memory on the platform.
`Platform.free-stack`	Bytes of free stack space on the platform.
`recursion-limit` / `recursion-limit bytes`	Get / set the budget, in bytes of C stack, that non-tail Scheme recursion may consume before a catchable error is raised. Both forms return the **previous** value — see Bounding native recursion depth below.
`Platform.verbosity` / `Platform.verbosity! n`	Get / set the logging verbosity level (0 = silent, higher = more logging).
`Platform.eval-progress`	An opaque token that advances as the evaluator works. Compare two samples to tell a *slow* computation from a *stopped* one — see below.
`Platform.progress-watch-start ms [poll-ms]`	Begin **observing** whether the evaluator stalls: prints one line if it has not progressed for `ms`. Samples every `poll-ms` (default 100, minimum 10) — this is the resolution of the margin below, not just a cost. Never resets or interrupts anything. `#t` if a sampler started. Embedded targets only.
`Platform.progress-watch-stats`	`(max-quiet-ms stalls poll-ms)` from the running sampler, or `#f` if none is running. The first number is how close your workload came to looking stopped — see below for the two conditions under which it means that.

Telling a slow computation from a stopped one

A program that must meet a deadline has no way, from the outside, to distinguish a long computation from a stuck one: both look like a device that has gone quiet. A timeout cannot separate them either, because the timeout would have to be longer than the longest legitimate computation — by which point a genuine hang has already cost you that much time.

From *inside* the interpreter the two are easy to separate: a long computation keeps allocating, and a stopped one does not. `Platform.eval-progress` exposes that:

```
(define (progress-delta thunk)
  (let ((a (Platform.eval-progress)))
    (thunk)
    (- (Platform.eval-progress) a)))

(progress-delta (lambda () (do-some-work)))   ;; tens of thousands
(progress-delta (lambda () (delay_ms 400)))   ;; a handful
```

Three things to know before building on it:

- **It is a token, not a count.** Compare two samples; do not attach meaning to the value. It wraps, which is harmless for comparison and meaningless for anything else.
- **Reading it allocates**, so two consecutive reads differ by a few counts with nothing in between. Measure an idle baseline first and compare the *size* of a delta against it, never its presence.
- **Sample it from somewhere other than the work being watched** — a timer, a supervisory task, a monitoring loop. Code that may be stuck cannot report that it is stuck.

`Platform.progress-watch-start` does this for you on an embedded target: a low-priority sampler reads the token every 100 ms and prints one line — once per stall, not once per sample — naming how long the evaluator was quiet. **It only reports.** Leave it running through a long soak and read what the worst *legitimate* quiet period on your workload actually is. That measurement is what makes any automatic recovery safe to configure later; a threshold chosen without it restarts healthy work, which is a worse failure than the hang it was meant to catch.

Reading the margin. A report that nothing crossed your threshold does not tell you how much room you had — it is equally true of a workload that peaked just under it and one that never came close. `Platform.progress-watch-stats` gives you the number itself:

```
(Platform.progress-watch-start 1000)       ;; observe; reports if 1 s passes with no progress
;; ... run your application for as long as you can ...
(Platform.progress-watch-stats)            ;; => (312 0 100)
```

Read all three values or none of them:

- `max-quiet-ms` (312 here) is the longest the evaluator went without allocating.
- `stalls` must be `0` for that first number to be a margin. The field records the largest gap observed, whatever caused it — so once a stall has been reported, `max-quiet-ms` is describing that stall rather than your workload.
- `poll-ms` is there because the sampler only sees a gap at poll boundaries, so a gap that ends just after a sample is recorded short. **The worst true gap is up to** `max-quiet-ms + poll-ms`, and that sum — not the raw number — is what a threshold should be chosen against. Using the raw figure is optimistic in the one direction that matters: it would set the threshold too low and interrupt healthy work.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

`#f` rather than `(0 0 0)` when no sampler is running, deliberately: a watch that never started and a watch that saw nothing produce the same numbers and mean opposite things.

If `max-quiet-ms` comes back equal to `poll-ms`, you have measured the floor, not your workload. The sampler only sees a gap at poll boundaries, so a workload that allocates at least once per poll reports exactly one poll interval every time — the same number a workload with 1 ms gaps would report. It is still a useful result (your workload is nowhere near the threshold), but to measure the peak itself, poll faster:

```
(Platform.progress-watch-start 1000 10)     ;; 10 ms resolution instead of 100
```

7.15.2 Bounding native recursion depth

All three execution tiers recurse on the **native C stack** when Scheme code recurses without a tail call. An unbounded non-tail recursion would therefore drive the C stack into the operating system's guard page, which is an uncatchable fault rather than a Scheme error.

LambLisp guards against that. It records the address of its outermost evaluation frame and, once a recursive call has grown more than a set budget of bytes past it, raises an ordinary catchable `"recursion too deep"` error:

```
(define (count-up n) (+ 1 (count-up (+ n 1))))   ;;; deliberately not a tail call

(guard (e (#t 'too-deep))
  (count-up 0))                                  ;;=> too-deep
```

`recursion-limit` reads and sets that budget:

```
(recursion-limit)          ;;=> the budget in bytes -- 40960 on an ESP32 build
(recursion-limit 20000)    ;;=> 40960: sets the budget to 20000 and returns the PREVIOUS value
```

Both forms return the value that was in force **before** the call, so a procedure can lower the limit around a suspect computation and put the old value back afterwards.

Two things to know before changing it:

- **The budget must stay below the real stack.** It ships a comfortable margin under the stack the platform actually gives the evaluator, so that the guard fires while there is still stack left. Raising it past the real stack size removes the guard's entire value: the recursion then overflows for real, and you get a fault instead of an error you can catch.
- **A very low value is clamped.** A budget smaller than the REPL's own frames would trip on the very expression that would put it back, leaving a session in which everything you type fails, so small values are raised to a floor rather than honoured.

How deep a recursion fits is the budget divided by the stack one evaluation level consumes, and that depends on the tier the procedure runs in: a natively compiled procedure uses a larger frame per level than the interpreter does, so the same procedure reaches the limit sooner once it has been compiled. Size a recursion for the deepest tier it may run in — or write it as a tail call, which consumes no stack at all.

7.15.3 Sizing the cell pool

By default LambLisp sizes its cell pool from **all the memory available to it**, less a reserve held back for heap objects — strings, bytevectors, bignum digits and compiled bytecode bodies, which are allocated from the same pool as cells and would otherwise be squeezed out by a heap that grew to fill it. The reserve is an eighth of the pool memory present at startup, with a floor of 128 KB.

The initial block is always a whole number of *expansion blocks*, so a heap rebuilt as several blocks holds the same number of cells as one block of the same total. The expansion block size is therefore the granularity of every allocation the collector makes, initial or later, and it defaults to 8192 cells.

Nothing needs to be configured for this. Two keys in `Settings-Lamb.scm` override it when you want a different arrangement — a fixed footprint, memory left free for another subsystem, or a coarser expansion granularity:

Key	Effect
`cell_block_size`	Cells in the initial block. **Omit it, or set it to 0, to size the pool from available memory** (the default). A positive value is used as given, and the automatic sizing is not applied.
`extension_block_size`	Cells per expansion block, and the granularity the initial block is rounded down to. Defaults to 8192; a value below 1024 is refused and 8192 is used instead.

Both require a reboot to take effect, because the pool is built before Scheme runs. The pool can still grow after startup — see `Platform.expand-to-n-blocks!` and `Platform.max-cell-blocks!` in the table below — and an explicit `cell_block_size` does not prevent that; it only decides where the heap starts.

A worked example, the same firmware on two targets:

```
8 MB PSRAM part : 8,384,176 B available, 1,048,022 B reserved -> 74 x 8192 = 606,208 cells
desktop host    : 134,217,728 B available, 16,777,216 B reserved -> 597 x 8192 = 4,890,624 cells
```

7.15.4 GC and memory tuning

Procedure	Description
`gc!`	Run one complete GC cycle (mark + sweep); useful to reset the GC phase before a benchmark.
`Platform.gc-idle-task!  us`	Run the incremental GC idle task for up to `us` microseconds.
`Platform.cell-block-size`	Number of cells in the first (fixed) memory block — the **effective** size the heap was built with, which is not always what was asked for. By default nothing asks: the first block is sized from available memory (see Sizing the cell pool below). If a block that size cannot be allocated, the VM halves its request until one can, so this procedure is the only reliable statement of what the running VM has.

Procedure	Description
`Platform.n-cell-blocks`	Current number of allocated memory blocks.
`Platform.nfree`	Current number of free cells in the GC heap.
`Platform.max-cell-blocks` / `Platform.max-cell-blocks! n`	Get / set the maximum number of GC blocks to allocate before the memory-cap warning fires.
`Platform.extension-block-size` / `Platform.extension-block-size! n`	Get / set the number of cells in each future extension (non-first) block.
`Platform.expand-to-n-blocks! n`	Grow the heap to at least `n` blocks now, allocating extension blocks as needed. Returns `#t` when `n` blocks are present, `#f` when the heap declined to grow (block cap reached, or the memory reserved for strings, vectors and bignums would be infringed); the reason is logged.
`Platform.gc-budget-ns` / `Platform.gc-budget-ns! ns`	Get / set the GC time budget per loop iteration, in nanoseconds.
`Platform.gc-budget-achieved-ns`	The budget the collector **actually gets** per quantum — one quantum's real cost. Compare it with `Platform.gc-budget-ns`, which is what was *requested*. They agree to within one cell's cost in normal operation; they diverge when the request is smaller than about twice the measured per-cell cost, because a quantum cannot be shorter than the cells it must visit. A request that cannot be honoured is not an error and is not reported anywhere else, so read this back after setting a tight budget.
`Platform.gcload-target-pct` / `Platform.gcload-target-pct! pct`	Get / set the target GC load as a percentage (0–100) of loop time; when exceeded, the heap grows toward the max.

7.15.5 Example

Reading the current GC-load target and giving the collector a larger per-loop time budget:

```
(Platform.gcload-target-pct)      ;; => e.g. 30   (grow memory once GC exceeds 30% of loop time)
(Platform.gc-budget-ns)           ;; => current per-loop GC budget in ns

(Platform.gc-budget-ns! 500000)   ;; allow up to 0.5 ms of GC per loop iteration
(Platform.gcload-target-pct! 20)  ;; keep GC under ~20% of loop time; expand memory sooner

(Platform.free-heap)              ;; check headroom after tuning
(Platform.n-cell-blocks)          ;; how many blocks are currently allocated
```

To force a full collection before timing a benchmark (so no partial GC cycle skews the result):

```
(gc!)                             ;; complete mark + sweep now
(let ((t0 (micros)))
  (run-benchmark)
  (- (micros) t0))                ;; elapsed microseconds
```

7.16 LLIP — LambLisp Interaction Protocol

LLIP is remote procedure calls for Lisp, and it is transport-agnostic. A procedure that lives on another node is called with the syntax it would have locally; arguments and results travel as S-expressions, so nothing

is marshalled into a foreign type system on the way out and nothing has to be parsed back out of one on the way home. `llip-make-remote` makes that literal — it returns an ordinary local procedure value that happens to execute somewhere else:

```
(define board     (llip-connect-tcp "10.0.0.11" 5555 "secret-token"))
(define get-sonar (llip-make-remote board 'get-sonar))

(get-sonar)          ; an ordinary call - the measurement is taken on the board
```

Nothing in that fragment names a transport, and that is the other half of the design. LLIP owns exactly one thing: the codec, one S-expression per line. Everything underneath is a `(cons send! recv)` pair — `llip-conn-transport` builds one from a TCP connection, `llip-serial.scm` supplies one over a UART, and a test harness supplies one that goes nowhere. Changing which pair is passed changes nothing in the code above it, which is why one protocol carries a supervisor's control loop over Ethernet, a firmware push down a USB cable, and a camera frame over a duty-cycled WiFi link.

On top of the call primitive the server offers remote filesystem management — upload, download, directory operations — so the two things a remote node is ever asked for, *run this* and *hold this file*, arrive over one connection, one wire format and one session lifecycle.

7.16.1 Wire Format

Every message — request or response — is a single S-expression in `write` format followed by a newline. The server reads one line per message using LambLisp's `read` procedure, so strings, symbols, numbers, lists, and booleans all round-trip correctly without escaping or framing overhead. There is no length prefix, no binary encoding, and no session multiplexing — one request at a time per connection.

7.16.2 Connection Lifecycle

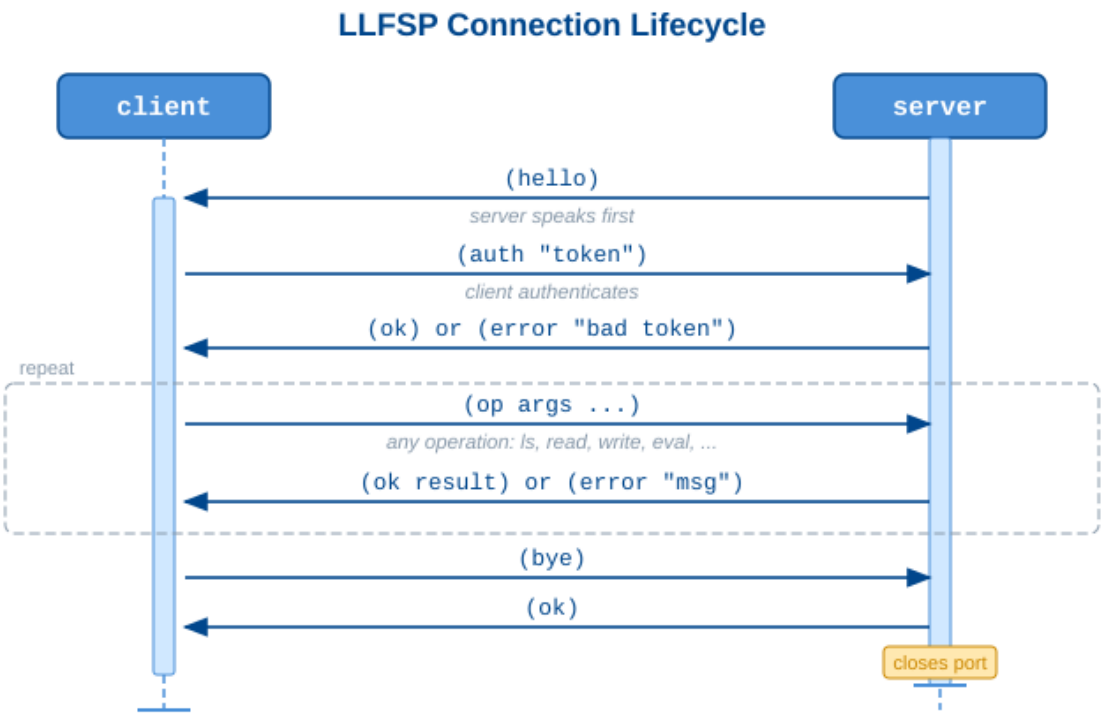


Figure 7.3 LLIP Connection Lifecycle

A session has three phases, and the first is fixed:

```
server -> client   (hello "llip/1.0")
client -> server   (auth "secret-token")
server -> client   (ok)                     ; accepted -- or the server closes, saying nothing

client -> server   <request>
server -> client   (ok <value> ...)   or   (error "message")
...
client -> server   (bye)
server -> client   (ok)                     ; then the server closes the connection
```

A rejected token gets no reply at all. The server does not answer `(error "bad token")`; it closes. That is deliberate: it is the difference between a scanner learning *this port speaks LLIP and my guess was wrong* and learning nothing it can act on. The cost is that a genuine misconfiguration looks identical to silence, so a client that dies at the handshake should be read as *the token is wrong* before anything else is suspected. On the client side that case is not silent — `llip-connect-tcp` raises `llip-connect: auth failed`, which `guard` catches like any other Scheme error.

7.16.3 Operations

Request	Response	Description
`(ping)`	`(ok)`	Liveness check.
`(ls path)`	`(ok ("f1" ...))`	List directory; returns list of filename strings.
`(stat path)`	`(ok size)`	File size in bytes.
`(read path)`	`(ok "content")`	Full file content as string.
`(read path offset n)`	`(ok "chunk")`	Partial read; `n` bytes from `offset`.
`(write path "content")`	`(ok)`	Overwrite or create file.
`(append path "content")`	`(ok)`	Append to existing file.
`(delete path)`	`(ok)`	Delete file.
`(rename from to)`	`(ok)`	Rename file; both paths must be absolute.
`(load path)`	`(ok result)`	Load and eval file; return last expression value.
`(eval expr)`	`(ok result)`	Eval arbitrary expression in server's interaction environment.
`(bye)`	`(ok)`	Close connection.

Errors are returned as `(error "message")` and raised as Scheme errors on the client side, so `guard` can catch them.

Every path is checked before any I/O touches the filesystem. A path must be a non-empty string that **begins with** `/`, contains no `..` and contains no NUL byte; anything else is answered `(error "bad path")` and nothing is opened. The leading slash is the rule that catches people out: `(read "setup.scm")` is refused and `(read "/setup.scm")` is not. There is no working directory on the wire and no relative resolution — a request names an absolute path or it is rejected.

`(ls)` with no argument lists `/`. `(stat)`, `(read)` and `(delete)` on a path that does not exist answer `(error "not found")` rather than an empty result, so a missing file can never be mistaken for an empty one.

`(read path)` and `(write path "content")` carry the whole file as a single Scheme string, which bounds the size they can move in one request; `(read path offset n)` — which is what `llip-upload` and `llip-download` use — is the form for anything larger than a small configuration file. The string form is also text-only: a LambLisp string cannot hold a NUL byte, so binary content does not survive it. See Carrying binary payloads for the frame that does.

7.16.4 The eval Operation

`(eval expr)` evaluates any Scheme expression in the server's interaction environment and returns the result as a Scheme value. This means the client can query live state, invoke procedures, reconfigure parameters, or upload and execute new behavior — all without restarting the board:

```
(llip-eval c '(Motor.speed))                      ; query live value
(llip-eval c '(Motor.motion 1000 0.5 0.5 0.5 0.5)) ; enqueue a motion command
(llip-eval c '(set! loop-target-ms 5))             ; reconfigure a parameter live
(llip-eval c '(load "/new-behavior.scm"))          ; deploy new code without reboot
(llip-eval c '(dict-keys (current-environment)))   ; introspect all bound names
```

7.16.5 Hot-Reload Without Reboot

The three operations above compose into the deployment story that motivates the protocol: edit a file on the development host, `(write)` it to the device, `(load)` it, and the new definitions replace the old ones in the interaction environment. Nothing reboots, nothing is reflashed, and the control loop never stops running.

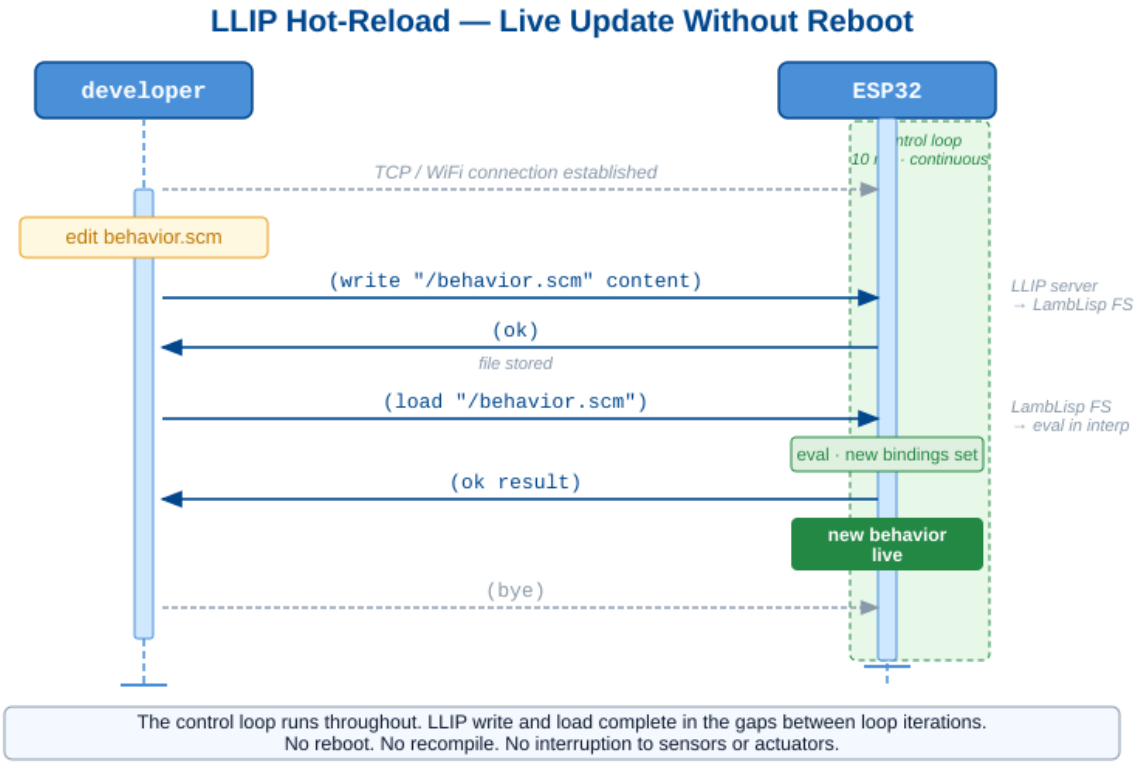


Figure 7.4 Hot-Reload: Live Update Without Reboot

The green band is the point of the figure. The control loop is servicing sensors and actuators for the whole exchange; the `write` and `load` round-trips complete in the slack between loop iterations, so a running machine changes behavior without a control gap. `load` returns the value of the last expression in the file, so the reply carries a result the client can check rather than a bare acknowledgement.

Paths must be absolute — the server rejects any path that does not begin with `/`, or that contains `..`, before it touches the filesystem.

7.16.6 Client API (`llip-client.scm`)

A connection is opened with one of three constructors and released with `llip-disconnect`, which sends `(bye)`, drains the acknowledgement and closes the port:

Constructor	Transport	Authentication
`(llip-connect-tcp host port token)`	plain TCP	shared token, **sent in clear** — a trusted LAN or a development bench, never an open network
`(llip-connect host port token ↩ [ca-cert-pem])`	TLS	shared token inside TLS. The server certificate is verified only if `ca-cert-pem` is supplied; omit it and the channel is encrypted but the peer is unauthenticated
`(llip-connect-auth host port)`	plain TCP	mutual public-key authentication, no shared secret on the wire — see Security

Every operation takes the connection as its first argument and raises a Scheme error when the server replies `(error ...)`, so a remote failure is caught with `guard` exactly like a local one:

Procedure	Returns
`(llip-ping conn)`	`#t`
`(llip-ls conn path)`	a list of filename strings
`(llip-stat conn path)`	the file size in bytes
`(llip-read conn path)`	the whole file as one string
`(llip-read-chunk conn path offset n)`	up to `n` bytes starting at byte offset `offset`
`(llip-write conn path content)` · `(llip-append conn path content)`	`#t`
`(llip-delete conn path)` · `(llip-rename conn from to)`	`#t`
`(llip-load conn path)`	the value of the last expression in the file
`(llip-eval conn expr)`	the value of `expr`, as a Scheme value
`(llip-make-remote conn 'name)`	a local procedure that calls `name` on the remote
`(llip-upload conn local remote)` · `(llip-download conn remote local)`	copy a file in `llip-chunk-size` (512-byte) pieces
`(llip-disconnect conn)`	ends the session

Two calling modes are provided.

Synchronous — blocks until the result arrives. Simple, suitable for scripting and one-shot operations:

```
(define c (llip-connect-tcp "192.168.1.42" 8080 "token"))
(llip-eval c '(+ 1 2))                          ; => 3
(define remote-fib (llip-make-remote c 'fib))
(remote-fib 30)                                  ; => 832040
(llip-disconnect c)
```

`llip-make-remote` is the piece worth dwelling on. It captures the connection and the name and returns a procedure *value*, so a remote capability can be stored in a variable, passed to a higher-order procedure, or bound under the name the local code already calls. Moving a computation to another board is then a change to one definition rather than a change to every call site — and because the argument list is forwarded untouched, the remote procedure takes whatever arguments the remote definition takes, with no signature declared anywhere in between.

Asynchronous — sends without blocking and polls for the result from the main loop. Suitable for managing multiple boards or remote GPUs concurrently:

```
(define fire-inference
  (llip-make-remote-async gpu-conn 'run-model
    (lambda (result) (apply-actuation board-conn result))))

(fire-inference sensor-vec)      ; returns immediately
;; in supervisor loop:
(llip-poll-async gpu-conn)       ; delivers result to callback when ready
```

Chunked file transfer helpers (`llip-upload`, `llip-download`) copy files between local and remote filesystems in fixed 512-byte chunks, keeping peak heap usage bounded on memory-constrained boards.

7.16.7 Server (`llip-server.scm`)

The server runs on the embedded board as a non-blocking subsystem polled from the main loop:

```
;; Plain token over TCP -- development and trusted LANs.  No encryption.
(define srv (llip-make-tcp-server 8080 "secret-token"))

;; TLS, with the board presenting a certificate.
(define srv (llip-make-server 8443 cert-pem key-pem "secret-token"))

;; Mutual public-key authentication -- no shared secret on the wire at all.
(define srv (llip-make-auth-server 8080 "/llip-privkey.scm" "/llip-trusted.scm"))

;; ... then, once per main-loop iteration:
(llip-server-poll srv)
```

`llip-server-poll` is non-blocking and bounded: it accepts at most one pending connection per call and returns immediately when none is waiting. An accepted session then runs to completion inside that call, so the size of the files you serve is charged against the loop budget you have — that is the knob that decides whether LLIP service is invisible to your control loop or shows up in its jitter. Every exception raised inside a session is caught, so a malformed request, a rejected path or a client that disappears mid-sentence cannot take the main loop down with it.

The mode is fixed when the server is created, and is never negotiated on the wire. Which constructor was called decides which handshake the server will accept; a client speaking the other one fails at the first message. This is the LLIP design decision most worth stating to a security reviewer. A protocol in which a peer can *ask* for the weaker mode can be pushed into it by an attacker who merely drops packets, at which point the strong mode is worth nothing. There is no downgrade path here because there is nothing to downgrade: the authenticated handler never sends the `(hello "llip/1.0")` banner at all, and the token handler reads an authenticated client's first message as a malformed auth and closes.

Keys are loaded once, when `llip-make-auth-server` is called, rather than per connection. A missing key file is therefore a startup error naming the file, instead of a connection refused mysteriously in the field months later — and several hundred digits of RSA parsing never land in the accept path. `llip-load-keys` reads a `(private-key ...)` form and a trusted-peer file holding either one `(public-key ...)` form or a `(trusted-keys ...)` list of them, and returns how many peers it trusts.

7.16.8 Transports: TCP/TLS vs Serial

LLIP is transport-agnostic: the same S-expression-per-line codec runs over TCP, TLS, or a raw serial line. The difference is *who supplies reliability*.

Over **TCP/TLS** the transport already guarantees ordered, error-checked delivery, so the LLIP layer is just the line codec — write one S-expression, flush, read one line back. Frame helpers `llip-frame-write!` and `llip-frame-read` (in `llip-transport.scm`) implement exactly this: `read-line` + `parse-from-string` rather than `read` directly off the socket, because the reader's `peek` can mistake a split TCP segment for EOF mid-datum.

Over a **raw serial line** there is no transport reliability at all — a UART can drop, flip, or reorder bytes silently. LLIP therefore layers its own **stop-and-wait ARQ** on top of the serial REPL (`llip-serial.scm`): every data frame carries a Fletcher-16 checksum and a sequence number, and the sender retransmits on NAK or timeout. This is the reliability LLIP gets for free over TCP but must provide itself over serial.

Property	TCP / TLS	Serial
Reliability	native (TCP checksums + ARQ)	app-layer stop-and-wait ARQ (Fletcher-16 + seq)
Framing	one S-expr per line	one S-expr per line (same codec)
Integrity	TCP checksum	per-frame Fletcher-16 (`ck`)
Session setup	connect / accept	reset → boot → quiet mode
Best for	networked boards, supervisor mesh	a board on a USB cable, no network

7.16.9 Serial Transport and ARQ (`llip-serial.scm`)

The serial receiver is not a blocking loop. The frame verbs (`op` / `da` / `cl`) are plain top-level procedures that the ordinary REPL — itself non-blocking, assembling a line across main-loop ticks — evaluates one line at a time. Nothing blocks: the main loop keeps ticking sonar, LEDs, buzzer, and motors between frames.

Reliability is **stop-and-wait ARQ**. Each chunk frame is acknowledged before the next is sent; a bad checksum or wrong sequence number is answered with a NAK so the host retransmits.

Verb	Reply	Description
`(llt)`	—	Enter quiet mode: silence per-char echo and the async log flood so frames are not garbled.
`(op "path")`	`ACK`	Begin a transfer into `path`. The bytes go to `path.part`; **the destination itself is not touched.** Sequence and counters reset.
`(da SEQ CRC "chunk")`	`ACK SEQ` / `NAK exp`	Write one chunk if the Fletcher-16 CRC checks and `SEQ` is the expected next one; otherwise `NAK <expected>`. A duplicate (lost ACK) is re-ACKed, so a lost reply costs one retransmission and not a resync.

Verb	Reply	Description
`(cl)`	`DONE N FC`	Close `path.part` and **rename it over** `path`; report the total character count `N` and the aggregate checksum `FC` for an end-to-end check.
`(en)`	—	Leave quiet mode, back to interactive.
any other S-expr	its own output	Evaluated normally; in quiet mode only its own `(display ...)` streams — this is how a host runs a command.

The checksum procedure `ck` computes Fletcher-16 over a string's codepoints and must match the host tool's `fletcher16` byte-for-byte:

```
(define (ck s)
  (let ((a 0) (b 0) (L (string-length s)))
    (let lp ((i 0))
      (if (< i L)
          (begin
            (set! a (modulo (+ a (char->integer (string-ref s i))) 255))
            (set! b (modulo (+ b a) 255))
            (lp (+ i 1)))
          (+ (* b 256) a)))))
```

Quiet mode is what makes literal sentinels safe on a serial line: with no per-char echo and no `Input: <form>` log, a host sends one frame line, reads one ACK line, and moves on without its own markers being echoed back into the stream.

`FC` is the sum of the per-chunk checksums, modulo 65521 — a second, independent end-to-end check on top of the per-chunk one, cheap enough to maintain incrementally on a microcontroller.

What the write-to-`.part`-and-rename rule buys you is the failure mode, not the success path. A transfer that dies partway leaves the old file intact and a `.part` file beside it; the destination is replaced only by a transfer that reached `(cl)`, which means every chunk passed its CRC and arrived in sequence. The cost is one file's worth of spare space: on a full filesystem the push now fails when `path.part` cannot be opened, before anything is destroyed, rather than half-overwriting the target. One residual is worth stating rather than implying — a transfer that *completes* with mismatched `N`/`FC` totals still replaces the destination; the host learns of the mismatch from `DONE N FC` and must push again.

The pull direction

The same board can send a file back, which is how you answer *what is actually on this unit?* without reflashing it to find out. The verbs mirror the push, with one addition:

Verb	Reply	Description
`(sf "path")`	`SF` / `SF-ERR <reason>`	Open `path` for reading; reset sequence and counters.
`(sd)`	`SD <seq> <ck> <hex>` / `SD-EOF <n> <fc>`	Emit the **next** chunk: up to 128 bytes, hex-encoded, with its Fletcher-16 and sequence number. `SD-EOF` reports the totals.
`(sr)`	`SD <seq> <ck> <hex>` / `SR-ERR none`	**Re-emit the last chunk without advancing** — the retry.

Verb	Reply	Description
`(sc)`	`SC-OK`	Close the read side. Safe to call when nothing is open.

`sr` exists because the retry is not symmetric with the push. On the way up the host holds the data, so a `NAK` simply asks it to send that chunk again. On the way down the device holds it and `sd` has already moved past it — without `sr` a single corrupted chunk forces the whole file to start over, and on a 14 KB file at 128 bytes a chunk that is 110 separate opportunities to loop forever on a noisy line. One chunk of buffer removes the whole failure class.

Chunks come back **hex-encoded**, which doubles the bytes on the wire and is the right trade for pulling a settings file or a log. It buys two things a quoted string cannot: nothing needs escaping, so there is no second parser for either end to get wrong, and a NUL byte is just `00` — a LambLisp string cannot hold one at all. One path therefore pulls text and binary alike.

7.16.10 Carrying Binary Payloads

The line codec is text, and that is a real limit rather than a stylistic one: a LambLisp string cannot hold a NUL byte — writing one does not set a byte, it *shortens the object* — so a firmware image, a bytevector or a JPEG cannot ride a quoted string. LLIP solves this the same way in both directions, by keeping the **header** an ordinary S-expression and putting the **payload** on the wire raw, immediately after it.

Over a serial line — the `db` frame. `(db SEQ LEN CRC)` is a legal S-expression, so the same non-blocking REPL that handles `op`/`da`/`cl` handles it with no mode switch at the parser. Evaluating it only *arms* the receiver; it does not read the payload. The bytes are then drained by `(db-tick!)`, called once per main-loop iteration, which takes whatever has arrived and returns — so sensors, actuators and the collector keep being serviced between chunks of a firmware push. While a frame is armed the line assembler stands aside, which is what lets a `\n` inside the payload be a data byte counted against `LEN` rather than a line terminator. When `LEN` bytes have arrived the chunk's Fletcher-16 is checked and the receiver answers `ACK <seq>` or `NAK <expected>`, and the arm is cleared on **both** arms — a receiver left armed is a console that looks alive and answers nothing.

Note that `db` checksums **bytes** while `da` checksums a string's **codepoints**. The two are not interchangeable, and a host tool must use the byte version for `db` and the codepoint version for `da`.

Over TCP — the vision frame. `llip-vision.scm` carries a camera frame as one header line followed by exactly `LEN` raw bytes with no trailing newline:

```
(vision-frame SEQ LEN FMT)\n<LEN bytes>

;; Sender: grab a JPEG and put it on an already-open LLIP connection.
(llip-vision-capture-send! conn seq)        ; -> bytes written, or #f if there is no camera

;; Receiver: read one frame -> (SEQ LEN FMT BYTEVECTOR), or #f on EOF or a truncated frame.
(llip-vision-read conn)
```

This is **not** base64, deliberately. LambLisp ports are already binary — `binary-port?` is `#t` for every port, and `read-bytevector`, `write-bytevector`, `read-u8`, `write-u8` and `u8-ready?` are all available — so a frame rides the same connection the control traffic uses, unencoded. Measured on a 240x240 sensor, a frame is 4,954 bytes; base64 would have spent 6,605 to carry the same picture. The header stays readable by `read`, so an operator watching the link still sees ordinary LLIP traffic for everything that is not the payload itself.

The receive side reads `LEN` bytes in a loop rather than in one call, and distinguishes three answers that are easy to conflate: end-of-file means the peer closed mid-frame and the frame is truncated; zero bytes means *not yet*, which is not an error; a positive count is progress and resets the stall budget. A truncated frame is reported as `#f` and never silently delivered short. A caller inside a tick budget can pass an absolute deadline, and must then treat an `#f` as leaving the connection **desynchronised** — the header and part of the body have been consumed, so the link can no longer be framed and must be reopened.

7.16.11 Device-Initiated Serving (`llip-transport.scm`)

LLIP is symmetric — *either* end may initiate an exchange. For the robot stack the roles are reversed from the filesystem case: the Jetson orchestrator is the TCP **server**, and the 4WD car is the TCP **client** (it duty-cycles its WiFi for power, so the client drives connect/disconnect). `llip-transport.scm` provides both sides:

Procedure	Description
`(llip-frame-write! port frame)`	Write one frame as a flat S-expression + newline, then flush.
`(llip-frame-read port)`	Read one line and parse it to a frame; `#f` on EOF or parse failure.
`(llip-accept server-port)`	Poll the non-blocking `server-accept` (20 ms) until a client connects.
`(llip-conn-transport conn)`	Wrap a connection as the `(cons send! recv)` pair the orchestrator expects.
`(llip-serve-once server-port goal . opts)`	Accept one 4WD session, run the orchestrator loop over it, close (the close tells the 4WD to drop WiFi); returns `(done N)` or `(halt R)`.
`(llip-4wd-serve host port run-fn)`	Client side: connect, then loop receiving move-programs, running `run-fn`, and sending telemetry until the orchestrator closes the link.

The transport is a plain `(cons send! recv)` pair, so a stub transport swaps out for real TCP with no change to the orchestrator:

```
;;; Jetson side: serve one car session toward a goal.
(define server (open-tcp-server-port 5555))
(llip-serve-once server "drive to the dock" 4)      ; ttl = 4 s

;;; 4WD side: run received move-programs and report telemetry.
(llip-4wd-serve "10.0.0.1" 5555 move-run-frame!)
```

7.16.12 Command-Line Client: `tools/llip`

`tools/llip` is the host-side LLIP client and listener — a single-file Python 3 tool (only `--serial` needs `pyserial`). Because LLIP is symmetric, it can both drive a device and serve a device that initiates.

Verb	Description
`send SRC [DEST]`	Push a text file into the device over the `op/da/cl` frames (chunked, ARQ-verified on serial).
`sendbin SRC [DEST]`	Push a **binary** file over the `db` frame — raw bytes, no escaping, so a firmware image or a JPEG survives intact.
`recv REMOTE [LOCAL]`	Pull a file off the device over the `sf/sd/sr/sc` frames, hex-encoded and checksummed per chunk.
`eval "FORM"`	Evaluate a form and stream its output up to a done marker.

Verb	Description
`repl`	Interactive line REPL relayed over the link (Ctrl-D to exit).
`listen DIR`	Serve a device that *initiates* — this host becomes the frame receiver, mirroring the device's `op/da/cl` handler, so a device can push files or drive eval whenever it connects.

Transport selection. `--serial PORT [--baud N]` uses a serial line (default 115200) and supplies its own Fletcher-16 stop-and-wait ARQ; `--tcp HOST:PORT` uses TCP, whose native reliability means no app-layer ARQ and no reset/boot/quiet dance. With neither flag, a single `/dev/ttyUSB*` or `/dev/ttyACM*` is auto-detected. On serial the tool performs a clean reset + boot wait, then enters quiet mode (`,quiet`) so frames are not garbled, and restores the interactive console (`,verbose`) on close.

```
# Push a behavior file to the only attached board (serial auto-detected), then run it.
tools/llip send new-behavior.scm /new-behavior.scm
tools/llip eval '(load "/new-behavior.scm")'

# Same over TCP to a running LLIP TCP server on the board.
tools/llip --tcp 10.0.0.11:5555 eval '(Motor.speed)'

# Serve a device that initiates a file push into ./inbound (":PORT" binds all interfaces).
tools/llip --tcp :6000 listen ./inbound
```

`tools/llip` implements the host end of all of the above: the `op/da/cl` push, the `sf/sd/ sr/sc` pull, the `db` binary frame and the TCP line codec. Its `fletcher16` must agree with the device's `ck` byte for byte; if you write your own host tool, that one function is the invariant to reproduce exactly.

7.16.13 HTTP / LLM Layer (`llip-http.scm`)

For AI-driven control, LambLisp talks to a local `llama-server` (llama.cpp) in-process over `open-tcp-client-port` — no subprocess, no Python: "only LL in the stack." JSON exists *only* here, as the thin envelope llama.cpp's `/completion` API requires; everything the orchestrator passes around is LLIP S-expressions, and the LLM's "content" is itself a grammar-constrained LLIP move-program.

Procedure	Description
`(llip-http-post-json host port path body)`	POST a JSON body (HTTP/1.1 + `Connection: close`) and return the response body string, or `#f` on connect failure.
`(llip-llm-complete prompt grammar [host [port]])`	The LLM call: send a prompt + a GBNF grammar, return the grammar-constrained move-program string (the JSON `content` field), or `#f`. Host/port default to `127.0.0.1:8080`.

```
(llip-llm-complete
  "go forward for two seconds then stop"
  llip-grammar)                      ; => "((forward (s 2)) (stop))"
```

7.16.14 AI Orchestration and the Robot Stack

The robot stack is a three-tier control loop built entirely in LambLisp. An LLM proposes a move-program, the grammar makes invalid output impossible, the 4WD re-validates and executes it through a bounded behaviour queue, and a reactive safety layer can veto any move at any instant. Intent flows down; safety is guaranteed at the bottom.

Deliberative tier — the orchestrator (`llip-orchestrator.scm`). Owns all I/O and state; the LLM stays a pure text function. The loop is: goal → plan (LLM) → move-program frame → send → recv telemetry → done / replan / halt.

Procedure	Description
`llip-grammar`	The GBNF grammar for the move DSL, defined in `scm/features/llip-orchestrator.scm`. Constraining generation to this grammar makes plans guaranteed-valid.
`(llip-build-prompt instruction)`	Wrap an instruction in the qwen ChatML system/↩ user/assistant prompt.
`(llip-plan instruction [host [port]])`	Deliberative tier: call the LLM, read the result, and return a validated move list, or `#f`.
`(llip-make-move-program seq ttl moves)`	Build a `(move-program (seq ...) (ttl ...) (moves ...))` frame.
`(llip-orchestrate goal transport . opts)`	The orchestration loop over a `(cons send! recv)` transport; returns `(done SEQ)` or `(halt REASON)`. Auto-recovers obstacle aborts by re-planning; deadman / planner-failure / unknown status → halt.

Executive tier — the executor (`llip-executor.scm`). Turns a validated move list into motor action on the shared cooperative motion queue. Safety by construction: it *interprets*, never `eval`s — a fixed `case` dispatch (verb → binder) is the entire restricted environment, so no move-program can express anything but the seven verbs.

Procedure	Description
`(move-run-frame! frame)`	Execute a full move-program frame: feed the deadman, record `seq`, then run its moves.
`(move-run-program! bq moves)`	Re-validate the whole program (defence in depth — the 4WD never trusts the link) and enqueue each move on behaviour queue `bq`.
`(move-exec! bq m)`	The dispatch that *is* the sandbox: verb → binder, nothing else reachable; unknown verb is an error.
`(move-tank! l r)`	Tank drive: left/right side speeds → the four motors.
`(mv-forward ...)` `(mv-backward ...)` `(mv-turn-left ...)` `(mv-turn-right ...)` `(mv-wait ...)` `(mv-stop ...)` `(mv-dock ...)`	The seven per-verb binders; each enqueues one timed `once-for` task so a long move holds its slot without blocking the main loop.

Procedure	Description
`(move-estop!)`	Reactive e-stop: abort the motion queue and cut the motors.

Reactive tier — safety (`llip-reactive.scm`). Lowest in the stack, highest in priority. Safety lives on the 4WD, not the planner: the robot stays safe even when the plan is wrong, stale, or the link drops. `(reactive-tick!)` runs every main-loop iteration *before* the motor tick, so a reflex or the deadman can veto the executor before it advances another move.

Procedure	Description
`(reactive-tick!)`	Run the reactive vetoes; call before `Motor.loop` each iteration.
`(reactive-veto! signal reason)`	Preempt the plan: e-stop, raise a signal, and report an aborted frame up so the planner can re-plan.
`(reactive-deadman!)`	Halt if no valid frame arrived within `deadman-ms` (default 1500); latched, clears on recovery.
`(llip-feed-deadman!)`	Reset the heartbeat — called whenever an LLIP frame arrives.

The obstacle reflex is edge-triggered with hysteresis (trip below 0.15 m, re-arm above 0.25 m) so a recovery maneuver can run without being e-stopped every tick.

Signals (`llip-signals.scm`). `(robot-signal! event)` maps a robot event to the LED ring: base modes (`idle` / `running` / `dock`) persist; momentary signals (`bad-move` / `obstacle` / `low-battery`) are bounded blinks that auto-restore the base; `deadman` is the one sticky, highest-priority signal and the only one that paints the whole ring — safety wins.

Framing and validation (`llip-frame.scm`). Pure data, no I/O, so it loads and tests on any LambLisp (Linux or ESP32):

Procedure	Description
`(llip-move-program? frame)`	Is `frame` a well-formed `(move-program (seq N) (ttl T) (moves ...))`?
`(llip-program-valid? moves)`	Is `moves` a valid move list — every move legal, length ≤ 32?
`(llip-make-telemetry seq move-index status reason sensors)`	Build a telemetry ("up") frame; status $\in$ `running/done/aborted/halted`.

The move vocabulary is seven verbs (`forward backward turn-left turn-right wait stop dock`) with unit-tagged magnitudes (`s m cm deg`); `stop` and `dock` are nullary. This vocabulary is defined once, in `scm/features/llip-orchestrator.scm`, and the GBNF grammar returned by `llip-grammar` is generated from it – so the LLM, the validator, and the executor all agree on exactly what a legal move is.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.16.15 Security

Start from what an LLIP session can do. `(eval expr)` evaluates an arbitrary expression in the server's interaction environment, and `(write path content)` places arbitrary content on the filesystem. There is no restricted subset and no sandbox: whoever completes the handshake can run anything the VM can run. Authentication is therefore the entire security boundary, not one control among several, and the choice of server constructor is the choice of how strong that boundary is.

Mode	What protects the session	Appropriate for
`llip-make-tcp-server`	a shared token, sent in clear and replayable by anyone who sees it	a development bench or a trusted, isolated LAN
`llip-make-server`	TLS, with the token inside it	a network you do not fully control, where the board can carry a certificate and key
`llip-make-auth-server`	mutual public-key authentication; no shared secret ever crosses the wire	a deployment where a stolen token would be unrecoverable

The authenticated mode performs RSA-2048 mutual authentication over fresh 32-byte nonces, derives a session key by Diffie-Hellman and HKDF-SHA256, and encrypts subsequent **requests** with AES-128-GCM. Each side signs the two nonces in the order its own role requires, so a signature captured from one side cannot be reflected back at the other. A peer whose public key is not in the trusted-keys file is closed on, before any request is read.

One limitation of that mode, stated rather than left to be discovered. In the current implementation the request side is authenticated and confidential; the server's **replies are still written in plaintext** on the same connection. Treat the authenticated mode as protecting *who may issue commands* and *what those commands are* — not yet as protecting what the board sends back. Where reply confidentiality matters, run the TLS server instead.

Failures are silent by design, in one specific place. A bad token, an unknown key or a failed signature closes the connection with no reply at all, so an unauthenticated peer learns nothing from the failure mode — not even that it guessed the right protocol. The price is paid by legitimate operators: a misconfigured client sees a connection that simply ends, and the first thing to check is the credential, not the network.

Path handling is checked, not trusted. Every path is rejected unless it is absolute, free of `..` and free of NUL bytes, and the check runs before anything is opened — so a traversal attempt never reaches the filesystem layer at all. This bounds *which* files a session touches; it does not bound *whether* it can touch files, which is the token's job.

The mode cannot be negotiated down. See Server above: a server runs one handshake because of how it was started, and a client speaking the other one fails at the first message. An attacker who can drop packets cannot force the weaker path, because there is no message in which the weaker path can be requested.

7.16.16 Limits

Worth knowing before you design around the protocol:

- **One request in flight per connection.** There is no multiplexing and no request id; a response belongs to the last request sent. The asynchronous client API enforces this — a second send while one is outstanding raises rather than interleaving. Run two connections if you need two conversations.
- **An accepted session runs to completion inside one `llip-server-poll` call.** Per-iteration cost is bounded by the work the session asks for, so a large whole-file read is charged to one loop tick. Chunked transfers exist partly for this reason.
- `read` / `write` / `append` **move a whole file as one Scheme string**, which bounds the size they can carry in a single request. Use `(read path offset n)`, or `llip-upload` / `llip-download`, for anything beyond small configuration and source files.
- **The string form is text-only.** A LambLisp string cannot hold a NUL byte, so binary content must use the binary frames described in Carrying binary payloads.
- **Serial transfer is stop-and-wait**, so throughput over a UART is bounded by the round trip per chunk rather than by the baud rate. That is the cost of needing no transport reliability underneath; over TCP the same codec runs with no ARQ at all.

7.16.17 Role in the Supervisory Architecture

LLIP is the uniform communication layer across all deployment tiers — embedded boards, edge supervisor (Jetson), and cloud GPU. The same client API is used regardless of whether the remote is an ESP32 on a LAN or a cloud LambLisp instance over WAN. Latency differences are absorbed by the async API: fast LAN connections resolve in the same loop tick, slow WAN connections resolve in a later tick, and the supervisor loop handles both transparently.

7.16.18 Worked Example: Supervisory Control of the Freenove 4WD Car

The following is drawn from `data/demo-supervisor.scm`. The Linux supervisor connects to the car over plain TCP, wraps each car operation as a thin Scheme procedure, and drives the car from the supervisor's own Scheme environment without any change to the car's on-board code:

```
;;; 1. Connect to the car from the Linux supervisor.
(define car (llip-connect-tcp "192.168.1.100" 8080 "demo-car"))

;;; 2. Wrap remote calls as local procedures.
;;;    llip-eval sends a quoted expression to the car and returns its result.
(define (car-move! dir ms)  (llip-eval car `(robot-move! ',dir ,ms)))
(define (car-stop)          (llip-eval car '(robot-stop)))
(define (car-turn! deg)     (llip-eval car `(robot-turn! ,deg)))
(define (car-sweep)         (llip-eval car '(robot-sweep)))
(define (car-set-leds! pat) (llip-eval car `(set-leds! ',pat)))

;;; 3. Drive the car from the supervisor -- same syntax as on-board code.
(car-set-leds! '(red green blue red green blue red green blue red green blue))
(car-move! 'forward 1000)   ; move forward for 1 s
(car-turn! 90)              ; turn 90 degrees
(car-sweep)                 ; trigger sonar sweep; returns scan result list
```

```
(car-stop)

;;; 4. Query live state.
(llip-eval car '(get-sonar))    ; => distance in cm
(llip-eval car '(get-battery))  ; => battery %

;;; 5. Deploy new behavior without rebooting the car.
(llip-upload car "new-behavior.scm" "/new-behavior.scm")
(llip-eval car '(load "/new-behavior.scm"))

;;; 6. Disconnect when done.
(llip-disconnect car)
```

The car runs `llip-server.scm` on its main loop and never needs to know anything about the supervisor — it just handles `eval` requests as they arrive. The supervisor can inspect live telemetry, tune parameters, upload new `.scm` files, and hot-reload them, all while the car remains under continuous control.

7.16.19 Digital Twin

A *digital twin* is a LambLisp process on the edge or cloud whose Scheme environment mirrors the live state of a physical board. Telemetry is pulled from the board via LLIP; a Scheme model evolves in lockstep. The result is a live, queryable, scriptable replica of the physical device. It can answer questions the board cannot answer without interrupting its control loop:

- Replay the last N seconds of sensor history.
- Simulate a fault condition and observe how the control law responds.
- Validate that new code produces the same trajectory on the model before deploying it to hardware.
- Run a faster-than-realtime forecast of future system state.

The twin is a Scheme program that holds the device model as ordinary variables and procedures. A sync procedure runs periodically, reading live values from the board and writing them into the twin's environment:

```
;;; twin-state: last-known board state as a closure-backed dict object.
(define twin-state
  (dict->obj (list->dict '((speed . 0.0) (heading . 0.0) (battery . 100.0)
                           (sonar . ()) (loop-ms . 0)))))

;;; Pull one tick of live telemetry from the board into the twin.
(define (twin-sync! board)
  (twin-state 'speed   (llip-eval board '(Motor.speed)))
  (twin-state 'heading (llip-eval board '(get-heading)))
  (twin-state 'battery (llip-eval board '(get-battery)))
  (twin-state 'sonar   (llip-eval board '(get-sonar)))
  (twin-state 'loop-ms (llip-eval board '(loop-elapsed-ms))))

;;; Simplified kinematic model: predict (dx dy) over interval dt.
(define (twin-predict-position dt)
  (let ((v   (twin-state 'speed))
        (hdg (twin-state 'heading)))
    (list (* v dt (cos hdg))
          (* v dt (sin hdg)))))
```

Validation before deployment. Before uploading new behavior to the board, run it against the twin. The twin loads the candidate file and evaluates the same sequence of inputs the board would receive, then compares outputs against an expected trajectory:

```
(define (validate-candidate! path)
  (llip-eval twin-conn `(load ,path))
  (let loop ((steps 100) (ok #t))
    (if (= steps 0)
```

```
        ok
        (let* ((predicted (llip-eval twin-conn '(step-model)))
               (expected  (expected-trajectory steps)))
          (loop (- steps 1) (and ok (close-enough? predicted expected)))))))

(if (validate-candidate! "new-behavior.scm")
    (begin
      (llip-upload board "new-behavior.scm" "/new-behavior.scm")
      (llip-eval board '(load "/new-behavior.scm")))
    (display "Validation failed -- not deploying\n"))
```

Anomaly replay. The twin records a rolling window of telemetry snapshots. When an anomaly is detected the window is frozen and replayed offline:

```
(define telemetry-log '())
(define log-window 200)           ; keep last 200 ticks

(define (twin-sync-and-log! board)
  (twin-sync! board)
  (set! telemetry-log
        (take (cons (twin-snapshot) telemetry-log) log-window)))

;;; On anomaly: print the frozen history.
(define (analyse-anomaly)
  (for-each (lambda (snap)
              (display (format "t=~a speed=~a sonar=~a\n"
                               (snap 'loop-ms)
                               (snap 'speed)
                               (snap 'sonar))))
            (reverse telemetry-log)))
```

The twin runs entirely in Scheme on the edge machine; the on-board code requires no changes.

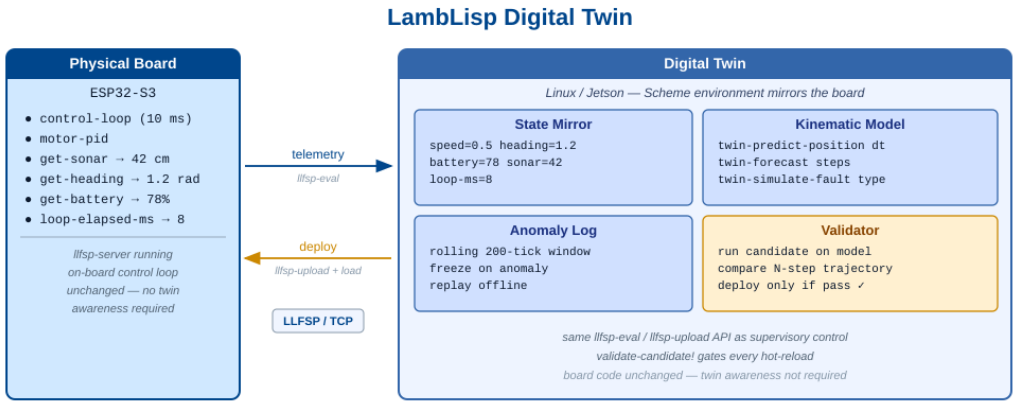


Figure 7.5 LambLisp Digital Twin

7.16.20 Unified Distributed Environment

When a supervisory controller manages hardware on multiple nodes — an ESP32 running sensors and actuators, a Jetson running a GPU inference engine, a Linux host running vision — the application logic should not have to know which function lives where. `llip-make-remote` projects a named procedure from a remote node into the local Scheme namespace. The result is an ordinary procedure: calling it sends the arguments over LLIP and returns the result. The distribution is invisible to the rest of the program.

```
;;; One-time setup: project remote bindings into the local namespace.
(define board  (llip-connect "192.168.1.42" 7700))
```

```
(define gpu    (llip-connect "192.168.1.43" 7700))

;;; Board bindings - sensor reads and motor commands execute on the ESP32.
(define get-sonar      (llip-make-remote board 'get-sonar))
(define robot-move!    (llip-make-remote board 'robot-move!))
(define Motor.speed    (llip-make-remote board 'Motor.speed))

;;; GPU bindings - inference executes on the Jetson.
(define run-inference  (llip-make-remote gpu 'run-model))
(define get-embeddings (llip-make-remote gpu 'get-embeddings))

;;; Local procedures run here on the supervisor.
(define (get-camera-frame) ...)
(define (choose-action embedding sonar) ...)
```

With the namespace unified, the perception–action loop reads as if everything were local:

```
(define (perception-action-loop)
  (let* ((frame     (get-camera-frame))           ; local
         (embedding (get-embeddings frame))        ; Jetson GPU
         (dist      (get-sonar))                   ; ESP32 sensor
         (action    (choose-action embedding dist)) ; local
         (speed     (cdr (assq 'speed action))))
    (Motor.speed speed)                           ; ESP32 actuator
    (robot-move! (assq 'direction action))        ; ESP32 actuator
    (perception-action-loop)))
```

Moving a binding to a different node requires changing exactly one `llip-make-remote` call at setup; the loop body is unchanged. Adding a new node — say a second camera on a Raspberry Pi — is adding new bindings at setup.

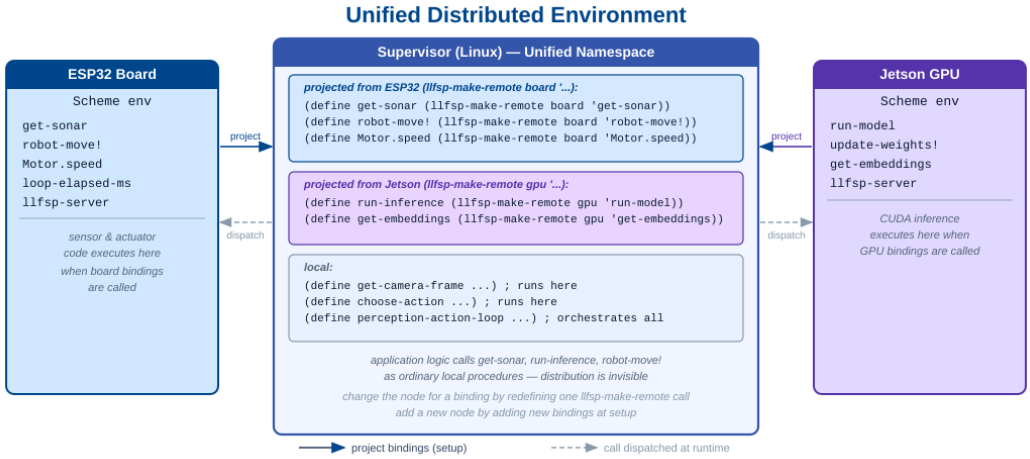


Figure 7.6 Unified Distributed Environment

7.17 LCD1602 Display

Feature	Description
`LCD1602.begin`	Initialize an LCD1602 character display over I2C.
`LCD1602.clear` / `LCD1602.home`	Clear the display or move the cursor to position (0, 0).

Feature	Description
`LCD1602.setCursor`	Move the cursor to a specific column and row.
`LCD1602.print` / `LCD1602.write`	Print a string or write a raw character code to the display.
`LCD1602.display` / `LCD1602.backlight` / `LCD1602.cursor`	Control display-on, backlight-on, and cursor-visibility.

7.18 Ultrasonic Sonar (HC-SR04)

Feature	Description
`Sonar.begin`	Initialize a sonar sensor on given trigger and echo GPIO pins.
`Sonar.start`	Trigger a new distance measurement (non-blocking; ISR-driven).
`Sonar.result`	Return the most recent echo duration in microseconds.
`Sonar.ping`	Trigger and synchronously wait for one measurement; returns echo time.
`Sonar.poke`	Schedule a sonar ping and register a callback for the result.
`Sonar.us->distance`	Convert an echo time in microseconds to a distance in centimetres.

7.19 PCA9685 PWM Controller

Feature	Description
`PCA9685.begin`	Initialize a PCA9685 16-channel PWM controller over I2C.
`PCA9685.setupSingleDevice`	Configure one PCA9685 device on the bus.
`PCA9685.setSingleDeviceToFrequency`	Set the PWM output frequency for a device.
`PCA9685.getSingleDeviceFrequency`	Read back the configured PWM frequency.
`PCA9685.setChannelDutyCycle`	Set one channel's duty cycle as a percentage.
`PCA9685.setChannelPulseWidth`	Set one channel's pulse width in microseconds.
`PCA9685.setChannelServoPulseDuration`	Set one channel's pulse width for standard servo control.
`PCA9685.setChannelOnAndOffTime`	Set raw on/off tick values for one channel.

7.20 WS2812 Addressable LED Strip (NeoPixel)

Feature	Description
`WS2812.begin`	Initialize an addressable LED strip on a given data GPIO pin.
`WS2812.setBrightness`	Set the global brightness scaling factor (0–255).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Feature	Description
`WS2812.setLedColorData`	Set the RGB color of one LED by index.
`WS2812.show`	Transmit buffered color data to the strip.
`WS2812.Wheel`	Convert a 0–255 position to an RGB colour traversing the hue wheel.

7.21 SPI Bus

The SPI procedures mirror the Arduino `SPIClass` API for full-duplex master-mode communication. The bus deliberately does **not** own the chip-select (CS) line — the caller drives CS explicitly with `digitalWrite` (from CommonIO), exactly as Adafruit's `SPIDevice` does — so a single bus can address any number of devices. A transaction (`beginTransaction ... endTransaction`) latches the clock rate and SPI mode (0–3) for the duration of one device exchange. On the Linux host build every procedure is a compile-clean no-op, so code loads and smoke-tests unchanged.

Procedure	Description
`SPI.begin`	Start the hardware SPI peripheral. Call with no arguments for the board defaults, or `(SPI.begin sck miso mosi)` to assign the clock, MISO, and MOSI GPIO pins (CS is left to the caller).
`SPI.end`	Release the SPI peripheral.
`SPI.beginTransaction`	Open a transaction: `(SPI.beginTransaction clock-hz [mode])` sets the clock rate in Hz and the SPI mode 0–3 (clock polarity/phase; default 0), MSB-first.
`SPI.endTransaction`	Close the current transaction.
`SPI.transfer`	Full-duplex single-byte exchange: `(SPI.transfer byte)` clocks one byte out on MOSI and returns the byte simultaneously read on MISO.
`SPI.transfer-bytevector!`	In-place full-duplex block transfer: `(SPI.transfer-bytevector! bv)` sends each byte of `bv` and overwrites it with the byte read back, returning the same bytevector.
`SPI.write-bytevector`	Write-only block send: `(SPI.write-bytevector bv)` clocks every byte of `bv` out on MOSI, discarding MISO, and returns the number of bytes written.

```
;; Talk to an SPI device whose chip-select is on GPIO 10.
(define cs 10)
(pinMode cs 3)                          ; 3 = OUTPUT
(digitalWrite cs 1)                      ; CS idle high

(SPI.begin 12 13 11)                     ; sck=12 miso=13 mosi=11
(SPI.beginTransaction 1000000 0)         ; 1 MHz, mode 0

(digitalWrite cs 0)                      ; select the device
(SPI.transfer #x9F)                      ; send a "read-ID" command byte
(define id (bytevector 0 0 0))
(SPI.transfer-bytevector! id)            ; read 3 ID bytes back in place
(digitalWrite cs 1)                      ; deselect

(SPI.endTransaction)
id                                       ; => the device's 3-byte ID
```

7.22 OneWire (1-Wire Bus)

The OneWire procedures implement the Maxim/Dallas 1-Wire protocol directly in software, with microsecond-precise, IRAM-resident bit slots timed under a critical section (interrupts masked) for reliable signalling. Every procedure takes the GPIO **pin** as its first argument, so a single build can drive several independent 1-Wire buses. The pin is configured as open-drain with the internal pull-up enabled, but an external 4.7 kΩ pull-up to 3V3 is still required by the bus. On the Linux host build every procedure is a compile-clean no-op.

Procedure	Description
`onewire-reset`	`(onewire-reset pin)` issues a reset pulse and returns `#t` if at least one device answered with a presence pulse, else `#f`.
`onewire-write-byte`	`(onewire-write-byte pin b)` writes one byte, LSB first.
`onewire-read-byte`	`(onewire-read-byte pin)` reads and returns one byte, LSB first.
`onewire-write-bit`	`(onewire-write-bit pin b)` writes a single bit (`0` or `1`).
`onewire-read-bit`	`(onewire-read-bit pin)` reads and returns a single bit (`0` or `1`).
`onewire-search`	`(onewire-search pin)` runs the Maxim ROM-search algorithm and returns a list of 8-byte ROM bytevectors, one per device found (family code / LSB first).

```
;; Read a DS18B20 temperature sensor on GPIO 4 (single device on the bus).
(define ow 4)

(onewire-reset ow)
(onewire-write-byte ow #xCC)              ; SKIP ROM (address the only device)
(onewire-write-byte ow #x44)             ; CONVERT T (start a conversion)
(delay-ms 750)                           ; 12-bit conversion takes ~750 ms

(onewire-reset ow)
(onewire-write-byte ow #xCC)             ; SKIP ROM
(onewire-write-byte ow #xBE)             ; READ SCRATCHPAD
(define lsb (onewire-read-byte ow))
(define msb (onewire-read-byte ow))
(define raw (+ lsb (* 256 msb)))         ; signed 1/16 °C units
(/ raw 16.0)                             ; => temperature in °C
```

7.23 Freenove 4WD Car Kit — Peripheral Summary

The Freenove 4WD Car Kit (ESP32) demo uses the following hardware. Implemented devices have full LambLisp Scheme API bindings. Devices present on the board but not yet implemented in LambLisp are noted explicitly.

Device	Implementation status
4x DC motors (via PCA9685 I2C, address 0x5F)	Implemented — `PCA9685.*` procedures
2x pan/tilt servos (PCA9685, channels 0–1)	Implemented — `PCA9685.setChannelServoPulseDuration`
12x WS2812 RGB LEDs (GPIO 32)	Implemented — `WS2812.*` procedures
HC-SR04 ultrasonic sonar (TRIG=12, ECHO=15)	Implemented — `Sonar.*` procedures
Photosensitive light sensor (ADC GPIO 33)	Implemented — `analogRead`

Device	Implementation status
Battery voltage divider (ADC GPIO 32)	Implemented — `analogRead`
Buzzer (GPIO 2, LEDC channel 0)	Implemented — `tone` / `noTone` / `ledc*` procedures
3x line-tracking IR sensors (PCF8574 I2C, address 0x20)	Implemented — `Wire.*` low-level I2C
IR receiver (GPIO 0)	*(not implemented)* — pin defined, no IR decode driver
OV2640 camera (DVP interface)	LambLisp camera driver implemented (`camera-init`, `camera-capture`, `camera->lcd`, ...) — currently targeted at the esp32-s3-eye; not yet wired to this board's camera

7.24 Camera and On-Board Display (esp32-s3-eye)

LambLisp drives the OV2640 camera and the built-in ST7789 240×240 IPS LCD on the **esp32-s3-eye** dev board (8 MB PSRAM). The binding follows a strict division of labour: **C++ owns the pixels, LambLisp owns the policy.** The frame hot-paths — grab a frame, byte-swap and blit it to the panel, compute a mean-luma scalar — stay in C++ (`ll_xmop3_EyeCam.cpp`), while Scheme (`scm/features/eye-cam.scm`) drives the demo choreography and the eye's own main loop. The C++ layer needs only the ESP-IDF components already bundled in the Arduino-ESP32 core (`esp_camera` for the OV2640 DVP sensor, `esp_lcd` for the ST7789 over SPI3) — no external Arduino libraries.

All framebuffers and blits are **RGB565**. The OV2640 emits RGB565 in **big-endian** (byte-swapped) byte order, and `esp_lcd` ships memory to the panel in that same order, so a camera frame blits straight through with no swap — but a colour built in native order (e.g. an `lcd-box` fill) is byte-swapped once on its way to the wire. These procedures ship on the esp32-s3-eye target only (they are compiled under `LL_CAMERA`); on other boards the symbols are not registered.

7.24.1 Camera / LCD primitives (C++)

Registered by the `EyeCam-install-mop3` installer. RGB565 colours are plain integers (0–65535).

Procedure	Description
`(lcd-init)` → `#t`	Bring up SPI3, the ST7789 panel, and the backlight. Idempotent; also reserves the internal-DMA band buffer used by every later draw.
`(lcd-backlight [percent])` → void	Set backlight brightness 0–100 (default 100). `0` blanks the screen without tearing down the panel; useful to isolate a backlight fault from a draw fault.
`(lcd-clear [rgb565])` → void	Clear the whole screen to one colour (default black).
`(lcd-box x y w h rgb565)` → void	Draw a filled axis-aligned rectangle in one RGB565 colour.

Procedure	Description
`(lcd-rgb565! bvec x y w h)` → void	Blit a big-endian RGB565 region from a bytevector to the panel at (x, y). Silently skips if the bytevector is shorter than `w*h*2` bytes.
`(lcd-testpat)` → `#t`	Blit a known 240×240 test pattern (colour bars, grayscale ramp, fine stripes) through the same path as `camera->lcd` — pinpoints display-path corruption with no camera involved.
`(camera-init [mode])` → `#t/#f`	Bring up the OV2640. `mode` is `'rgb565` (default, drives the LCD) or `'jpeg` (for `camera-capture`). Re-calling in the same format is a no-op that keeps the reserved DMA line buffer.
`(camera->lcd)` → `#t/#f`	Grab one RGB565 frame and blit it to the LCD — the live-preview hot path. `#f` if the camera or LCD is not up, or no frame is available.
`(camera-preview [n])` → `n`	Blit `n` frames to the LCD in a tight blocking loop (default 300), then return. Freezes on the last frame; the eye-cam app-loop is the non-blocking alternative.
`(camera-thumb [n] [x0 y0])` → `#t/#f`	Grab one RGB565 frame and print an `n×n` thumbnail over serial as hex RGB565 rows (`n` default 24, clamped 4–48). With `x0 y0`, prints a raw `n×n` crop at that origin instead of a subsample. For off-device visual verification without the LCD. Output is gated by the quiet REPL — send `,verbose` first.
`(camera-set key val)` → void	Tune the OV2640 sensor. `key` is one of `framesize`, `brightness`, `contrast`, `hmirror`, `vflip`, `special-effect`; `val` is an integer.
`(camera-capture)` → bytevector	Return one JPEG frame as a bytevector (requires `(camera-init 'jpeg)`); `#f` if no frame is available.
`(frame-brightness)` → integer	Cheap mean luma (Rec. 601 weights) over a downsampled RGB565 frame, 0–255 — a scalar for trigger-policy demos ("brighter than N → fire"). Returns `-1` if the camera is down or not in RGB565 mode.

7.24.2 eye-cam.scm live-preview layer

Portable one-liners and the eye's application loop, built on top of the C++ primitives. This file also defines the RGB565 colour constants `red`, `green`, `blue`, `black`, and `white`.

Procedure	Description
`(eye-orient!)`	Correct the sensor orientation for the lab mount (`vflip` + `hmirror`, i.e. a 180° flip). Adjust here if the physical mount changes.
`(eye-preview)`	The one-liner demo: `lcd-init`, `camera-init`, `eye-orient!`, then a blocking `camera-preview`.
`(eye-demo)`	Flash a red/green/blue/white/black colour splash on the LCD to prove the panel is alive, then hand over to the live camera preview.
`eye-live?`	Boolean flag (initially `#f`); when true the app-loop blits a fresh camera frame every tick.

Procedure	Description
`(eye-live-start)` → `'live`	Init the LCD + camera once, apply `eye-orient!`, and set `eye-live?` so the app-loop begins live blitting.
`(eye-live-stop)` → `'stopped`	Clear `eye-live?` — freezes on the current frame while the loop keeps running (no blit).
`(eye-make-loop)`	The eye's main-loop *maker*. The eye target setup installs it via `(set! app-loop eye-make-loop)`; setup.scm calls it once to build the loop that replaces the shared robot loop. The loop auto-starts the live preview on the first tick (safely after boot-time flash reads finish), blits when `eye-live?`, and donates spare time to the incremental GC.

Bring up the panel and camera and run a non-blocking live preview (the app-loop keeps the panel live while the REPL stays responsive):

```
(eye-live-start)     ; => live   -- LCD + camera up, blitting each loop tick
;; ... panel now shows the live camera feed ...
(eye-live-stop)      ; => stopped -- freeze on the current frame
```

Capture a thumbnail over serial for off-device verification (no LCD needed):

```
,verbose             ; un-gate log output so the thumbnail prints
(camera-init)        ; => #t   -- RGB565 mode
(eye-orient!)        ; correct the sensor orientation
(camera-thumb 32)    ; => #t   -- prints a 32x32 hex RGB565 thumbnail
```

7.25 Bundled Scheme Libraries

The low-level device primitives documented above (`PCA9685.*`, `WS2812.*`, `Sonar.*`, `Wire.*`, `tone`, ...) are C++ *mops* — thin, fast, and unopinionated. On top of them LambLisp ships a set of `.scm` **convenience layers** that turn those primitives into a friendly, board-aware API: named servos, a four-motor drive with an I2C write-cache, a fuel-gauge LED ring, buzzer patterns, drift-free timers, and I2C bus scanners. These files live under `scm/core/` and are selected into each target's filesystem image by its manifest (`scm/targets/<env>.manifest`), so a given board ships only the layers relevant to it.

Every layer **degrades gracefully**: if the underlying mop or the board's pin map is absent, the public procedures are rebound to a no-op (`Lambda0`) rather than erroring, so the same application code loads on a board that lacks the peripheral. The flagship Freenove 4WD Car worked example (below) is built almost entirely from these APIs.

7.25.1 Servos and Motors (`PCA9685.scm`)

Wraps the raw `PCA9685.*` PWM mops into normalized servos (input range −1..+1) and a four-wheel drive whose commands are queued as timed *behaviours* (see Behaviors) and de-duplicated against a per-channel write-cache so the I2C bus is touched only when a command actually changes.

Procedure	Description
`pca9685-info`	Object accessor for board config: `(pca9685-info 'Xservo)`, `'MotorPins`, `'servo-frequency`, etc.
`make-servo`	`(make-servo chan pwmin pwmax)` → a `(lambda (-1..1) ...)` mapping the normalized range onto a channel's pulse-width span.
`xservo` / `yservo`	Ready-made pan / tilt servos (channels 0 / 1); call with −1..+1. `servo2...servo7` also defined.
`Motor.setpins`	Low-level: drive one motor from a forward/back channel pair at a signed fraction.
`Motor.set`	Set motor `n` (0–3) to −1..+1; writes I2C only when the value changes.
`Motor.set4`	Set all four motors at once: `(Motor.set4 m0 m1 m2 m3)`.
`Motor.setAll`	Set all four motors to one speed.
`Motor.motion`	Queue a timed move: hold `(m0 m1 m2 m3)` for `duration` ms, then run the next queued motion.
`Motor.loop`	Tick the motion queue once per main loop; auto-stops the motors when the queue drains.
`Motor.estop!`	Emergency stop: flush the motion queue and cut all four motors immediately.
`Motor.last`	Vector of the last value written to each motor channel (the change-cache).

```
(xservo 0) (yservo 0)                ; center the pan/tilt head
(Motor.motion 1000  0.5 0.5 0.5 0.5)  ; forward at half speed for 1 s
(Motor.motion 1000 -0.5 -0.5 -0.5 -0.5); then reverse for 1 s
;; ... then, once per main-loop iteration:
(Motor.loop)                         ; advances the queue; stops when idle
```

7.25.2 Addressable LEDs (`WS2812.scm`)

A ring/strip helper over the `WS2812.*` mops: whole-strip and per-LED colour setters, a symbolic colour palette, nautical navigation lights, and non-blocking blink behaviours driven by a shared LED queue (`led-bq`).

Procedure	Description
`WS2812.setAll`	Set every LED to one `r g b` colour and show.
`WS2812.off`	Turn the whole strip off.
`WS2812.set`	Set one LED by index to `r g b` and show.
`WS2812.setColor`	Set one LED to a named colour: `(WS2812.setColor 0 'rgb-red)`.
`WS2812.test`	Queued startup self-test: flash all LEDs red → green → blue → off.
`WS2812.navLights`	Port red, starboard green, fore/aft white.
`rgb-colors`	Palette accessor: `(rgb-colors 'rgb-cyan)` → `(0 128 128)`. Colours: black, red, green, blue, cyan, yellow, magenta, white.
`ws2812-info`	Config accessor: `'Nleds`, `'ledchan`, `'ledtype`, colour-order type codes.
`Led.blinkOnce`	Queue one blink of LED `nled`: on `color` for `on-time` ms, off `off-time` ms.
`Led.blinkAll`	Queue one blink of the whole ring.
`Led.loop`	Tick the LED queue once per main loop.

```
(WS2812.navLights)                    ; running lights
(Led.blinkAll 'rgb-red 200 200)       ; queue a 200/200 ms red blink
;; ... once per main-loop iteration:
(Led.loop)
```

7.25.3 LED Ring Service (`Leds.scm`)

(Ships only on the Freenove 4WD Car target.) A higher-level, robot-agnostic service layered on `WS2812.scm`: it splits the ring into an illumination arc and a signal arc, runs a **priority arbiter** (transient signals preempt a persistent base mode, then restore it), and paints the ring as a **battery fuel gauge**.

Procedure	Description
`led-illuminate!`	Turn the forward white illumination arc on/off (`#f` clears it).
`led-emit!`	`(led-emit! pri task)` — play a transient signal task if its priority $\geq$ the current one, then restore the base mode.
`led-clear!`	Hard-clear the arbiter: drop any signal, reset priority, restore base mode.
`led-blink`	Build a blink-pattern task for a zone: `n`$>$`0` blinks then done, `n`=`0` blinks forever.
`battery-check`	Read the battery ADC, paint the ring green/yellow/red as a charge gauge, log volts, return raw ADC.
`battery-lit`	Compute how many LEDs to light for a raw ADC reading (the gauge math).

```
(battery-check)          ; ring shows remaining charge; logs "ADC ... N/12 LEDs ... ~ V"
(led-illuminate! #t)     ; front arc white for the camera
```

7.25.4 Buzzer (`Buzzer.scm`)

Non-blocking tone patterns over the `tone` mop, queued on a dedicated behaviour queue (`Buzzer.bq`).

Procedure	Description
`Buzzer.tone`	The underlying tone primitive (rebound to no-op if the board has no buzzer pin).
`Buzzer.on` / `Buzzer.off`	Start a continuous tone at `freq` Hz / silence it.
`Buzzer.click`	A single short 8 ms acknowledgement click (queued).
`Buzzer.onoff`	Queue tone-on for `on-time` ms, then off for `off-time` ms.
`Buzzer.offon`	Queue off then on (the inverse phase order).
`Buzzer.arf`	A rising chirp "bark" (60→4000 Hz). Built on `Buzzer.chirp`.
`Buzzer.loop`	Tick the buzzer queue once per main loop.

```
(Buzzer.click)                ; short acknowledgement beep
(Buzzer.onoff 2000 100 100)   ; queue a 2 kHz 100/100 ms beep
;; ... once per main-loop iteration:
(Buzzer.loop)
```

7.25.5 Timers (`Timers.scm`)

Closure-based, drift-free timing helpers. Each constructor returns a procedure you poll; none of them block.

Procedure	Description
`Stopwatch_ms` / `Stopwatch_us`	Return a thunk giving elapsed ms / µs since the stopwatch was created.
`AutoTimer_ms`	Countdown timer: returns remaining ms while running, `#f` when expired (auto-restarts). Starts expired.
`KitchenTimer_ms` / `KitchenTimer_us`	Return an object with `'remaining`, `'set!`, `'expired?`, `'running?` messages.
`Oneshot`	Return a thunk that yields `#t` the first call and `#f` forever after.
`uptime_ms`	Milliseconds since boot (a shared stopwatch).
`make-idler`	Rate-limiter: returns a thunk that yields `#t` when its interval has elapsed *and* the loop has spare time.

```
(define t (Stopwatch_ms))
(do-some-work)
(info "took ~a ms\n" (t))

(define beat (AutoTimer_ms 1000))
;; in the loop: (unless (beat) (heartbeat))   ; fires ~once/second
```

7.25.6 I2C GPIO Expander (`PCF8574.scm`)

Driver for the PCF8574 8-bit I2C port expander (on the 4WD car it reads the line-tracking IR sensors). `PCF8574` builds a per-address object with `'read8` / `'write8` messages.

Procedure	Description
`PCF8574`	`(PCF8574 addr)` → an object with `'read8` (returns last input byte) and `'write8` messages.
`pcf8574`	A ready-made instance at the default address `0x20`.
`PCF8574-IO`	Low-level raw `read8` / `write8` taking an explicit address (over `Wire`).

```
(define ir pcf8574)
((dict-ref ir 'read8))          ; read the 8-bit input byte (e.g. IR line sensors)
```

7.25.7 Light Sensor (`LightSensor.scm`)

Procedure	Description
`light-sensor`	Read the photosensor ADC, normalized to 0.0–1.0. No-op returning nothing if the board has no `pin-photo`.

```
(light-sensor)      ; => e.g. 0.42  (brighter = larger)
```

7.25.8 I2C Bus Utilities (`I2C.scm`)

Convenience helpers layered on the native `Wire.*` mops for bringing up and scanning the bus.

Procedure	Description
`I2C.begin`	Set the SDA/SCL pins and begin the bus as master: `(I2C.begin sda scl addr)`.
`I2C.ping`	Probe one address; returns the transmission error code (`#f`/0 = present).
`I2C.inventory`	Scan addresses 0x00–0x7F, log expected vs. unexpected devices, return the count found.

```
(I2C.begin pin-sda pin-scl 0)   ; 0 = we are bus master
(I2C.inventory)                 ; => N ; logs each device found
```

7.25.9 Sonar Convenience Wrappers (`Sonar.scm`)

The low-level `Sonar.*` mops (`Sonar.begin`, `Sonar.start`, `Sonar.result`, `Sonar.us->distance`, ...) are documented under Ultrasonic Sonar. This `.scm` layer adds board-aware setup and a **continuous, non-blocking** ranging loop that keeps the latest distance in metres.

Procedure	Description
`Sonar.pins`	The `(trig echo)` pin pair from the board map (or `(0 0)`).
`Sonar.range-m`	Rated maximum range in metres (reported when nothing is in range).
`Sonar.latest`	The most recent measured distance in metres (updated by `Sonar.loop`).
`Sonar.setup`	Initialize the sensor from `Sonar.pins` and arm the first ping.
`Sonar.loop`	Send-or-expect tick: resolve the in-flight ping, update `Sonar.latest`, launch the next. Call once per main loop. Never blocks.

```
(Sonar.setup)
;; ... once per main-loop iteration:
(Sonar.loop)
(when (< Sonar.latest 0.30) (Motor.estop!))   ; obstacle reflex
```

7.26 Behaviors: Cooperative Scheduling

`scm/core/Behaviors.scm` is the cooperative, non-blocking scheduling DSL that the device layers above are built on. It exists because a `(loop)`-driven robot must never block: every timed move, blink, or beep has to interleave with sensor reads, comms, and reflexes.

The whole protocol is a **step-thunk**: a zero-argument procedure, called once per main-loop tick, that returns `#t` when it is finished (pop it) or `#f` to keep running. A *behaviour queue* runs the head step-thunk each tick and advances when it returns `#t`; you drive one queue per subsystem with `('tick)` once per main loop. Motor moves, the LED ring, the buzzer, and the LLIP executor are all just consumers of this one module.

7.26.1 Task Constructors

Procedure	Description
`while-do`	`(while-do pred action)` — run `action` every tick while `pred`; done when `pred` is false. The basis of all the rest.
`until-do`	Run `action` every tick until `pred` becomes true.
`once`	Do `action` exactly once, then pop.
`behavior-hold`	A task that does nothing and never finishes (pure placeholder).
`for-ms`	Run `action` every tick for `ms` milliseconds.
`once-for`	Do `action` once, then hold the slot for `ms` ms before popping (e.g. one motor move held for a duration).
`after-ms`	Wait `ms` ms, then do `action` once and pop (deferred action / start delay).
`wait-ms`	Hold the slot for `ms` ms doing nothing (inter-move gap).
`wait`	The canonical drift-free timer task: hold the queue head for `ms` ms, then pop. Deadline latches on the first tick at the head.
`forever`	Run `action` every tick, forever (ended only by queue abort).
`cycle`	Cycle through `(list duration-ms thunk)` phases forever — a whole blink/animation as one task.
`seq`	Compose sub-tasks into one task, run in order; done when all are consumed. Lets a multi-step sequence occupy a single queue slot.

7.26.2 The Queue

Procedure / Message	Description
`make-behavior-queue`	Return a message-dispatch queue procedure.
`(bq 'push task)`	Enqueue a step-thunk.
`(bq 'tick)`	Run one slice; call once per main loop.
`(bq 'abort)`	Reactive e-stop: drop the running task and flush everything behind it.
`(bq 'idle?)`	`#t` when nothing is running and the queue is empty.
`behavior-queue-seq`	Push several tasks in order: `(behavior-queue-seq bq t1 t2 ...)`.

```
(define bq (make-behavior-queue))
;; A start-delayed, timed motion sequence, all non-blocking:
(behavior-queue-seq bq
  (after-ms 500 (lambda () (Motor.setAll 0.5)))   ; wait 0.5 s, then go
  (wait 1000)                                      ; ...run for 1 s
  (once (lambda () (Motor.setAll 0))))             ; then stop
;; drive it every main-loop iteration:
(bq 'tick)
;; ... and to bail out at any moment:
(bq 'abort)
```

7.27 ESP32 System

Feature	Description
`millis` / `micros`	Elapsed time since boot (millisecond / microsecond resolution).
`esp32-efuse-mac-get-default`	Read the factory-programmed 48-bit MAC address from eFuse.
`esp32-efuse-pkg-ver`	Read the ESP32 package version from eFuse.
`esp32-efuse-read-block` / `esp32-efuse-write-block`	Read or write raw eFuse data blocks.

7.27.1 Memory Tuning: Stack Size, DMA Memory, and Pre-Reserve

On the ESP32 the scarce resource is **internal DRAM** (∼320–350 KB on the S3), *not* PSRAM. The LambLisp cell heap lives in PSRAM (megabytes), so it is rarely the constraint; the tasks that fail on a tight board are the ones that need **internal DRAM**, and specifically its **DMA-capable** subset. Three levers interact — get them wrong and a board with plenty of PSRAM still can't read its own filesystem.

1. loopTask stack size. The application runs on the Arduino `loopTask`, whose stack is carved from internal DRAM. LambLisp overrides the framework's weak `getArduinoLoopTaskStackSize()` (in `main.cpp`) — e.g. 48 KB. Bigger is not free: every KB of stack is a KB of internal DRAM denied to DMA. Two things bound how small it can go:

- `LL_EVAL_STACK_BUDGET` (`ll_platform_generic.h`) — this guard raises a *catchable* "recursion too deep" error once `eval()` grows this far past its outermost frame. **Keep it below the loopTask stack size** (e.g. 40 KB budget under a 48 KB stack, ∼8 KB headroom). If the budget exceeds the real stack, deep recursion overflows the C stack — a hard SIGSEGV/panic — instead of a clean error.
- **The reader is *not* guarded.** `read_list` recurses once per paren-nesting level *and* runs a read-time `macroexpand` at each level, so a single deeply-nested top-level form can overflow the stack while it is being *read* (`array.scm`'s transpose nested ∼12 deep and blew the 96 KB stack while other files were fine). Fix by keeping `.scm` forms shallow — factor deep helpers into flat top-level definitions rather than one monster expression.

2. Flash reads need DMA memory. `esp_flash_read` (and therefore every LittleFS read) needs a `MALLOC_CAP_DMA`-capable buffer in **internal** SRAM. PSRAM is *not* DMA-capable for flash, so a read into a PSRAM destination allocates an internal bounce buffer. Consequence: a read can fail with `ESP_ERR_NO_MEM` (**err 257** in the `esp_littlefs: failed to read ... err 257` message) even with **megabytes of PSRAM free** — what matters is the *largest contiguous free `MALLOC_CAP_DMA` block*, which can be a few KB while total internal free still looks healthy (100 KB+). This is the failure class behind the eye and N8R2 (2 MB PSRAM): once the DMA-capable block is fragmented or starved, the filesystem is effectively unreadable and boot-time `(load ...)` silently produces nothing.

3. The pre-reserve technique — and its trap. To guarantee a peripheral or the filesystem always has DMA memory, **reserve a contiguous** `MALLOC_CAP_DMA` **block early** — at install/setup time, before the heap and other subsystems fragment internal SRAM — and hold it for the code that needs it (or free it just-in-time around the operation). `ll_xmop3_EyeCam.cpp` does this for the camera/LCD band buffer.

The trap: an over-eager early reserve *starves* the very reads it was meant to protect. In that case the camera's early DMA-hole reserve dropped the largest free DMA block to ∼16 KB, so **boot-time LittleFS reads returned garbage**; the fix was to gate that reserve *off* (`EYE_CAM_EARLY_DMA_RESERVE 0`), leaving ∼32 KB DMA free through boot. So the technique cuts both ways:

- Reserve **enough** DMA for what genuinely needs it (camera frame/LCD band buffers).
- But **not so much** that boot-time flash reads have no DMA block left.
- On memory-tight boards (2 MB-PSRAM N8R2), that balance is the whole game — shrinking the loop↩ Task stack frees *general* internal DRAM but does not by itself defragment or enlarge the DMA-capable block, so a small reserved-and-held DMA buffer for the FS-read path is usually what actually fixes it.

Rule of thumb. Keep the loopTask stack as small as the reader/eval recursion allows (shallow `.scm` forms let you go smaller), keep `LL_EVAL_STACK_BUDGET` a notch under it, and protect the filesystem's DMA needs with a small, early, held reserve rather than hoping enough DMA-capable memory survives boot.

7.28 LambLisp Modbus Interface

LambLisp includes a full Modbus binding for both controller (master) and peripheral (slave) roles over TCP and RTU transports. This chapter covers the API for both roles, platform-specific build requirements, and worked examples for common industrial IoT scenarios.

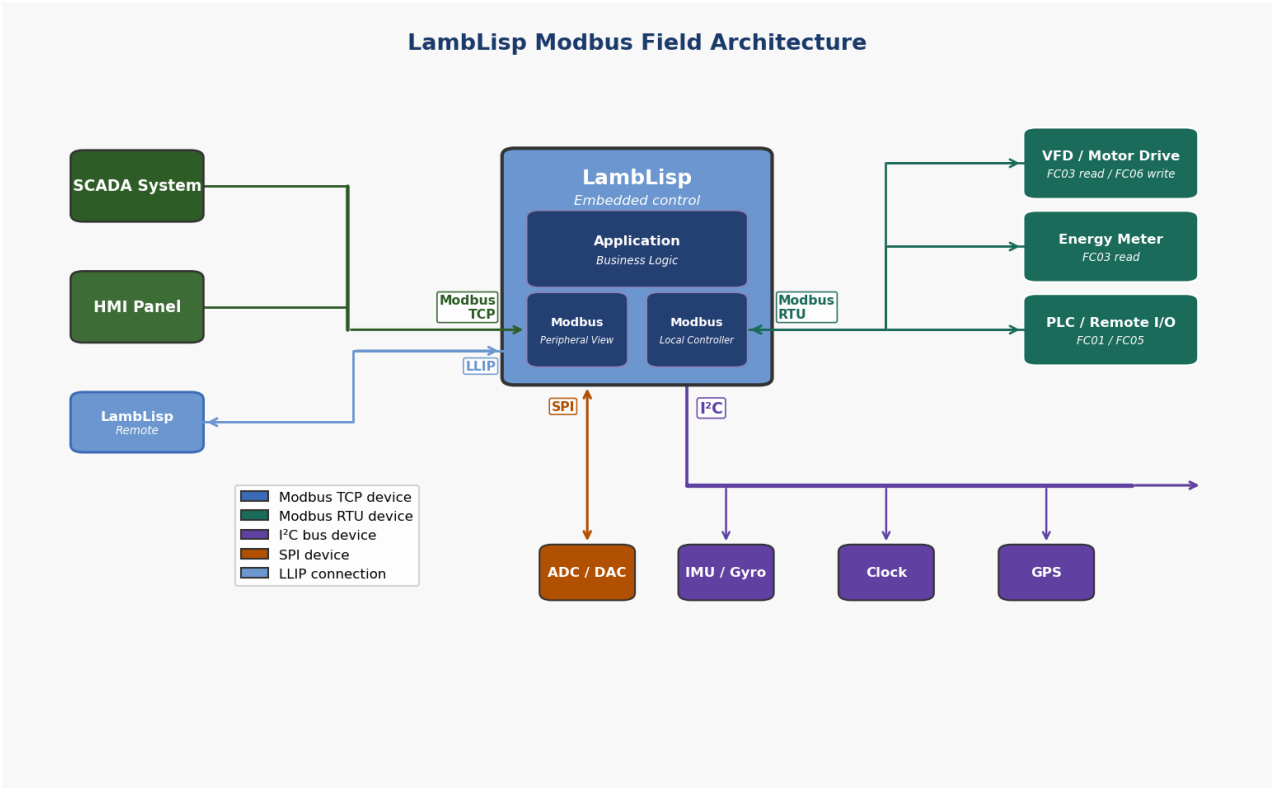


Figure 7.7 Typical Modbus Field Architecture

7.28.1 1. Overview

Modbus is a serial communication protocol published in 1979 and now ubiquitous in industrial automation. It defines a request/response model between a **controller** (the device that initiates requests) and one or more **peripherals** (devices that respond to requests). The protocol is simple enough to be embedded in PLCs, variable-frequency drives (VFDs), energy meters, temperature controllers, and thousands of other field devices — many of which still ship with Modbus as their primary communication interface.

LambLisp supports two roles and two transports:

Role	Description
Controller	LambLisp initiates read/write requests to peripheral devices (PLCs, VFDs, meters, sensors)
Peripheral	LambLisp exposes a register bank to an external SCADA, HMI, or PLC controller

Transport	Description
TCP	Modbus framing over TCP/IP, port 502. Used over Ethernet and WiFi.
RTU	Binary framing over RS-485/RS-232 serial. Used for wired field buses.

The same function calls work regardless of transport; only the connect call differs.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.28.2 2. LambLisp as Modbus Controller

In controller mode, LambLisp initiates requests to one or more peripheral devices and reads back the results. The binding uses a **non-blocking request/poll model** designed for real-time embedded loops.

2.1 Connecting

```
;; TCP: connect to a device at 192.168.1.10, Modbus unit ID 1
(define conn (modbus-tcp-connect "192.168.1.10" 502 1))

;; RTU: connect over UART2, 9600 baud, unit ID 3
(define conn (modbus-rtu-connect 2 9600 3))

;; Close when done
(modbus-close conn)
```

`(modbus-tcp-connect host port unit-id)` — Opens a TCP connection to `host:port` targeting peripheral `unit-id`. Returns a connection object. Raises an error if the connection cannot be established.

`(modbus-rtu-connect uart-num baud unit-id)` — Opens an RTU connection on the specified UART at the given baud rate targeting peripheral `unit-id`. On ESP32, configure the DE/RE pin with `modbus-rtu-set-de-pin` before opening (see Platform Notes).

`(modbus-close conn)` — Closes the connection and releases associated resources.

2.2 The Non-Blocking Request/Poll Model

Modbus devices — especially RTU peripherals — can take 50–500 ms to respond. On an embedded system running a real-time loop at 10–100 Hz, blocking that entire loop to wait for a sensor response is unacceptable. The LambLisp Modbus binding therefore uses a **two-phase model**: `modbus-request` queues the request and returns immediately; `modbus-poll` checks whether the response has arrived.

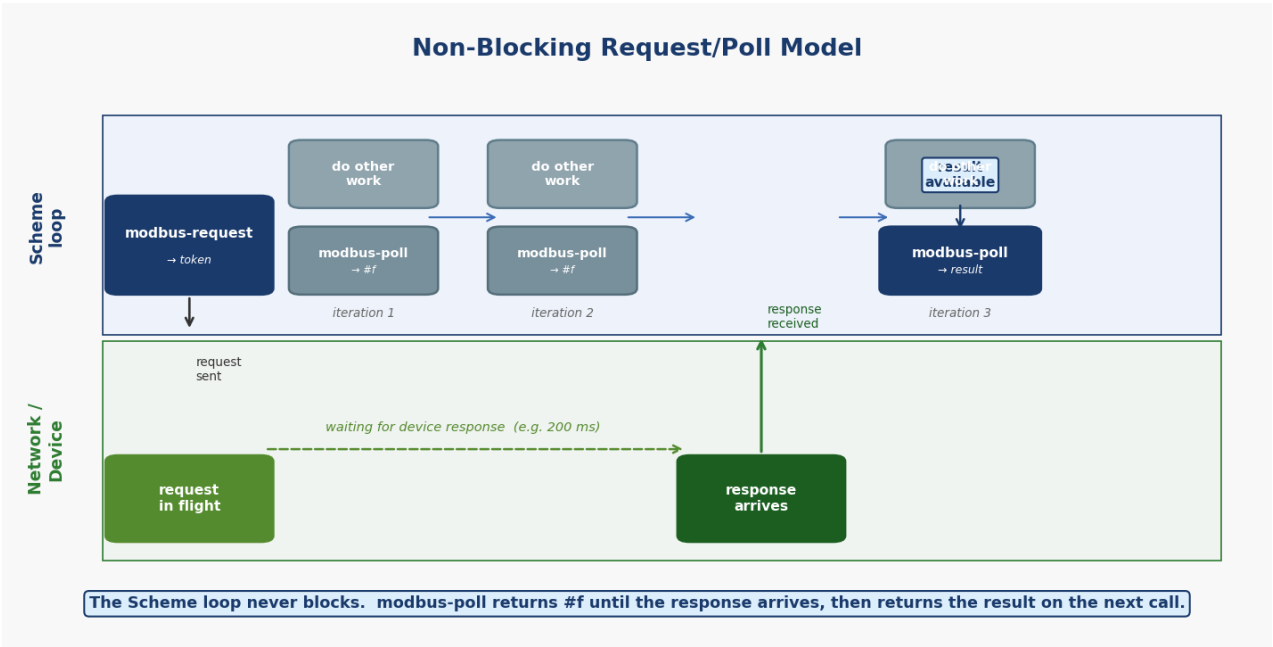


Figure 7.8 Non-blocking request/poll flow

```
;; Phase 1: issue request - returns a token immediately, never blocks
(define tok (modbus-request conn 'read-holding 40 2))

;; Phase 2: poll on each loop iteration
(define (check-result tok)
  (let ((result (modbus-poll tok)))
    (if result
        (begin (display "got: ") (display result) (newline))
        (display "not yet\n"))))
```

`modbus-poll` returns `#f` when the response has not yet arrived, and the result vector when it has. The calling loop simply does other work between polls and checks again on the next iteration. The loop never blocks.

`(modbus-request conn function-code start-address count)` — Queues a request and returns a token. `function-code` is a symbol: `read-coils`, `read-discrete-inputs`, `read-holding`, `read-input`, `write-coil`, `write-register`, `write-coils`, `write-registers`. `start-address` is the 0-based register address. `count` is the number of registers (for reads) or omitted for single-register writes.

`(modbus-poll token)` — Returns `#f` if the response has not arrived, or the result on completion. The result for read operations is a vector of integers or booleans; see FC-specific sections below. Raises an error if the device returned an exception code.

`(modbus-set-timeout conn ms)` — Sets the response timeout in milliseconds. Default is 1000 ms. If `modbus-poll` is called after the timeout has elapsed without a response, it raises an error.

2.3 Function Codes

All reads and writes use `modbus-request` with a symbol for the function code (see the Quick Reference for the full table). Reads return a vector; writes return `#t` on success.

```
;; Read 4 holding registers from address 0
(define tok (modbus-request conn 'read-holding 0 4))
(let ((regs (modbus-poll tok)))
  (display (/ (vector-ref regs 0) 10.0))   ; e.g. VFD frequency scaled ×10
  (display " Hz\n"))

;; Write a single register; poll returns #t
(define tok2 (modbus-request conn 'write-register 9 1500))
(modbus-poll tok2)   ; -> #t
```

For multi-value writes pass a vector as the value argument:

```
(modbus-request conn 'write-registers 10 (vector 1500 0))
(modbus-request conn 'write-coils      0 (vector #t #f #t #t))
```

2.4 Float32 Helpers

Many industrial devices store 32-bit floats in two consecutive 16-bit registers. Word order varies by manufacturer: `'abcd` (most Schneider/ABB/Siemens equipment) or `'cdab` (some Modicon PLCs and meters). Check your device manual.

`(modbus-request-float32 conn addr word-order)` — FC03 read of 2 registers. `(modbus-poll-float32 token)` — Returns the reassembled inexact real, or `#f`.

```
(define tok (modbus-request-float32 conn 100 'cdab))
(let loop ()
  (let ((temp (modbus-poll-float32 tok)))
    (or temp (begin (do-other-work) (loop)))))
```

2.5 Error Handling

`modbus-poll` raises when the device returns a Modbus exception, the request times out, or the connection is lost. Use `guard` to handle errors without crashing the loop:

```
(guard (e (#t (display (error-object-message e)) (newline) #f))
  (let loop ()
    (or (modbus-poll tok) (begin (do-other-work) (loop)))))
```

Exception codes appear in the message string, e.g. `"Modbus exception 02:  illegal data address"`.

7.28.3 3. LambLisp as Modbus Peripheral

In peripheral mode, LambLisp maintains a register bank that an external controller (SCADA system, HMI panel, or PLC) can read and write using standard Modbus function codes. The Scheme code updates the register bank from sensor readings, and optionally receives callbacks when the controller writes new setpoints or commands.

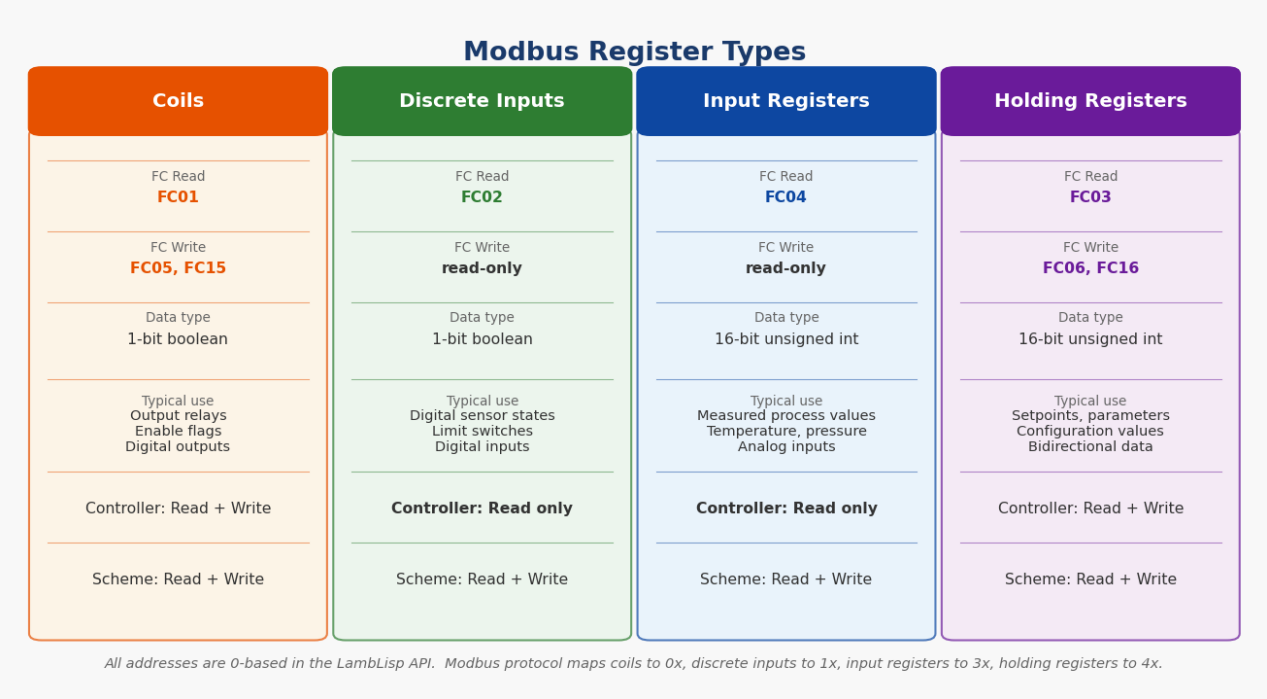


Figure 7.9 Modbus register types

3.1 Opening a Peripheral

```
;; TCP peripheral: listen on port 502, present as unit ID 1
(define peripheral (modbus-tcp-peripheral-open 502 1))

;; RTU peripheral: UART 2, 9600 baud, unit ID 5
(define peripheral (modbus-rtu-peripheral-open 2 9600 5))
```

`(modbus-tcp-peripheral-open port unit-id)` — Opens a TCP listening socket on `port`. On Linux, port 502 requires root or a capability grant; using port 5502 is common during development. Returns a peripheral object.

`(modbus-rtu-peripheral-open uart-num baud unit-id)` — Opens the specified UART as a Modbus RTU peripheral. Returns a peripheral object.

`(modbus-peripheral-close peripheral)` — Closes the peripheral and releases the port/UART.

3.2 The Four Register Tables

Modbus defines four data tables. The table below shows FC read/write codes and access permissions from both sides.

Table	FC Read	FC Write	Controller access	Scheme access
Coils	FC01	FC05, FC15	Read + Write	Read + Write
Discrete Inputs	FC02	—	Read only	Read + Write
Input Registers	FC04	—	Read only	Read + Write
Holding Registers	FC03	FC06, FC16	Read + Write	Read + Write

3.3 Register Accessors

All addresses are 0-based.

```
;;; --- Holding registers ---
(modbus-peripheral-hreg!  peripheral address value)   ; write from Scheme
(modbus-peripheral-hreg   peripheral address)         ; read from Scheme

;;; --- Input registers ---
(modbus-peripheral-ireg!  peripheral address value)   ; write from Scheme
(modbus-peripheral-ireg   peripheral address)         ; read from Scheme

;;; --- Coils ---
(modbus-peripheral-coil!  peripheral address bool)    ; write from Scheme
(modbus-peripheral-coil   peripheral address)         ; read from Scheme -> #t or #f

;;; --- Discrete inputs ---
(modbus-peripheral-dinput! peripheral address bool)   ; write from Scheme
(modbus-peripheral-dinput  peripheral address)        ; read from Scheme -> #t or #f
```

Values for registers must be exact integers in [0, 65535]. Coil and discrete input values must be booleans.

3.4 The Serve Loop

`modbus-peripheral-poll` processes any pending incoming requests from the controller. It is non-blocking: if no request is pending it returns `#f` immediately; if a request was processed it returns `#t`. Call it on each iteration of the main loop.

```
(define (main-loop)
  ;; Update process data in the register bank
  (modbus-peripheral-ireg! peripheral 0
    (inexact->exact (round (* (read-temperature) 10))))
  (modbus-peripheral-ireg! peripheral 1
    (inexact->exact (round (* (read-pressure) 100))))
  ;; Serve any pending controller requests
  (modbus-peripheral-poll peripheral)
  ;; ... other real-time work ...
  (main-loop))
```

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

3.5 Write Callbacks

To be notified when the controller writes to coils or holding registers, register a callback with `modbus-peripheral-on-write!`. The callback receives the table symbol, the start address, and a vector of written values.

```
(modbus-peripheral-on-write! peripheral
  (lambda (table start-addr values)
    (cond
      ((eq? table 'holding)
       (when (= start-addr 10)
         ;; Controller wrote a new speed setpoint to HR40011
         (set-motor-speed! (/ (vector-ref values 0) 10.0))))
      ((eq? table 'coils)
       (when (= start-addr 0)
         ;; Controller toggled output relay 1
         (set-relay! 0 (vector-ref values 0)))))))
```

The callback is invoked synchronously inside `modbus-peripheral-poll` after the register bank has been updated and the response has been sent to the controller.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.28.4 4. Platform Notes

4.1 Linux (x86_64 and aarch64)

The Linux backend uses **libmodbus** (https://libmodbus.org).

Install the development package:

```
sudo apt-get install libmodbus-dev
```

Enable Modbus in the build by defining `LL_MODBUS 1` in the board configuration header (e.g. `ll_board_linux_x86.h`):

```
#define LL_MODBUS 1
```

The build system links `-lmodbus` automatically when `LL_MODBUS` is set.

For TCP peripheral mode on port 502, either run as root or grant the capability:

```
sudo setcap cap_net_bind_service=+ep ./lamblisp
```

Alternatively, use port 5502 during development and configure the SCADA simulator to connect to that port.

4.2 ESP32-S3

The ESP32 backend uses **no external library**. Modbus frames are built and parsed directly using `WiFiClient` (TCP) and `HardwareSerial` (RTU) from the Arduino framework. No `lib_deps` entry is required for the Modbus environment.

Enable Modbus in the board header:

```
#define LL_MODBUS 1
```

RTU wiring for RS-485: RS-485 transceivers (e.g. MAX485, SN65HVD) require a direction-enable pin (DE/RE). Configure it before issuing any RTU requests:

```
(modbus-rtu-set-de-pin conn 4)   ; example: GPIO 4 drives DE/RE on ESP32-S3 DevKit
```

`modbus-rtu-set-de-pin` sets the GPIO pin asserted HIGH before transmit and released LOW after `flush()`. Connect DE and RE of the transceiver together and wire to the specified GPIO. The default pin (`LL_MODBUS_RTU_DE`) is `-1` (disabled), suitable for RS-232 or auto-direction transceivers.

Recommended UART-to-RS-485 pin assignments for ESP32-S3 DevKit:

Signal	GPIO
UART2 TX	17
UART2 RX	18
DE/RE	4

WiFi must be initialized before using Modbus TCP. Call `(wifi-connect ssid pass)` (or equivalent board setup) before `modbus-tcp-connect`.

TCP peripheral on ESP32: `modbus-tcp-peripheral-open` allocates a `WiFiServer` and calls `begin()`. `modbus-peripheral-poll` calls `server->available()` once per iteration to accept new clients. No FreeRTOS task is created.

4.3 Build Flag Summary

Flag	Effect
`LL_MODBUS 1`	Enables the Modbus binding; links libmodbus (Linux only)
`LL_MODBUS 0` or undefined	Modbus functions are not available; calls raise "undefined"

7.28.5 5. Quick Reference

Controller Functions

Function	Description
`(modbus-tcp-connect host port unit)`	Connect to a TCP Modbus peripheral
`(modbus-rtu-connect uart baud unit)`	Connect to an RTU Modbus peripheral
`(modbus-close conn)`	Close connection
`(modbus-set-timeout conn ms)`	Set response timeout in milliseconds
`(modbus-request conn fc addr count)`	Issue a Modbus request; returns token
`(modbus-poll token)`	Check for response; returns result or `#f`
`(modbus-request-float32 conn addr order)`	Issue FC03 read of 2-register float
`(modbus-poll-float32 token)`	Poll float32 response; returns inexact or `#f`
`(modbus-rtu-set-de-pin conn gpio)`	Set RS-485 direction-enable pin (ESP32)

Controller Function Codes (symbols for `modbus-request`)

Symbol	FC	Direction	Result
`read-coils`	FC01	Read	Vector of booleans
`read-discrete-inputs`	FC02	Read	Vector of booleans
`read-holding`	FC03	Read	Vector of integers
`read-input`	FC04	Read	Vector of integers
`write-coil`	FC05	Write	`#t` on success
`write-register`	FC06	Write	`#t` on success
`write-coils`	FC15	Write	`#t` on success
`write-registers`	FC16	Write	`#t` on success

Peripheral Functions

Function	Description
`(modbus-tcp-peripheral-open port unit)`	Open TCP peripheral on port
`(modbus-rtu-peripheral-open uart baud unit)`	Open RTU peripheral
`(modbus-peripheral-close peripheral)`	Close peripheral
`(modbus-peripheral-poll peripheral)`	Process pending controller requests
`(modbus-peripheral-on-write! peripheral proc)`	Register write callback
`(modbus-peripheral-hreg! peripheral addr val)`	Write holding register from Scheme

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Function	Description
`(modbus-peripheral-hreg peripheral addr)`	Read holding register from Scheme
`(modbus-peripheral-ireg! peripheral addr val)`	Write input register from Scheme
`(modbus-peripheral-ireg peripheral addr)`	Read input register from Scheme
`(modbus-peripheral-coil! peripheral addr bool)`	Write coil from Scheme
`(modbus-peripheral-coil peripheral addr)`	Read coil from Scheme
`(modbus-peripheral-dinput! peripheral addr bool)`	Write discrete input from Scheme
`(modbus-peripheral-dinput peripheral addr)`	Read discrete input from Scheme

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.29 LambLisp PROFIBUS and PROFINET Interface

LambLisp implements the two fieldbuses of the Siemens automation world: **PROFIBUS DP** over RS-485, and **PROFINET** over Ethernet. This chapter covers both, the API for each role, the build flags, and what you need to build locally.

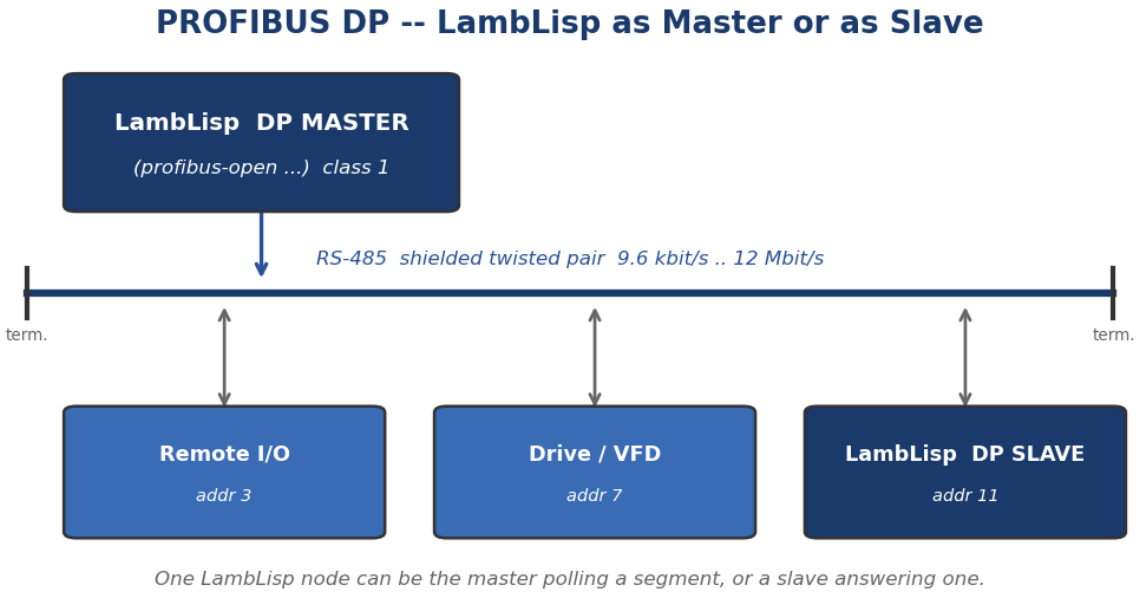


Figure 7.10 PROFIBUS DP: LambLisp as master or as slave

7.29.1 1. What You Get

Capability	Enable with	Also needs
PROFIBUS DP master (class 1, cyclic)	`LL_PROFIBUS=1`	an RS-485 transceiver
PROFIBUS DP slave	`LL_PROFIBUS=1`	as above; baud-capped, see §3
PROFINET IO-Device	`LL_PROFINET=1`	the `pnd` daemon, built locally per §5.1

PROFIBUS is self-contained: set the flag, wire up a transceiver, and it works.

PROFINET comes in two pieces. The `profinet-*` bindings are compiled into your LambLisp binary by `LL_PROFINET=1`; the protocol stack runs beside it as a separate daemon, `pnd`, which you build from the source in `ll_profinet/`. Section 5.1 has the commands.

7.29.2 2. PROFIBUS DP as a Master

A DP master polls each slave in turn, exchanging a fixed-size process image every cycle. LambLisp performs the whole handshake for you: a slave must be found, parameterised, configured, checked for diagnostics, and only then does cyclic data exchange begin.

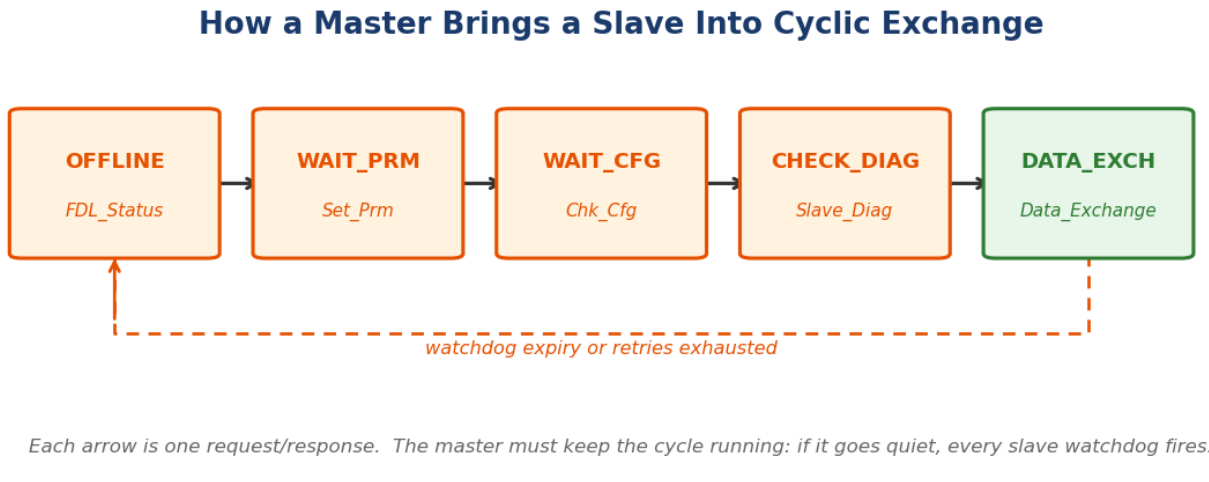


Figure 7.11 The state machine the master drives, per slave

2.1 Opening the Bus

```
(define bus (profibus-open "/dev/ttyUSB0" 500000))   ; POSIX: a tty path
(define bus (profibus-open 1 500000))                ; ESP32: a UART index
(profibus-set-address bus 1)                         ; this station's FDL address
```

PROFIBUS uses **8E1** framing — 1 start bit, 8 data bits, **even parity**, 1 stop bit — not the 8N1 that Modbus RTU uses. LambLisp configures this for you; you only supply the baud rate.

Standard rates are 9600, 19200, 45450, 93750, 187500, 500000, and 1.5/3/6/12 Mbit/s. Higher rates mean shorter cable: 1200 m at 9.6 kbit/s, 100 m at 12 Mbit/s.

2.2 Declaring Slaves

Each slave needs its address, its **ident number** (from the vendor's GSD file), and its input/output lengths:

```
(define io (profibus-add-slave bus 3 #x0801 4 2))    ; addr 3, 4 bytes in, 2 out
(profibus-set-prm io (bytevector #x88 #x0A #x0A #x0B #x08 #x01 #x00))
(profibus-set-cfg io (bytevector #x13 #x21))         ; 4 bytes in, 2 bytes out
(profibus-set-watchdog io 1000)                      ; ms; 0 disables
```

The **configuration identifier bytes** passed to `profibus-set-cfg` must match the slave's GSD exactly, or it will refuse to leave `wait-cfg`. This is the single most common commissioning failure. The encoding is:

Bit	Meaning
0–3	length − 1
4	inputs present
5	outputs present
6	word (set) or byte (clear)
7	consistency over the whole module

So `#x10` is one input byte, `#x20` one output byte, `#x30` one of each, `#x13` four input bytes, `#x50` one input word.

2.3 The Service Loop

```
(profibus-start bus)

(let loop ()
  (profibus-service bus)                    ; advances the bus; never blocks

  (let drain ()                             ; 1. handle events
    (let ((ev (profibus-poll-event bus)))
      (when ev
        (case (car ev)
          ((slave-up)   (log "slave ~a online" (cadr ev)))
          ((slave-down) (log "slave ~a DOWN: ~a" (cadr ev) (caddr ev)))
          ((diag)       (log "slave ~a diagnostics ~a" (cadr ev) (caddr ev))))
        (drain))))

  (let* ((in    (profibus-inputs io))         ; 2. read a consistent snapshot
         (level (bytevector-u16-ref in 0)))
    (profibus-outputs-set! io                 ; 3. stage the next outputs
      (u16->bytevector (control-law level))))
  (loop))
```

Three properties of this loop are worth understanding:

- `profibus-service` **never blocks.** Call it as often as you like; it does whatever work is possible and returns.
- `profibus-inputs` **returns a consistent snapshot.** The image is double-buffered, so you never see half of one cycle and half of the next.
- `profibus-outputs-set!` **stages, it does not transmit.** The staged image is published at a cycle boundary, so a partially-written image is never sent.

2.4 Events, Not Exceptions

A slave dropping off the bus is **not** an error. It is an event:

```
(slave-up   3)              ; reached cyclic exchange
(slave-down 3 timeout)      ; also: prm, cfg, ident, fcs
(diag       3 #u8(...))     ; diagnostics available
(cfg-reject 3 #u8(...))     ; your cfg bytes, so you can see what was refused
```

Errors are raised only for programming mistakes — a bad handle, an undeclared slave, an unsafe baud rate. A control loop should not need `guard` around every cycle just to survive a station failing.

2.5 Diagnostics and Global Control

```
(profibus-diag io)                    ; -> bytevector, or #f
(profibus-global-control bus #x02 0)  ; CLEAR: zero all outputs, group 0
(profibus-stats bus)                  ; -> (cycles retries timeouts fcs min mean max)
```

`profibus-stats` reports retries and timeouts separately from errors. A bus that silently retries is a bus that is about to fail in the field, so watch these counters rather than waiting for a station to drop.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.29.3 3. PROFIBUS DP as a Slave

LambLisp can also *be* a DP slave, answering a third-party master:

```
(define self (profibus-slave-open "/dev/ttyUSB0" 19200 3 #x0801 2 1))
(profibus-slave-set-cfg self (bytevector #x11 #x20))    ; what we will accept
(profibus-slave-inputs-set! self (bytevector #xDE #xAD))

(let loop ()
  (profibus-slave-service self)
  (let ((out (profibus-slave-outputs self)))
    (apply-actuators out))
  (loop))
```

The baud ceiling, and why it exists

`profibus-slave-open` **refuses** a baud rate above 500 kbit/s, raising `pb-baud-unsafe`.

This is deliberate. A DP slave must begin its reply within `T_SDR`, a minimum of 11 bit times. That is 1.15 ms at 9.6 kbit/s but only **0.92 µs at 12 Mbit/s** — far below what any software responder can guarantee. Devices that run at 12 Mbit/s use a dedicated ASIC for exactly this reason.

Baud	`T_SDR` minimum	Software slave
9.6 kbit/s	1.15 ms	comfortable
93.75 kbit/s	117 µs	fine
500 kbit/s	22 µs	the supported ceiling
1.5 Mbit/s	7.3 µs	needs hardware turnaround
12 Mbit/s	0.92 µs	ASIC only

Refusing loudly at open time is better than a segment that works on the bench and fails intermittently in a plant.

7.29.4 4. RS-485 Wiring

The standard violet DB9:

Pin	Signal	Note
3	B (RxD/TxD-P)	data +
8	A (RxD/TxD-N)	data −
5	DGND	terminator reference
6	VP (+5 V)	**powers the terminator network**
4	RTS	direction, for repeaters
1, 9	shield	to the connector housing

Both ends of a segment need the active terminator (390 Ω to VP, 220 Ω across A/B, 390 Ω to DGND); proper PROFIBUS connectors have it switchable.

> **A segment with unpowered terminators appears to work at low baud and fails intermittently at high baud.** Rule this out before debugging software timing.

On ESP32, LambLisp uses the UART's hardware RS-485 half-duplex mode so the transceiver turnaround is handled in hardware. If your board cannot do this, set a direction pin with `(profibus-set-de-pin bus gpio)` and expect a lower baud ceiling. `(profibus-hardware-turnaround? bus)` tells you which you got.

For a plant segment use an isolated transceiver. For bench work a MAX3485 is fine; do not daisy-chain through a breadboard at 500 kbit/s.

7.29.5 5. PROFINET

PROFINET is Ethernet-based and needs a full protocol stack: DCP for discovery and commissioning, DCE/↩ RPC for connection establishment, and cyclic real-time frames.

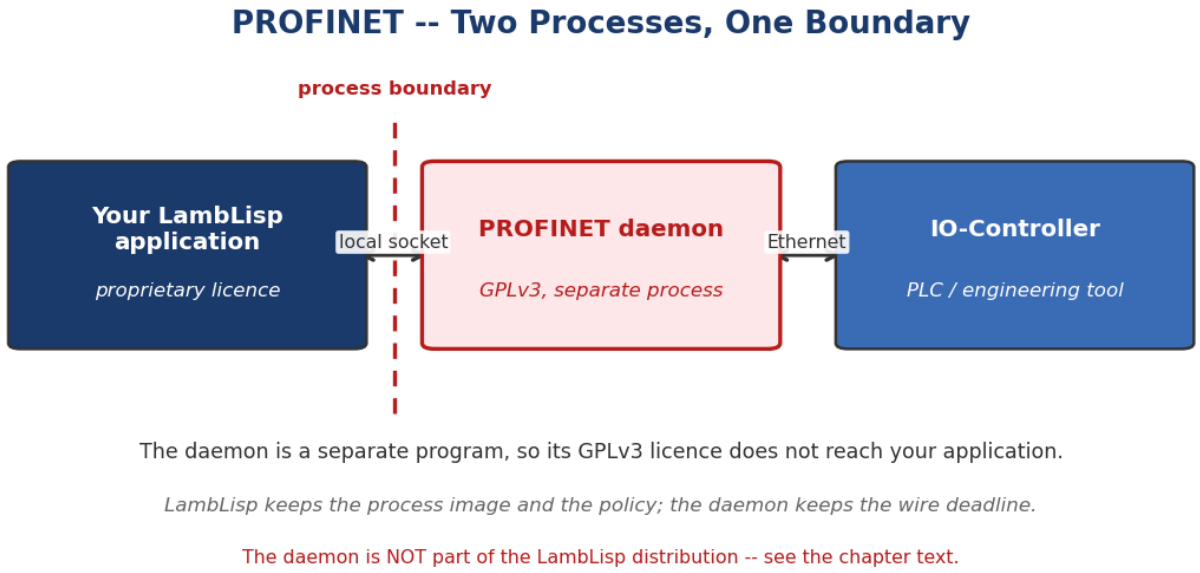


Figure 7.12 PROFINET runs as two processes

5.1 Building the daemon

The PROFINET stack runs as a separate program, `pnd`, and LambLisp talks to it over a local socket. The daemon holds the cyclic deadline, so a garbage collection in your Scheme code cannot make the device miss a bus cycle.

The daemon is shipped in source code form and needs to be built locally. Its source is in `ll_profinet/`, and the build needs the rt-labs `p-net` and `osal` sources, which you clone into `extern/`:

```
cd ll_profinet
mkdir -p extern && cd extern
git clone --depth 1 https://github.com/rtlabs-com/p-net.git
git clone --depth 1 https://github.com/rtlabs-com/osal.git
cd ..
./build.sh
```

`build.sh` is the entire build — the public `p-net` drop carries no CMake files, so the script compiles the stack, `osal`, and the LambLisp adaptation layer directly. It produces:

Output	What it is
`build/pnd`	the daemon
`build/↩ libprofinet.a`	the stack, if you would rather link your own

Output	What it is
`build/linktest`	calls `pnet_init()`; run it to prove the build works

Run the daemon on the interface carrying PROFINET traffic. It needs `CAP_NET_RAW` to open an `AF_PACKET` socket and `CAP_NET_ADMIN` to honour a DCP Set-IP, so give it the capability or start it as root:

```
sudo setcap cap_net_raw,cap_net_admin+ep build/pnd
build/pnd --if pn0 --station lamb-io-1 &
```

`--socket PATH` moves the control socket (default is shared with the `profinet-*` bindings), and `--vendor N` / `--device N` set the identity reported in DCP. Start `pnd` before the LambLisp program that calls `profinet-open`.

5.2 The API

The bindings *are* in your LambLisp binary, and mirror the PROFIBUS shape:

```
(define dev (profinet-device-open "/tmp/lamblisp-profinet.sock" #x0493 #x0002))
(profinet-add-module    dev 1 #x00000010)
(profinet-add-submodule dev 1 1 #x00000001 4 4)      ; 4 bytes in, 4 out
(profinet-start dev "lamb-io-1")                     ; our NameOfStation

(let loop ()
  (let drain ()
    (let ((ev (profinet-poll-event dev)))
      (when ev
        (case (car ev)
          ((dcp-set-name) (save-station-name! (cadr ev)))   ; commissioning!
          ((dcp-set-ip)   (save-ip-config!    (cadr ev)))
          ((connect)      (log "controller connected"))
          ((param-end)    (log "parameterised"))
          ((dcp-signal)   (flash-led!)))                    ; "where are you?"
        (drain))))

  (profinet-inputs-set! dev 1 1 (read-sensors))
  (when (eq? (profinet-iocs dev 1 1) 'good)
    (apply-actuators (profinet-outputs dev 1 1)))
  (loop))
```

`(dcp-set-name "...")` arriving is the moment an engineering tool commissions your device. Persisting that name is your application's job — which is exactly why it is surfaced to Scheme rather than handled silently.

5.3 Device model

PROFINET describes a device as **slots** holding **modules**, each with **subslots** holding **submodules** that carry the actual I/O. Your declaration must match the **GSDML** file you give the engineering tool, or the controller will refuse the connection at `Connect`. Generate the GSDML from the same definition you use in code; hand-maintaining both is a reliable way to lose an afternoon.

7.29.6 6. Certification and Naming

Both protocols are trademarks of PROFIBUS & PROFINET International (PI), and the right to use the names and marks on a product is tied to certification.

Item	Requirement	Cost
Vendor ID	one per company, from PI	free
Device Ident Number	required for every DP device **except** a master class 2	registration fee
GSD / GSDML file	one per device, matching the firmware	your effort
Certification	mandatory for PROFINET devices, per Conformance Class, at a PI Test Lab	lab + certificate fees

Two practical notes. A **DP master class 2** (an engineering or diagnostic tool) needs no Device Ident Number, which makes it the least encumbered thing to build first. And using a pre-certified communication module reduces your certification effort substantially but does **not** remove it — PI certifies devices, not chips.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.29.7 7. Build Flags

Features are compiled in per target. Your package ships a `platformio.ini` already locked to the target you downloaded, so to change the feature set, edit its `build_flags` directly and rebuild.

Flag	Enables
LL_PROFIBUS=1	`profibus-*` and `profibus-slave-*`
LL_PROFINET=1	`profinet-*` (needs the daemon at run time)

Add them to the `build_flags` of your target's environment:

```
build_flags =
    -O3
    -DNDEBUG
    -DLL_PROFIBUS=1
    -DLL_PROFINET=1
```

Verify a feature actually reached your binary:

```
(defined? 'profibus-open)     ; => #t when LL_PROFIBUS was set
```

7.29.8 8. Quick Reference

PROFIBUS DP master

Function	Purpose
`(profibus-open port baud)`	open the bus; returns a handle
`(profibus-close bus)`	close it
`(profibus-set-address bus addr)`	this station's FDL address, 0–125
`(profibus-set-timing bus tsl retries)`	slot time (ms) and retry count
`(profibus-set-de-pin bus gpio)`	RS-485 direction pin, if not hardware
`(profibus-hardware-turnaround? bus)`	`#t` if the UART drives direction
`(profibus-add-slave bus addr ident in out)`	declare a slave
`(profibus-set-prm slave bv)`	parameterisation bytes
`(profibus-set-cfg slave bv)`	configuration identifiers
`(profibus-set-watchdog slave ms)`	slave watchdog; 0 disables
`(profibus-start bus)` / `(profibus-stop bus)`	begin/end cyclic exchange
`(profibus-service bus)`	advance the bus; non-blocking
`(profibus-state slave)`	`offline` `wait-prm` `wait-cfg` `check-diag` `data-exch`
`(profibus-inputs slave)`	consistent input snapshot
`(profibus-outputs-set! slave bv)`	stage outputs
`(profibus-diag slave)`	diagnostics bytevector, or `#f`
`(profibus-global-control bus cmd group)`	sync/freeze/clear broadcast
`(profibus-poll-event bus)`	next event, or `#f`
`(profibus-stats bus)`	cycles, retries, timeouts, FCS errors, timing

PROFIBUS DP slave

Function	Purpose
`(profibus-slave-open port baud addr ident in out)`	become a slave
`(profibus-slave-set-cfg self bv)`	the configuration we accept
`(profibus-slave-inputs-set! self bv)`	data we provide to the master
`(profibus-slave-outputs self)`	data received from the master
`(profibus-slave-set-diag self bv)`	extended diagnostics
`(profibus-slave-service self)`	answer pending telegrams
`(profibus-slave-state self)`	current state
`(profibus-slave-poll-event self)`	next event, or `#f`

PROFINET IO-Device

Function	Purpose
`(profinet-device-open path vendor device)`	connect to the daemon
`(profinet-add-module dev slot ident)`	declare a module
`(profinet-add-submodule dev slot sub ident in out)`	declare a submodule
`(profinet-start dev name)`	start with a NameOfStation
`(profinet-inputs-set! dev slot sub bv)`	provider data to the controller
`(profinet-outputs dev slot sub)`	consumer data from the controller
`(profinet-iops-set! dev slot sub good?)`	our provider status
`(profinet-iocs dev slot sub)`	`good` or `bad`, from the controller
`(profinet-state dev)`	`offline` `dcp-only` `connected` `operate`
`(profinet-connected? dev)`	`#t` when cyclic exchange is running
`(profinet-poll-event dev)`	next event, or `#f`
`(profinet-alarm dev slot sub type bv)`	send a process alarm
`(profinet-stats dev)`	cycles, missed, cycle counter, jitter, alarms

Error tags

Raised as the first element of the error message, so you can dispatch without string matching.

Tag	Meaning
`pb-baud-unsafe`	baud above the software responder ceiling
`pb-timeout`	no reply within the slot time after retries
`pb-fcs`	checksum or delimiter error
`pb-state`	operation invalid in this state
`pb-cfg-reject`	slave refused the configuration
`pb-ident`	slave's ident number does not match
`pn-transport`	lost the PROFINET daemon
`pn-not-connected`	I/O access before a controller connected
`pn-submodule`	slot/subslot not declared

7.30 LambLisp BACnet/IP Interface

LambLisp includes a BACnet/IP binding (ASHRAE 135 / ISO 16484-5, Annex J) written in pure Scheme. It lets a LambLisp node discover BACnet equipment, read and write the properties of that equipment, and — in the other direction — answer discovery and property reads as a BACnet Device object of its own.

The implementation is `scm/features/bacnet.scm` in the source package, and ships as `bacnet.scm` on the device filesystem of every target whose manifest includes it. It needs no build flag and no native library.

7.30.1 1. Overview and conformance

BACnet does not divide the world into client and server devices. It assigns a role **per service**: side **A** originates a confirmed request, side **B** executes it and answers. Conformance is therefore stated as BIBBs (BACnet Interoperability Building Blocks) rather than as "client" or "server".

LambLisp implements both sides of discovery and the A side of data sharing:

Call	BIBB	Role	What it does
`bacnet-who-is`	DM-DDB-A	originates	Sends Who-Is, collects the I-Am replies
`bacnet-read-property`	DS-RP-A	originates	ReadProperty against another device
`bacnet-write-property`	DS-WP-A	originates	WriteProperty against another device
`bacnet-device-poll`	DM-DDB-B	executes	Answers Who-Is with I-Am
`bacnet-device-poll`	DS-RP-B	executes	Serves ReadProperty on its Device object

The B-side pair is the difference between a tool that *talks to* BACnet equipment and a node that *is* BACnet equipment: a workstation scanning the network will find a LambLisp node (DM-DDB-B) and can then read its Device object (DS-RP-B).

Conformance profile. This implementation **meets the BIBB requirements of B-SS** (BACnet Smart Sensor), the smallest standard device profile, which requires DS-RP-B and DM-DDB-B. That wording is deliberate: it is *not* a certified B-SS device, because certification is a test-laboratory matter and not a property of source code. Treat the BIBB names above as a description of what the code does, and read them against the standard before they enter a compliance table.

Not implemented. DS-WP-B (nothing can write *to* a LambLisp node), ReadPropertyMultiple, COV (change-of-value) subscriptions, alarm and event handling, and segmentation. The transport is **BACnet/IP over UDP only** — MS/TP is not supported. The B side serves its Device object; it does not expose a general object database.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.30.2 2. Sessions and discovery

Load the library, open a session, and broadcast a Who-Is. A session is a UDP port plus the invoke-id counter that keeps concurrent requests apart.

Call	Result
`(bacnet-open [port])`	Session object, or `#f`. Port defaults to 47808 (0xBAC0).
`(bacnet-who-is s bcast [timeout-ms])`	List of `(instance max-apdu vendor-id)`, default 2000 ms
`(bacnet-close s)`	Closes the session's port

```
(load "bacnet.scm")

(define s (bacnet-open))                    ; UDP 47808
(define found (bacnet-who-is s "192.168.1.255" 2000))
;; => ((201 1476 260) (307 480 0))          ; instance, max APDU, vendor id

(for-each (lambda (d) (display "device ") (display (car d)) (newline)) found)
```

`bacnet-who-is` collects replies for the whole timeout rather than stopping at the first, because Who-Is is a broadcast and every device answers independently. Duplicate I-Am frames from the same device are discarded.

7.30.3 3. Reading and writing properties

Both calls take the peer's IP address, an object type, an object instance number, and a property identifier.

Call	Default timeout
`(bacnet-read-property s addr objtype objinst propid [timeout-ms])`	3000 ms
`(bacnet-write-property s addr objtype objinst propid value [priority [timeout-ms]])`	3000 ms

Object types and property identifiers have names, so a call reads as the standard writes it:

Object type constant	Value	Property constant
`bacnet-object-analog-input`	0	`bacnet-prop-present-value`
`bacnet-object-analog-output`	1	`bacnet-prop-object-name`
`bacnet-object-analog-value`	2	`bacnet-prop-description`
`bacnet-object-binary-input`	3	`bacnet-prop-units`
`bacnet-object-binary-output`	4	`bacnet-prop-status-flags`
`bacnet-object-binary-value`	5	`bacnet-prop-object-list`
`bacnet-object-device`	8	`bacnet-prop-vendor-name`
`bacnet-object-multi-state-value`	19	`bacnet-prop-firmware-revision`

```
;; Read analog input 1's present value from the device at 192.168.1.42
(bacnet-read-property s "192.168.1.42"
                      bacnet-object-analog-input 1
                      bacnet-prop-present-value)
;; => 21.5

;; Command analog output 3 to 50% at priority 8
(bacnet-write-property s "192.168.1.42"
                       bacnet-object-analog-output 3
                       bacnet-prop-present-value 50.0
                       8)
;; => #t
```

The `priority` argument is BACnet's command-priority array slot (1 is highest, 16 lowest); omit it to write without a priority. A read returns the decoded value, or `#f` if the device answered with an Error PDU or did not answer within the timeout.

7.30.4 4. Answering as a Device object

To be discoverable on a BACnet network, a node needs a Device object and something that services incoming frames. `bacnet-make-device` builds the object; `bacnet-device-poll` services one frame and returns.

Call	Notes
`(bacnet-make-device instance name [vendor-id [vendor-name]])`	Device object instance and name
`(bacnet-device-poll dev s bcast [timeout-ms])`	Services one frame; default 100 ms
`(bacnet-device-prop dev propid)`	Reads a property of our own Device
`(bacnet-device-set-prop! dev propid value)`	Sets one; `#f` if the property is absent

```
(define dev (bacnet-make-device 4711 "LambLisp Node"))
(define s   (bacnet-open))

(bacnet-device-set-prop! dev bacnet-prop-description "Boiler room sensor")

;; Service the network in the application's own loop.
(let loop ()
  (bacnet-device-poll dev s "192.168.1.255" 100)
  (do-application-work)
  (loop))
```

`bacnet-device-poll` blocks for at most its timeout, so it can sit inside an existing control loop without a thread. A short timeout keeps the loop responsive; the node answers Who-Is and ReadProperty on its Device object between iterations.

7.30.5 5. Requirements and limits

Requirement	Detail
Transport	BACnet/IP (Annex J) over UDP, port 47808 (0xBAC0). No MS/TP.
Runtime primitives	`udp-recv-from`, and `SO_BROADCAST` on POSIX UDP sockets
Network	Who-Is is a **broadcast**; the caller passes the subnet broadcast address
Frame size	Bounded by the max APDU the node advertises; segmentation is not supported

Who-Is cannot work without broadcast: a LambLisp build whose UDP sockets lack `SO_BROADCAST` will send the frame and hear nothing back. Both primitives are present in every build that ships this library.

Because segmentation is not implemented, a ReadProperty whose answer exceeds one APDU — a large `object-list`, for instance — will be refused by the peer rather than reassembled. Read such properties with an array index if the device supports one.

7.31 Bundled Demos

Alongside the reusable convenience layers, LambLisp ships a handful of complete **example applications** under `scm/demos/`. Where the libraries give you an API, the demos show that API assembled into something that runs end-to-end: a supervisory-control server for the Freenove 4WD car, a capability-limited eval sandbox, an occupancy-grid telemetry mapper, and two tiny character/data sketches. Each is selected into a target's filesystem image by its manifest (`scm/targets/<env>.manifest`); all five demos ship on the Linux hosts (`linux_x86_64`, `linux_x86_64_profile`, `linux_x86_64_cuda`, `linux_x86_64_hip`, `linux_aarch64`) and none of the ESP32 targets — they are host-side orchestration and teaching code, not on-board firmware.

The supervisor, sandbox, and telemetry demos are three tiers of one system and are used together (they underpin the Worked Examples); the two sketches stand alone.

7.31.1 Supervisory Control (`demo-supervisor.scm`)

The demo supervisor is a three-tier orchestrator: a browser talks to a small Python web bridge (part of the demo rig, not shipped here), which connects over LLIP to this supervisor, which in turn drives the Freenove 4WD car over a second LLIP link. It runs a non-blocking main loop that accepts bridge clients, polls telemetry, sweeps the sonar into the map, dispatches sandboxed viewer expressions to the car, and broadcasts results and map/telemetry updates to every client. Its load order and the required companion files (`llip-client.scm`, `llip-server.scm`, `demo-sandbox.scm`, `demo-telemetry.scm`) are documented in the file header.

`demo-start` is the single entry point; it opens the car link and bridge server (both configured by the `demo-car-*` / `demo-bridge-*` defines at the top of the file) and enters the loop. The **car wrappers** are thin conveniences over LLIP eval that you can also call directly from a REPL attached to the supervisor:

Procedure	Description
`demo-start`	Entry point: connect to the car, start the bridge LLIP server, run the main loop.
`car-eval`	`(car-eval expr)` — evaluate an arbitrary form on the car over LLIP; the primitive the others build on.
`car-move!`	`(car-move! dir ms)` — drive in direction `dir` for `ms` milliseconds.
`car-turn!`	`(car-turn! deg)` — turn in place by `deg` degrees.
`car-sweep`	`(car-sweep)` — run a sonar sweep; returns `(angle . dist)` readings for the map.
`car-set-leds!`	`(car-set-leds! pat)` — set the LED strip to pattern `pat`.

Also defined: `car-stop`, `car-leds-off`.

```
(demo-start)              ; connect, serve the bridge, and run the loop
;; ...or drive the car directly over the supervisor's LLIP link:
(car-move! 'forward 500)  ; forward for half a second
(car-turn! 90)            ; quarter turn in place
(car-sweep)               ; => ((angle . dist) ...)
```

7.31.2 Eval Sandbox (`demo-sandbox.scm`)

The sandbox is what makes it safe to let an anonymous web viewer type Scheme that runs on your robot. Every submitted expression is parsed and checked before it is forwarded to the car, and each client is rate-limited. The safety model has three layers:

- **Whitelist.** Only calls whose head is a symbol in `sandbox-whitelist` (a fixed set of robot verbs and sensor getters — `robot-move!`, `get-sonar`, `set-leds!`, ...) are permitted, and the check recurses into nested argument calls, so every call anywhere in the form must be whitelisted. Bare atoms are allowed as data.
- **Blacklist.** Heads in `sandbox-forbidden-heads` (`define`, `set!`, `load`, `eval`, `apply`, file/TCP open, `system`, `shell`, ...) are rejected outright — this closes the obvious escapes before the whitelist is even consulted.
- **Rate limit.** Each `client-id` may submit at most `sandbox-rate-limit` (default 3) expressions per `sandbox-rate-window-ms` (default 2000 ms) window, tracked in an alist.

`sandbox-eval` is the public entry point; `sandbox-safe?` is the reusable predicate.

Procedure	Description
`sandbox-eval`	`(sandbox-eval expr-string client-id car-conn now-ms)` — rate-check, parse, safety-check, then eval on the car. Returns `(ok result-string)` or `(error msg-string)`.
`sandbox-safe?`	`(sandbox-safe? expr)` — `#t` if a parsed form passes the whitelist/blacklist rules; useful on its own.

```
(sandbox-safe? '(robot-move! 'forward 500))   ; => #t
(sandbox-safe? '(load "evil.scm"))            ; => #f  (forbidden head)
(sandbox-safe? '(car (robot-pose)))           ; => #f  (car not whitelisted)
```

7.31.3 Telemetry & Floor Mapping (demo-telemetry.scm)

This layer turns raw sensor polls into a live occupancy-grid map. Telemetry is kept as a flat alist (sonar, battery, light, heading, pose-x/y, loop-ms, ncells); the map is a 100×100 flat vector of cells at 10 cm resolution, each 0 unknown, 1 free, or 2 occupied. Sonar sweeps are ray-cast from the car's current pose to mark free space and wall hits, and the grid can be rendered as ASCII for a web view.

Procedure	Description
telem-poll!	(telem-poll! car-conn now-ms) — one LLIP round-trip that reads all sensors and updates the telemetry alist. Errors are swallowed (the car may be momentarily busy).
grid-update-sweep!	(grid-update-sweep! sweep-result) — fold a sweep ((angle . dist) pairs) into the occupancy grid from the current pose/heading.
grid->ascii	(grid->ascii) — render the grid as an ASCII string (? unknown, . free, # wall).
telem->json	(telem->json) — render current telemetry as a JSON-like string for the web bridge.

```
(telem-poll! conn (elapsed-ms))       ; refresh sensor readings
(grid-update-sweep! (car-sweep))      ; fold a sonar sweep into the map
(display (grid->ascii))               ; print the map
```

7.31.4 Musical Notes (Music.scm)

A minimal note-table sketch for buzzer tunes: it defines the concert-pitch reference a4 (440, the frequency of A4) and notes, the twelve chromatic note names. It is a starting point for buzzer melodies rather than a finished player.

Definition	Description
a4	440 — reference frequency of A in octave 4.
notes	List of the twelve chromatic note-name symbols.

7.31.5 Personality (Personality.scm)

A one-line character sketch: Personality names the device's identity (here 'Freenove-4WD-Car). It is the smallest possible example of a per-target behavior/identity file that other demos or applications can key off of.

Definition	Description
Personality	The device's identity symbol, e.g. 'Freenove-4WD-Car.

Chapter 8

Worked Examples

8.1 Worked Example: Autonomous Floor Mapping with the Freenove 4WD Car

This chapter walks through a complete three-tier LambLisp application that uses the Freenove 4WD Car to map the free space in a room using its HC-SR04 ultrasonic sonar. The example is self-contained and intended to be run as-is; every code snippet is a fragment of a working file in `data/`.

The three tiers are:

Tier	Host	Role
Car	ESP32 (on the 4WD car)	Real-time sensor/actuator loop; autonomous gap-filling; map accumulation
Supervisor	Jetson Orin Nano	Mission orchestration; waypoint injection; map aggregation and display
Strategy	Cloud LLIP server (Linux)	Coverage-path planning; global map storage; mission replay

The tiers communicate exclusively over LLIP. The car runs as an LLIP server and never initiates a connection; the Jetson connects to the car and to the cloud server.

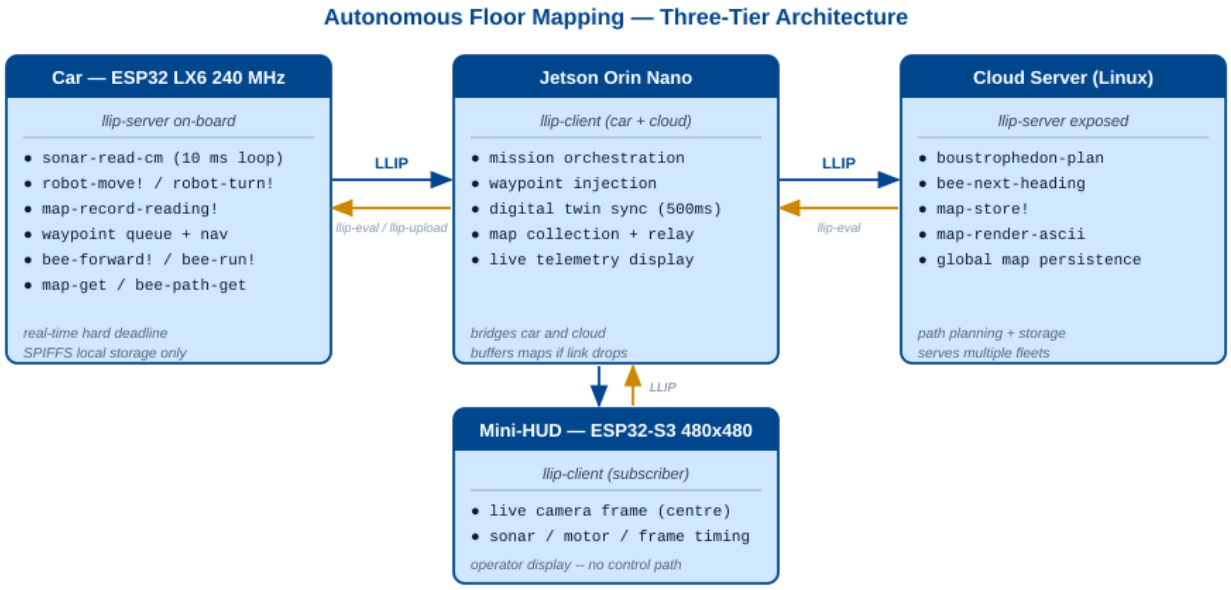


Figure 8.1 Three-Tier Architecture

8.1.1 1. Overview

The **floor-mapping mission** proceeds in three phases:

1. **Plan** — the cloud server uses the room outline (provided by the user) to compute a boustrophedon coverage path: a set of parallel strip waypoints that guarantee complete coverage.
2. **Execute** — the Jetson feeds waypoints to the car one at a time via LLIP. Between waypoints the car drives autonomously, taking a sonar reading at each step and storing `(x y dist-cm heading-deg)` tuples in an on-board map list. If the sonar detects an unexpected obstacle the car halts and waits for the Jetson to issue a new waypoint.
3. **Collect** — the Jetson periodically pulls the accumulated map list from the car and relays it to the cloud server, which merges it into a persistent grid and can serve it back to any connected display client.

A fourth, continuous thread runs on the Jetson: the **digital twin**, which mirrors live car state (speed, heading, battery, sonar) so that the supervisor can make decisions without polling the car in the middle of a move.

8.1.2 2. Hardware Setup

The only hardware modification required beyond the stock Freenove 4WD Car Kit is ensuring that the HC-SR04 sonar module is mounted facing forward on the pan servo bracket. The servo can be commanded to sweep left/right to widen the field of view; for the floor-mapping example the servo is fixed at 90° (forward-facing).

```
Room coordinate system:

        +Y (north)
         |
  ------+------ +X (east)
         |
    car starts at (0,0)
    facing +X (east = 0 deg)
```

All distances and positions are in centimetres. Headings are in degrees, measured counter-clockwise from east, matching standard mathematical convention.

Required LambLisp modules (loaded by `setup.scm` or `(load ...)` on demand):

File	Purpose
`llip-server.scm`	LLIP server (already loaded on car)
`robot.scm`	Motor helpers: `robot-move!`, `robot-turn!`, `robot-stop`
`sonar.scm`	HC-SR04 wrapper: `sonar-read-cm`
`floor-map.scm`	Map accumulator (new, defined below)
`llip-client.scm`	LLIP client (Jetson and cloud side)
`coverage.scm`	Boustrophedon planner (cloud side)

8.1.3 3. On-Board Car Code

3.1 Map Accumulator (`floor-map.scm`)

The map is a plain Scheme list of readings. Each reading is a four-element list `(x y dist-cm heading-deg)`. The car's current pose `(x y heading)` is maintained as mutable top-level variables so the Jetson can query them at any time via LLIP.

```
;;; floor-map.scm -- on-board map accumulator for the 4WD car
;;; Copyright 2026 by Frobenius Norm LLC

;;; Pose state -- updated by move! and turn! wrappers below.
(define car-x       0.0)    ; centimetres from origin
(define car-y       0.0)
(define car-heading 0.0)    ; degrees CCW from east

;;; Accumulated readings: list of (x y dist-cm heading-deg).
(define floor-map '())

;;; Record one sonar reading at the current pose.
(define (map-record-reading!)
  (let ((d (sonar-read-cm)))
    (set! floor-map
          (cons (list car-x car-y d car-heading) floor-map))
    d))

;;; Clear the map and reset pose.
(define (map-reset!)
  (set! floor-map '())
  (set! car-x       0.0)
  (set! car-y       0.0)
  (set! car-heading 0.0))

;;; Return the map as a list (for LLIP retrieval).
(define (map-get) floor-map)

;;; Update pose after a forward move of dist-cm.
(define (pose-advance! dist-cm)
  (let ((rad (* car-heading (/ (* 4 (atan 1)) 180))))
    (set! car-x (+ car-x (* dist-cm (cos rad))))
    (set! car-y (+ car-y (* dist-cm (sin rad))))))

;;; Update pose after a turn of delta-deg (CCW positive).
(define (pose-turn! delta-deg)
  (set! car-heading (modulo (+ car-heading delta-deg) 360)))
```

3.2 Motion Primitives

The motion wrappers combine hardware commands with pose tracking and sonar sampling. A sonar reading is taken before every forward move; if an obstacle is closer than `map-obstacle-cm` the move is cancelled and `'obstacle` is returned.

```
;;; map-obstacle-cm -- tunable threshold; default 20 cm.
(define map-obstacle-cm 20)

;;; Move forward step-cm, recording a sonar reading first.
;;; Returns 'moved or 'obstacle.
(define (map-step-forward! step-cm)
  (let ((d (map-record-reading!)))
    (if (< d map-obstacle-cm)
        'obstacle
        (begin
          (robot-move! 'forward (round (* step-cm 10)))  ; ms ~= cm * 10
          (pose-advance! step-cm)
          'moved))))

;;; Turn in place, updating heading.
(define (map-turn! delta-deg)
  (robot-turn! delta-deg)
  (pose-turn! delta-deg))
```

3.3 Autonomous Fill Behaviour

When no waypoint is pending the car enters **autonomous fill** mode: it continues taking sonar readings and nudging forward in small steps, building map density around its current position. If it hits an obstacle it rotates in 10° increments until it finds a clear path, or waits if no path exists within a 360° sweep.

This behaviour runs inside the car's `loop` procedure and yields to the Jetson whenever a new waypoint arrives.

```
;;; Autonomous fill step -- called from loop when no waypoint is pending.
;;; Returns 'ok, 'obstacle-all-around, or 'at-waypoint.
(define (map-fill-step!)
  (let ((result (map-step-forward! 5)))  ; 5 cm per fill step
    (if (eq? result 'moved)
        'ok
        ;; Try rotating to find a clear direction.
        (let search ((tries 36))
          (if (= tries 0)
              'obstacle-all-around
              (begin
                (map-turn! 10)
                (if (eq? (map-step-forward! 5) 'moved)
                    'ok
                    (search (- tries 1)))))))))

;;; Waypoint queue -- list of (x y) targets, set by the Jetson via LLIP.
(define waypoint-queue '())

(define (waypoint-pending?) (not (null? waypoint-queue)))

(define (waypoint-next!)
  (let ((wp (car waypoint-queue)))
    (set! waypoint-queue (cdr waypoint-queue))
    wp))

;;; Add a waypoint (called remotely by the Jetson).
(define (waypoint-push! x y)
  (set! waypoint-queue
        (append waypoint-queue (list (list x y)))))
```

3.4 Waypoint Navigation

Navigating to a waypoint uses a simple proportional heading controller: compute the bearing to the target, turn to face it, then drive forward in steps until the car is within `map-arrival-cm` of the target.

```
(define map-arrival-cm 10)   ; close-enough radius

(define (bearing-to x y)
  (let* ((dx (- x car-x))
         (dy (- y car-y))
         (b  (* (atan dy dx) (/ 180 (* 4 (atan 1))))))
    (modulo b 360)))

(define (distance-to x y)
  (let ((dx (- x car-x))
        (dy (- y car-y)))
    (sqrt (+ (* dx dx) (* dy dy)))))

;;; Navigate to (tx ty).  Returns 'arrived or 'blocked.
(define (map-navigate-to! tx ty)
  (let loop ()
    (let ((d (distance-to tx ty)))
      (if (< d map-arrival-cm)
          'arrived
          (let* ((bearing (bearing-to tx ty))
                 (delta   (let ((raw (- bearing car-heading)))
                            (cond ((> raw  180) (- raw 360))
                                  ((< raw -180) (+ raw 360))
                                  (else raw)))))
            (when (> (abs delta) 5)
```

```
            (map-turn! delta))
          (let ((result (map-step-forward! 10)))
            (if (eq? result 'obstacle)
                'blocked
                (loop))))))))

;;; Main loop procedure -- replaces the application's (loop).
(define (loop)
  (if (waypoint-pending?)
      (let* ((wp     (waypoint-next!))
             (tx     (car wp))
             (ty     (cadr wp))
             (result (map-navigate-to! tx ty)))
        (when (eq? result 'blocked)
          (set! waypoint-queue (cons wp waypoint-queue))))  ; re-queue
      (map-fill-step!)))
```

3.5 Bee Strategy Module

The **bee strategy** is a reactive alternative to the pre-planned boustrophedon path. The car drives forward until it hits an obstacle, then asks the cloud strategy server for the next heading. It records every unblocked cell it traverses — a separate list from the sonar-obstacle map — so the cloud can choose the next heading that biases toward unexplored territory.

```
;;; bee.scm -- reactive coverage for the 4WD car
;;; Copyright 2026 by Frobenius Norm LLC

;;; Traversed (unblocked) cells -- separate from the sonar obstacle map.
(define bee-path '())

(define (bee-path-record!)
  (set! bee-path (cons (list car-x car-y) bee-path)))

(define (bee-path-get) bee-path)

(define (bee-path-reset!)
  (set! bee-path '()))

;;; One forward step in bee mode.
;;; Returns 'moved or 'blocked.
(define (bee-forward! step-cm)
  (let ((d (map-record-reading!)))
    (if (< d map-obstacle-cm)
        'blocked
        (begin
          (robot-move! 'forward (round (* step-cm 10)))
          (pose-advance! step-cm)
          (bee-path-record!)
          'moved))))

;;; Run the bee strategy for n-steps total forward moves.
;;; After each obstacle the car asks the cloud for a new heading.
(define (bee-run! cloud n-steps step-cm)
  (let loop ((remaining n-steps))
    (when (> remaining 0)
      (when (eq? (bee-forward! step-cm) 'blocked)
        (let* ((new-h  (llip-eval cloud
                         `(bee-next-heading ,car-heading ',bee-path)))
               (delta  (let ((raw (- new-h car-heading)))
                         (cond ((> raw  180) (- raw 360))
                               ((< raw -180) (+ raw 360))
                               (else raw)))))
          (map-turn! delta)))
      (loop (- remaining 1)))))
```

8.1.4 4. Jetson Supervisor

The Jetson runs `data/floor-map-supervisor.scm`. It connects to both the car and the cloud server, requests a coverage plan, and feeds it to the car as a waypoint stream. It also runs the digital twin sync loop in parallel using LambLisp's **timed-queue** deferred task facility.

4.1 Connections

```
;;; floor-map-supervisor.scm -- Jetson side
;;; Copyright 2026 by Frobenius Norm LLC

(load "llip-client.scm" 0)

;;; Connect to the car and the cloud strategy server.
(define car    (llip-connect "192.168.1.100" 5555))
(define cloud  (llip-connect "llip.example.com" 5555))

;;; Upload the on-board modules and reset the map.
(llip-upload car "floor-map.scm" "/floor-map.scm")
(llip-eval   car '(load "/floor-map.scm"))
(llip-eval   car '(map-reset!))
(display "Car ready\n")
```

4.2 Requesting a Coverage Plan

The cloud server runs a boustrophedon planner that accepts the room outline as a list of `(x y)` corner coordinates and returns a list of `(x y)` waypoints.

```
;;; Describe the room outline (clockwise, centimetres from origin).
(define room-outline
  '((0 0) (400 0) (400 300) (0 300)))

;;; Ask the cloud for a complete coverage plan.
;;; Returns a list of (x y) waypoints.
(define coverage-plan
  (llip-eval cloud `(boustrophedon-plan ',room-outline 40)))
                                          ;; 40 cm strip width

(display (string-append "Plan: " (number->string (length coverage-plan))
                        " waypoints\n"))
```

4.3 Feeding Waypoints to the Car

Waypoints are pushed to the car's queue via `waypoint-push!`. The Jetson sends them in batches: a new batch is sent whenever the queue drains below a low-water mark, allowing the car's autonomous fill behaviour to run between waypoints while the Jetson is computing the next batch.

```
(define wp-low-water 3)   ; refill when fewer than this many waypoints remain

(define (queue-length!)
  (llip-eval car '(length waypoint-queue)))

;;; Push one waypoint to the car.
(define (push-waypoint! wp)
  (llip-eval car `(waypoint-push! ,(car wp) ,(cadr wp))))

;;; Feed the full plan, refilling the queue as it drains.
(define (run-mission! plan)
  (let loop ((remaining plan))
    (unless (null? remaining)
      (let ((qlen (queue-length!)))
        (when (< qlen wp-low-water)
```

```
          (let refill ((wps remaining) (sent 0))
            (if (or (null? wps) (= sent 5))  ; send up to 5 at a time
                (set! remaining wps)
                (begin
                  (push-waypoint! (car wps))
                  (refill (cdr wps) (+ sent 1))))))
        (loop remaining)))))

(run-mission! coverage-plan)
```

4.4 Digital Twin Sync

A timed task runs every 500 ms on the Jetson, pulling live telemetry from the car and updating a local twin dict. The twin is available for query at any time without blocking the mission loop.

```
;;; Local twin state -- updated by sync-twin! every 500 ms.
(define twin
  (list->dict '((x . 0.0) (y . 0.0) (heading . 0.0)
                (sonar . 0.0) (battery . 100.0) (loop-ms . 0))))

(define (sync-twin!)
  (dict-rebind-bang twin 'x       (llip-eval car 'car-x))
  (dict-rebind-bang twin 'y       (llip-eval car 'car-y))
  (dict-rebind-bang twin 'heading (llip-eval car 'car-heading))
  (dict-rebind-bang twin 'sonar   (llip-eval car '(sonar-read-cm)))
  (dict-rebind-bang twin 'battery (llip-eval car '(get-battery))))

;;; Schedule twin sync every 500 ms using the timed queue.
(tq-repeat! 500 sync-twin!)
```

4.5 Collecting the Map

After the mission completes the Jetson retrieves the full map from the car and relays it to the cloud for storage and visualisation.

```
;;; Pull the completed map from the car.
(define raw-map (llip-eval car '(map-get)))

(display (string-append "Map: " (number->string (length raw-map))
                        " readings\n"))

;;; Send the map to the cloud for storage and rendering.
(llip-eval cloud `(map-store! 'floor-run-1 ',raw-map))

;;; Request an ASCII render for quick inspection.
(display (llip-eval cloud '(map-render-ascii 'floor-run-1 80 24)))
```

8.1.5 5. Cloud Strategy Server

The cloud server is a Linux process running a LambLisp LLIP server. It provides the boustrophedon planner and the map store. Any number of Jetson supervisors can connect simultaneously; maps from different runs accumulate in a global dict keyed by run name.

5.1 Boustrophedon Coverage Planner

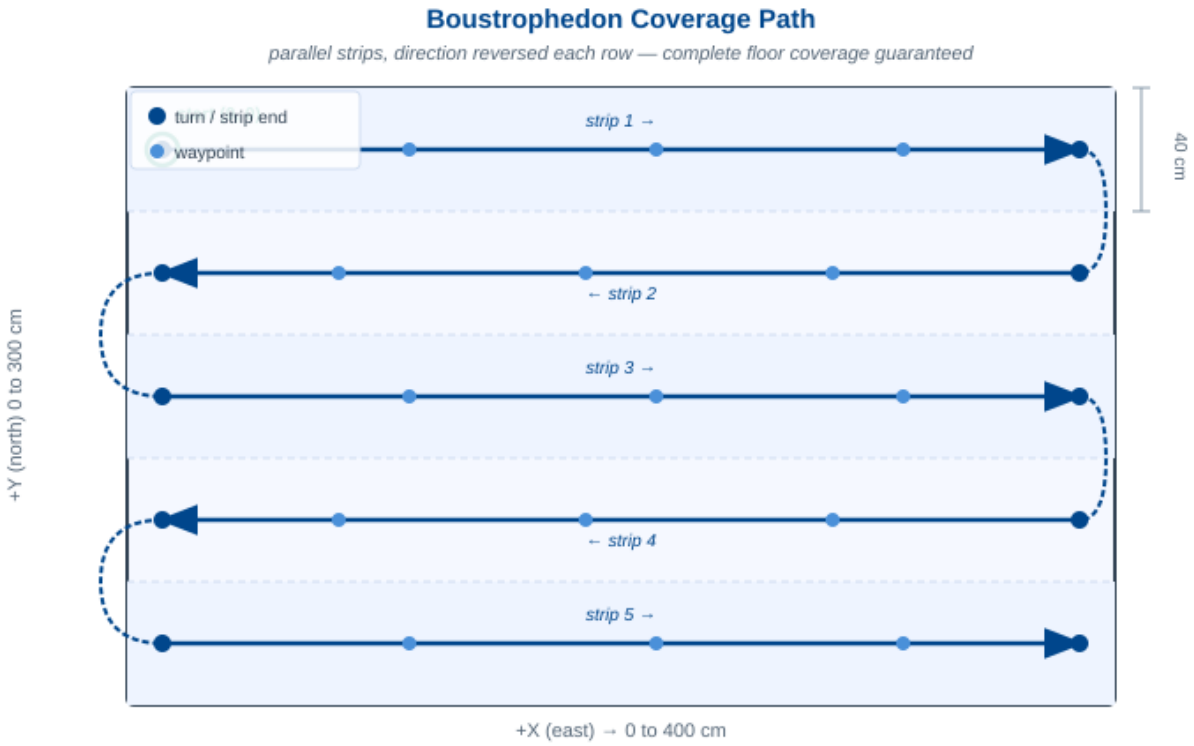


Figure 8.2 Boustrophedon Coverage Path

A boustrophedon path covers a rectangular area in parallel horizontal strips, reversing direction at the end of each strip. The planner takes the room outline and strip width and returns a list of `(x y)` waypoints.

For this example the room outline is assumed to be axis-aligned; a full polygon planner would decompose non-convex rooms into monotone cells first.

```
;;; coverage.scm -- boustrophedon coverage planner (cloud side)
;;; Copyright 2026 by Frobenius Norm LLC

;;; Compute axis-aligned bounding box of a polygon.
(define (bbox corners)
  (let ((xs (map car  corners))
        (ys (map cadr corners)))
    (list (apply min xs) (apply min ys)
          (apply max xs) (apply max ys))))

;;; Generate boustrophedon waypoints for the bounding box.
;;; strip-cm: strip width in centimetres.
;;; Returns a flat list of (x y) waypoints.
(define (boustrophedon-plan corners strip-cm)
```

```
  (let* ((box  (bbox corners))
         (x0   (+ (car  box) (/ strip-cm 2)))
         (y0   (+ (cadr box) (/ strip-cm 2)))
         (x1   (- (caddr  box) (/ strip-cm 2)))
         (y1   (- (cadddr box) (/ strip-cm 2)))
         (wps  '()))
    (let strip-loop ((y y0) (left->right #t))
      (when (<= y y1)
        (let row-loop ((x (if left->right x0 x1)))
          (when (if left->right (<= x x1) (>= x x0))
            (set! wps (cons (list x y) wps))
            (row-loop (if left->right (+ x strip-cm) (- x strip-cm)))))
        (strip-loop (+ y strip-cm) (not left->right))))
    (reverse wps)))
```

5.2 Map Store and ASCII Render

Maps are stored as association lists keyed by run name. The ASCII renderer projects the `(x y dist-cm)` readings onto a character grid, marking obstacle hits with `#` and free space with `.`.

```
;;; Global map store: dict of run-name -> readings list.
(define map-store (list->dict '()))

(define (map-store! run-name readings)
  (dict-bind-bang map-store (string->symbol (symbol->string run-name))
                  readings map-store))

(define (map-get-run run-name)
  (dict-ref map-store (string->symbol (symbol->string run-name))))

;;; ASCII render: cols x rows character grid.
(define (map-render-ascii run-name cols rows)
  (let* ((readings (map-get-run run-name))
         (xs   (map car  readings))
         (ys   (map cadr readings))
         (x0   (apply min xs))  (x1 (apply max xs))
         (y0   (apply min ys))  (y1 (apply max ys))
         (xrng (max 1 (- x1 x0)))
         (yrng (max 1 (- y1 y0)))
         (grid (make-vector (* cols rows) #\.)))
    (for-each
      (lambda (r)
        (let* ((rx (car r))  (ry (cadr r))  (rd (caddr r))
               (col (inexact->exact (round (* (/ (- rx x0) xrng) (- cols 1)))))
               (row (inexact->exact (round (* (/ (- ry y0) yrng) (- rows 1)))))
               (idx (+ (* (- rows 1 row) cols) col)))
          (vector-set! grid idx (if (< rd 20) #\# #\.))))
      readings)
    (let build ((i 0) (out ""))
      (if (= i (* cols rows))
          out
          (build (+ i 1)
                 (string-append out
                   (string (vector-ref grid i))
                   (if (= (modulo (+ i 1) cols) 0) "\n" ""))))))))
```

5.3 Bee Strategy Server

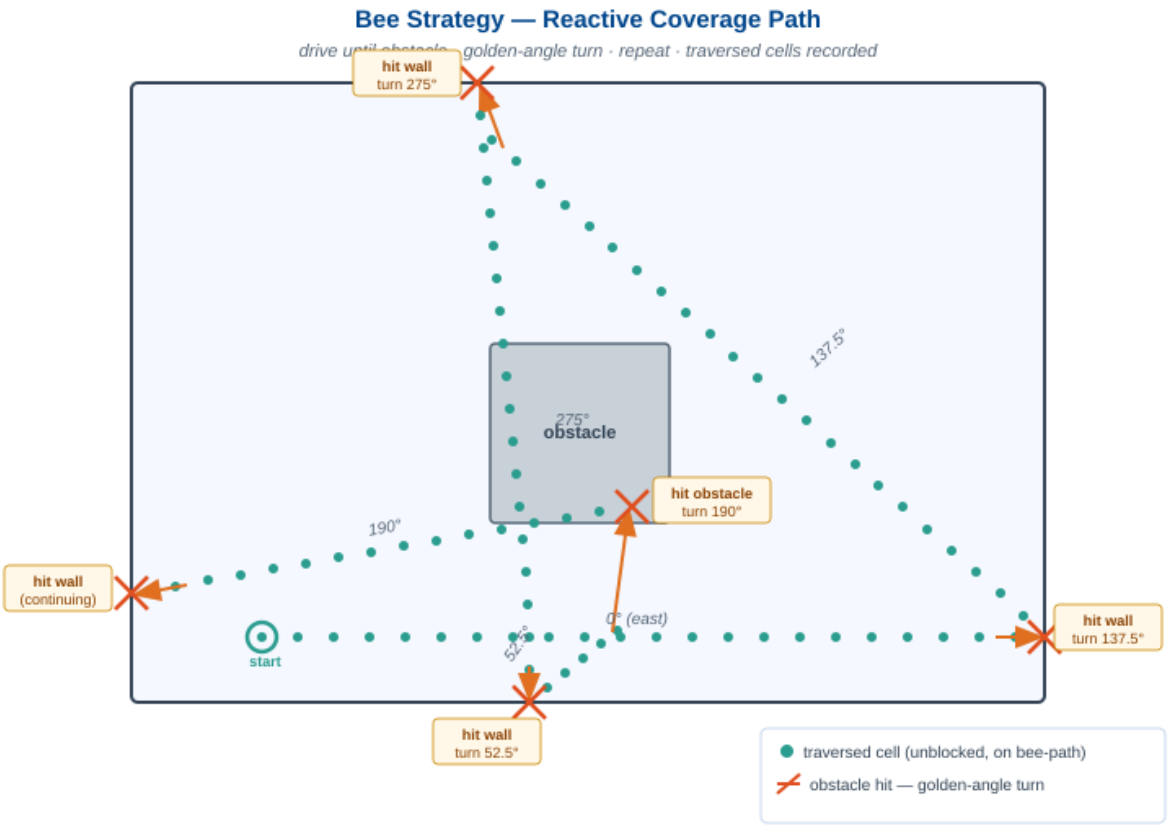


Figure 8.3 Bee Strategy Path

The bee strategy server exposes a single function `bee-next-heading` that the car calls after every obstacle hit. It receives the car's current heading and the full list of traversed (unblocked) cells, and returns the next heading in degrees.

Heading selection uses the **golden angle** (360 / phi^2 = 137.5 degrees) as the base increment. Rotating by the golden angle on each successive call distributes headings evenly around the circle — no direction is revisited until all others have been tried. On top of that a spatial check avoids headings that point back toward already-dense areas of the bee-path, biasing the car toward unexplored territory.

```
;;; bee-strategy.scm -- heading selection for bee coverage (cloud side)
;;; Copyright 2026 by Frobenius Norm LLC

;;; Golden angle: 360 / phi^2 ~= 137.508 degrees.
;;; Successive rotations by this amount distribute evenly over the circle
;;; without ever repeating a direction until the full circle is covered.
(define bee-golden-angle 137.508)

;;; Absolute angular difference between two headings (0-180).
(define (heading-diff a b)
  (let ((d (abs (- (modulo a 360) (modulo b 360)))))
    (min d (- 360 d))))

;;; Count how many path cells lie within cone-deg of heading from origin.
(define (cells-in-cone heading cone-deg path)
  (length
    (filter (lambda (p)
              (< (heading-diff (bearing-to (car p) (cadr p)) heading)
```

```
                cone-deg))
            path)))

;;; Select the next heading.
;;; Tries golden-angle increments 1..12; returns the first candidate
;;; whose 30-degree forward cone contains fewer than 4 visited cells.
;;; Falls back to the 1x golden-angle rotation if all cones are dense.
(define (bee-next-heading current-heading path)
  (let try ((n 1))
    (let ((candidate (modulo (+ current-heading (* n bee-golden-angle)) 360)))
      (if (or (> n 12)
              (< (cells-in-cone candidate 30 path) 4))
          candidate
          (try (+ n 1))))))
```

The 12-step limit means the algorithm tries a full 1650 degrees of rotation (12 x 137.5) before giving up, which reliably finds an unvisited sector in any room that is not completely surrounded by walls.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

8.1.6 6. Running the Example

6.1 Start the Cloud Server

On any Linux host with LambLisp built for `linux_x86_64`:

```
# Upload the cloud-side files and start the LLIP server.
pio run -e linux_x86_64 -t exec -- \
    --load coverage.scm --load bee-strategy.scm \
    --load llip-server.scm --server 5555
```

The server accepts connections on port 5555 and exposes `boustrophedon-plan`, `bee-next-heading`, `map-store!`, and `map-render-ascii` to any LLIP client. Both planners are loaded simultaneously; the supervisor chooses which to use.

6.2 Flash and Start the Car

Flash the ESP32 with the standard Freenove 4WD build environment:

```
pio run -e Freenove-4WD-Car-Kit-ESP32
pio run -e Freenove-4WD-Car-Kit-ESP32 --target uploadfs
pio run -e Freenove-4WD-Car-Kit-ESP32 --target upload
```

On boot, `setup.scm` loads `robot.scm`, `sonar.scm`, and `llip-server.scm` and starts the control loop. The car is now reachable at its WiFi IP address on port 5555.

6.3 Run the Supervisor on the Jetson

```
ssh jetson
cd ~/Lamb-v01
pio run -e linux_aarch64 -t exec -- --load floor-map-supervisor.scm
```

The supervisor prints progress as it connects, retrieves the plan, and feeds waypoints to the car:

```
Car ready
Plan: 48 waypoints
 ... mission running ...
Map: 312 readings
+------------------------------------------------------------------------+
|.....................................................#####............|
|.....................................................#########........|
|.................................................###############........|
|.....................................##.....####################.....|
|....................................####.....######################....|
|..............................#########.....########################..|
|             (west wall)                       (obstacle cluster)    |
+------------------------------------------------------------------------+
```

6.4 Tuning Parameters

All key parameters are top-level Scheme variables and can be adjusted live from the Jetson without reflashing the car:

```
;;; From the Jetson REPL or any LLIP client:

;;; Widen the obstacle clearance zone.
(llip-eval car '(set! map-obstacle-cm 30))

;;; Slow down the coverage drive speed (ms per cm).
(llip-eval car '(set! map-step-ms-per-cm 15))

;;; Increase sonar averaging.
(llip-eval car '(set! sonar-samples 5))

;;; Inject a manual waypoint to investigate an area of interest.
(llip-eval car '(waypoint-push! 200 150))

;;; Switch to bee strategy mid-mission (no reflash needed).
;;; Upload bee.scm, reset bee state, then run 200 steps at 5 cm each.
(llip-upload car "bee.scm" "/bee.scm")
(llip-eval   car '(load "/bee.scm"))
(llip-eval   car '(bee-path-reset!))
(llip-eval   car `(bee-run! ,cloud 200 5))

;;; Tune the golden-angle cone sensitivity on the cloud side.
(llip-eval cloud '(set! bee-golden-angle 120.0))  ; denser spiral
```

8.1.7 7. Architecture Notes

7.1 Why Three Tiers?

The three-tier split follows naturally from where compute, latency, and connectivity requirements differ:

Concern	Car (ESP32)	Jetson	Cloud
Real-time loop	yes (hard deadline)	no	no
Sensor/actuator I/O	yes	no	no
Mission orchestration	no	yes	no
Map aggregation	partial	yes	yes
Path planning	no	capable	yes (preferred)
Persistent storage	no	local	yes (global)
Multi-fleet visibility	no	no	yes

The car's 240 MHz Xtensa LX6 core is powerful enough to run the full LambLisp stack and real-time sensor loops simultaneously, but its SPIFFS filesystem is not suited to persistent large-map storage. The Jetson bridges the gap: it has a full Linux filesystem and can buffer map data indefinitely even if the cloud link is intermittent.

7.2 LLIP as the Universal Glue

Every inter-tier call in this example is a plain `llip-eval`. There is no custom protocol, no serialisation library, and no schema. A list is a list whether it lives on the ESP32, the Jetson, or the cloud server. Updating the map format — adding a timestamp or a confidence value to each reading — requires changing one line in `floor-map.scm` and nothing else.

7.3 Extending the Example

Natural extensions to this worked example include:

- **Multi-robot coordination** — two or more cars upload maps to the same cloud run; the planner partitions the coverage path between them and prevents redundant work.
- **Obstacle classification** — replace the binary obstacle/free reading with a machine-learning classifier running on the Jetson that labels obstacle types (wall, furniture, person) based on sonar return profiles.
- **Replanning on the fly** — if the car reports `'obstacle-all-around`, the Jetson requests a new partial plan from the cloud starting at the car's current pose, circumventing the blocked area.
- **Persistent map merging** — subsequent runs load the previous cloud map and bias the coverage planner toward unexplored or low-density areas, building up a progressively more detailed map over multiple sessions.

See also: LLIP — LambLisp Interaction Protocol · Digital Twin · Freenove 4WD Car Kit — Peripheral Summary · Ultrasonic Sonar (HC-SR04)

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 9

LambLisp Compatibility Matrix

9.1 Legend

Symbol	Meaning
Y	Fully supported
~	Partial – implemented but incomplete, or stubbed to return `#<undef>`
N	Not supported
--	Not defined by this specification
ext	LambLisp extension (not in any Scheme standard)

9.2 Coverage Summary

One row per section. Y and ext count as covered; ~ as partial; N as missing.

Section	R5RS defined	R7RS adds	LambLisp covers	Gaps
Language (syntax, binding, macros)	core	extensions incl. `case-lambda`	~ most, incl. `case-lambda`, `cond-expand` and `parameterize`	`define-library/export`
Type predicates	full	`null?` `bytevector?`	Y all	–
Numbers	integer, real, rational, complex	`exact-integer?` `finite?` `infinite?` `nan?`	Y integer, real, rational, complex (native types), all predicates	–
Pairs & Lists	full	`make-list` `list-copy`	Y full + extras	–
Characters	full	`char-ci` `digit-value` `char-foldcase`	Y all	–
Strings	full	`string-ci` `string-copy!` `string-map` `string-foldcase`	Y all	–
Vectors	full	`vector->string` `vector-append` `vector-map`	Y full + extras	–
Bytevectors	–	full	Y full	–
Control features	`apply` `map` `for-each` `values` `call/cc` `dynamic-wind`	string/vector map/for-each	~ all; `call/cc` is escape-only	re-entrant continuations (generators, coroutines)
Environments & eval	full	–	Y full	–
Ports	partial	full	Y full	–
I/O	partial	full	Y full, incl. datum-label output for shared and circular structure	–
System interface	`load`	`file-exists?` `command-line` `exit`	Y `load`, time functions, file ops, `command-line`, `exit`, `features`	–
Exception handling	–	full	Y full	`raise-continuable` resumes at the raise point

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Section	R5RS defined	R7RS adds	LambLisp covers	Gaps
LambLisp extensions	–	–	ext	OTA updates

9.3 Detailed Compatibility

9.3.1 Core Language

Feature	R5RS	R7RS	LambLisp	Notes
Procedures				
`define lambda`	Y	Y	Y	
`nlambda`	--	--	ext	LambLisp/InterLisp extension
`macro`	--	--	ext	LambLisp extension
Conditionals				
`if cond case and or not else`	Y	Y	Y	
`when unless`	--	Y	Y	
`cond-expand`	--	Y	Y	
`case-lambda`	--	Y	Y	Generates dispatch lambda at expansion time; rest-arg formals supported
Binding forms				
`set! let let* letrec letrec* =>`	Y	Y	Y	
Named `let`	Y	Y	Y	
`do`	Y	Y	Y	
Syntax definition				
`define-syntax`	Y	Y	Y	Evaluates transformer spec; binds in current scope only

Feature	R5RS	R7RS	LambLisp	Notes
`let-syntax` `letrec-syntax`	Y	Y	Y	`let-syntax` evals transformers in outer env; `letrec-syntax` in new env
`syntax-rules`	Y	Y	Y	All tests pass
`syntax-error`	--	Y	Y	Raises as a runtime error; syntax context not tracked
Multiple values				
`values` `call-with-values`	Y	Y	Y	
`let-values` `let*-values` `define-values`	--	Y	Y	
Quotation				
`quote` `quasiquote` `unquote` `unquote-splicing`	Y	Y	Y	
Reader abbreviations `' ` , ,@`	Y	Y	Y	
Sequencing				
`begin`	Y	Y	Y	
Delayed evaluation				
`delay` `force`	Y	Y	Y	Implemented in Scheme
`delay-force` `promise?` `make-promise`	--	Y	Y	Implemented in Scheme
Dynamic binding				
`make-parameter` `parameterize`	--	Y	Y	
File inclusion				
`include`	--	Y	Y	Via `load`
`include-ci`	--	Y	Y	Loads with `char-foldcase` applied to symbols
Libraries / modules				
`define-library` `import` `export`	--	Y	~	`import` is supported; `define-library` and `export` are not
Records				

Feature	R5RS	R7RS	LambLisp	Notes
`define-record-type`	--	Y	Y	Implemented in Scheme; dict-backed records, O(1) field access
Continuations				
`call-with-current-continuation`	Y	Y	~	Escape-only (upward): a captured continuation may be invoked to RETURN from its `call/cc`, not to re-enter it afterwards. Generators and coroutines are not supported. **An enclosing `guard` with a catch-all clause will swallow the escape** – see below.
`dynamic-wind`	Y	Y	Y	Before/after thunks run on normal exit and on an escape

A catch-all `guard` between a `call/cc` and its `k` will capture the escape instead of letting it out. This is inherent to how escape continuations are built, not a defect, and it is the one call/cc behaviour most likely to be mistaken for a bug.

An escape is implemented as a *tagged raise*: invoking `k` raises a uniquely-tagged value, and `call/cc` installs a handler that recognises its own tag and returns the value. Any exception handler that catches **everything** sits in that path and gets there first:

```
(call/cc (lambda (k) (+ 1 (k 42))))                    ;=> 42     -- the escape lands

(call/cc (lambda (k)
           (guard (e (#t (list 'caught e)))             ; catch-all
             (k 42))))                                  ;=> (caught ((cont) . 42))
```

The second form does not return 42 and does not error. The handler runs with the escape's own internal token as its condition – `((cont) . 42)` above – so the symptom is a `guard` clause firing for an exception the program never raised, carrying a value that appears in no source file. Recognise that shape and the cause is immediate.

Avoid it by not catching what you did not raise. Match specific conditions rather than using `#t`/`else` as a clause, or move the `guard` so it does not sit between the `call/cc` and the point where `k` is invoked. `raise` and `error` are unaffected – only a clause that catches *everything* intercepts an escape.

9.3.2 Type System

Feature	R5RS	R7RS	LambLisp	Notes
Type predicates				
`boolean? char? number? symbol?`	Y	Y	Y	
`pair? vector? procedure? string?`	Y	Y	Y	

Feature	R5RS	R7RS	LambLisp	Notes
`port? eof-object?`	Y	Y	Y	
`null?`	Y	Y	Y	
`bytevector?`	--	Y	Y	
`atom?`	--	--	ext	
Equivalence				
`eq? eqv? equal?`	Y	Y	Y	
Datum labels				
`#0= #0#`	--	Y	~	Partially supported by reader

9.3.3 The Numeric Tower

Scheme defines a numeric tower with four levels, each a superset of the level below:

```
integer  ->  rational  ->  real  ->  complex       (each a subset of the next)
```

In LambLisp all four levels are **native VM cell types**:

Type	Cell	Storage	Exactness
Integer	`T_INT32`, `T_INT64`	`LL_int32` in car; a `T_INT64` spans car+cdr where a word is 32 bits	always exact
Real	`T_FLOAT32`, `T_FLOAT64`	`LL_float32` in car; a `T_FLOAT64` spans car+cdr where a word is 32 bits	always inexact
Rational	`T_RATIONAL`	`(numerator . denominator)`, GCD-reduced, denominator >= 2. Each part is an embedded `LL_int32`, or a reference to a boxed `T_INT64`/`T_BIGNUM` when it does not fit	always exact
Complex	`T_COMPLEX`	`(real . imaginary)` as an `LL_float32` pair	always inexact

The pair notation in the Storage column describes **storage**, not type: a rational and a complex number are *atoms* to your program. `(pair? 1/2)` is `#f` and `(car 1/2)` is an error — see Cell Slots and What the Collector Traverses.

The standard predicates `integer?`, `rational?`, `real?`, `complex?`, `exact?`, and `inexact?` all dispatch on these native types. Coercion follows the tower upward automatically: integer arithmetic stays exact; mixing exact and inexact promotes to inexact; `exact->inexact` and `inexact->exact` cross the boundary explicitly.

Higher-level arithmetic for rational and complex numbers is provided in the loadable libraries:

- `rxrs_rational.scm` — `rationalize`, arithmetic operators extended to handle `T_RATIONAL` values
- `rxrs_complex.scm` — complex arithmetic, `make-rectangular`, `make-polar`, `magnitude`, `angle`

For embedded applications where rational or complex numbers are not required, these libraries need not be loaded.

9.3.4 Numbers

Feature	R5RS	R7RS	LambLisp	Notes
Types				
Integer	Y	Y	Y	
Real (inexact float)	Y	Y	Y	
Rational (exact fractions)	Y	Y	Y	Native `T_RATIONAL` cell type; GCD-reduced `(numerator . denominator)` pair. Extended arithmetic in loadable `rxrs_rational.scm` (`rationalize`, `gcd`, `lcm`).
Complex	Y	Y	Y	Native `T_COMPLEX` cell type; single-precision `(real . imaginary)` float pair. Extended arithmetic in loadable `rxrs_complex.scm`; rectangular and polar construction, `magnitude`, `angle`, `sqrt` for negative reals.
Exactness				
`exact? inexact?`	Y	Y	Y	
`exact->inexact inexact->exact`	Y	--	Y	R5RS names
`exact inexact`	--	Y	Y	R7RS names (aliases)
`exact-integer?`	--	Y	Y	
Numeric predicates				
`number? integer? real?`	Y	Y	Y	
`rational? complex?`	Y	Y	Y	

Feature	R5RS	R7RS	LambLisp	Notes
`zero? positive? negative? odd? even?`	Y	Y	Y	
`finite? nan?`	--	Y	Y	
`infinite?`	--	Y	Y	
Arithmetic				
`+ - * / abs max min`	Y	Y	Y	
`< <= = >= >`	Y	Y	Y	
`quotient remainder modulo`	Y	Y	Y	
`expt log square sqrt`	Y	Y	Y	
`sin cos tan asin acos atan`	Y	Y	Y	
`floor` (single-arg)	Y	Y	Y	Returns exact integer from inexact input (R5RS compliant)
`ceiling`	Y	Y	Y	Returns exact integer
`truncate`	Y	Y	Y	Returns exact integer
`round`	Y	Y	Y	Banker's rounding (half-to-even) per R5RS
`floor/ floor-quotient floor-remainder`	--	Y	Y	Integer args; `floor/` returns two values via `values`
`truncate/ truncate-quotient truncate-remainder`	--	Y	Y	Integer args; identical to `quotient`/`remainder`
`gcd lcm`	Y	Y	Y	
`numerator denominator`	Y	Y	Y	
`rationalize`	Y	Y	Y	Stern-Brocot simplest-in-interval algorithm
`exact-integer-sqrt`	--	Y	Y	Returns two values via `values`
`real-part imag-part magnitude angle`	Y	Y	Y	

Feature	R5RS	R7RS	LambLisp	Notes
`make-rectangular make-polar`	Y	Y	Y	
`number->string string->number`	Y	Y	Y	

9.3.5 Symbols

Feature	R5RS	R7RS	LambLisp	Notes
`symbol? symbol->string string->symbol`	Y	Y	Y	
`symbol=?`	--	Y	Y	
`gensym`	--	--	ext	Used internally by `syntax-rules` hygiene

9.3.6 Pairs and Lists

Feature	R5RS	R7RS	LambLisp	Notes
`pair? cons car cdr set-car! set-cdr!`	Y	Y	Y	
`null? list?`	Y	Y	Y	
`atom?`	--	--	ext	
`list`	Y	Y	Y	
`make-list`	--	Y	Y	
`caar` ... `cddddr` (all compositions)	Y	Y	Y	
`length`	Y	Y	Y	
`append reverse`	Y	Y	Y	

Feature	R5RS	R7RS	LambLisp	Notes
`reverse!`	--	--	ext	Destructive in-place reversal
`list-tail list-ref list-set!`	Y	Y	Y	
`list-copy`	--	Y	Y	Preserves improper list tail
`memq memv member`	Y	Y	Y	
`assq assv assoc`	Y	Y	Y	
`vector->alist alist->vector`	--	--	ext	

9.3.7 Characters

Feature	R5RS	R7RS	LambLisp	Notes
`char?`	Y	Y	Y	
`char=? char<? char>? char<=? char>=?`	Y	Y	Y	
`char-ci=? char-ci<? char-ci>? char-ci<=? char-ci>=?`	Y	Y	Y	
`char-alphabetic? char-numeric? char-whitespace?`	Y	Y	Y	
`char-upper-case? char-lower-case?`	Y	Y	Y	
`char-upcase char-downcase`	Y	Y	Y	
`char-foldcase`	--	Y	Y	`tolower()` (ASCII); in `ll_vm_ai_↩ rxrs.cpp`
`char->integer integer->char`	Y	Y	Y	
`digit-value`	--	Y	Y	

9.3.8 Strings

Feature	R5RS	R7RS	LambLisp	Notes
`string?` `make-string` `string`	Y	Y	Y	
`string-length` `string-ref` `string-set!`	Y	Y	Y	
`string=?` `string<?` `string>?` `string<=?` `string>=?`	Y	Y	Y	
`string-ci=?` `string-ci<?` `string-ci>?` `string-ci<=?` `string-ci>=?`	Y	Y	Y	
`string-foldcase`	--	Y	Y	ASCII downcase loop; in `ll_vm_ai_rxrs.cpp`
`substring` `string-append`	Y	Y	Y	
`string->list` `list->string`	Y	Y	Y	
`string-copy`	Y	Y	Y	
`string-copy!`	--	Y	Y	
`string-fill!`	Y	Y	Y	

A string may contain NUL, and `string-length` is the authority

A LambLisp string stores its byte length and its character count explicitly rather than deriving either from a terminator, so a `#\null` may appear anywhere in a string, including in the middle, and `string-length` counts it like any other character:

```
(define s (list->string (list #\a (integer->char 0) #\b)))
(string-length s)          ; => 3   -- the NUL is a character, not an end marker
```

Printing is the documented exception. The payload is kept NUL-terminated so that a string can be handed to a C API without being copied first – that is what makes the FFI and the logging paths cheap on a microcontroller – and every path that crosses into C consequently stops at the first NUL. That covers `display` and `write`, the text of an error message, and any `%s` logging. So a string with an embedded NUL reports its true length and shows less than that when printed. This is a deliberate trade, not an oversight: the data stays intact and reachable, and only one representation of it is lossy.

> **Current limitation.** Several procedures that derive a length while building a new value do not yet read the stored count and so also stop at an embedded NUL. Until that is resolved, treat `string-length`, `string-ref`, `string-set!` and `string-fill!` as the operations that are NUL-clean, and do not rely on a NUL surviving `substring`, `string-copy`, `string->list`, `string->vector` or `string->utf8`.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

9.3.9 Vectors

Feature	R5RS	R7RS	LambLisp	Notes
`vector? make-vector vector`	Y	Y	Y	
`vector-length vector-ref vector-set!`	Y	Y	Y	
`vector-fill!`	Y	Y	Y	
`vector->list list->vector`	Y	Y	Y	
`vector-copy`	--	Y	Y	
`vector->string string->vector`	--	Y	Y	
`vector-append`	--	Y	Y	
`vector->alist alist->vector`	--	--	ext	

9.3.10 Bytevectors

Feature	R5RS	R7RS	LambLisp	Notes
`bytevector? make-bytevector bytevector`	--	Y	Y	
`bytevector-length`	--	Y	Y	
`bytevector-u8-ref bytevector-u8-set!`	--	Y	Y	
`bytevector-copy bytevector-copy! bytevector-append`	--	Y	Y	
`utf8->string string->utf8`	--	Y	Y	ASCII-only; multi-byte UTF-8 sequences not supported
`#u8(...)` literal	--	Y	Y	

9.3.11 Control Features

Feature	R5RS	R7RS	LambLisp	Notes
`procedure? apply`	Y	Y	Y	
`map for-each`	Y	Y	Y	Multi-list variants supported
`string-map string-for-each`	--	Y	Y	Multi-string variants supported
`vector-map vector-for-each`	--	Y	Y	Multi-vector variants supported
`values call-with-values`	Y	Y	Y	
`delay force`	Y	Y	Y	Implemented in Scheme
`delay-force promise? make-promise`	--	Y	Y	Implemented in Scheme
`call-with-current-continuation`	Y	Y	~	Escape-only; a catch-all `guard` between the `call/cc` and its `k` swallows the escape
`dynamic-wind`	Y	Y	Y	Before/after thunks run on normal exit and on an escape

9.3.12 Environments and Evaluation

Feature	R5RS	R7RS	LambLisp	Notes
`eval`	Y	Y	Y	
`scheme-report-environment`	Y	Y	Y	
`null-environment`	Y	Y	Y	
`interaction-environment`	Y	Y	Y	
`environment`	--	Y	Y	

9.3.13 Ports

Feature	R5RS	R7RS	LambLisp	Notes
`port? input-port? output-port?`	Y	Y	Y	
`textual-port? binary-port?`	--	Y	Y	
`input-port-open? output-port-open?`	--	Y	Y	
`current-input-port current-output-port current-error-port`	Y	Y	Y	
`close-port close-input-port close-output-port`	Y	Y	Y	
`open-input-file open-output-file`	Y	Y	Y	
`open-input-binary-file open-output-binary-file`	--	Y	Y	
`open-input-string`	--	Y	Y	
`open-output-string get-output-string`	--	Y	Y	
`open-input-bytevector open-output-bytevector`	--	Y	Y	
`with-input-from-file with-output-to-file`	Y	Y	Y	
`call-with-input-file call-with-output-file`	Y	Y	Y	
`call-with-port`	--	Y	Y	

9.3.14 Input and Output

Feature	R5RS	R7RS	LambLisp	Notes
`read read-char peek-char`	Y	Y	Y	
`eof-object?`	Y	Y	Y	
`char-ready?`	Y	Y	Y	

Feature	R5RS	R7RS	LambLisp	Notes
`read-line`	--	Y	Y	
`read-string`	--	Y	Y	Reads *k* CODEPOINTS, not *k* bytes (UTF-8 aware)
`read-u8 peek-u8 u8-ready?`	--	Y	Y	
`set-port-nonblocking! port-nonblocking?`	--	--	Y	LambLisp extension, Chez-compatible; see How blocking works on a port
`read-bytevector read-bytevector!`	--	Y	Y	
`write display newline`	Y	Y	Y	
`write-char`	Y	Y	Y	
`write-u8`	--	Y	Y	
`write-bytevector flush-output-port`	--	Y	~	
`write-shared write-simple`	--	Y	Y	`write-shared` datum-labels every node appearing more than once

9.3.15 How blocking works on a port

A port can be in three states, and only two of them are what most Scheme code expects. A file either has bytes for you or is at end of file – those are the two everyone knows. **A socket has a third: open, connected, and with nothing in it yet.** That third state is not an error and it is not end of file; it means *the bytes have not arrived yet*. Every rule below follows from keeping those two kinds of "nothing" apart.

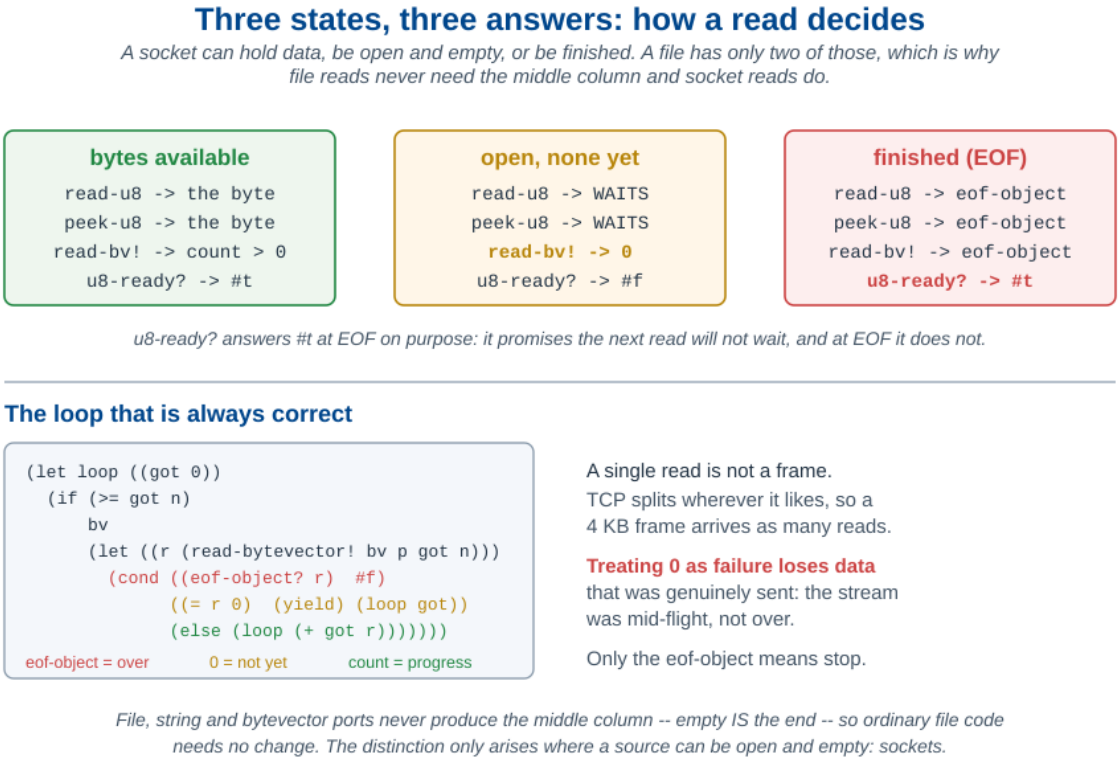


Figure 9.1 How a port read decides what to do

The blocking procedures

`read-u8`, `peek-u8`, `read-char`, `peek-char` and `read-line` **block**, as R7RS requires. On a socket with nothing in it yet they wait until a byte arrives or the peer closes; they never return a "nothing happened" value, because R7RS gives them only two outcomes – a datum, or the eof object – and there is no third slot to put it in.

On an embedded target this waiting **yields**: the radio, the collector and the watchdog keep running while a read waits. It is still a wait, though, and on a single cooperative loop that means everything else your program owes is not being done. A blocking read is appropriate when the next byte is genuinely the only thing that matters; when it is not, use the bulk form below.

The bulk form, and the meaning of `0`

`read-bytevector!` is the one that can tell you about the middle state, because a count has somewhere to put the answer:

return	meaning	what to do
a count $>$ `0`	that many bytes were read	keep going
`0`	**open, nothing arrived yet**	yield, then try again

return	meaning	what to do
the eof object	the stream is over	stop

A `0` is not a failure and not an end of file. It is the port saying *ask me again*. Treating it as either of the other two is the single most common way to lose data that was genuinely sent, because a TCP frame is split wherever the network chooses: a 4 KB message routinely arrives as a dozen reads with pauses between them.

This is a deliberate extension. R7RS specifies a count or the eof object and never mentions `0`, so LambLisp follows **Chez Scheme**, whose `get-bytevector-some!` on a non-blocking port returns `0` when nothing has arrived and the eof object only at a real end of stream. Gambit's `read-subu8vector` returns `0` for *both*, which cannot be told apart; that ambiguity is what is being avoided.

If you are porting from Chez

The **bulk** behaviour is the same in both. The **single-byte** behaviour is not, and the difference is worth knowing before you rely on it:

	Chez, non-blocking port	LambLisp
`get-bytevector-some!` / `read-bytevector!`, nothing yet	`0`	`0`
... at end of stream	eof object	eof object
`get-u8` / `read-u8`, nothing yet	**eof object**	**waits**
`lookahead-u8` / `peek-u8`, nothing yet	**eof object**	**waits**

Chez's single-byte reads report an end of file that has not happened; the caller is expected to know the port is non-blocking and interpret the eof accordingly. LambLisp does not do that – its single-byte reads wait for a real answer, so an eof object from `read-u8` or `peek-u8` always means the stream is genuinely over. The cost is that LambLisp has no *non-blocking single-byte* read: when you need one, ask `u8-ready?` first, or use `read-bytevector!` and read the count.

LambLisp has `set-port-nonblocking!` and `port-nonblocking?` with Chez's contract: the default is `#f`, and the flag is accepted on every port type – a string or bytevector port takes it and simply never has a third state to report.

Its value is mostly in TESTING. Whether a socket reports "nothing yet" otherwise depends on the target, so a program exercised only against a host would never meet the middle state until it ran on a device. Setting the flag lets a host test produce a `0` deliberately:

```
(define p (open-tcp-client-port host port))
(set-port-nonblocking! p #t)
(read-bytevector! bv p 0 8)   ; => 0 while the peer has sent nothing
```

`u8-ready?` and `char-ready?`

These are the probes, and they never block. `#t` means *the next read will not wait*.

Note what follows from that wording: **at end of file they return** `#t`, not `#f`. A read at EOF returns the eof object immediately, so it does not wait, so the port is "ready". This surprises people, and code that loops `while (not (u8-ready? p))` waiting for a closed stream to become ready will spin forever if it gets the

port kind	third state?
file, string, bytevector	**no** – empty *is* the end
serial console	no
TCP and TLS sockets	**yes**

So ordinary file-reading code needs no changes and will never see a `0`. **The distinction matters only where a source can be open and empty**, which in practice means the network. Code written and tested against files or strings on a host can therefore meet the middle state for the first time on a device – which is exactly the case to write a test for before shipping.

9.3.16 System Interface

Feature	R5RS	R7RS	LambLisp	Notes
`load`	Y	Y	Y	`(load "file.scm" verbosity)` – second arg: 1=verbose, 0=quiet
`file-exists? delete-file`	--	Y	Y	
`current-jiffy jiffies-per-second`	Y	Y	Y	
`current-second`	Y	Y	Y	Returns `millis()/1000.0` as inexact real
`command-line`	--	Y	Y	Returns the real argv on host targets; `("lamblisp")` on embedded
`exit emergency-exit`	--	Y	Y	
`get-environment-variables`	--	Y	Y	Host targets only; empty list on embedded
`features`	--	Y	Y	Returns `(lamblisp r7rs ieee-float)`
`system`	--	--	Y	**LambLisp extension** (no shell primitive exists in R7RS). `(system COMMAND)` runs COMMAND under `/bin/sh` and returns its exit status; stdout and stderr are inherited. Host targets only – returns `#f` on embedded, so a script can probe for the capability.

Feature	R5RS	R7RS	LambLisp	Notes
`system-capture`	--	--	Y	**LambLisp extension.** `(system-capture COMMAND)` returns `(exit-status . output-string)`, collecting stdout; append `2>&1` to COMMAND to include stderr. Returns `#f` on embedded and if the command cannot be launched.

9.3.17 Exception Handling

Feature	R5RS	R7RS	LambLisp	Notes
`raise`	--	Y	Y	
`raise-continuable`	--	Y	Y	The handler's value becomes the value of the `raise-continuable` expression
`with-exception-handler`	--	Y	Y	
`guard`	--	Y	Y	Implemented as a `syntax-rules` macro
`error`	--	Y	Y	

`error-object?`
`error-object-message`
`error-object-irritants`
	--	Y	Y	

`read-error? file-error?`	--	Y	Y	

Exception Cost in Real-Time Loops

Exception handling is correct on every target, and its cost is **asymmetric** in a way worth designing around: installing a handler is comparatively cheap, raising through one is not.

what the code does	cost on an ESP32-class target	cost on a Linux or Jetson host
enter a `guard` or `with-exception-handler` that never fires	a few hundred microseconds	a few microseconds

what the code does	cost on an ESP32-class target	cost on a Linux or Jetson host
raise one exception and catch it	**several to tens of milliseconds**	a few microseconds

The gap is structural rather than incidental. Garbage collection is divided into bounded slices by design, so a collection quantum stays in the low hundreds of microseconds however full the heap is (see Real-time Guarantees). Unwinding is not sliced: raising walks the native stack through compiler-generated tables held in flash, and the work scales with the depth of the stack being unwound rather than with a budget. On current microcontroller targets a caught exception therefore costs **roughly two orders of magnitude more than a collection quantum** – the one operation in the runtime that is slower than collection, not faster.

Three consequences for application code:

Do not raise on a path that runs every iteration of a control loop. A single caught exception can exceed the entire timing budget of a 1 kHz loop. Reserve `raise` for genuinely exceptional conditions, where its rarity is what makes the cost acceptable. Where a loop needs an early exit or a not-found result, return a sentinel value and test it – `#f`, or a value the caller distinguishes – rather than raising and catching. Implementing ordinary control flow by raising is an idiom that carries over from other scripting languages and does not carry over to this runtime's cost model.

Arm the handler outside the loop, not inside it. Handler installation is per entry, so a `guard` written around the loop body pays its setup on every iteration, while the same `guard` written around the whole loop pays it once:

```
;; pays handler setup n times
(let loop ((i 0))
  (if (< i n)
      (begin (guard (e (#t (handle e))) (step i))
             (loop (+ i 1)))))

;; pays it once, and still catches anything any iteration raises
(guard (e (#t (handle e)))
  (let loop ((i 0))
    (if (< i n) (begin (step i) (loop (+ i 1))))))
```

The second form is not merely faster; it is the form that expresses "this loop may fail" rather than "each step may fail and the loop continues regardless". Choose the first only when an iteration really must recover and continue, and then accept the per-iteration cost deliberately.

Measure on the part you ship, and do not size from host figures. The host is several orders of magnitude faster at raising than a microcontroller is, so a design validated on Linux or on a Jetson can meet its deadlines there and miss them on an ESP32 by a wide margin. Absolute figures differ by part, by clock, and by execution tier; bracket the operation with `(millis)` or `(micros)` on the target itself and read the on-device numbers.

9.4 *See:* Real-time Guarantees · Runtime and GC Tuning

Chapter 10

LambLisp Frequently Asked Questions

10.1 Getting Started

10.1.1 How do I try LambLisp right now?

No hardware required, and no build. Fastest first:

```
docker run --rm -it ghcr.io/lamblisp/lamblisp:runtime
```

That boots straight to `LL=>` and installs nothing on your machine. Alternatives, in order of how much they ask of you: open `demo/index.html` from any release package to run *LambLisp* in your browser; download the Linux or Windows package and run the prebuilt binary; or, if you already have *platformio* and want to build from source, `pio run -e linux_x86_64 -t exec`.

Load any `.scm` file with `(load "file.scm" 1)`. The same source files run unchanged on ESP32 once deployed.

See: Running in a Container · Quick Start: Linux · Quick Start: Windows · Deploy to ESP32

10.1.2 What hardware does LambLisp run on?

The primary target is the **ESP32-S3** (240 MHz dual-core Xtensa LX7, 8 MB PSRAM, 4 MB flash filesystem (SPIFFS or LittleFS)). The Freenove 4WD car kit uses the original **ESP32** (single-core), which is also supported. A secondary target is the **NVIDIA Jetson Orin Nano** (ARM64, Linux). LambLisp also builds and runs on **Linux x86-64**, which is the primary development and testing platform.

See: Supported Platforms and Boards

10.1.3 Is LambLisp open source?

LambLisp is commercial software. It may be used for non-commercial purposes at no cost and with no support or warranty. Commercial use — generating revenue, promoting products or services, or building production systems — requires a license. Licensing includes customization, hardware application development assistance, post-deployment support, and warranty.

See: License

10.1.4 How do I update code without reflashing?

Upload a Scheme file to the running ESP32 over WiFi using LLIP, then evaluate `(load)` in the running loop — no reboot, no flash erase, no control gap:

```
;; From a supervisory host (Linux or Jetson):
(define board (llip-connect "10.0.0.11" 5555))
(llip-write board "/new-policy.scm" source-string)
(llip-eval  board '(load "/new-policy.scm"))
```

The new definitions replace the old ones in the interaction environment immediately. The GC reclaims the previous code on the next sweep.

See: Code Lifecycle: Incremental Hot-Reload · LLIP · Hot-Reload Without Reboot

10.2 Concepts

10.2.1 What is a real-time control system?

A real-time system guarantees that outputs are updated within a bounded time after inputs change — the *deadline*. LambLisp uses **loop-based control**: the application's `loop` procedure runs repeatedly, reading sensors and driving actuators each cycle. The GC runs incrementally in quanta so no single allocation event blocks the loop for more than a configurable slice. Deadlines may be *hard* (missing one requires a fallback) or *soft* (graceful degradation); most control problems can be factored to eliminate the hard aspect entirely.

See: Real-time Guarantees · Memory and GC Tuning

10.2.2 What is LLIP?

LLIP (LambLisp Interaction Protocol) is a line-oriented S-expression protocol over TCP that connects any two LambLisp nodes — ESP32 to Linux, ESP32 to Jetson, or any combination. It provides two services over one connection: remote filesystem operations (upload, download, directory listing) and remote evaluation (`llip-eval`). A supervisory controller on a Jetson or Linux host can query live sensor state, tune PID parameters, and hot-load new algorithms into a fleet of ESP32 boards without interrupting their control loops.

See: LLIP — LambLisp Interaction Protocol

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

10.2.3 How fast is LambLisp?

Benchmark	AST interpreter	Bytecode	NCG native	NCG/BC	NCG/Interp
tak(7,4,1) — mutual recursion	1,223 ms	28 ms	3.8 ms	**7×**	**324×**
sieve(200) — loop-heavy	558 ms	4.8 ms	1.6 ms	**3×**	**341×**
fib-memo(25) — memoization	1,346 ms	4.9 ms	1.8 ms	**3×**	**742×**
tree(8) — allocation-heavy	299 ms	92 ms	14 ms	**7×**	**22×**
ackermann(3,1) — deep recursion	36 ms	9.5 ms	1.5 ms	**6×**	**24×**
qsort(200) — vector in-place	2,293 ms	49 ms	10 ms	**5×**	**219×**
fib-iter(1000) — tail recursion	720 ms	18 ms	9.3 ms	**2×**	**77×**
closure(500) — closure alloc	582 ms	113 ms	26 ms	**4×**	**22×**

The NCG compiles recursive and arithmetic-heavy procedures to native Xtensa instructions with direct call dispatch, achieving 2–10× over bytecode and 22–742× over the interpreter on recursion-dominated workloads.

See: Execution Tiers: AST → Bytecode → Native Code

10.2.4 What is bytecode / NCG compilation?

LambLisp has three execution tiers that mix at the procedure level:

Tier	Invocation	Typical speedup vs AST
AST interpreter	Default; no compile step	1×
Bytecode (BC)	`(procedure->bytecode fn)`	2–10×
Native Code Generator (NCG)	`(ncg-compile fn)`	2–10× over BC, up to 742× over Interp on ESP32

Each procedure can be compiled independently; the rest of the program runs interpreted. Recursive calls between compiled procedures bypass the interpreter entirely, which is why deeply recursive benchmarks show the largest NCG speedups. On ESP32, NCG→NCG direct calls skip argument-list allocation entirely.

See: Execution Tiers: AST → Bytecode → Native Code · Bytecode Virtual Machine · Native Code Generator (NCG)

10.2.5 Does the same Scheme code run on all platforms?

Yes, with a small exception for hardware I/O calls. All language semantics — closures, tail calls, the macro system, the exception system, the GC, the REPL — behave identically on ESP32, Jetson, and Linux. Any `.scm` file that does not call hardware-specific functions (`digitalRead`, `Wire.begin`, CUDA bindings) runs without modification on all three targets.

For the small amount of code that must branch on platform, use the boolean predicates `linux?`, `esp32?`, and `jetson?` defined in `setup.scm`, or inspect the `lamb-board` string directly.

See: Multi-Platform Support · Deploy to ESP32

10.3 Language and Conformance

10.3.1 Why is it called LambLisp instead of Scheme?

Scheme emerged in the 1970s as a minimal, principled dialect of Lisp that unified lexical scoping, tail recursion, and continuations. LambLisp implements R5RS/R7RS-small and adds practical extensions (`nlambda`, `macro`, the R7RS exception system). Its `call-with-current-continuation` is **escape-only**: it does non-local exit, but not the re-entrant continuations that let a captured `k` resume an extent after it has been left.

For many Scheme enthusiasts full `call/cc` is the *sine qua non* of the language – it is what makes generators, coroutines and backtracking expressible – so calling this *Scheme* without qualification would overstate what it does. The name is a statement about that one gap, not a distancing from the standard: everything else about R5RS is here. See *Why only escape-only* `call/cc`*?* below.

See: Brief history of Lisp and Scheme

10.3.2 Why only escape-only `call-with-current-continuation`?

Full `call/cc` requires capturing the entire C call stack as a first-class object — either by copying the stack, or by converting all code to continuation-passing style (CPS). On a 240 MHz microcontroller with tight memory, the overhead of full CPS transformation is prohibitive, and stack copying is incompatible with the C-stack trampoline that makes the real-time guarantees possible.

So `call/cc` is provided, escape-only. It is built on the R7RS exception system: `k` raises a uniquely-tagged value and `call/cc` installs a handler that recognises its own tag and returns the value. That covers the majority of real-world uses — early return, structured loop exit, and error handling.

How much is actually given up is measurable, and it is small. Across the 1370-test R7RS conformance suite, re-entrant continuations account for **10 exceptions — under 1%**, and nothing else. That is the whole cost of the trade: the remaining 99.3% of the standard is implemented, and in exchange every procedure call in the system avoids the CPS overhead that full continuations would impose on it — including the calls made by programs that never use `call/cc` at all. A penalty paid on every call, for a feature required by fewer than one test in a hundred, is the wrong trade for a real-time runtime on a 240 MHz part. `dynamic-wind` is built on the same mechanism and is complete for the escapes this implementation can produce, because no re-entrant continuation exists that could resume into the extent again.

Two limits, both inherent and neither hidden:

1. **Re-entrant continuations are absent, and this does not fake them.** Storing `k` with `(set! cont k)` and invoking it after its extent has exited cannot work at any price here; that needs real `call/cc`. Generators, coroutines and backtracking built on resumable continuations are therefore out of reach.
2. **A `guard` with a catch-all clause sitting between a `call/cc` and its `k` will swallow the escape**, because the escape *is* a raise and the guard catches it first. This is inherent to exceptions-as-escapes, and it is the one behaviour most likely to be mistaken for a bug. It is documented in the Compatibility Matrix notes with a worked example.

The Scheme community has itself debated whether full `call/cc` is worth the implementation cost; it is possible that a future R8RS specification will optionalize or remove it.

See: Language Extensions

Copyright 2026 FrobeniusNorm LLC · All specifications subject to change.

Feature	LispBM / uLisp &c	LambLisp	ChezScheme / Racket
Runs entirely on-board	no	**yes**	yes
Real-time incremental GC	no	**yes**	no
R5RS / R7RS conformance	no	**yes** (few exceptions)	yes (few exceptions)
File system, WiFi, BLE	no	**yes**	n/↩ a
`call/cc`	no	**escape-only**	yes (full)
Target RAM	$<$ 64 KB	**2–8 MB PSRAM**	GB-class
Bytecode + NCG option	bytecode only	**yes**	yes

LambLisp was designed around these constraints: substantially conform to R5RS/R7RS-small; large enough to exploit 32-bit PSRAM but not a *Big Lisp*; real-time GC with no stop-the-world pauses; scalable execution from AST to NCG; easy reuse of Arduino-style drivers.

See: Other Scheme Implementations

10.3.4 Why not use embedded Python derivatives?

MicroPython and CircuitPython are useful for simple scripting but do not thrive as embedded control platforms. Their garbage collectors are tuned for throughput, not latency, and provide no real-time guarantees. A Python derivative called Serpent comes closest, with a real incremental collector, but it is not Python, requires off-board processing, has no generalized hardware interface, and does not fill the same solution space as Lamb↩ Lisp.

Python's indentation-sensitive block syntax makes tokenizing and parsing substantially heavier than Lisp's uniform prefix-parenthesis grammar. A Lisp reader is a few dozen lines of code; a Python parser requires tracking indent levels, dedent tokens, and a grammar with many special cases. That overhead compounds inside a microcontroller.

Part of Python's strength is its large library ecosystem, but the embedded Python variants cannot take full advantage of those libraries. LambLisp draws on lessons from Python where they apply — dictionaries are a first-class type in both languages — and extends them with hierarchical parent/child frames.

See: Other Scheme Implementations

10.3.5 Why not use LLVM for native code generation?

The NCG JIT-compiles Scheme lambdas **on the device at runtime**. The ESP32-S3 has 240 MHz and roughly 2 MB of PSRAM total. LLVM is a multi-megabyte library that requires a hosted environment with far more memory than an ESP32 provides. LLVM on the device is not a viable option regardless of output quality.

For the Linux and Jetson targets LLVM would fit in memory, but using a different code generator per platform would split the runtime semantics and make the codebase much harder to maintain. The hand-written NCG runs identically on all three targets, compiles a function in microseconds, and has no external dependencies. What it sacrifices in optimizer sophistication it gains in predictability — compilation latency is bounded and does not vary with loop complexity.

10.4 Integration and Operation

10.4.1 What are the advantages of *LambLisp* in combination with C/C++ on micro-controllers?

- **Interactive development** — the REPL lets you inspect state, redefine procedures, and test changes without reflashing.
- **Live code updates** — new definitions can be loaded over WiFi while the control loop runs.
- **Real-time safe GC** — incremental tricolor collector with configurable quanta keeps loop times bounded.
- **C++ interop** — any C++ function registered as a `mop3` appears as a native Scheme procedure; existing Arduino drivers need no changes.
- **Scalable execution** — bytecode and NCG compilation deliver 2–10× over BC and up to 742× over interpreter on hot recursive paths without touching the rest of the program.
- **AI-friendly** — uniform S-expression syntax makes LambLisp code easy to generate, modify, and verify programmatically.

See: Why ∗LambLisp∗? · C++ Integration · Scalability

10.4.2 Does LambLisp interoperate with C/C++ code?

Yes. Any C++ function with the `mop3` signature appears as a native Scheme procedure in the interaction environment:

```
Sexpr_t my_fn(Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec);
```

Evaluated-argument procedures (`T_MOP3_PROC`) and syntax/unevaluated-argument procedures (`T_MOP3_NPROC`) are both supported. An installer function registers them in the interaction environment at startup. Existing Arduino libraries, hardware drivers, and C++ classes require no modification.

See: C++ Integration

10.4.3 Can I use LambLisp without an RTOS?

Yes. LambLisp runs directly on top of the Arduino `loop()` / `setup()` framework with no operating system. There is no dependency on FreeRTOS, Zephyr, or any other RTOS. The GC and the control loop share a single thread; incremental GC quanta are interleaved with Scheme evaluation so that loop deadlines are met without cooperative multitasking.

See: Real-time Guarantees

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

10.4.4 What is the memory footprint?

On the ESP32-S3 with 8 MB PSRAM:

- **Cell pool** — configurable; by default, cells are allocated from PSRAM until the pool is full.
- **GC heap** — the same cell pool; there is no separate allocation arena.
- **Scheme stack** — held as a linked list of cells on the GC heap; no fixed C stack limit.
- **C++ stack** — 96 KB in the current build (overridden from the 8 KB ESP-IDF default via `LL_STACK_SIZE`). This larger size is needed for deep recursion in the conformance test suite; most production applications can safely reduce it to 16–32 KB.
- **Flash filesystem** — `.scm` source files and `setup.scm`; stored on SPIFFS or LittleFS, selected via `LL_SPIFFS=1` / `LL_LITTLEFS=1`.

A fully loaded `setup.scm` with `syntax-rules`, the R7RS exception system, and standard library definitions uses a few thousand cells; the cell pool is large enough that this is not a concern in practice.

See: Memory and GC Tuning

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 11

Other Scheme Implementations

This project was started after a review of available options for run-time languages to augment C++ embedded applications, and the likely candidates were all on the Lisp/Scheme spectrum. Some of the existing implementations that were examined include: TinyScheme, ulisp, microlisp, picolisp, T, L, Pre-Scheme, LispBM, chicken, bigloo, racket, chibi and others. Plug those into your favorite search engine to find even more.

General objections to existing Lisp/Scheme implementations
Required off-board compilation.
Out of scale for a microprocessor.
Non-portable hardware tricks (e.g., using the bottom bits of addresses as flag bits).
Non-Scheme lisps (e.g., no lexical scoping or tail recursion).
Bad fit on the interpret<->compile spectrum (e.g., low-level interpreter or huge compiler).
Lisp-on-Python, Lisp-on-JavaScript, and other such proofs.
Garbage collection: Real-time applications must avoid pauses, so stop-the-world, reference counting and generational GC cannot be used.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 12

Topic Documentation

12.1 Hardware & Platform Builtins (xmop3)

Topics

- GPIO & Basic I/O
- ESP32 System
- Camera (ESP32-S3-EYE)
- LCD1602 Character Display
- 1-Wire & DS18B20
- Over-the-Air Firmware Updates
- PCA9685 PWM / Servo Driver
- Platform (generic)
- Ultrasonic Sonar (HC-SR04)
- SPI Bus
- WiFi
- I2C (Wire)
- WS2812 / NeoPixel LEDs
- CUDA GPU
- HIP / ROCm GPU
- LLArduino – LambLisp's own Arduino API

12.1.1 Detailed Description

The C++ `ll_xmop3_*` builtins that expose hardware and platform features to Scheme – GPIO, buses (I2C/↩ SPI/1-Wire), sensors, displays, LEDs, WiFi, camera, GPU, and platform control.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

12.1.2 GPIO & Basic I/O

Functions

- Sexpr_t mop3_cio_digitalRead (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_digitalWrite (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_pinMode (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_analogRead (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_analogWrite (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_delay_ms (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_delay_us (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_analogReadResolution (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_analogReference (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_analogWriteResolution (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_noTone (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_pulseIn (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_pulseInLong (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_shiftIn (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_shiftOut (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_tone (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_bit (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_bitClear (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_bitRead (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_bitSet (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_bitWrite (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_highByte (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_lowByte (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_attachInterrupt (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_detachInterrupt (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_digitalPinToInterrupt (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_interrupts (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cio_noInterrupts (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t CommonIO_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp GPIO & Basic I/O builtins.

Function Documentation

mop3_cio_digitalRead()

```
Sexpr_t mop3_cio_digitalRead (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read digital pin state; returns #t if HIGH, #f if LOW.

mop3_cio_digitalWrite()

```
Sexpr_t mop3_cio_digitalWrite (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write digital pin HIGH/LOW. Level is Scheme-truthy (only #f is LOW) with one pragmatic extension: a NUMERIC ZERO also drives LOW. So (digitalWrite p #f) and (digitalWrite p 0) both give LOW; (digitalWrite p #t) and (digitalWrite p 1) give HIGH.

mop3_cio_pinMode()

```
Sexpr_t mop3_cio_pinMode (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Configure a pin as INPUT, INPUT_PULLUP, or OUTPUT.

mop3_cio_analogRead()

```
Sexpr_t mop3_cio_analogRead (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read analog pin; returns raw ADC integer value.

mop3_cio_analogWrite()

```
Sexpr_t mop3_cio_analogWrite (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write analog PWM value to pin.

mop3_cio_delay_ms()

```
Sexpr_t mop3_cio_delay_ms (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Delay execution by the given number of milliseconds.

mop3_cio_delay_us()

```
Sexpr_t mop3_cio_delay_us (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Delay execution by the given number of microseconds.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_cio_analogReadResolution()

```
Sexpr_t mop3_cio_analogReadResolution (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set ADC resolution in bits (stub – platform-dependent).

mop3_cio_analogReference()

```
Sexpr_t mop3_cio_analogReference (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set ADC voltage reference (stub – platform-dependent).

mop3_cio_analogWriteResolution()

```
Sexpr_t mop3_cio_analogWriteResolution (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set DAC/PWM resolution in bits (stub – platform-dependent).

mop3_cio_noTone()

```
Sexpr_t mop3_cio_noTone (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Stop tone generation on a pin (stub). noTone had a backend all along – both the vendor core and LLArduino provide it – so the stub was silently discarding a call that would have worked.

mop3_cio_pulseIn()

```
Sexpr_t mop3_cio_pulseIn (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Measure a pulse on a pin; returns duration in microseconds (stub).

mop3_cio_pulseInLong()

```
Sexpr_t mop3_cio_pulseInLong (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Like pulseIn but supports longer pulses (stub).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_cio_shiftIn()

```
Sexpr_t mop3_cio_shiftIn (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Shift in a byte of data one bit at a time (stub).

mop3_cio_shiftOut()

```
Sexpr_t mop3_cio_shiftOut (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Shift out a byte of data one bit at a time (stub).

mop3_cio_tone()

```
Sexpr_t mop3_cio_tone (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Generate a square wave tone on a pin at the given frequency and optional duration (ms).

mop3_cio_bit()

```
Sexpr_t mop3_cio_bit (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a bitmask with only bit n set (stub). These seven were registered but returned #UNDEF on EVERY target, including the ESP32 boards – callable, documented, and silently useless. They are pure arithmetic with no hardware behind them, so there was never a platform reason for the stub. Found by the LLArduino conformance suite on its first run. DIVERGENCE FROM ARDUINO, STATED: Arduino's bitSet/bitClear/bit↩ Write are MACROS that MUTATE their first argument in place. Scheme has no integer to mutate, so these RETURN the new value and the caller rebinds. Same result, different shape; a sketch ported verbatim must assign.

mop3_cio_bitClear()

```
Sexpr_t mop3_cio_bitClear (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Clear a specific bit in an integer (stub).

mop3_cio_bitRead()

```
Sexpr_t mop3_cio_bitRead (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read a specific bit from an integer (stub).

mop3_cio_bitSet()

```
Sexpr_t mop3_cio_bitSet (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set a specific bit in an integer (stub).

mop3_cio_bitWrite()

```
Sexpr_t mop3_cio_bitWrite (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write a specific bit in an integer (stub).

mop3_cio_highByte()

```
Sexpr_t mop3_cio_highByte (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the high byte of a two-byte integer (stub).

mop3_cio_lowByte()

```
Sexpr_t mop3_cio_lowByte (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the low byte of a two-byte integer (stub).

mop3_cio_attachInterrupt()

```
Sexpr_t mop3_cio_attachInterrupt (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Attach an ISR to a digital pin interrupt (stub).

mop3_cio_detachInterrupt()

```
Sexpr_t mop3_cio_detachInterrupt (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Detach the ISR from a digital pin interrupt (stub).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_cio_digitalPinToInterrupt()

```
Sexpr_t mop3_cio_digitalPinToInterrupt (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the interrupt number for a given digital pin (stub).

mop3_cio_interrupts()

```
Sexpr_t mop3_cio_interrupts (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Re-enable interrupts globally (stub).

mop3_cio_noInterrupts()

```
Sexpr_t mop3_cio_noInterrupts (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Disable interrupts globally (stub).

CommonIO_install_mop3()

```
Sexpr_t CommonIO_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`s + ascii.dec()`, never `d`, for an LL_int32 – `int32_t` is `long` on the xtensa toolchain and `int` on x86-64, so a fixed specifier is UB on one of them.

12.1.3 ESP32 System

Functions

- Sexpr_t mop3_esp32_heapinfo (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_gcmem (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_flash_probe (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_efuse_mac_get_default (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_efuse_pkg_ver (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_efuse_read_block (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_esp32_efuse_write_block (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **ESP32_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp ESP32 System builtins.

esp_efuse_mac_get_default() and friends MOVED OUT of esp_system.h into esp_mac.h in ESP-IDF 5. esp_mac.h DOES NOT EXIST in IDF 4.4, so this cannot be an unconditional include – hence __has_↩ include rather than a version test, which also keeps it correct if the header ever moves again. The symptom without it is a bare "'esp_efuse_mac_get_default' was not declared in this scope" plus a did-you-mean pointing at THIS FILE'S OWN wrapper (mop3_esp32_efuse_mac_get_default), which is a convincing wrong lead: it reads as a self-reference or a typo rather than as a header that moved.

Function Documentation

mop3_esp32_heapinfo()

```
Sexpr_t mop3_esp32_heapinfo (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Report heap free/largest-block for the three caps that matter to flash reads. Returns (internal-free internal-largest dma-free dma-largest spiram-free spiram-largest) and also logs it. Diagnostic: littlefs reads ENOMEM (esp_partition_read rc 257).

mop3_esp32_gcmem()

```
Sexpr_t mop3_esp32_gcmem (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Diagnostic: print GC-buffer / cell-block memory placement (INTERNAL vs PSRAM) + sizes.

mop3_esp32_flash_probe()

```
Sexpr_t mop3_esp32_flash_probe (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Reproduce a raw esp_partition_read under our own logging, to isolate the ENOMEM. (esp32-flash-probe "label" offset len where) where: 0=internal DMA buffer, 1=PSRAM buffer. Logs heap + destination region + rc; returns the esp_partition_read rc as an integer.

mop3_esp32_efuse_mac_get_default()

```
Sexpr_t mop3_esp32_efuse_mac_get_default (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read the default factory MAC address from eFuse; returns a 6-byte bytevector.

mop3_esp32_efuse_pkg_ver()

```
Sexpr_t mop3_esp32_efuse_pkg_ver (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the chip package version from eFuse.

mop3_esp32_efuse_read_block()

```
Sexpr_t mop3_esp32_efuse_read_block (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read Nbits bits from an eFuse block at a bit offset; returns a bytevector or error code.

mop3_esp32_efuse_write_block()

```
Sexpr_t mop3_esp32_efuse_write_block (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write Nbits bits from a bytevector or integer into an eFuse block; returns esp_err_t.

12.1.4 Camera (ESP32-S3-EYE)

Macros

- #define EYE_CAM_EARLY_DMA_RESERVE 0
- #define EYE_CAM_XCLK 15
- #define EYE_CAM_SIOD 4
- #define EYE_CAM_SIOC 5
- #define **EYE_CAM_VSYNC** 6
- #define **EYE_CAM_HREF** 7
- #define **EYE_CAM_PCLK** 13
- #define EYE_CAM_D0 11
- #define **EYE_CAM_D1** 9
- #define **EYE_CAM_D2** 8
- #define **EYE_CAM_D3** 10
- #define **EYE_CAM_D4** 12
- #define **EYE_CAM_D5** 18
- #define **EYE_CAM_D6** 17
- #define **EYE_CAM_D7** 16
- #define LL_CAM_SET_I(FN)

Functions

- Sexpr_t mop3_camera_init (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_camera_thumb (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_camera_set (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_camera_get (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_camera_capture (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_frame_brightness (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t EyeCam_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp Camera (ESP32-S3-EYE) builtins.

Macro Definition Documentation

EYE_CAM_EARLY_DMA_RESERVE

```
#define EYE_CAM_EARLY_DMA_RESERVE 0
```

THE PANEL LEFT THIS FILE. It is `ll_xmop3_Panel.cpp` now, behind the `LL_Panel` interface in `ll_platform_generic.h`, and this driver asks only ONE question of it: is there a panel to send a frame to? That is `LL_PANEL`.

What used to be here was `LL_CAM_HAS_LCD`, derived as `LL_ESP32S3` – correct only by accident, because the one S3 board with a camera happened to be the one board with the ST7789. Worse, the panel could not exist WITHOUT this file: all 98 LCD references were inside `#if LL_CAMERA`, so a board with a panel and no camera could not compile a line of it. A camera driver is the wrong owner for a display.

EYE_CAM_XCLK

```
#define EYE_CAM_XCLK 15
```

TWO BOARDS, TWO WIRINGS, AND THEY SHARE NO PIN. These are not interchangeable and a build that picks the wrong set configures GPIOs that either do not exist or belong to something else. The Freenove numbers are NOT guessed: they are exactly what scm/boards/pins/Freenove-4WD-Car-Kit-ESP32.scm already declares (pin-XCLK 21, pin-PCLK 22, pin-HREF 23, pin-CSI_VYSNC 25, pin-SIOD 26, pin-SIOC 27, pin-CSI_Y2..Y9 = 4 5 18 19 36 39 34 35), which is the Freenove WROVER-CAM mapping and differs from the AI-Thinker ESP32-CAM one that a web search hands you first. The Scheme map and this table must agree; if the board is ever re-wired, change BOTH.

EYE_CAM_SIOD

```
#define EYE_CAM_SIOD 4
```

SCCB SDA.

EYE_CAM_SIOC

```
#define EYE_CAM_SIOC 5
```

SCCB SCL.

EYE_CAM_D0

```
#define EYE_CAM_D0 11
```

D0..D7 are the sensor's Y2..Y9.

LL_CAM_SET_I

```
#define LL_CAM_SET_I( FN)
```

Value:

```
do { if (s->FN) rc = s->FN(s, (int) val); else have = false; } while (0)
```

(camera-set key val) -> #t if the sensor took it, #f if it did not.

Tune the sensor through its control block. Keys, with the range each one accepts:

EXPOSURE AND GAIN – the controls that decide whether a dim scene is usable at all: exposure-ctrl 0/↩ 1 automatic exposure on (1) or off (0) aec-value 0..1200 manual exposure time, in sensor row periods; needs exposure-ctrl 0 aec2 0/1 the DSP's second-stage auto exposure ("night mode") ae-level -2..2 bias the automatic exposure darker or brighter gain-ctrl 0/1 automatic gain on (1) or off (0) agc-gain 0..30 manual gain; needs gain-ctrl 0 gainceiling 0..6 how far automatic gain may go: 0 is 2x, 6 is 128x

IMAGE: framesize, quality (10..63, LOWER is better), brightness/contrast/saturation (-2..2), sharpness, denoise, special-effect, wb-mode, whitebal, awb-gain, raw-gma, lenc, dcw, bpc, wpc, colorbar, hmirror, vflip.

RAISING THE GAIN CEILING IS THE LEVER FOR A DIM SCENE, AND THE TRADE IS NOISE AND MOTION BLUR. Automatic exposure will not use a gain it is not permitted, so a camera indoors at dusk typically sits at its ceiling with the image still under-exposed; the default ceiling is low because a moving camera cannot afford the blur a long exposure brings. A camera that does not move can: there is nothing in the frame to smear, and a noisy bright frame carries far more for a recogniser than a clean black one. So a fixed installation should expect to raise `gainceiling`, and may set `aec-value` directly once it knows its scene.

BRIGHTNESS IS NOT EXPOSURE. It is a digital offset applied to pixels the sensor has already captured, so it lifts the black level and the noise together and collects no extra light. Measured on an under-exposed indoor wall: brightness 2 with contrast 2 left the scene darker still, because the added contrast crushed what little shadow detail there was. Reach for the exposure and gain keys above instead.

IT REPORTS WHETHER IT WORKED, AND THE THREE WAYS IT CAN FAIL ARE DISTINGUISHED IN THE LOG: the camera is not initialised, the key is not one of the above, or this sensor does not implement that control. A control block carries a null pointer for anything the attached part cannot do – the OV2640 and the OV5640 do not implement the same set – so each call is checked before it is made; an unchecked call through a null entry is a reset, not an error. Returning a value rather than nothing matters because the three failures and a success used to be indistinguishable from the caller: a typo in a key silently did nothing.

DEFINED AT FILE SCOPE, NOT INSIDE THE GC FRAME BELOW, AND IT MUST STAY THERE. The gc frame macros are variadic, so the braced block passed to one is a macro ARGUMENT – and a `#define` written inside a macro argument is not in effect for that argument's own tokens. Move this back in and all ∼20 uses below fail to compile with "'LL_CAM_SET_I' was not declared in this scope", naming a definition that is plainly visible above them. `#if` inside the same block DOES work (the preprocessor evaluates conditionals there), so a block can contain working directives alongside one that silently did nothing.

Function Documentation

mop3_camera_init()

```
Sexpr_t mop3_camera_init (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(camera-init [mode]) -> #t/#f. mode 'rgb565 (default, drives the LCD) or 'jpeg (for capture).

< 160x120

< 320x240

< 240x240 – the historical hardcoded default

< 480x320

< 640x480

< 800x600

< 1024x768

< 1600x1200, the OV2640's maximum

REFUSE AN UNKNOWN NAME RATHER THAN FALLING BACK TO THE DEFAULT. A typo that quietly captured at 240x240 would read as "the larger size made no difference" – a null result from an experiment that never ran, which this tree has paid for repeatedly.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_camera_thumb()

```
Sexpr_t mop3_camera_thumb (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(camera->lcd) -> #t/#f. Grab one RGB565 frame and blit it to the LCD (the hot path).

mop3_camera_set()

```
Sexpr_t mop3_camera_set (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

THE KEY CELL IS CONSUMED BEFORE THE VALUE IS FETCHED, AND THAT ORDER IS THE POINT. `car` and `cdr` ALLOCATE. A pair slot can hold a small integer or float EMBEDDED in the slot itself, so the accessor must BOX that value into a fresh cell on the way out – a cell the caller's argument list never contained, so the root the interpreter holds on that list does NOT cover it. Fetching the value is itself an accessor, hence itself an allocation: holding the key cell across it would let a collection cycle reclaim the box and hand the cell out again with a different type, after which the symbol test below passes on an unrelated object and the character read follows a pointer that is not a symbol. That is a fault with no Scheme error to catch, because the check that should have refused it has already run. So: test the key, take its characters, and only then fetch the value. `key_cell` MUST NOT be dereferenced after the `val` line below – it is named for the cell rather than for the key so that a reader looking for its uses finds all of them.

Interned symbol characters outlive the call; `""` keeps the comparisons below well-defined on the rejected path rather than relying on the early return alone.

< the key is one of ours

< this sensor implements it (the entry is not null)

< what the sensor's own setter answered; 0 is success

ONE LINE PER CONTROL, EACH CHECKING ITS OWN ENTRY FIRST. The repetition is deliberate: the setters do not share a signature (framesize and gainceiling take enums), so a table would need a cast per row and would hide exactly the null check that must not be skipped.

mop3_camera_get()

```
Sexpr_t mop3_camera_get (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(camera-get) -> an alist of the sensor's CURRENT exposure and gain state. A fixed installation has to know whether automatic exposure has already run out of room before it decides to raise the ceiling or to ask for more light, and the status block carries what the sensor settled on. Built by prepending and reversed once, so the list reads in the order written here.

mop3_camera_capture()

```
Sexpr_t mop3_camera_capture (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(camera-capture) -> bytevector. Return one JPEG frame (needs (camera-init 'jpeg)); returns #f if no frame is available.

mop3_frame_brightness()

```
Sexpr_t mop3_frame_brightness (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(frame-brightness) -> integer 0..255. Cheap mean luma over a downsampled RGB565 frame – a scalar for trigger-policy demos ("brighten past N -> fire").

EyeCam_install_mop3()

```
Sexpr_t EyeCam_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Installer – always compiles; registers the eye mops only under LL_CAMERA.

(lcd-backlight [percent]) -> void. Set backlight brightness 0..100 (default 100). Handy for demos: (lcd-backlight 0) blanks the screen without tearing down the panel, (lcd-backlight 100) restores it. Independent of panel content, so it also isolates a backlight fault from a draw fault when bringing up new hardware. `sym` is read by dict_bind_bang with no allocation in between, so it needs no root.

THE LCD BAND USED TO BE RESERVED HERE TOO, AND IS NOT ANY MORE: it belongs to the panel driver and is claimed lazily at (lcd-init). The two claims still COMPETE for the same internal-DMA pool, so whichever runs second can still come back NULL – that coupling is real and did not move with the code. If a blank screen returns after a camera change, this is the interaction to suspect.

12.1.5 LCD1602 Character Display

Typedefs

- typedef LCD1602_I2C **LCD1602**

Functions

- Sexpr_t LCD1602_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_clear (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_home (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_display (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_cursor (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_backlight (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_write (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_print (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t LCD1602_mop3_setCursor (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **LCD1602_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- LCD1602 * **lcd1602** = 0

Detailed Description

LambLisp LCD1602 Character Display builtins.

Function Documentation

LCD1602_mop3_begin()

```
Sexpr_t LCD1602_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Initialize the LCD1602 at the given I2C address with cols x rows dimensions.

LCD1602_mop3_clear()

```
Sexpr_t LCD1602_mop3_clear (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Clear the LCD display and return cursor to home.

LCD1602_mop3_home()

```
Sexpr_t LCD1602_mop3_home (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Move cursor to the home position (0, 0).

LCD1602_mop3_display()

```
Sexpr_t LCD1602_mop3_display (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Turn the display on or off; optional arg defaults to #t (on).

LCD1602_mop3_cursor()

```
Sexpr_t LCD1602_mop3_cursor (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Show or hide the cursor; optional arg defaults to #t (show).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

LCD1602_mop3_backlight()

```
Sexpr_t LCD1602_mop3_backlight (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Turn backlight on or off; optional arg defaults to #t (on).

LCD1602_mop3_write()

```
Sexpr_t LCD1602_mop3_write (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write a string at the current cursor position.

LCD1602_mop3_print()

```
Sexpr_t LCD1602_mop3_print (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Move cursor to (0,0) then write a string – convenience for full-line refresh.

LCD1602_mop3_setCursor()

```
Sexpr_t LCD1602_mop3_setCursor (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Move cursor to column col and row row (zero-based).

12.1.6 1-Wire & DS18B20

Functions

- Sexpr_t OneWire_mop3_reset (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_mop3_write_byte (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_mop3_read_byte (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_mop3_write_bit (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_mop3_read_bit (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_mop3_search (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t OneWire_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp 1-Wire & DS18B20 builtins.

Function Documentation

OneWire_mop3_reset()

```
Sexpr_t OneWire_mop3_reset (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-reset pin) -> #t/#f

OneWire_mop3_write_byte()

```
Sexpr_t OneWire_mop3_write_byte (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-write-byte pin b)

OneWire_mop3_read_byte()

```
Sexpr_t OneWire_mop3_read_byte (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-read-byte pin) -> byte

OneWire_mop3_write_bit()

```
Sexpr_t OneWire_mop3_write_bit (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-write-bit pin b)

OneWire_mop3_read_bit()

```
Sexpr_t OneWire_mop3_read_bit (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-read-bit pin) -> 0/1

OneWire_mop3_search()

```
Sexpr_t OneWire_mop3_search (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(onewire-search pin) -> (list rom-bytevector ...), each 8 bytes.

`reverse_bang` only rplacd's in place – it allocates no cell – so `acc` needs no root here.

OneWire_install_mop3()

```
Sexpr_t OneWire_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Install all OneWire symbols in base environment.

12.1.7 Over-the-Air Firmware Updates

Macros

- #define **OTA_NS** "ota-p125"

Functions

- Sexpr_t ota_mop3_request_install (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_request_recover (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_request_console (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_set_network (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- int ll_ota_boot_factory_raw (void)
- Sexpr_t ota_mop3_reboot_to_loader (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_boot_ok (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- int ll_ota_request_console_raw (void)
- int ll_ota_record_death_raw (const char *reason)
- Sexpr_t ota_mop3_record_death (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_clear_death (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_mop3_death_record (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t ota_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

< GENERATED from scm/features/llloader-vocab.scm

LambLisp Boot-Loader Handshake (OTA) builtins.

Function Documentation

ota_mop3_request_install()

```
Sexpr_t ota_mop3_request_install (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-request-install!)` — ask the loader to INSTALL (`pending_update=1`: fetch + write a new `ota_0`). Returns #t on success, #f off-device.

ota_mop3_request_recover()

```
Sexpr_t ota_mop3_request_recover (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-request-recover!)` — ask the loader to RECOVER (`pending_update`=2: re-fetch a known-good image). Returns #t on success, #f off-device.

ota_mop3_request_console()

```
Sexpr_t ota_mop3_request_console (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-request-console!)` — ask the loader for the CONSOLE fallback (`pending_update`=3: operator BLE/↩ UART). Returns #t on success, #f off-device.

ota_mop3_set_network()

```
Sexpr_t ota_mop3_set_network (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-set-network! ssid pass url)` — write the network-rung config keys `wifi_ssid` / `wifi_pass` / `img↩ _url` (all strings) to NVS. Returns #t on success, #f off-device.

ll_ota_boot_factory_raw()

```
int ll_ota_boot_factory_raw (void )
```

`(ota-reboot-to-loader)` — point otadata at the `factory` lloader partition and `esp_restart`; never returns on device. Returns #f if there is no factory partition (or off-device). Defined below, beside its siblings; declared here because the mop3 above uses it.

Point the next boot at the FACTORY partition. True on success.

ASKING FOR THE CONSOLE IS NOT ENOUGH ON ITS OWN, AND THIS IS THE HALF THAT IS EASY TO OMIT. `pending_update` is read by the LOADER – so it is read only if the loader RUNS. On a unit whose otadata selects `ota_0`, the second-stage bootloader starts `ota_0` directly and the factory loader is never entered; the pending request then survives in NVS until `(ota-boot-ok!)` erases it while marking the application healthy. A reset without this redirect therefore boots the very application that asked to be taken out of service, and the request leaves no trace of having been ignored, because nothing ignored it.

So a handover is TWO writes and a reset, in this order: the pending action, the boot partition, then restart. Verified by omitting the middle one: `(ota-request-console!)` succeeds, the board resets, and the transcript carries the application's own startup banner and not one loader line.

NO ALLOCATION AND NO VM CALLS, for the same reason as its two siblings: this path runs when the evaluator cannot.

ota_mop3_reboot_to_loader()

```
Sexpr_t ota_mop3_reboot_to_loader (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

ONE COPY OF THE REDIRECT, shared with the watchdog path – see ll_ota_boot_factory_raw(). It reports only success or failure, so the two reasons are distinguished here by asking again rather than by a second implementation.

ota_mop3_boot_ok()

```
Sexpr_t ota_mop3_boot_ok (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-boot-ok!)` — confirm THIS image is healthy: cancel the native OTA rollback and clear `pending_update` + `install_tries` (the loader's anti-loop counter). The loader's install-retry guard waits for this. Returns #t (vacuously #t off-device).

DO NOT CLEAR THE DEATH COUNT HERE, however natural it looks. This runs EARLY in a boot, to say "this image starts"; the death count exists to say "this image does not keep WORKING". Clearing it here would reset the streak before the failure that increments it, so a unit that dies every cycle would flap between the VM and the loader forever and never reach the latch threshold. Starting is not health. `(ota-clear-death!)` is the explicit reset, for an application that has evidence it is working.

ll_ota_request_console_raw()

```
int ll_ota_request_console_raw (void )
```

`(ota-record-death! reason)` — persist `reason` and bump the death count; returns the NEW count, or #f if the reason is not a legal symbol (or off-device). Records only: it does not request the console and does not reboot, because what to do after the Nth death is policy and policy belongs in Scheme. Call it through `(dead! reason)`.

Ask the loader for its CONSOLE rung without the evaluator. True on success.

The companion to ll_ota_record_death_raw: `dead!` in Scheme records, then requests console, then reboots, and the staleness watch must do the same three things in the same order or a wedge stops being equivalent to a declared death. Recording alone leaves the count bumped and nobody told: the count reaches the limit and the unit boots normally, because `pending_update` was never set.

ll_ota_record_death_raw()

```
int ll_ota_record_death_raw (const char * reason)
```

Record a death WITHOUT the evaluator: validate `reason`, bump the count, persist both. Returns the new count, or -1 if the reason is rejected or this is not an ESP32 build.

THE WATCHDOG PATH CANNOT USE THE MOP3, because the watchdog fires precisely when the evaluator cannot run – and the mop3 takes a `Lamb &` and logs through it. So the policy lives here, once, and both callers use it: `(ota-record-death! reason)` for a DECLARED death, and the staleness sampler for a WEDGE. Two copies of "how a death is recorded" would be the hand-maintained-second-copy defect, drifting in the direction that matters most – the wedge path is the one nobody exercises by hand.

NO ALLOCATION AND NO VM CALLS: NVS only. The sampler that calls this is watching the allocator, so allocating here would move the very counter it is reading. < saturate: a wrap would read as healthy

ota_mop3_record_death()

```
Sexpr_t ota_mop3_record_death (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

REFUSED, NOT SUBSTITUTED. A default reason here would be this function's opinion about a failure it knows nothing about, recorded as if the application had said it.

DELEGATES, so a declared death and a wedge cannot diverge – see ll_ota_record_death_raw().

ota_mop3_clear_death()

```
Sexpr_t ota_mop3_clear_death (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-clear-death!)` — erase the death record and reset the death count to zero. Call it when the application has evidence it is WORKING, not merely that it has started. Returns #t.

ota_mop3_death_record()

```
Sexpr_t ota_mop3_death_record (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(ota-death-record)` — the persisted death record as `(reason . count)`, or #f if this unit has not declared itself dead since the last `(ota-boot-ok!)`. Lets a VM that came back up report why it went down.

ota_install_mop3()

```
Sexpr_t ota_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Installer: bind the OTA loader-handshake builtins (`ota-request-install!` / `-recover!` / `-console!`, `ota-set-network!`, `ota-reboot-to-loader`, `ota-boot-ok!`) into `env_target`.

12.1.8 PCA9685 PWM / Servo Driver

Functions

- Sexpr_t PCA9685_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t PCA9685_mop3_setupSingleDevice (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t PCA9685_mop3_setSingleDeviceToFrequency (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_↩ exec)
- Sexpr_t PCA9685_mop3_getSingleDeviceFrequency (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t PCA9685_mop3_setChannelDutyCycle (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t PCA9685_mop3_setChannelPulseWidth (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t PCA9685_mop3_setChannelServoPulseDuration (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env↩ _exec)
- Sexpr_t PCA9685_mop3_setChannelOnAndOffTime (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **PCA9685_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- PCA9685 * **pca9685** = 0

Detailed Description

LambLisp PCA9685 PWM / Servo Driver builtins.

Function Documentation

PCA9685_mop3_begin()

```
Sexpr_t PCA9685_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Initialize PCA9685 at I2C addr via Wire; sets 50 Hz PWM frequency.

PCA9685_mop3_setupSingleDevice()

```
Sexpr_t PCA9685_mop3_setupSingleDevice (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Configure a single PCA9685 device on the Wire bus at the given I2C address.

PCA9685_mop3_setSingleDeviceToFrequency()

```
Sexpr_t PCA9685_mop3_setSingleDeviceToFrequency (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set PWM oscillator frequency (Hz) for a single device at the given I2C address.

PCA9685_mop3_getSingleDeviceFrequency()

```
Sexpr_t PCA9685_mop3_getSingleDeviceFrequency (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read back the PWM oscillator frequency (Hz) for a single device at the given address.

PCA9685_mop3_setChannelDutyCycle()

```
Sexpr_t PCA9685_mop3_setChannelDutyCycle (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set channel duty cycle as a fraction [0.0, 1.0] with optional phase shift fraction.

PCA9685_mop3_setChannelPulseWidth()

```
Sexpr_t PCA9685_mop3_setChannelPulseWidth (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set channel pulse width in raw 12-bit ticks (0–4095).

PCA9685_mop3_setChannelServoPulseDuration()

```
Sexpr_t PCA9685_mop3_setChannelServoPulseDuration (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set channel servo pulse duration in microseconds.

PCA9685_mop3_setChannelOnAndOffTime()

```
Sexpr_t PCA9685_mop3_setChannelOnAndOffTime (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set channel on-time and off-time in raw 12-bit ticks; on-time anchored at 0.

12.1.9 Platform (generic)

Functions

- Sexpr_t Platform_mop3_free_heap (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_cpu_mhz (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_cell_pool_free (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_free_internal** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_free_dma** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_largest_free_internal** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_largest_free_dma (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_fs_total (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_fs_used** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_free_stack (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_root_depth (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_root_depth_max** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_root_depth_max_reset (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_eval_progress (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- bool ll_progress_watch_start (uint32_t stale_ms, uint32_t poll_ms)
- bool **ll_progress_watch_stats** (uint32_t *max_quiet_ms, uint32_t *stalls, uint32_t *poll_ms)
- bool **ll_progress_watch_arm** (uint32_t stale_ms, uint32_t poll_ms)
- Sexpr_t Platform_mop3_progress_watch_start (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_progress_watch_stats (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_progress_watch_arm (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_micros (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_llarduino_backend (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

- Sexpr_t **Platform_mop3_llarduino_fmt_int** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_llarduino_fmt_real** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_llarduino_eol** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_millis** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_loop_elapsed_ms (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_loop_elapsed_us (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_lamb_reboot (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_exit (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_delay_ms (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_gc_idle_task (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_gc_collect (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_random_bytevector (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_random_integer (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_random_real (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_cell_block_size (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_n_cell_blocks (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_total_cells (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_nfree (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_max_cell_blocks (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_set_max_cell_blocks (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Platform_mop3_extension_block_size** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_set_extension_block_size (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_expand_to_n_blocks (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_sysmem_inuse (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_gc_budget_ns (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_gc_budget_achieved_ns (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_set_gc_budget_ns (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_gcload_target_pct (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_set_gcload_target_pct (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_verbosity (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_set_verbosity (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_repl_quiet (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Platform_mop3_build_utc (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- const char ∗const ∗ ll_feature_names (int ∗count)
- Sexpr_t __lamb_platform_generic_install_mop3 (Lamb &lamb, Sexpr_t list_containing_env_target, Sexpr_t env_exec)

Variables

- LambPlatform **lambPlatform**

Detailed Description

LambLisp Platform (generic) builtins.

Function Documentation

Platform_mop3_free_heap()

```
Sexpr_t Platform_mop3_free_heap (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return bytes of free heap memory on the platform.

Platform_mop3_cpu_mhz()

```
Sexpr_t Platform_mop3_cpu_mhz (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

CPU clock in MHz as the running system reports it, or 0 where the platform reports none.

A timing is only comparable with another timing taken at the same clock, and the clock a target boots at is a property of its boot configuration rather than of the firmware it is running: an image compiled for one frequency can come up at another. The platform prints its frequency once during start-up, which is of no use to a measurement taken minutes later.

So ask for it from inside the measurement you mean to keep, and record it with the result:

```
(display "cpu-mhz ") (display (Platform.cpu-mhz)) (newline)
```

Returns 0 on hosts, where the clock moves continuously under frequency scaling and no single figure would still be true when it was read.

Platform_mop3_cell_pool_free()

```
Sexpr_t Platform_mop3_cell_pool_free (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

THE POOL CELL BLOCKS AND PAYLOADS SHARE. `Platform.free-heap` is `esp_get_free_heap_size()` – the AGGREGATE of internal DRAM and PSRAM – which cannot see the pool that actually runs out. `cell_pool_free()` reports the pool cell blocks are allocated from, and `ll_platform_generic.h` already names the failure this exposes: the cell heap grows until PSRAM is full and "starves the PAYLOADS (strings, bignum digits, vectors, bytecode) that share that pool".

EXPOSED BECAUSE THE COLLAPSE IS UNINSTRUMENTED. 951 KB can disappear between the last reading and the first `alloc_exec` failure, with ZERO samples in between – every diagnostic that fires is downstream of the event. A Scheme-visible reading lets `bench-timed/run` bracket the loss to one benchmark and one phase, which is the one thing none of the existing instrumentation can do.

Platform_mop3_largest_free_dma()

```
Sexpr_t Platform_mop3_largest_free_dma (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.largest-free-dma) – largest CONTIGUOUS DMA-capable block, in bytes.

WHY THIS EXISTS, AND WHY IT REPLACED A PROBE BUILT ON `tone`. Measuring whether a board is still USABLE after something has eaten its internal memory needs an instrument with two properties, and a driver's init call has neither:

IT MUST BE REPEATABLE. `tone()` reaches `tone_init()`, which creates a FreeRTOS queue ONCE. After the first successful call it can never fail again, whatever the memory state – so an instrument built on it reports "fine" forever. Measured on a classic ESP32: a mass NCG pass drove internal free from 26,532 B to 628 B and the tone probe still reported ZERO failures. A ONE-SHOT INITIALISER CANNOT BE A REPEATABLE PROBE. IT MUST NAME THE RESOURCE THAT ACTUALLY STARVES. DMA-capable is STRICTER than internal, and it is what a camera, an I2S ring or a queue needs. Internal headroom can look adequate while every DMA request refuses.

AND LARGEST, NOT FREE, for the reason `ncg_exec_pool_init()` already records: free is not the number that decides a CONTIGUOUS request. It once printed `INTERNAL free=85448` to explain a refusal whose real blocker was a 14,324-byte largest block. Read this beside `Platform.free-dma`, never instead of it – the pair distinguishes exhaustion from fragmentation, and they need different fixes.

Platform_mop3_fs_total()

```
Sexpr_t Platform_mop3_fs_total (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Filesystem space, in bytes. `Platform.fs-total` is the size of the mounted filesystem and `Platform.fs-used` is what is currently occupied; free space is the difference. Both report 0 where no filesystem is mounted, so "no filesystem" is distinguishable from "full filesystem".

WHY AN APPLICATION SHOULD READ THESE BEFORE A SUSTAINED WRITE. Flash write latency is not flat: it rises sharply as the filesystem fills, because the allocator has to work harder to find and erase space. Measured on an ESP32-S3 with a 128-block partition, one chunked file transfer: 76.7 ms per chunk round trip with the filesystem clean, and 1,211 ms per chunk with ∼82 KB of other files present – about sixteen times slower, appearing when those files are added and disappearing when they are removed. The writes still succeed and their checksums still match, so nothing fails and nothing warns; a program with a deadline simply starts missing it, and the delay looks like it came from somewhere else entirely.

So a program that must meet a deadline while writing should check free space first and degrade deliberately – write less, write elsewhere, or decline – rather than discover the cost.

Platform_mop3_free_stack()

```
Sexpr_t Platform_mop3_free_stack (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return bytes of free stack space on the platform.

Platform_mop3_root_depth()

```
Sexpr_t Platform_mop3_root_depth (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.root-depth) and (Platform.root-depth-max) – how much of the GC root stack is in use now, and the deepest it has ever been. (Platform.root-depth-max!) re-baselines the peak to the current depth so one piece of work can be measured on its own.

WHAT THE ROOT STACK IS, IN ONE SENTENCE: the runtime records there every heap value it is holding on behalf of code that is mid-execution, so the collector cannot reclaim something still in use. Its capacity is fixed at build time, and exhausting it raises a catchable error rather than corrupting anything – so this pair is a capacity-planning instrument, not a fault detector.

READ THEM AS A PAIR, AND PREFER THE PEAK. The instantaneous depth is whatever happens to be held at the moment you ask, which on an idle REPL is near nothing; it cannot tell you what a workload DID. Two very different programs look identical in a single reading – one that nests procedure calls deeply, and one that iterates a long loop accumulating held values – and they have opposite remedies. The peak distinguishes them: deep nesting is bounded by how far the program recurses, while a loop's peak grows with its ITERATION COUNT, so a peak that scales with the size of the input names a loop.

HOW TO USE IT. Call (Platform.root-depth-max!) immediately before the work and (Platform.root-depth-max) immediately after; the difference is that work's demand. Run it at two input sizes. A figure that stays flat as the input grows is bounded by the program's structure and will stay safe on a larger input; one that doubles when the input doubles will not, and the input size at which it would exhaust the stack is worth computing before a deployment rather than discovering in one.

THE PEAK IS NOT RESET BY A COLLECTION and does not fall when values are released – it is a high-water mark, deliberately, because the number that matters for capacity is the worst moment the program has ever reached and not the comfortable one it is in now.

Platform_mop3_root_depth_max_reset()

```
Sexpr_t Platform_mop3_root_depth_max_reset (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

$<$ the new baseline, so a caller can record it

Platform_mop3_eval_progress()

```
Sexpr_t Platform_mop3_eval_progress (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.eval-progress) – a monotonic token that advances as the evaluator allocates.

WHAT IT IS FOR. A long computation and a stalled interpreter are indistinguishable from outside the VM: both look like a board that is not answering. From inside they are easy to separate – a long computation keeps allocating and a stalled one does not. Sampling this value, waiting, and sampling again therefore distinguishes SLOW from STOPPED, which no timer can do on its own.

HOW TO USE IT. Compare two samples for equality and nothing else: the value is an opaque token, not a count, and it wraps. Sample it from a timer, a supervisory task or a monitoring loop – never from the computation being watched, which is the thing that may be stuck.

READING IT ALLOCATES, so two consecutive reads differ by a few counts even with nothing in between. Compare the SIZE of a delta against a known-idle baseline, not its mere presence: a delta of a handful means nothing happened, a delta of thousands means work is being done.

ll_progress_watch_start()

```
bool ll_progress_watch_start (uint32_t stale_ms, uint32_t poll_ms)
```

Declared here rather than #included from "ll_vm_progress_watch.h".

THIS FILE SHIPS IN THE CUSTOMER PACKAGE AND THAT HEADER DOES NOT. The release cut copies `src/` minus every `ll_vm_*` and `w3*` file, so an `#include "ll_vm_<anything>.h"` here compiles fine in this tree and fails in the customer's with *"ll_vm_progress_watch.h: No such file or directory"* – a build we do not run and they do. (`ll_wasm_browser.cpp` includes `ll_vm_term.h` and is safe only because the include sits inside its `#if LL_EMSCRIPTEN`.) The SYMBOL is fine: it is compiled into the prebuilt `liblamblisp.a` that ships alongside, so the link resolves – it is only the header text that is absent. Keep shipped files declaring what they need from the VM, or get it from `LambLisp.h`, which does ship.

Platform_mop3_progress_watch_start()

```
Sexpr_t Platform_mop3_progress_watch_start (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.progress-watch-start MS) – begin OBSERVING whether the evaluator stalls.

Starts a low-priority sampler that reads the progress token every POLL-MS milliseconds (100 by default, floored at 10) and prints one line if the evaluator has not allocated for MS milliseconds. POLL-MS is the RESOLUTION of the margin that `Platform.progress-watch-stats` reports, not merely a cost: a gap shorter than one poll cannot be seen at all. **It never resets or interrupts anything.** Returns #t if a sampler started, #f if one was already running, if the task could not be created, or on a platform where sampling is not provided.

WHAT IT IS FOR. A deadline-driven program cannot tell a long computation from a stuck one by watching the clock, and neither can an operator watching a serial port. This reports the difference: one line, naming how long the evaluator was quiet, with no effect on the program. Leave it running during a long soak and read what the worst legitimate quiet period actually is – that measurement is what makes any later automatic recovery safe to set, and guessing it is how a watchdog ends up restarting healthy work.

MS IS FLOORED AT 100. Below that it measures its own sampling noise rather than the program. A sensible starting point is several times the longest uninterruptible step the application performs; a flash erase or a large allocation can legitimately occupy the evaluator for hundreds of milliseconds. POLL-MS IS SETTABLE BECAUSE IT IS THE MARGIN'S RESOLUTION, NOT JUST A COST KNOB. The sampler records the gap between allocations only at poll boundaries, so a workload that allocates at least once per poll reports a gap of exactly POLL-MS every time – the FLOOR, indistinguishable from any smaller gap. Measured on an S3 at the 100 ms default: a 19-second allocating workload reported max_quiet_ms=100, i.e. nothing resolvable. To measure the peak rather than the floor, poll faster; the cost is a wakeup per interval and nothing else, since the task reads one word. Floored at 10 ms in ll_progress_watch_start.

< the floor that matters is enforced below; this only stops a negative cast

Platform_mop3_progress_watch_stats()

```
Sexpr_t Platform_mop3_progress_watch_stats (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.progress-watch-stats) – the MARGIN, as `(max-quiet-ms stalls poll-ms)`, or #f.

Returns #f when no sampler is running, which is deliberately not a list of zeros: a watch that never started and a watch that saw nothing produce the same numbers and mean opposite things.

READ ALL THREE OR NONE. `max-quiet-ms` is the longest interval the evaluator went without allocating, and it is a MARGIN only while `stalls` is 0 – the field records the largest gap observed, so a deliberate stall raises it too and would then be reporting the stall back to you. And it is an UNDERESTIMATE by up to `poll-ms`, because a gap that ends just after a sample is recorded short; the worst true gap is at most `max-quiet-ms + poll-ms`. A threshold taken from the raw figure is optimistic by that much, in the direction that reboots healthy work.

WHAT IT IS FOR: arm the watch, run the real application for as long as you can stand, then read this. The number it gives is how close your workload came to looking stopped – which is the only honest basis for choosing a threshold that is later allowed to reset the unit. BUILT BACK TO FRONT WITH EACH PARTIAL LIST PROTECTED. cons() protects its own arguments, but the list being accumulated is a local value not reachable from `sexpr`, so it needs a root across the next allocation – see the mop3 rule in CLAUDE.md. ONE FRAME, THREE PUSHES, NO POPS. A frame records the rootstack depth on entry and restores it on exit, so each re-push needs no matching pop and the depth cannot drift. The re-push after every assignment is required, not redundant: a push records the VALUE at that instant, and `res` is a different cell after each `cons`.

Platform_mop3_progress_watch_arm()

```
Sexpr_t Platform_mop3_progress_watch_arm (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.progress-watch-arm MS [POLL-MS]) – let a stall RESET the unit.

Until this is called the watch only reports. Armed, a stall of MS milliseconds records the reason `wedged`, bumps the same death count a declared `(dead!)` uses – one anti-loop limit, not two – and resets.

THE ACTUATORS ARE LEFT WHERE THE RESET LEAVES THEM. That is a deliberate decision, not an oversight: the mechanism that knows how to stop this machine runs IN the evaluator, and the watch fires precisely when the evaluator cannot run. Of the three ways out – a separate safing task, a board that de-energises on reset, or accepting it – the owner chose to accept it. On a machine that can move, weigh that before arming.

SO ARM IT LATE AND SET MS GENEROUSLY. A false fire costs a reboot with the outputs wherever they were, which is the expensive direction. Measured on an S3: a real workload's worst quiet period is 330-385 ms during STARTUP and 10-30 ms once running, so arming after the application has loaded puts a 1000 ms threshold ∼30x away instead of ∼2.6x. Read `(Platform.progress-watch-stats)` on your own workload before choosing.

Platform_mop3_micros()

```
Sexpr_t Platform_mop3_micros (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return microseconds since boot as an integer.

Platform_mop3_llarduino_backend()

```
Sexpr_t Platform_mop3_llarduino_backend (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return milliseconds since boot as an integer. LLArduino introspection. These exist so the layer's Arduino-fidelity guarantees are ASSERTED rather than asserted-in-a-comment: (llarduino-fmt-int 255 16) must be "ff" with no 0x prefix, (llarduino-fmt-real 1.5 2) must be "1.50" not "1.5", and (llarduino-eol) must be "\r\n" – all rendered through the real Print, so a divergence from a device shows up as a test failure.

Platform_mop3_loop_elapsed_ms()

```
Sexpr_t Platform_mop3_loop_elapsed_ms (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return elapsed milliseconds since the start of the current Arduino loop() call.

Platform_mop3_loop_elapsed_us()

```
Sexpr_t Platform_mop3_loop_elapsed_us (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return elapsed microseconds since the start of the current Arduino loop() call.

Platform_mop3_lamb_reboot()

```
Sexpr_t Platform_mop3_lamb_reboot (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Reboot the platform (embedded) or do nothing (POSIX).

Platform_mop3_exit()

```
Sexpr_t Platform_mop3_exit (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Flush stdio and exit (POSIX) or reboot (embedded).

< release platform resources

< embedded: reboot after cleanup

< unreachable

Platform_mop3_delay_ms()

```
Sexpr_t Platform_mop3_delay_ms (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Delay for the given number of milliseconds using platform delay().

Platform_mop3_gc_idle_task()

```
Sexpr_t Platform_mop3_gc_idle_task (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Run the GC idle task for up to the given number of microseconds.

Platform_mop3_gc_collect()

```
Sexpr_t Platform_mop3_gc_collect (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Run one complete GC cycle (mark + sweep); use before benchmarks to reset GC phase.

Platform_mop3_random_bytevector()

```
Sexpr_t Platform_mop3_random_bytevector (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a bytevector of n cryptographically random bytes.

Platform_mop3_random_integer()

```
Sexpr_t Platform_mop3_random_integer (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a cryptographically random integer.

Platform_mop3_random_real()

```
Sexpr_t Platform_mop3_random_real (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a cryptographically random float32 in [0, 1).

$<$ [0, 2^64-1] $*$ 2^-64 $\rightarrow$ [0,1) exact

Platform_mop3_cell_block_size()

```
Sexpr_t Platform_mop3_cell_block_size (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the number of cells in the first (fixed) memory block.

Platform_mop3_n_cell_blocks()

```
Sexpr_t Platform_mop3_n_cell_blocks (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the current number of allocated memory blocks.

Platform_mop3_total_cells()

```
Sexpr_t Platform_mop3_total_cells (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

THE CELL POOL, computed ONCE in the VM. Callers used to reconstruct it as nblocks ∗ cell_block_size which is right only if every block is the same size. With a 65,536-cell base block and 8,192-cell expansions that overstated the pool by 7.2x (4,194,304 reported against a true 581,632), and every occupancy figure derived from it was wrong by the same factor. total_cells() is baseBlockSize + NexpansionBlocks ∗ expansionBlock↩ Size – ask the VM, do not re-derive it.

Platform_mop3_nfree()

```
Sexpr_t Platform_mop3_nfree (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the current number of free cells in the GC heap.

Platform_mop3_max_cell_blocks()

```
Sexpr_t Platform_mop3_max_cell_blocks (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the maximum number of GC blocks allowed before the memory_cap_reached warning fires.

Platform_mop3_set_max_cell_blocks()

```
Sexpr_t Platform_mop3_set_max_cell_blocks (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the maximum number of GC blocks before memory_cap_reached warning fires.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Platform_mop3_set_extension_block_size()

```
Sexpr_t Platform_mop3_set_extension_block_size (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the number of cells in each future extension memory block.

Platform_mop3_expand_to_n_blocks()

```
Sexpr_t Platform_mop3_expand_to_n_blocks (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Expand the heap to at least n blocks; allocates extension blocks as needed. Returns #t when n blocks are present, #f when expansion stopped short – the heap may decline to grow when the memory reserved for strings, vectors and bignums would be infringed, and logs why.

Platform_mop3_sysmem_inuse()

```
Sexpr_t Platform_mop3_sysmem_inuse (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Platform.gc-diag) -> (pass-max-us urgent-max-us urgent-calls). Reported through the RESULT of a mop3 so the bench can `display` it – warn() output never survives the embedded path (the REPL quiet flag at the source, plus the serial harness dropping any line naming LambMemory or gc_pass), so a warn-based probe silently reports nothing and looks like a clean result. (Platform.sysmem-inuse) -> bytes currently held by the SYSTEM allocator, or -1.

THE CELL POOL CANNOT ANSWER THE FINALIZATION QUESTION, which is why this exists. A vector, bytevector or heap string keeps its ELEMENTS in a malloc'd block and only its handle in a cell – measured: a `(make-vector 5000)` moved the free-cell count by FIVE. So a backing block that is never freed when its cell is collected is a real leak that every cell-based counter reports as clean. This reads the allocator directly.

-1 means "this build cannot tell", NOT "zero leaked": mallinfo2 is glibc-only, so wasm, MinGW and the ESP32 return -1 and a caller must treat that as UNKNOWN and skip, never as a pass.

Platform_mop3_gc_budget_ns()

```
Sexpr_t Platform_mop3_gc_budget_ns (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the GC time budget per loop iteration in nanoseconds.

Platform_mop3_gc_budget_achieved_ns()

```
Sexpr_t Platform_mop3_gc_budget_achieved_ns (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

The budget the collector ACTUALLY gets per quantum, in nanoseconds: `mark_quantum x t_mark`. Compare it with `Platform.gc-budget-ns`, which is what was REQUESTED. They diverge when the request is below twice the measured per-cell cost, because a quantum cannot be smaller than one cell – so a tighter request is silently not honoured, and this is how a caller sees that.

Platform_mop3_set_gc_budget_ns()

```
Sexpr_t Platform_mop3_set_gc_budget_ns (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the GC time budget per loop iteration in nanoseconds.

Platform_mop3_gcload_target_pct()

```
Sexpr_t Platform_mop3_gcload_target_pct (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the target GC load as a percentage of loop time.

Platform_mop3_set_gcload_target_pct()

```
Sexpr_t Platform_mop3_set_gcload_target_pct (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the target GC load percentage (0–100).

Platform_mop3_verbosity()

```
Sexpr_t Platform_mop3_verbosity (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the current verbosity level (0 = silent, higher = more logging).

Platform_mop3_set_verbosity()

```
Sexpr_t Platform_mop3_set_verbosity (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the verbosity level.

Platform_mop3_repl_quiet()

```
Sexpr_t Platform_mop3_repl_quiet (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Quiet REPL: enable/disable line-mode quiet console (suppress async log() output and the per-char redisplay() prompt-reprint). The LLIP tunnel uses this – (llt) sets it on, (en) clears it – so a tool's frames aren't garbled by the prompt-redraw flood.

Platform_mop3_build_utc()

```
Sexpr_t Platform_mop3_build_utc (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

`(lamb-build-utc)` -> the compile-time build stamp of the RUNNING binary.

WHY THIS NEEDS TO BE REACHABLE FROM SCHEME. It is the one value that identifies WHICH BINARY is executing, and until now it existed only in the boot banner – readable solely from a serial transcript, which requires a RESET to capture. That makes it unusable for the thing it is most needed for: confirming an OTA replaced the image, over the network, on a board with no cable attached. Resetting to find out defeats the purpose.

IT IS THE ONE OBSERVABLE NO TOOL IN THE CHAIN CAN FABRICATE. The binary reports its own identity, so a verb cannot claim a successful update by exiting 0 – and several verbs in this toolset have been found doing exactly that. A changed stamp is proof the image changed; an unchanged stamp is proof it did not, whatever anything else reported.

NOTE it dates the BUILD, not the source: a dirty tree produces a binary that is no commit at all. Use it to answer "is this a different binary than before", which is a question it answers exactly, and never "which commit is this".

ll_feature_names()

```
const char *const * ll_feature_names (int * count)
```

The feature identifiers reported by (features) and matched by cond-expand.

(features) – R7RS (scheme base): the list of feature identifiers this implementation provides (kept consistent with cond-expand).

ONE list, two consumers – see the declaration in `ll_platform_generic.h` for why.

Only MECHANICALLY DERIVABLE identifiers belong here: the implementation, the platform, the CPU and the byte order, each read from a compile-time macro so it cannot disagree with the binary it is compiled into. A feature that asserts a LANGUAGE property – `full-unicode`, `exact-closed`, `ratios` – is a promise about behaviour and is added only once something tests that the promise holds. Claiming one that is only PARTLY true is worse than omitting it: a program guarding on it takes the wrong branch and says nothing, whereas a missing feature makes it take the conservative branch. `full-unicode` is the live example – the reader and writer handle every codepoint to U+10FFFF, but `char-upcase` and friends cover ASCII and Latin-1 only, so the feature is NOT reported. Platform. LL_WINDOWS and LL_WASM both imply LL_POSIX and then subtract from it, so they are tested FIRST and the posix/unix/linux arm excludes them – see the LL_WINDOWS and LL_WASM commentary in ll_platform_generic.h.

CPU.

Byte order.

__lamb_platform_generic_install_mop3()

```
Sexpr_t __lamb_platform_generic_install_mop3 (Lamb & lamb, Sexpr_t list_containing_env_target, Sexpr_t env_exec)
```

REGISTERED ONLY WHEN THE INSTRUMENT IS COMPILED IN, so `(defined? 'Platform.root-depth-max)` is a TRUE answer about this build. A -1 return is the right signal for a C++ caller and the wrong one as the ONLY signal for a Scheme caller: root-depth-tests.scm guards its depth-boundedness arm on `defined?` and carries a correct skip branch ("depth boundedness is unmeasured on this build"), and leaving these bound with LL_ROOT_HIGH_WATER=0 made that skip UNREACHABLE – the arm then compared p-base/p-sm/p-big/p-loop, every one of them -1, so every comparison held and the suite reported depth boundedness as PROVEN on a board while measuring nothing. A degenerate pass is worse than a failure: it is indistinguishable from a real result and it removes the reason to look again. Caught before it ran on hardware. `Platform.root-depth` STAYS REGISTERED unconditionally – it reads Nroots directly, costs nothing and is always true.

12.1.10 Ultrasonic Sonar (HC-SR04)

Macros

- #define LL_SONAR_NO_DEVICE_GUARD 0
- #define SONAR_REQUIRE_DEVICE()

Functions

- void IRAM_ATTR sonar_echo_isr (void *arg)
- Sexpr_t Sonar_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_start (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_result (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_ping (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_poke (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_us_to_distance (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_set_timeout_us (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_set_min_interval_us (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_set_rest_after_echo_us (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_mop3_defaults (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Sonar_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- Sonar * sonar = 0

Detailed Description

Copyright 2026 by Frobenius Norm LLC 2026-04-23 00:00:00.

< esp_timer_get_time(): the IRAM-resident clock the ISR needs

LambLisp Ultrasonic Sonar (HC-SR04) builtins.

Macro Definition Documentation

LL_SONAR_NO_DEVICE_GUARD

```
#define LL_SONAR_NO_DEVICE_GUARD 0
```

Return `#f` from a sonar procedure when the device is absent or not yet begun. `#f` is already these procedures' "no reading", so a caller that handles a timeout handles a missing sensor.

SONAR_REQUIRE_DEVICE

```
#define SONAR_REQUIRE_DEVICE()
```

Value:

```
do { if (!sonar || !sonar->begun) return HASHF; } while (0)
```

Function Documentation

sonar_echo_isr()

```
void IRAM_ATTR sonar_echo_isr (void * arg)
```

FALLING-edge ISR: ECHO pin went LOW – echo received (or timeout on sensor side).

`esp_timer_get_time()` AND NOT `micros()`, AND THE FUNCTION BEING `IRAM_ATTR` IS WHY THAT MATTERS RATHER THAN WHY IT DOES NOT. An ISR marked IRAM_ATTR is placed in IRAM so it can run while the flash cache is disabled – but the marking covers THIS function only, not what it calls. Arduino's `micros()` lives in FLASH on this core (`esp32-hal-misc.c`; measured at `0x40170ba4` against `esp_timer_get_time` at `0x4008393c`), so calling it from here jumped out of IRAM the moment a flash operation had the cache off, and the core panicked:

```
Guru Meditation Error: Core 1 panic'ed (Cache error).
Cache disabled but cached memory region accessed
```

The backtrace comes out CORRUPTED, so the panic names neither this file nor the sonar – the only clue is the GPIO number sitting in A2/A4. Reproduced by `(sd-mount)` with a floating echo pin: `esp_vfs_fat_↩sdmmc_mount` disables the cache and the floating input supplies the edge. Any flash write will do it – LittleFS, NVS, OTA – so this is not SD-specific, and a board that has never mounted a filesystem can carry the fault indefinitely without showing it.

`micros()` is itself just `(unsigned long) esp_timer_get_time()`, so this is the same number with the same 32-bit wrap, and `result()`'s arithmetic is unchanged.

THE GENERAL FORM, worth carrying to any other ISR here: `IRAM_ATTR` places THIS function in IRAM and says nothing about its callees. An ISR is only as interrupt-safe as the deepest thing it calls. < IRAM-safe; micros() is not

Sonar_mop3_begin()

```
Sexpr_t Sonar_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Initialize the Sonar singleton with trigger and echo pin numbers.

Sonar_mop3_start()

```
Sexpr_t Sonar_mop3_start (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Start a ping. On ESP32: non-blocking. On other platforms: blocks on pulseIn.

Sonar_mop3_result()

```
Sexpr_t Sonar_mop3_result (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Returns: #f – echo not yet received (still within timeout window) 0 – timed out, no echo; caller should re-arm with Sonar.start integer – elapsed microseconds; caller should convert and re-arm

Sonar_mop3_ping()

```
Sexpr_t Sonar_mop3_ping (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Blocking ping: fire trigger, wait for echo, return elapsed microseconds or #f on timeout.

Sonar_mop3_poke()

```
Sexpr_t Sonar_mop3_poke (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Blocking ping converted to distance in meters; returns #f on timeout.

Sonar_mop3_us_to_distance()

```
Sexpr_t Sonar_mop3_us_to_distance (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Convert elapsed microseconds to distance in meters; optional Celsius temperature arg.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Sonar_mop3_set_timeout_us()

```
Sexpr_t Sonar_mop3_set_timeout_us (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Sonar.timeout-us! <us>) – set the no-echo timeout at runtime; returns the effective (clamped-to-range-floor) value so the caller can confirm it.

Sonar_mop3_set_min_interval_us()

```
Sexpr_t Sonar_mop3_set_min_interval_us (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Sonar.min-interval-us! <us>) – set the HC-SR04 inter-ping settle gap at runtime; returns the effective value. Larger = steadier readings, lower max ping rate.

Sonar_mop3_set_rest_after_echo_us()

```
Sexpr_t Sonar_mop3_set_rest_after_echo_us (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Sonar.rest-after-echo-us! <us>) – set the required REST measured from the ECHO (or from a declared timeout) to the next SEND; returns the effective value. 0 DISABLES it, leaving min-interval-us! the only gate. The default is NOT 0 – this gate is on; the value is SONAR_REST_AFTER_ECHO_DEFAULT_US. Never shortens min-interval-us!: both gates must pass, so this can only make the driver wait longer.

Sonar_mop3_defaults()

```
Sexpr_t Sonar_mop3_defaults (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Sonar.defaults!) – restore every runtime tunable to its compiled default. Returns the three effective values as (timeout-us min-interval-us rest-after-echo-us), so a caller can assert the reset rather than trusting it.

Sonar_install_mop3()

```
Sexpr_t Sonar_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Install all Sonar symbols in base environment.

Variable Documentation

sonar

```
Sonar* sonar = 0
```

Sonar singleton.

12.1.11 SPI Bus

Functions

- Sexpr_t SPI_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_beginTransaction (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_endTransaction (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_transfer (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_transfer_bytevector (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_write_bytevector (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_mop3_end (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t SPI_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp SPI Bus builtins.

Function Documentation

SPI_mop3_begin()

```
Sexpr_t SPI_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.begin [sck miso mosi]) – start the HW SPI peripheral on the given pins (or defaults).

SPI_mop3_beginTransaction()

```
Sexpr_t SPI_mop3_beginTransaction (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.beginTransaction clock-hz [mode]) – open a transaction; mode 0..3 (default 0).

SPI_mop3_endTransaction()

```
Sexpr_t SPI_mop3_endTransaction (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.endTransaction) – close the current transaction.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

SPI_mop3_transfer()

```
Sexpr_t SPI_mop3_transfer (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.transfer byte) -> byte – full-duplex single-byte exchange.

SPI_mop3_transfer_bytevector()

```
Sexpr_t SPI_mop3_transfer_bytevector (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.transfer-bytevector! bv) – in-place full-duplex block transfer; returns bv.

SPI_mop3_write_bytevector()

```
Sexpr_t SPI_mop3_write_bytevector (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.write-bytevector bv) – write-only (MOSI), MISO discarded; returns byte count.

SPI_mop3_end()

```
Sexpr_t SPI_mop3_end (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(SPI.end) – release the SPI peripheral.

SPI_install_mop3()

```
Sexpr_t SPI_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Install all SPI symbols in base environment.

12.1.12 WiFi

Enumerations

- enum LL_WifiPref { **LL_WIFI_PREF_ABSENT** = 0 , **LL_WIFI_PREF_OFF** , **LL_WIFI_PREF_ON** }

Functions

- Sexpr_t WiFi_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_disconnect (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_reconnect (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_config (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_setDNS (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_waitForConnectResult (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_isConnected (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_setAutoReconnect (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_SSID (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_BSSID (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_RSSI (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_scanChannel (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_localIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_subnetMask (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_gatewayIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_broadcastIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_networkID (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_dnsIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_subnetCIDR (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_psk (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_macAddress (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_encryptionType (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_status (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_mode (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_getHostname (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_setHostname (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_setSleep (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_channel (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_scanNetworks (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_hostByName (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPConfig (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPdisconnect (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPgetStationNum (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPBroadcastIP (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPSSID (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPmacAddress (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPgetHostname (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WiFi_mop3_softAPsetHostname (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- void ll_wifi_boot_reserve ()
- Sexpr_t WiFi_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- WiFiClass * LL_WiFi = &WiFi

Detailed Description

LambLisp WiFi builtins.

Enumeration Type Documentation

LL_WifiPref

```
enum LL_WifiPref
```

WILL THIS BOARD ACTUALLY RAISE A RADIO? Answered HERE, in the installer, because the reservation is made here and a speculative reservation is not free.

WHY THIS IS NOT "PARSING SCHEME IN C++": it is not evaluating anything. setup.scm reads Settings.scm and then Settings-local.scm (which OVERRIDES it – setup.scm:120-129) long after this point, and `(setting 'wifi 0)` is a plain assq over the result. All we need is the value of one key, and LittleFS is already mounted (ll_heap_note("after LittleFS.begin") precedes the installer loop), so we read the file and look for it.

IT FAILS TOWARD TODAY'S BEHAVIOUR. Cannot open the file, cannot find the key, cannot parse the value -> return false -> NO reservation -> the radio allocates as it would with no reservation mechanism at all. A wrong answer costs ordering quality; it can never cost correctness. That matters because the cost of guessing WRONG is asymmetric: on a part whose internal DRAM is tight, reserving ∼48 KB for a radio that never starts can push the NCG exec pool's claim past the largest free block, and the board then runs with NO native code at all – having held memory for a radio it does not use.

KEEP IN STEP WITH scm/core/WiFi.scm:19, which is the other half of this decision: (when (positive? (setting 'wifi 0)) (WiFi.begin ...)) If that guard changes, this must change with it. Two readers of one key is a drift risk, and the mitigation is that this one fails safe while that one is authoritative.

TRI-STATE, BECAUSE "NO KEY" AND "KEY SAYS 0" ARE DIFFERENT FACTS. Collapsing them into one bool is what made the boot message assert a conclusion about the BOARD ("this board does not auto-connect at boot") when what had actually happened was that the settings could not be read. A correct configuration and an unreadable one printed the same sentence, so the operator's next move was to check the hardware rather than the file.

Function Documentation

WiFi_mop3_begin()

```
Sexpr_t WiFi_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Connect to an access point with SSID and optional password; returns wl_status_t.

HAND THE RESERVATION BACK IMMEDIATELY BEFORE THE RADIO CLAIMS, so the stack lands in the hole taken at install time rather than in whatever the .scm load chain has left. Same shape as EyeCam's camera hole. Nothing allocates internal DRAM between here and begin().

WiFi_mop3_disconnect()

```
Sexpr_t WiFi_mop3_disconnect (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Disconnect from WiFi; optional args wifioff and eraseap.

WiFi_mop3_reconnect()

```
Sexpr_t WiFi_mop3_reconnect (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Reconnect to the last access point; returns #t on success.

WiFi_mop3_config()

```
Sexpr_t WiFi_mop3_config (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set static IP configuration; all args are IP strings or #f for INADDR_ANY.

WiFi_mop3_setDNS()

```
Sexpr_t WiFi_mop3_setDNS (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Reconfigure DNS servers without changing IP/GW/subnet.

WiFi_mop3_waitForConnectResult()

```
Sexpr_t WiFi_mop3_waitForConnectResult (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Block until connected or timeout (default 60000 ms); returns wl_status_t integer.

WiFi_mop3_isConnected()

```
Sexpr_t WiFi_mop3_isConnected (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

NOT STARTED IS NOT CONNECTED. A predicate named `isConnected` must answer the question, never abort the VM – returning #f here is both correct and what any caller expects.

WiFi_mop3_setAutoReconnect()

```
Sexpr_t WiFi_mop3_setAutoReconnect (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Enable or disable automatic reconnection; optional arg defaults to #t (enable).

WiFi_mop3_SSID()

```
Sexpr_t WiFi_mop3_SSID (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(WiFi.SSID) -> the CONNECTED access point's SSID. (WiFi.SSID i) -> the SSID of scan result i, after (WiFi.scanNetworks).

THE INDEX FORM WAS MISSING AND THE ARGUMENT WAS SILENTLY IGNORED, which made `WiFi.scanNetworks` useless: it returns a COUNT, and nothing could read the results behind it. `(WiFi.SSID 0)` returned the connected SSID – an empty string when not associated – so a scan reported "2 networks" and then two blanks, which reads as a broken radio rather than a missing accessor. The question it answers: "which networks can this board actually see" is the first question about a board that will not associate, and the runtime could not answer it. Arduino has had the overload all along; this just stops throwing it away.

WiFi_mop3_BSSID()

```
Sexpr_t WiFi_mop3_BSSID (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the BSSID (AP MAC address) as a colon-separated hex string.

WiFi_mop3_RSSI()

```
Sexpr_t WiFi_mop3_RSSI (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(WiFi.RSSI) -> the connected AP's signal in dBm; (WiFi.RSSI i) -> scan result i. Same missing-overload story as WiFi.SSID above.

WiFi_mop3_scanChannel()

```
Sexpr_t WiFi_mop3_scanChannel (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(WiFi.channel i) -> the channel of scan result i. NEW. Channel is how you tell a 2.4 GHz AP (1-14) from a 5 GHz one (36+) – and an ESP32-S3 is 2.4 GHz ONLY, so an SSID on channel 36 is one this part can never join no matter how strong it is. Worth having when a board "cannot see" a network everything else can.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

WiFi_mop3_localIP()

```
Sexpr_t WiFi_mop3_localIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the local IP address as a dotted-decimal string.

WiFi_mop3_subnetMask()

```
Sexpr_t WiFi_mop3_subnetMask (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the subnet mask as a dotted-decimal string.

WiFi_mop3_gatewayIP()

```
Sexpr_t WiFi_mop3_gatewayIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the gateway IP address as a dotted-decimal string.

WiFi_mop3_broadcastIP()

```
Sexpr_t WiFi_mop3_broadcastIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the broadcast IP address as a dotted-decimal string.

WiFi_mop3_networkID()

```
Sexpr_t WiFi_mop3_networkID (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the network base address as a dotted-decimal string.

WiFi_mop3_dnsIP()

```
Sexpr_t WiFi_mop3_dnsIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the DNS server IP string; optional arg n selects primary (0) or secondary (1).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

WiFi_mop3_subnetCIDR()

```
Sexpr_t WiFi_mop3_subnetCIDR (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the subnet prefix length as an integer (CIDR notation).

WiFi_mop3_psk()

```
Sexpr_t WiFi_mop3_psk (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the WPA/WPA2 pre-shared key as a string.

WiFi_mop3_macAddress()

```
Sexpr_t WiFi_mop3_macAddress (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the station MAC address as a colon-separated hex string.

WiFi_mop3_encryptionType()

```
Sexpr_t WiFi_mop3_encryptionType (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the encryption type of the connected network as an integer.

WiFi_mop3_status()

```
Sexpr_t WiFi_mop3_status (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a list (status-int status-string) describing the current WiFi connection state.

status() reads the same NetworkInterface state isConnected() does, so it is the direct equivalent and gets the same guard. WL_NO_SHIELD is the honest answer for "no interface".

< "%s" even though every `msg`

< above is a literal with no % in it: this overload is PRINTF, so the safety is a property of the TABLE rather than of the call, and the next case added would not announce the difference.

WiFi_mop3_mode()

```
Sexpr_t WiFi_mop3_mode (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set WiFi mode: 0=NULL 1=STA 2=AP 3=APSTA; returns #t on success.

WiFi_mop3_getHostname()

```
Sexpr_t WiFi_mop3_getHostname (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the mDNS hostname as a string.

WiFi_mop3_setHostname()

```
Sexpr_t WiFi_mop3_setHostname (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the mDNS hostname; returns #t on success.

WiFi_mop3_setSleep()

```
Sexpr_t WiFi_mop3_setSleep (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set power-save sleep mode: #t/#f or integer 0=none 1=modem 2=light.

WiFi_mop3_channel()

```
Sexpr_t WiFi_mop3_channel (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the current WiFi channel number.

WiFi_mop3_scanNetworks()

```
Sexpr_t WiFi_mop3_scanNetworks (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Scan for nearby networks and return the count found.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

WiFi_mop3_hostByName()

```
Sexpr_t WiFi_mop3_hostByName (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Resolve a hostname to an IP string; returns #f on failure.

< printf overload – the

< literal format keeps data out of it.

WiFi_mop3_softAP()

```
Sexpr_t WiFi_mop3_softAP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Start a soft access point with SSID and optional password/channel/hidden/max-conn.

WiFi_mop3_softAPConfig()

```
Sexpr_t WiFi_mop3_softAPConfig (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set soft-AP IP configuration; ip, gateway, subnet are dotted-decimal strings.

WiFi_mop3_softAPdisconnect()

```
Sexpr_t WiFi_mop3_softAPdisconnect (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Shut down the soft-AP; optional arg wifioff also disables the radio.

WiFi_mop3_softAPgetStationNum()

```
Sexpr_t WiFi_mop3_softAPgetStationNum (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the number of stations currently connected to the soft-AP.

WiFi_mop3_softAPIP()

```
Sexpr_t WiFi_mop3_softAPIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the soft-AP IP address as a dotted-decimal string.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

WiFi_mop3_softAPBroadcastIP()

```
Sexpr_t WiFi_mop3_softAPBroadcastIP (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the soft-AP broadcast IP address as a dotted-decimal string.

WiFi_mop3_softAPSSID()

```
Sexpr_t WiFi_mop3_softAPSSID (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the soft-AP SSID as a string.

WiFi_mop3_softAPmacAddress()

```
Sexpr_t WiFi_mop3_softAPmacAddress (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the soft-AP MAC address as a colon-separated hex string.

WiFi_mop3_softAPgetHostname()

```
Sexpr_t WiFi_mop3_softAPgetHostname (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the soft-AP mDNS hostname as a string.

WiFi_mop3_softAPsetHostname()

```
Sexpr_t WiFi_mop3_softAPsetHostname (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the soft-AP mDNS hostname; returns #t on success.

ll_wifi_boot_reserve()

```
void ll_wifi_boot_reserve ()
```

Reserve the radio's fast RAM at boot, BEFORE the VM claims internal DRAM.

ORDER IS THE WHOLE POINT OF THIS FUNCTION EXISTING SEPARATELY. The radio needs one CONTIGUOUS block, and the reservation has to be made while a block that size still exists. Taking it at installer time is too late: the VM's internal-DRAM residents are claimed when the interpreter is constructed, which is before any installer runs, so by then the largest free block has already been reduced and the reserve is refused – measured on a classic ESP32 as `wifi reserve 49152 FAILED` with 34,804 bytes largest, against 110,580 before construction.

The refusal is a DEGRADATION, not an error: the consumer then allocates unordered and the radio works. That is why the old ordering survived – nothing fails loudly, and the one line scrolls past in a boot log.

RELEASED TWO WAYS, unchanged: to WiFi.begin() if the radio starts, and by the unclaimed sweep after setup.scm if it does not. A reserve that could not be met is not held at all.

Take the radio's boot-time fast-RAM reservation. Called from `Lamb::setup()` BEFORE the VM claims its internal-DRAM residents, so the reserve is made while a contiguous block that size still exists. No-op where there is no radio or the part is not an ESP32.

WiFi_install_mop3()

```
Sexpr_t WiFi_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

CLAIM THE RADIO'S FAST RAM HERE, NOT WHERE WiFi.scm HAPPENS TO RUN. Measured on esp32-s3-eye: the `when` form in WiFi.scm costs 48,752 bytes and takes the largest contiguous block from 69,620 to 31,732 – mid-way through the .scm chain, after every earlier claimant has already budgeted against a number that is about to be wrong. Reserving at install time means the claim lands while the pool is whole, and the barrier after the installer loop reports a remainder a later claimant can actually trust. RELEASED THREE WAYS: to WiFi.begin() if the radio starts, by the unclaimed sweep after setup.scm if it does not, and not at all if the reserve could not be met (best-effort).

SAY WHAT WAS READ, NOT WHAT IT MEANS. "no key found" is a failure to read the settings; "(wifi . 0)" is an instruction someone wrote. Printing one sentence for both is the defect – it sends whoever is holding a correctly-configured board off to test the hardware. The radio reserve is taken at BOOT, before the VM claims internal DRAM. See ll_wifi_boot_reserve() and its call site in Lamb::setup().

Variable Documentation

LL_WiFi

```
WiFiClass* LL_WiFi = &WiFi
```

POINT THIS AT THE ARDUINO GLOBAL. It was declared and NEVER ASSIGNED anywhere in the repo – so it was a NULL pointer, and every one of the ~40 `LL_WiFi->` call sites below dereferenced null. It did not look like that from the crash: Guru Meditation Error: Core 1 panic'ed (LoadProhibited), EXCVADDR 0x00000008 NetworkInterface::getStatusBits() NetworkInterface.cpp:231 WiFiSTAClass::isConnected() WiFiSTA.cpp:222 which reads as an Arduino 3.x defect – a "null netif in the new NetworkInterface layer" – and was first diagnosed that way. It is ours. `STAClass STA` is a MEMBER of WiFiSTAClass, so with `this == nullptr` the call reaches `_interface_event_group` at offset 8 of a null object: that IS the 0x00000008 in the report. getStatusBits() already null-checks its event group, so a real object cannot panic there. WHY IT SURVIVED ARDUINO 2.0.17: `isConnected()` there read a global and touched no member, so calling it through a null `this` was undefined but harmless in practice. Arduino 3.x moved status behind a member, and the latent null became a reboot. So this is a migration TRIGGER, not a migration regression – the defect predates it. Constant-initialised: `&WiFi` is a link-time address, so this does not depend on WiFi's constructor having run (no static-init-order hazard); the calls all happen after setup(). Fixing it HERE fixes all ~40 sites at once, which per-site readiness guards would not.

12.1.13 I2C (Wire)

Functions

- Sexpr_t Wire_mop3_setTimeout (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_setPins (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_requestFrom (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

- Sexpr_t Wire_mop3_setClock (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_beginTransmission (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_available (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_read (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_end (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_onReceive (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_onRequest (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_clearWireTimeoutFlag (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_getWireTimeoutFlag (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_endTransmission (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Wire_mop3_write (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Wire_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- TwoWire * **LL_Wire** = 0

WiFi and Wire, in case we need to rationalize conflicting implementations.

- WiFiClass * LL_WiFi

Detailed Description

LambLisp I2C (Wire) builtins.

Function Documentation

Wire_mop3_setTimeout()

```
Sexpr_t Wire_mop3_setTimeout (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set Wire timeout; no args = 1000 ms default; (timeout reset-on-timeout-flag) form also accepted.

(Wire.setWireTimeout) OR (Wire.setWireTimeout timeout reset-on-timeout-flag)

With no parameters, some generally useful (but as yet unknown) values are used.

Wire_mop3_setPins()

```
Sexpr_t Wire_mop3_setPins (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Assign SDA and SCL pins before calling Wire.begin.

Wire_mop3_begin()

```
Sexpr_t Wire_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(begin [addr]) If address is provided , it is our slave bus address. If no address is provided, then we are the bus controller.

Wire_mop3_requestFrom()

```
Sexpr_t Wire_mop3_requestFrom (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

(Wire.requestFrom addr quantity [release-bus-after]) OR (Wire.requestFrom addr quantity)

If not provided, release-bus-after defaults to #t.

Wire_mop3_setClock()

```
Sexpr_t Wire_mop3_setClock (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set the I2C clock frequency in Hz.

Wire_mop3_beginTransmission()

```
Sexpr_t Wire_mop3_beginTransmission (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Begin a transmission to the I2C device at the given address.

Wire_mop3_available()

```
Sexpr_t Wire_mop3_available (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the number of bytes available to read from the I2C buffer.

Wire_mop3_read()

```
Sexpr_t Wire_mop3_read (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Read one byte from the I2C receive buffer; returns -1 if none available.

Wire_mop3_end()

```
Sexpr_t Wire_mop3_end (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Release the Wire bus and free resources; returns #t on success.

Wire_mop3_onReceive()

```
Sexpr_t Wire_mop3_onReceive (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Register a receive callback for slave mode (stub).

Wire_mop3_onRequest()

```
Sexpr_t Wire_mop3_onRequest (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Register a request callback for slave mode (stub).

Wire_mop3_clearWireTimeoutFlag()

```
Sexpr_t Wire_mop3_clearWireTimeoutFlag (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Clear the Wire timeout flag (stub).

Wire_mop3_getWireTimeoutFlag()

```
Sexpr_t Wire_mop3_getWireTimeoutFlag (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the Wire timeout flag (stub).

Wire_mop3_endTransmission()

```
Sexpr_t Wire_mop3_endTransmission (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

End the current transmission; optional stop arg; returns #f on success or error string.

`s + ascii.dec()`, never `d`, for an LL_int32: the type names a WIDTH, not a C spelling, and `int32_t` is `int` on x86-64 but `long` on the xtensa toolchain – so no fixed conversion specifier is right on every target and the wrong one is UB.

Wire_mop3_write()

```
Sexpr_t Wire_mop3_write (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write a byte, character, string, or bytevector to the I2C transmit buffer; returns bytes written.

WRITE THE STRING AS AN EXPLICIT (bytes, length) PAIR. This was `LL_Wire->write(sexpr->any_str_get_chars())`, which was CORRECT on Arduino 2.x and does not compile on 3.x: TwoWire in 2.0.17 declared its own convenience overloads – inline size_t write(const char s) { return write((uint8_t *) s, strlen(s)); } (Wire.h:136) plus write(long) / write(unsigned int) / etc. – and Arduino 3.x DELETED all of them, leaving only write(uint8_t) and write(const uint8_t *, size_t). With the char overload gone the call binds to write(uint8_t) and fails as invalid conversion from 'char*' to 'uint8_t' [-fpermissive] The [-fpermissive] tag makes this look like mere compiler tightening, and it is NOT: the overload it needed genuinely no longer exists. Behaviour on 2.x was right, so this rewrite is behaviour-preserving there (identical strlen-based call) and merely explicit. Passing the pair directly is correct on BOTH cores and does not depend on convenience overloads that a future core may drop again.

Variable Documentation

LL_WiFi

```
WiFiClass* LL_WiFi  [extern]
```

POINT THIS AT THE ARDUINO GLOBAL. It was declared and NEVER ASSIGNED anywhere in the repo – so it was a NULL pointer, and every one of the ~40 `LL_WiFi->` call sites below dereferenced null. It did not look like that from the crash: Guru Meditation Error: Core 1 panic'ed (LoadProhibited), EXCVADDR 0x00000008 NetworkInterface::getStatusBits() NetworkInterface.cpp:231 WiFiSTAClass::isConnected() WiFiSTA.cpp:222 which reads as an Arduino 3.x defect – a "null netif in the new NetworkInterface layer" – and was first diagnosed that way. It is ours. `STAClass STA` is a MEMBER of WiFiSTAClass, so with `this == nullptr` the call reaches `_interface_event_group` at offset 8 of a null object: that IS the 0x00000008 in the report. getStatusBits() already null-checks its event group, so a real object cannot panic there. WHY IT SURVIVED ARDUINO 2.0.17: `isConnected()` there read a global and touched no member, so calling it through a null `this` was undefined but harmless in practice. Arduino 3.x moved status behind a member, and the latent null became a reboot. So this is a migration TRIGGER, not a migration regression – the defect predates it. Constant-initialised: `&WiFi` is a link-time address, so this does not depend on WiFi's constructor having run (no static-init-order hazard); the calls all happen after setup(). Fixing it HERE fixes all ~40 sites at once, which per-site readiness guards would not.

12.1.14 WS2812 / NeoPixel LEDs

Functions

- Sexpr_t mop3_neopixelWrite (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WS2812_mop3_begin (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WS2812_mop3_setBrightness (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WS2812_mop3_setLedColorData (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WS2812_mop3_Wheel (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t WS2812_mop3_show (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **WS2812_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Variables

- WS2812 * **ws2812** = 0

Detailed Description

LambLisp WS2812 / NeoPixel LEDs builtins.

Function Documentation

mop3_neopixelWrite()

```
Sexpr_t mop3_neopixelWrite (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Write a single NeoPixel directly: (neopixelWrite pin r g b).

WS2812_mop3_begin()

```
Sexpr_t WS2812_mop3_begin (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Initialize WS2812 strip: (WS2812.begin num-leds pin channel led-type).

WS2812_mop3_setBrightness()

```
Sexpr_t WS2812_mop3_setBrightness (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set global brightness scale (0–255) applied to all subsequent show() calls.

WS2812_mop3_setLedColorData()

```
Sexpr_t WS2812_mop3_setLedColorData (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Set LED color data: (WS2812.setLedColorData index rgb) or (... index r g b).

WS2812_mop3_Wheel()

```
Sexpr_t WS2812_mop3_Wheel (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Map a wheel position (0–255) to an RGB packed integer cycling through the color wheel.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

WS2812_mop3_show()

```
Sexpr_t WS2812_mop3_show (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Push the color buffer to the physical LED strip; returns 0 on success.

12.1.15 CUDA GPU

Functions

- __global__ void **cudakernel_vectorAdd** (const LL_float32 *A, const LL_float32 *B, LL_float32 *C, int n)
- __global__ void **kernel1** (LL_float32 *data, int n)
- cudaError_t **launch_kernel1** (int nblocks, int thrPerBlk, int shared, cudaStream_t stream, LL_float32 *data, int n)
- __global__ void **kernel2** (LL_float32 *data, int n)
- void cudaDevicePtrDestroyer (Ptr_t p)
- void cudaHostPtrDestroyer (Ptr_t p)
- int array_matmul_cublas_f32 (const LL_float32 *A, const LL_float32 *B, LL_float32 *C, int m, int k, int n)
- int array_matmul_cublas_f64 (const LL_float64 *A, const LL_float64 *B, LL_float64 *C, int m, int k, int n)
- Sexpr_t mop3_cpp_vector_add (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaLaunch_kernel1 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudakernel_vectorAdd (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMalloc (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMallocHost (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaFree (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMemcpy (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMemset (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- void **cudaStreamDestroyer** (Ptr_t p)
- Sexpr_t mop3_cudaStreamCreate (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaStreamSynchronize (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaDeviceSynchronize (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaGetDeviceCount (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaGetDeviceProp (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t **Cuda_install_mop3** (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp CUDA GPU builtins.

Function Documentation

cudaDevicePtrDestroyer()

```
void cudaDevicePtrDestroyer (Ptr_t p)
```

GC destructor for device memory.

cudaHostPtrDestroyer()

```
void cudaHostPtrDestroyer (Ptr_t p)
```

GC destructor for pinned host memory.

array_matmul_cublas_f32()

```
int array_matmul_cublas_f32 (const LL_float32 * A, const LL_float32 * B, LL_float32 * C, int m, int k, int n)
```

Row-major C(m x n) = A(m x k) * B(k x n) via cuBLAS, on HOST buffers. Our arrays are host-resident (shape . data) pairs, so we round-trip through the device: cudaMalloc + H2D + gemm + D2H + cudaFree.

cuBLAS is COLUMN-major but our buffers are ROW-major. Rather than copy/transpose, use the identity (A*B)^T = B^T * A^T. A row-major (m x k) buffer, reinterpreted column-major, already IS A^T (k x m). So computing, in cuBLAS column-major terms, Ct(n x m) = B^T(n x k) * A^T(k x m) — i.e. sgemm(N,N, n, m, k, dB, n, dA, k, dC, n) — writes exactly our desired row-major C(m x n) into dC with no transposes. Returns 0 on success, nonzero on any CUDA/cuBLAS error.

array_matmul_cublas_f64()

```
int array_matmul_cublas_f64 (const LL_float64 * A, const LL_float64 * B, LL_float64 * C, int m, int k, int n)
```

Double-precision counterpart of array_matmul_cublas_f32 (uses cublasDgemm).

mop3_cpp_vector_add()

```
Sexpr_t mop3_cpp_vector_add (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

CPU-side vector add: C[i] = A[i] + B[i] for i in [0,n). Args: (A B C n) – A, B, C are realvectors; n is element count. Available with or without CUDA; used in three-way correctness tests.

mop3_cudaLaunch_kernel1()

```
Sexpr_t mop3_cudaLaunch_kernel1 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Launch kernel1 (element ∗= 2) on a device buffer; args: (stream data [n]).

mop3_cudakernel_vectorAdd()

```
Sexpr_t mop3_cudakernel_vectorAdd (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

GPU vector add C[i] = A[i]+B[i]; args: (A B C n) as device/pinned pointers or bytevectors.

mop3_cudaMalloc()

```
Sexpr_t mop3_cudaMalloc (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Allocate n bytes of device memory; returns a T_CPP_HEAP object with cudaFree destructor.

mop3_cudaMallocHost()

```
Sexpr_t mop3_cudaMallocHost (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Allocate n bytes of page-locked host memory; returns a T_CPP_HEAP object with cudaFreeHost destructor.

mop3_cudaFree()

```
Sexpr_t mop3_cudaFree (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Free one or more device memory objects returned by cudaMalloc.

mop3_cudaMemcpy()

```
Sexpr_t mop3_cudaMemcpy (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Combines cudaMemcpy() and cudaMemcpyAsync(). If the stream argument is provided then the asynchronous copy will be used. count is in bytes. dst and src may be device pointers (cudaMalloc), pinned host pointers (cudaMallocHost), or bytevectors.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_cudaMemset()

```
Sexpr_t mop3_cudaMemset (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Fill count bytes of device memory with a byte value: (cudaMemset ptr val count).

mop3_cudaStreamCreate()

```
Sexpr_t mop3_cudaStreamCreate (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Create a CUDA stream; returns a T_CPP_HEAP object with cudaStreamDestroy destructor.

mop3_cudaStreamSynchronize()

```
Sexpr_t mop3_cudaStreamSynchronize (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Block until all operations in the given CUDA stream have completed.

mop3_cudaDeviceSynchronize()

```
Sexpr_t mop3_cudaDeviceSynchronize (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Block until all CUDA work on the current device has completed.

mop3_cudaGetDeviceCount()

```
Sexpr_t mop3_cudaGetDeviceCount (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the number of CUDA-capable devices available on the host.

mop3_cudaGetDeviceProp()

```
Sexpr_t mop3_cudaGetDeviceProp (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return an association list of device properties for the given device index (default 0).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

12.1.16 HIP / ROCm GPU

Functions

- __global__ void **cudakernel_vectorAdd** (const LL_float32 *A, const LL_float32 *B, LL_float32 *C, int n)
- __global__ void **kernel1** (LL_float32 *data, int n)
- hipError_t **launch_kernel1** (int nblocks, int thrPerBlk, int shared, hipStream_t stream, LL_float32 *data, int n)
- __global__ void **kernel2** (LL_float32 *data, int n)
- void cudaDevicePtrDestroyer (Ptr_t p)
- void cudaHostPtrDestroyer (Ptr_t p)
- Sexpr_t mop3_cpp_vector_add (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaLaunch_kernel1 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudakernel_vectorAdd (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMalloc (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMallocHost (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaFree (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMemcpy (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaMemset (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- void **cudaStreamDestroyer** (Ptr_t p)
- Sexpr_t mop3_cudaStreamCreate (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaStreamSynchronize (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaDeviceSynchronize (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaGetDeviceCount (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t mop3_cudaGetDeviceProp (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t Hip_install_mop3 (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)

Detailed Description

LambLisp HIP / ROCm GPU builtins.

Function Documentation

cudaDevicePtrDestroyer()

```
void cudaDevicePtrDestroyer (Ptr_t p)
```

GC destructor for device memory.

cudaHostPtrDestroyer()

```
void cudaHostPtrDestroyer (Ptr_t p)
```

GC destructor for pinned host memory.

mop3_cpp_vector_add()

```
Sexpr_t mop3_cpp_vector_add (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

CPU-side vector add: C[i] = A[i] + B[i] for i in [0,n). Args: (A B C n) – A, B, C are realvectors; n is element count. Available with or without CUDA; used in three-way correctness tests.

mop3_cudaLaunch_kernel1()

```
Sexpr_t mop3_cudaLaunch_kernel1 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Launch kernel1 (element ∗= 2) on a device buffer; args: (stream data [n]).

mop3_cudakernel_vectorAdd()

```
Sexpr_t mop3_cudakernel_vectorAdd (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

GPU vector add C[i] = A[i]+B[i]; args: (A B C n) as device/pinned pointers or bytevectors.

mop3_cudaMalloc()

```
Sexpr_t mop3_cudaMalloc (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Allocate n bytes of device memory; returns a T_CPP_HEAP object with hipFree destructor.

mop3_cudaMallocHost()

```
Sexpr_t mop3_cudaMallocHost (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Allocate n bytes of page-locked host memory; returns a T_CPP_HEAP object with hipHostFree destructor.

mop3_cudaFree()

```
Sexpr_t mop3_cudaFree (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Free one or more device memory objects returned by hipMalloc.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mop3_cudaMemcpy()

```
Sexpr_t mop3_cudaMemcpy (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Combines hipMemcpy() and hipMemcpyAsync(). If the stream argument is provided then the asynchronous copy will be used. count is in bytes. dst and src may be device pointers (hipMalloc), pinned host pointers (hipHostMalloc), or bytevectors.

mop3_cudaMemset()

```
Sexpr_t mop3_cudaMemset (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Fill count bytes of device memory with a byte value: (hipMemset ptr val count).

mop3_cudaStreamCreate()

```
Sexpr_t mop3_cudaStreamCreate (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Create a CUDA stream; returns a T_CPP_HEAP object with hipStreamDestroy destructor.

mop3_cudaStreamSynchronize()

```
Sexpr_t mop3_cudaStreamSynchronize (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Block until all operations in the given CUDA stream have completed.

mop3_cudaDeviceSynchronize()

```
Sexpr_t mop3_cudaDeviceSynchronize (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Block until all CUDA work on the current device has completed.

mop3_cudaGetDeviceCount()

```
Sexpr_t mop3_cudaGetDeviceCount (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return the number of CUDA-capable devices available on the host.

mop3_cudaGetDeviceProp()

```
Sexpr_t mop3_cudaGetDeviceProp (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Return an association list of device properties for the given device index (default 0).

Hip_install_mop3()

```
Sexpr_t Hip_install_mop3 (Lamb & lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

Installer: bind the HIP/ROCm GPU builtins into `env_target` (AMD-GPU counterpart of the CUDA bindings); host builds register safe no-ops.

12.1.17 LLArduino – LambLisp's own Arduino API

Classes

- struct LLArduinoBackend
- struct LLGpiodPin
- struct LLGpiodLine

Macros

- #define LL_LLA_NPINS 256

Functions

- bool ll_gpiod_available ()
- void **ll_gpiod_pin_mode** (int, int)
- void **ll_gpiod_digital_write** (int, int)
- int **ll_gpiod_digital_read** (int)
- void **ll_gpiod_analog_write** (int, int)
- int ll_gpiod_analog_read (int)
- void **ll_gpiod_tone_on** (int, unsigned int)
- void **ll_gpiod_tone_off** (int)
- int **ll_gpiod_pin_mode_get** (int)
- int **ll_gpiod_pin_duty_get** (int)
- int **ll_gpiod_pin_tone_get** (int)
- int **ll_gpiod_npins** ()
- void pinMode (int pin, int mode)
- void **digitalWrite** (int pin, int val)
- int **digitalRead** (int pin)
- void **analogWrite** (int pin, int val)

- int **analogRead** (int pin)
- void **tone** (int pin, unsigned int freq)
- void **tone** (int pin, unsigned int freq, unsigned long dur)
- void **noTone** (int pin)
- int ll_llarduino_pin_mode (int pin)
- int **ll_llarduino_pin_duty** (int pin)
- int **ll_llarduino_pin_tone** (int pin)
- int **ll_llarduino_npins** ()
- const char * **ll_llarduino_backend** ()
- virtual size_t **Print::write** (const uint8_t *buf, size_t n)
- size_t **Print::write** (const char *s)
- size_t **Print::print** (char c)
- size_t **Print::print** (const char *s)
- size_t **Print::print** (const String &s)
- size_t **Print::print** (long n, int base=10)
- size_t **Print::print** (int n, int base=10)
- size_t **Print::print** (unsigned long n, int base=10)
- size_t Print::print (double v, int digits=2)
- size_t Print::println ()
- size_t **Print::println** (char c)
- size_t **Print::println** (const char *s)
- size_t **Print::println** (const String &s)
- size_t **Print::println** (int n, int base=10)
- size_t **Print::println** (long n, int base=10)
- size_t **Print::println** (unsigned long n, int base=10)
- size_t **Print::println** (double v, int digits=2)
- size_t Print::printf (const char *fmt,...) CHECKPRINTF_pos2
- void **LLArduinoSerial::begin** (unsigned long baud=115200)
- void **LLArduinoSerial::end** ()
- int **LLArduinoSerial::availableForWrite** ()
- void **LLArduinoSerial::flush** () override
- size_t LLArduinoSerial::write (uint8_t c) override
- int LLArduinoSerial::available () override
- int **LLArduinoSerial::read** () override
- int LLArduinoSerial::peek () override

Variables

- LLArduinoSerial **Serial**

Detailed Description

The NULL (simulated) backend: pin state in memory, no hardware.

Print / Stream / Serial – REAL character I/O, not a simulation.

libgpiod backend – REAL pins on a Raspberry Pi or Jetson.

This is the backend that runs where there is no GPIO to drive – a developer laptop, an x86 host, a container – so that the 26 `commonio` procedures (pinMode, digitalWrite, digitalRead, analogRead/Write, tone, ...) can be CALLED AND ASSERTED ON without a board.

WHY THAT MATTERS HERE, concretely: of the seven boards where commonio is compiled in, most are unavailable most of the time – shared with other sessions, unplugged to save power, or in a failed USB port. On one occasion exactly ONE of seven was reachable. Anything that can only be tested with a board in hand is, in practice, tested rarely; a recent Arduino 3.x WiFi (regression) reached customers as a boot loop for that reason.

IT ANNOUNCES ITSELF AS SIMULATED. A simulated backend is honest only while nobody can mistake it for hardware, so `llarduino-backend` returns 'null here and the loopback semantics are documented below rather than left to be discovered. The rule – never invent a plausible sensor reading – still binds the REAL backends: on a Pi or Jetson, analogRead has no ADC behind it and must say so instead of returning a number. Here every value is one the caller itself wrote, which is a different thing from a fabricated measurement.

This is what makes LLArduino a port rather than a simulation: `(digitalWrite 18 1)` in Scheme drives an actual header pin. Selected at RUNTIME (see the dispatch table in ll_llarduino_null.cpp) because whether real pins exist is a property of the MACHINE, not the build – the same aarch64 binary runs on a Jetson with a 40-pin header and on a box whose gpiochips are not header pins at all.

libgpiod v1 API, PINNED DELIBERATELY. v1.6.3 is what both this workstation and the Jetson carry, and libgpiod v2 is SOURCE-INCOMPATIBLE with v1 (different handle types, different request API). A distro upgrade that silently moved to v2 would break this file, which is the same class of trap as the esptool 4.x/5.x subcommand split and the Arduino 2.x/3.x break behind If v2 support is wanted it goes beside this, not through it.

Print and Stream are pure character I/O, and LambStdio already implements exactly that on POSIX, so this is a facade over working plumbing rather than a stand-in. That is the same footing as millis(): genuinely implemented on the host, not faked.

ARDUINO'S QUIRKS ARE MATCHED ON PURPOSE. Each of these is a place where a host test could pass while a device disagrees, which is the failure criterion 3 is designed to catch:

- println() ends "\r\n", NOT "\n" – Arduino has always done this, and a serial monitor shows it
- write() returns a BYTE COUNT – callers legitimately sum it
- print(double) defaults to 2 dp – not g, not full precision
- print(int, base) takes BIN/OCT/DEC/HEX and prints NO prefix ("ff", not "0xff")

Macro Definition Documentation

LL_LLA_NPINS

```
#define LL_LLA_NPINS 256
```

Simulated pin file. LL_LLA_NPINS is generous because pin NUMBERING is board-specific (a Pi's BCM numbers, an ESP32's GPIO numbers) and this backend must not reject a number that is legal somewhere. Out-of-range is CLAMPED AWAY FROM MEMORY, never written: a bad pin number is a caller error, not a reason to corrupt the heap.

Function Documentation

ll_gpiod_available()

```
bool ll_gpiod_available ()  [extern]
```

The libgpiod backend lives in ll_llarduino_gpiod.cpp; only its entry points are needed here.

ll_gpiod_analog_read()

```
int ll_gpiod_analog_read (int pin)  [extern]
```

NO ADC AND NO PWM HERE, AND THEY MUST NOT PRETEND. Neither a Pi nor a Jetson has an ADC on the header – analogRead conventionally means an EXTERNAL converter (MCP3008 over SPI, ADS1115 over I2C) – and analogWrite needs a PWM chip wired to that specific pin. The rule holds: a fabricated sensor reading is worse than an error, because it is plausible. -1 says "unsupported on this backend"; it is never a measurement.

pinMode()

```
void pinMode (int pin, int mode)
```

Public Arduino surface – one line each, straight through the table.

ll_llarduino_pin_mode()

```
int ll_llarduino_pin_mode (int pin)
```

Backend inspection – lets a test assert on state the Arduino API cannot report (mode, duty, tone) and name which backend answered. See ll_llarduino_null.cpp.

print()

```
size_t Print::print (double v, int digits = 2)
```

Arduino's default is TWO decimal places, not g and not full precision.

println()

```
size_t Print::println ()
```

CRLF, as Arduino does – not "\n".

CRLF – Arduino's println has always emitted "\r\n". A host that emits bare "\n" looks correct in a terminal and differs from the device, which is exactly the silent divergence to avoid.

printf()

```
size_t Print::printf (const char * fmt, ...)
```

HardwareSerial::printf, used by LL itself.

HardwareSerial::printf is not standard Arduino Print, but LambLisp's own Stdio layer calls it (ll_platform_↩ Stdio.cpp), so the facade owes it. < truncate, never overrun

write()

```
size_t LLArduinoSerial::write (uint8_t c)  [override], [virtual]
```

Implements Print.

available()

```
int LLArduinoSerial::available ()  [override]
```

available() must count the pushed-back byte, or peek() would hide input from a caller that polls available() before reading – the classic Stream bug.

peek()

```
int LLArduinoSerial::peek ()  [override]
```

One byte of pushback, which is all Stream promises. LambStdioClass has no peek of its own.

Chapter 13

Class Documentation

13.1 autobuf_t< _T_ > Class Template Reference

Public Member Functions

- void **reset** ()
- void use_fastram ()
- unsigned long size (unsigned long minsz, unsigned long incr, unsigned long maxsz)
- unsigned long size (unsigned long needed)
- _T_ * fill (_T_ f, unsigned long needed)
- _T_ * buf (unsigned long needed)
- unsigned long size (void)
- unsigned long maxsize (void)
- _T_ * fill (_T_ f)
- _T_ * buf (void)

13.1.1 Detailed Description

template<typename _T_>
class autobuf_t< _T_ >

This template type provides a simple auto-sizing buffer, using the target type copy constructor. DEBT make special autobuf types for char and byte to reduce resizing time.

13.1.2 Member Function Documentation

use_fastram()

```
template<typename _T_>
void autobuf_t< _T_ >::use_fastram ()  [inline]
```

Ask for this buffer's storage to prefer FAST RAM, with automatic migration to SLOW RAM if it ever outgrows what fast can give. OFF BY DEFAULT: fast RAM is the scarce pool, so a buffer takes it only by saying so. < idempotent

< cannot move – keep the buffer AND the old policy

< still the OLD policy's allocator here, by construction

< flipped only once the storage actually matches it

size() [1/3]

```
template<typename _T_>
unsigned long autobuf_t< _T_ >::size (unsigned long minsz, unsigned long incr, unsigned long maxsz)  [inline]
```

Set the size limits for this autobuf. Buffer will be resized if needed.

size() [2/3]

```
template<typename _T_>
unsigned long autobuf_t< _T_ >::size (unsigned long needed)  [inline]
```

Ensure that the buffer is at least the given size, and return the size.

fill() [1/2]

```
template<typename _T_>
_T_ * autobuf_t< _T_ >::fill (_T_ f, unsigned long needed)  [inline]
```

Fill this autobuf with a given value.

buf() [1/2]

```
template<typename _T_>
_T_ * autobuf_t< _T_ >::buf (unsigned long needed)  [inline]
```

Return the address of this autobuf's elements.

size() [3/3]

```
template<typename _T_>
unsigned long autobuf_t< _T_ >::size (void )  [inline]
```

Return the current size of this autobuf.

maxsize()

```
template<typename _T_>
unsigned long autobuf_t< _T_ >::maxsize (void )  [inline]
```

Return the hard upper bound (*max*) on this autobuf's size.

fill() [2/2]

```
template<typename _T_>
_T_ * autobuf_t< _T_ >::fill (_T_ f)  [inline]
```

Fill this autobuf with a given value.

buf() [2/2]

```
template<typename _T_>
_T_ * autobuf_t< _T_ >::buf (void )  [inline]
```

Return a pointer to the elements in this autobuf.

13.2 Cell Class Reference

Classes

- struct CellFeatures

Public Types

Cell type enumeration

Note the following properties of this enumeration:
NIL is a singleton, in the middle of the enumeration.
Types <= NIL are atoms.
Types >= NIL are lists.
Types > NIL are pairs.
Types <= T_PORT_HEAP are heap-allocated objects, requiring specialized finalizing but not specialized marking.
Types <= T_ANY_HEAP_SVEC are heap allocated vectors requiring specialized marking in addition to finalizing.
For Scheme purposes, there is one `pair` type, but there are several other specialized `pair` types that facilitate execution.
These additional `pair` types are atoms, but list operations such as `car`, `cdr`, and `append` can operate on them.

- enum {
  T_SVEC_HEAP =0 , T_SVEC2N_HEAP , T_ANY_HEAP_SVEC = T_SVEC2N_HEAP ,
  T_SYM_HEAP ,
  T_BVEC_HEAP , T_BIGNUM , T_STR_HEAP , T_CPP_HEAP ,
  T_PORT_HEAP , T_NEEDS_FINALIZING = T_PORT_HEAP , T_BVEC_EXT , T_STR_EXT ,
  T_BVEC_IMM , T_STR_IMM , T_GENSYM , T_BOOL ,
  T_CHAR , T_INT32 , T_INT64 , T_FLOAT32 ,
  T_FLOAT64 , T_MOP3_PROC , T_MOP3_NPROC , T_VOID ,
  T_UNDEF , T_NIL , T_COMPLEX , T_RATIONAL ,
  T_FREE , T_PAIR , T_SVEC_IMM , T_PROC ,
  T_NPROC , T_MACRO , T_DICT , T_THUNK_SEXPR ,
  T_THUNK_BODY , T_ERROR , T_VALUES , T_BYTECODE ,
  T_BYTECODE_N , Ntypes }

Public Member Functions

- LL_int32 car_int32 () const
- LL_float32 car_float32 () const
- LL_int32 cdr_int32 () const
- LL_float32 cdr_float32 () const
- Sexpr_t mk_error (const char *fmt,...) CHECKPRINTF_pos2
- Sexpr_t mk_error (Sexpr_t irritants, const char *fmt,...) CHECKPRINTF_pos3
- CharVec_t prechecked_str_ext_get_chars ()
- CharVec_t prechecked_str_imm_get_chars ()
- void **prechecked_bvec_imm_set_length** (LL_int32 l)
- Charst_t prechecked_gensym_get_chars ()
- void **prechecked_gensym_get_info** (LL_int32 &hsh, Charst_t &chars)
- Sexpr_t **prechecked_error_get_irritants** ()
- Sexpr_t **prechecked_error_get_str** ()
- Charst_t **prechecked_error_get_chars** ()

Static Cell constructors, used only for "well-known" atomic cells (nil, true, false etc).

In general, it is best not to do anything complex at static construction time. There is no guarantee that dependencies will be ready. In particular, use of the terminal may cause the system to crash. Most applications should leave any construction activities to more purposeful code at runtime, using only cells obtained through cons.

- **Cell** (Word_t typ, Word_t w1, Word_t w2)
- **Cell** (Word_t typ, Sexpr_t w1, Sexpr_t w2)

Low-level cell field manipulation.

Note that the type field occupies all of byte 0, although not all bits are required. This speeds up the very common type-checking operation, because no mask is required to extract the type bits.

The flags are in byte 1, and a mask operation is required to access the individual flag fields.

- void zero ()
- LL_int32 type (void)
- void type (LL_int32 t)
- LL_int32 flags (void)
- void flags_set (LL_int32 f)
- void flags_clr (LL_int32 f)
- void rplaca (Word_t p)
- void rplacd (Word_t p)

PAIR subtype accessors – the four subtype bits as pairs use them.

Meaningful for T_PAIR and the slot-typed types (T_RATIONAL, T_COMPLEX) only. When another type starts using the subtype bits it will name its own predicates for them; do not reach for these there.

- bool is_slot_typed () const
- bool car_is_int () const
- bool car_is_float () const
- bool cdr_is_int () const
- bool cdr_is_float () const
- bool car_is_embedded () const
- bool cdr_is_embedded () const
- bool **car_is_ptr** () const
- bool **cdr_is_ptr** () const

Cell value extractors, dependent on Cell type.

- Ptr_t get_car_addr ()
- Word_t get_car (void)
- Word_t get_cdr (void)
- Bool_t as_Bool_t ()
- Char_t as_Char_t ()
- ll_codepoint_t as_codepoint ()
- LL_int32 as_int32 () const
- LL_int64 as_int64 () const
- LL_float32 as_float32 () const
- LL_float64 as_float64 () const
- Ptr_t as_Ptr_t ()
- Charst_t as_Charst_t ()
- Bytest_t as_Bytest_t ()
- CharVec_t as_CharVec_t ()
- ByteVec_t as_ByteVec_t ()
- Portst_t as_Portst_t ()

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Cell hash value

Hashing is used extensively throughout LambLisp. Each Cell has a hash value, calculated as follows:

- *If the Cell is a symbol, the Cell's hash value is the hash value of the symbol.*
- *Otherwise, the address of the Cell is hashed, and that is the result.*

- LL_int32 hash_sexpr (void)
- LL_int32 hash_contents (void)
- LL_int32 hash (void)

Cell setters converting the C types into S-expressions.

- Sexpr_t set (LL_int32 typ, Word_t w1, Word_t w2)
- Sexpr_t set (LL_int32 typ, LL_int32 a, Sexpr_t b)
- Sexpr_t make_free (Sexpr_t next)
- Sexpr_t set (LL_int32 typ, Word_t a, Sexpr_t b)
- Sexpr_t set (LL_int32 typ, Sexpr_t a, Sexpr_t b)
- Sexpr_t set (Bool_t b)
- Sexpr_t set (Char_t c)
- Sexpr_t set (LL_int32 i)
- Sexpr_t set (LL_int64 v)
- Sexpr_t set (LL_float32 r)
- Sexpr_t set (LL_float64 d)
- Sexpr_t set (LL_float32 r, LL_float32 i)
- LL_float32 cpx_real () const
- LL_float32 cpx_imag () const
- Sexpr_t set (LL_int32 num, LL_int32 den)
- void rat_set_car_i32 (LL_int32 v)
- void rat_set_car_ptr (Sexpr_t p)
- void rat_set_cdr_i32 (LL_int32 v)
- void rat_set_cdr_ptr (Sexpr_t p)
- LL_int64 int_as_i64 () const
- Sexpr_t rat_car_ptr () const
- Sexpr_t rat_cdr_ptr () const
- bool **rat_car_big** () const
- bool **rat_cdr_big** () const
- bool rat_is_big () const
- bool exact_is_big () const
- LL_int64 as_numerator () const
- LL_int64 as_denominator () const
- int exact_sign ()
- Sexpr_t set (Port_t &p)
- Sexpr_t set (LL_int32 typ, LL_int32 a, Charst_t b)
- Sexpr_t set (LL_int32 typ, LL_int32 a, Bytest_t b)
- Sexpr_t set (LL_int32 typ, LL_int32 a, CharVec_t b)
- Sexpr_t set (LL_int32 typ, LL_int32 a, ByteVec_t b)

Accessors for use when the cell type is unverified.

*The **any** and **mustbe** accessors will perform type checking and throw an error if an improper access is attempted. The **coerce** operators will perform C coercion on its operand if possible, otherwise throw an error.*

- Bool_t mustbe_Bool_t ()
- Char_t mustbe_Char_t ()
- ll_codepoint_t mustbe_codepoint ()
- LL_int32 mustbe_int32 ()

- LL_float32 mustbe_float32 ()
- Sexpr_t mustbe_any_str_t ()
- Sexpr_t mustbe_cppobj_t ()
- CPPDeleterPtr prechecked_cppobj_get_deleter ()
- Ptr_t prechecked_cppobj_get_ptr ()
- CPPDeleterPtr **any_cppobj_get_deleter** ()
- Ptr_t **any_cppobj_get_ptr** ()
- void **any_cppobj_get_info** (CPPDeleterPtr &d, Ptr_t &p)
- LL_float32 coerce_float32 ()
- LL_float64 coerce_float64 ()
- LL_int32 **coerce_int32** ()
- LL_int64 **coerce_int64** ()
- int exact_rank () const
- int inexact_rank () const
- bool is_exact () const
- bool is_inexact () const
- bool is_complex () const
- int num_rank () const
- LL_int64 numerator64 () const
- LL_int64 denominator64 () const
- LL_float64 coerce_cpx_real ()
- LL_float64 coerce_cpx_imag ()
- Sexpr_t **error_get_str** ()
- Sexpr_t **error_get_irritants** ()
- Charst_t **error_get_chars** ()
- Sexpr_t any_error_get_str ()
- Sexpr_t **any_error_get_irritants** ()
- LL_int32 **any_sym_get_hash** ()
- Charst_t **any_sym_get_chars** ()
- void **any_sym_get_info** (LL_int32 &hsh, Charst_t &chars)

Operations on strings

*LambLisp supports several subtypes of **strings**. At the time of writing, there are heap-allocated strings (read-write), strings whose characters outside of LambLisp's managed memory, (aka external or EXT strings), and short **immediate** strings that are contained completely within a single cell. Additional subtypes (such as load-on-demand strings) may be added in future.*

There are functions of the form any_xxx() that can be used with any subtype, and there are type-specific functions of the form any_xxx_yyy() for use where the type is already known, with yyy being a code hint.

- CharVec_t any_str_get_chars ()
- LL_int32 any_str_get_length ()
- void **any_str_get_info** (LL_int32 &len, CharVec_t &chars)
- void str_invalidate_charlength ()
- LL_int32 any_str_get_charlength ()

Operations on vectors and sub-types of vectors

Within the LambLisp virtual machine, a Lisp vector is referred to as svec. This is an array of S-expressions having a fixed dimension. There are also bytevectors; these are an array of bytes, also of fixed dimension.

As with strings, there are several subtypes of S-expression vectors. There is a heap-allocated vector, which may be of any size. There is a second type of heap-allocated vector, that is always sized to be a power of 2. These are provided to support efficient hash tables. There are immediate vectors, which may be of 0, 1, or 2 elements. The 2-element immediate vector can also be used as a hash table. This can reduce search time by half without requiring any heap allocation, at the cost of 1 extra cell allocation.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Bytevectors are also diverse, having heap, external, and immediate variants. Heap bytevectors are allocated, obviously, on the system heap, and the heap space is freed when the bytevector is garbage-collected.

External bytevectors operate on bytes provided externally to LambLisp's memory manager. This space may have been dynamically allocated from outside LambLisp, or may be located in read-only memory. When a C++ object os injected into LambLisp, it can optionally be provided with a garbage collector callback; in that case the external object can be automatically garbage collected when no it's longer used in the Lisp program.

Immediate bytevectors are contained within a LambLisp Cell. The maximum size is limited by the word size of the underlying platform.

Within the LambLisp virtual machine, there are generic functions of the form any_xvec_xxx() that can operate on any xvec (svec or bvec) subtype, as well as type-specific functions of the form xvec_yyy_xxx(), where yyy is a code hint for the cell storage type (heap, immediate, ROM).

- void any_svec_get_info (LL_int32 &Nelems, Sexpr_t *&elems)
- Sexpr_t * **any_svec_get_elems** ()
- LL_int32 any_bvec_get_length ()

Byte access to STRINGS.

strings are indexed by CHARACTER (R7RS), which is correct but removed the only way to reach the underlying UTF-8 BYTES from Scheme. Both views are wanted – a character index for text, a byte index for encoding work – so the bytevector operations accept a string and expose its bytes. `(bytevector-length "<alpha>X")` is 3 where `(string-length "<alpha>X")` is 2, and that is the point, not an inconsistency. `bytevector?` deliberately still answers #f for a string: R7RS type predicates partition the types, and a string is not a bytevector. What is shared is the OPERATIONS, not the type.

THIS IS AN EXTENSION, NOT R7RS – SAY SO IN ANY DOCUMENTATION. R7RS 6.9 defines the bytevector procedures over bytevectors; a string is a type error, and the standard bridge is `string->utf8` / `utf8->string`, both of which LambLisp already provides. Verified against two independent R7RS implementations rather than assumed:

chibi-scheme (R7RS reference) guile –r7rs (bytevector-length "abc") -> ERROR ERROR (bytevector-u8-ref "abc" 0) -> ERROR ERROR (bytevector-length (string->utf8 "<alpha>X")) -> 3 3 (string-length "<alpha>X") -> 2 2

So portable code must use `string->utf8`, and anything written against this extension will not move to another Scheme. What the extension buys, and the reason it is worth having HERE, is that it is a ZERO-COPY VIEW: `string->utf8` allocates a bytevector and copies the bytes, which on a microcontroller with a fixed cell pool is exactly the allocation you were trying to avoid when you asked for the bytes in the first place. The length is strlen – strings here are NUL-terminated UTF-8 byte buffers – so a string containing a NUL byte reports short. That is the same limit `string-length` has always had on this representation, not a new one introduced here.

- ByteVec_t any_bvec_get_elems ()
- void any_bvec_get_info (LL_int32 &Nelems, ByteVec_t &elems)

Array element-type tag – HEAP AND EXT SHAPES ONLY. NOT VALID ON T_BVEC_IMM.

These read and write Cell byte 3, which is free on T_BVEC_HEAP / T_BVEC_EXT (car holds the length, cdr the payload pointer) – but on T_BVEC_IMM BYTE 3 IS ELEMENT 0. The immediate layout is [0]=type [1]=flags [2]=element count [3..]=elements, so calling these on an immediate READS A DIMENSION BYTE AS A TYPE, or OVERWRITES the first element with one. There is no type check here to stop you: `_contents._byte[3]` is taken unconditionally. NOT valid for all bvec subtypes, which is the opposite of the truth for the one subtype where it matters. A PLAIN BYTEVECTOR HAS NO ELEMENT TYPE AT ALL.

mk_bytevector() sets no tag; an R7RS bytevector is just bytes. This tag belongs to the ARRAY feature (ll_vm_mop3_extra.cpp), where an array is a SHAPE bytevector plus a DATA bytevector: the shape carries the element type and the dims, the data is untagged. `array-type` reads it from the shape. HOW THE ARRAY CODE HANDLES THE IMMEDIATE CASE, and copy it if you add a caller: it does NOT use these accessors. When the shape fits immediate `(1 + 4*ndim <= sizeof(Cell) - 3)` it stores the type as sdata[0] – element 0 of the payload – and puts the dims at sdata+1, sizing the vector with that leading byte included. Only the heap branch calls bvec_set_elem_type(). See ll_vm_mop3_extra.cpp:533, which is the whole convention. Element types are Cell::T_INT32, T_INT64, T_FLOAT32, T_FLOAT64.

- LL_int32 bvec_elem_type (void) const
- void bvec_set_elem_type (LL_int32 t)

Cell conversions to printable representation

These functions convert a Cell (or parts of a Cell) to a printable representation of the S-expression contents of the Cell. Because environments are often included in the descendants of the Cell being printed, the depth of environment recursiveness is limited.

- String cell_name (void)
- Charst_t type_name (LL_int32 typ)
- Charst_t **type_name** (void)
- Charst_t gcstate_name (void)
- String dump ()
- String **str** (Sexpr_t sx, Bool_t as_write_or_display, DatumLabels *lbl, LL_int32 max_depth)
- String **str** (Sexpr_t sx, Bool_t as_write_or_display, LL_int32 env_depth, LL_int32 max_depth)
- String **str** (Bool_t as_write_or_display, LL_int32 env_depth, LL_int32 max_depth)
- String **str** (Bool_t as_write_or_display, LL_int32 env_depth)
- String **str** (Bool_t as_write_or_display)
- String **str** (LL_int32 env_depth)
- String **str** (void)

Static Public Attributes

- static const int GC_STATE_MASK = F_GC01 | F_GC02 | F_GC04

Cell flags, including gc states for multi-pass incremental gc, tail marker, and spares.

- static const int F_GC01 = 0x01
- static const int F_GC02 = 0x02
- static const int F_GC04 = 0x04
- static const int F_TAIL = 0x08

THE SUBTYPE BITS – flags bits 4-7 (<tt>flags \& 0xF0</tt>).

```
bit   7   6   5   4   3   2   1   0
+---+---+---+---+---+---+---+---+
|  subtype bits |TAI|  gc state |
+---+---+---+---+---+---+---+---+
0xF0          0x08   0x07
```

Four bits whose meaning is decided by the cell's top-level type. TODAY ONLY PAIRS USE THEM, as the four booleans below: is the car an embedded int, is it an embedded float, and the same two for the cdr. There is deliberately no generic subtype accessor – the shared part can be factored out when a second type actually uses these bits.

- static const int F_SUBTYPE_MASK = 0xF0
- static const int F_CAR_IS_INT = 0x10
- static const int F_CAR_IS_FLOAT = 0x20
- static const int F_CDR_IS_INT = 0x40
- static const int F_CDR_IS_FLOAT = 0x80

Flag testing & setting for garbage collection and tail recursion.

The collector is incremental, and a cell's `gc_state()` records its position in a LIFECYCLE rather than a colour: idle, issued, stacked, marked, free. A cell surviving a collection returns to idle. The original set of 3 colors was enlarged to 4 with *Kung and Song 1977*, with their correctness proof of the incremental GC algorithm. In LambLisp, is is useful to have a fifth state, and to think of the state or color as a stage in the Cell life cycle. When GC is in progress, Cells may advance in their life cycle, but may also be moved back to an earlier gc state as the result of an assignment. This enumeration allows some tests to be combined in an inequality rather than a sequence of equality tests or a C `switch`.

LambLisp∗ uses a **trampoline technique** to implement tail recursion. The **tail** of a series of expressions is the last one evaluated; this is the expression that will return the value of the series.

In the *trampoline*, instead of evaluating the last expression in the series and returning the value (as in C/C++), the expression is returned unevaluated, with the **tail flag** set. The evaluator checks every result to see if it is a *tail* that needs additional evaluation, or is a final result to be returned.

This removes the need for an additional stack frame during recursion.

- enum {
  gcst_idle , **gcst_issued** , **gcst_stacked** , **gcst_marked** ,
  gcst_free , **Ngcstates** }
- static const int F_BC_VARIADIC = 0x01
- LL_int32 gc_state (void)
- Sexpr_t gc_state (LL_int32 st)
- LL_int32 tail_state (void)
- Sexpr_t tail_state_set (void)
- Sexpr_t tail_state_clr (void)
- bool bytecode_is_variadic () const
- void bytecode_set_variadic ()

Cell type testing

The cell type enumeration has been designed to group together cell types according to their most common operations.

The main groups are those needing special sweep and finalizing during garbage collection, those that are treated as pairs, and simple atoms. There is a subgroup of those needing special sweep, that also need specialised marking during the garbage collection mark phase.

There are some types that have optimized subtypes (such as **immediate** types), and they differ in their processing at GC time. Type-testing functions below have `is_any_x()` predicates that group all subtypes together (e.g., `is_any_str_atom()` returns **true** for any type of string (heap, immediate, external). There are also functions further down that will return the contents of complex cells, and throw an error if called with incorrect type. Therefore it is often not necessary to use the predicates before accessing the atom internals with `any_x_get_info()`.

Note also: it is sometimes faster to check the types directly, rather than check the cell feature table. Preliminary testing indicates more than 2 type tests should use the table instead.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

- static const CellFeatures **features** [Ntypes]
- void **init_static_data** ()
- Bool_t **is_atom** (void)
- Bool_t is_pair (void)
- Bool_t is_any_pair (void)
- Bool_t is_any_svec_atom ()
- Bool_t is_any_svec2n_atom ()
- Bool_t is_any_str_atom (void)
- Bool_t is_any_sym_atom (void)
- Bool_t is_any_bvec_atom (void)
- Bool_t is_any_int (void)
- Bool_t **is_integer_valued** (void)
- Bool_t is_bignum (void)
- Bool_t is_any_real (void)

Accessors for use when the type is known.

If the cell type is already known, then these accessors can be used to efficiently access the cell contents.

- static const int LL_STR_HDR = (int) sizeof(LL_int32)
- Sexpr_t **prechecked_anypair_get_car** ()
- Sexpr_t **prechecked_anypair_get_cdr** ()
- LL_int32 **prechecked_sym_heap_get_hash** ()
- Charst_t **prechecked_sym_heap_get_chars** ()
- void **prechecked_sym_heap_get_info** (LL_int32 &hsh, Charst_t &chars)
- Ptr_t prechecked_str_heap_get_block ()
- CharVec_t prechecked_str_heap_get_chars ()
- LL_int32 prechecked_str_heap_get_charlength ()
- void prechecked_str_heap_set_charlength (LL_int32 n)

13.2.1 Detailed Description

The Cell class is the foundational class for the LambLisp Virtual Machine.

Nearly all the Cell methods are *inline* for performance. A Cell has only getters and setters; there are no other side effects. Cell fields are changed only through explicit requests, and not as the result of any other mutations. This means that Cells do not participate in garbage collection, which is managed from outside the Cell class. Indeed, there is no concept within the Cell that there might be a plurality of them; only the behavior of a single Cell is defined.

A Cell may be one of several types. Historically, the number of types varied with the variant of Lisp. In Lamb↩ Lisp, there are Cell types that correspond directly to the types described in the *Scheme RxRS* specifications (integers, procedures etc), and there are additional types that implement underlying behavior to support the higher-level language (thunks, dictionaries).

For example, LambLisp supports several types of *strings* internally, depending on whether the character are stored on the heap, in externally-provided memory, or immediately within the cell (for fast operations on short strings). Each of these *string* types is compatible with the string type in the *Scheme RxRS* specifications. Likewise, bytevectors have external and immediate variants.

LambLisp also supports specialized *pair* types, such as the LambLisp *dictionary*. These act as *Lisp* pairs for purposes of allocation and garbage collection, but their specialized operators are executed by the underlying LambLisp virtual machine and therefore operate at C++ speed.

The Cell type enumeration is purposefully ordered in such a way as to allow efficient integer comparisons instead of a C switch in most cases, enhancing runtime performance as well as garbage collection.

Cell enumeration characteristics:
Cells requiring submarking
Cells requiring finalizing
Simple atoms: immediate cell types like bool char int real etc
External (non-GC) objects and pointers to native C++ code
Cells that point to C++-allocated objects, paired with optional GC finalizers.
"well-known singleton atoms" such as NIL, undefined, and void types.
Pair - the Cell type that responds to the Lisp *pair?* predicate.
Other pairs - specialized pair types

With this ordering of Cell types, the solution to many common cases during expression evaluation can be obtained with an inequality, rather than a C++ *switch* statement. This provides a significant performance boost, because the most common cases are checked first, and the total number of cases is reduced.

Cell optimized type tests
(Types < "simple atoms") need specialized GC marking and heap reclamation.
(Types <= "cells requiring finalizing") need specialized GC heap reclamation.
(Types >= T_NIL) are lists.
(Types >= T_PAIR) are pairs (but not only *Lisp* pairs, extended pairs too).
(Types != T_PAIR) are atoms, including numbers, strings, vectors, and singletons such as NIL.
(Types > T_PAIR) are extended pairs, used for specialized lists known to LambVM, but receiving regular *pair* GC processing.

There are cases where the ordering does not help so much. For example the LambLisp type system allows for several types of strings. Because the type system is ordered according to GC requirements, these are not adjacent in the enumeration and less amenable to inequality tests. For these cases, there is a cell feature matrix available to be queried by type and feature. This allows the determination to be made with an array lookup, which is faster than several sequential tests or a C *switch*, putting a permanent cap on the cost of a type check. It also allows other details such as `is-immediate?` to be implemented inexpensively.

The garbage collector states are ordered for similar reasons. See the garbage collector chapter for details on the Cell life cycle.

13.2.2 Member Enumeration Documentation

anonymous enum

```
anonymous enum
```

Enumerator

T_SVEC_HEAP	(LL_int32 Sexpr_t[]) A vector of Sexprs is a (len Sexpr_t*) pair.
T_SVEC2N_HEAP	(LL_int32 Sexpr_t[]) Same as vector, but length must be a power of 2; useful for hash tables.
T_ANY_HEAP_SVEC	Any types less than or equal to this are heap-stored vectors of S-expressions. They need specialized marking at GC time.
T_SYM_HEAP	(LL_int32 Charst_t) Symbol is a (hash char*) pair.
T_BVEC_HEAP	(LL_int32 Bytest_t) Bytevector is a (len byte*) pair.
T_BIGNUM	(LL_int32 Bytest_t) Arbitrary-precision integer; same layout as T_BVEC_↩ HEAP: car=byte-length, cdr=ptr to payload [sign_nlimbs int32][limb0 LL_↩ int32]...
T_STR_HEAP	(LL_int32 Charst_t) car=BYTE length, cdr=ptr to a payload allocated len+1 and ZEROED – EXACTLY the shape of T_BVEC_HEAP above, which is the physical half of "a string is a bytevector". The car IS maintained at every site that builds or resizes a string; what is missing is a READER – any_str_get_length() still calls strlen(), so string-length scans the buffer twice while the byte length sits here. The CHARACTER length is genuinely absent.
T_CPP_HEAP	(ptr-to-cpp-deleter ptr-to-cpp-object) Deleter is a function `void f(void *cpp_obj)` of the appropriate type T of the C++ object, and performs `delete (T *) cpp_obj;`
T_PORT_HEAP	(reserved Ptr_t) car is unused, cdr is ptr to underlying C++ port instance
T_NEEDS_FINALIZING	Any types less than or equal to this have data stored in the heap, and need specialized finalizing at GC time.
T_BVEC_EXT	(LL_int32 Bytest_t) Same as T_BYTEVEC but externally allocated; the byte array is not freed at GC time.
T_STR_EXT	(reserved Charst_t) Same as T_STR_HEAP but externally allocated; the character array is not freed at GC time. UNLIKE T_STR_HEAP the car really is unused here – *OBJ_SYS_STR* (ll_vm_cell.cpp:23) is built with car=0 – so an external string's length comes from strlen and nowhere else.
T_BVEC_IMM	Special format: type and flag bytes as usual, byte 2 is vector length, remaining bytes are vector elements.
T_STR_IMM	Special format: type and flag bytes as usual, remaining bytes are 0-terminated string embedded in the cell.
T_GENSYM	Runtime symbol generation with no heap operations.
T_BOOL	(Bool_t reserved) Boolean atom
T_CHAR	(Char_t reserved) Character atom
T_INT32	(LL_int32 reserved) Exact 32-bit integer; value in car via memcpy(4).

T_INT64	(LL_int64 wide) Exact 64-bit integer; 8-byte value at &car (spans car+cdr on ESP32). GC-safe: type < T_NIL.
T_FLOAT32	(LL_float32 reserved) Inexact 32-bit LL_float32; value in car via memcpy(4).
T_FLOAT64	(LL_float64 wide) Inexact 64-bit LL_float64; 8-byte value at &car (spans car+cdr on ESP32). GC-safe: type < T_NIL.
T_MOP3_PROC	(reserved *Mop3st_t) pointer to native function - args are evaluated before calling
T_MOP3_NPROC	(reserved *Mop3st_t) pointer to native macro processor - args are not evaluated before calling
T_VOID	(don't care) VOID has its own type and a singleton instance OBJ_VOID
T_UNDEF	(ERROR ERROR) UNDEF has its own type and a singleton instance OBJ_↩ UNDEF
T_NIL	(ERROR ERROR) NIL has its own type and a singleton instance. • The NIL singleton is both a list and an atom, but not a pair. • Types <= T_NIL are atoms, types >= T_NIL are lists, types > T_NIL are pairs, but only T_PAIR responds to the Scheme *pair?* predicate.
T_COMPLEX	(slot-typed) Complex number atom; car=real part, cdr=imaginary part. EACH SLOT is an embedded LL_int32/LL_float32 OR a reference to another number cell, so the collector DOES traverse the slots – a complex is an atom to your program but not to the collector.
T_RATIONAL	(slot-typed) Exact rational atom; car=numerator, cdr=denominator (>=2, GCD-reduced). EACH SLOT is an embedded LL_int32 OR a traced pointer to a boxed T_INT32/T_INT64/T_BIGNUM – so ALONE among the types below T_PAIR, the GC DOES scan it; see the T_RATIONAL case in mark_one_↩ quantum().
T_FREE	(Sexpr_t Sexpr_t) Every cell on the free list. Below T_PAIR, so `car`/`cdr` raise on a stale read instead of walking it. car is OBJ_FREED, cdr is the next free cell. [P298]
T_PAIR	(slot Sexpr_t) Normal cons cell. EACH SLOT holds a reference to another cell OR an embedded LL_int32 or LL_float32 – `cons` packs small numbers directly into the slot rather than allocating a cell for them, and Lamb::car()/↩ cdr() materialise them again on the way out. Types below T_PAIR are atoms. • Pair types. Types > NIL and types >= T_PAIR are pairs, but only T_↩ PAIR responds to Scheme (pair?). • All pair types can be garbage collected without type-specific GC marking or finalizing.
T_SVEC_IMM	(Sexpr_t Sexpr_t) Vector of 0, 1 or 2 elements.
T_PROC	(Sexpr_t Sexpr_t) Procedure pair; car is lambda (formals + body containing free variables), cdr is environment.
T_NPROC	(Sexpr_t Sexpr_t) Non-evaluating procedure pair; car is nlambda (formals + body containing free variables), cdr is environment.
T_MACRO	(Sexpr_t Sexpr_t) Macro; car is transformer (proc of 1 arg) and cdr is env.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

T_DICT	(Sexpr_t Sexpr_t) Dictionary pair; car is local frame, cdr is list of parent frames, used for environments & object instances.
T_THUNK_SEXPR	(Sexpr_t Sexpr_t) S-expression thunk pair; car is sexpr, cdr is environment with all variable bindings.
T_THUNK_BODY	(Sexpr_t Sexpr_t) Code body thunk pair; car is body (i.e., list of sexprs), cdr is environment with all variable bindings.
T_ERROR	(Sexpr_t Sexpr_t) An error cell; car is a T_STRING, cdr is a pointer to *irritants*.
T_VALUES	(Sexpr_t Sexpr_t) Multiple return values; car is first value, cdr is list of remaining values.
T_BYTECODE	(Sexpr_t Sexpr_t) Compiled bytecode procedure; car is T_SVEC_HEAP↩ [bvec,arity,maxloc,maxstk,const...], cdr is closure env.
T_BYTECODE_N	(Sexpr_t Sexpr_t) Compiled bytecode nlambda (non-evaluating); same layout as T_BYTECODE, but caller does NOT evaluate arguments.
Ntypes	Number of LambLisp virtual machine types.

13.2.3 Member Function Documentation

zero()

```
void Cell::zero ()  [inline]
```

Set all cell bits to zero.

type() [1/2]

```
LL_int32 Cell::type (void )  [inline]
```

Return the type of the cell as a small integer.

type() [2/2]

```
void Cell::type (LL_int32 t)  [inline]
```

Set the type of this cell.

flags()

```
LL_int32 Cell::flags (void )  [inline]
```

Return the entire set of cell flags.

flags_set()

```
void Cell::flags_set (LL_int32 f)  [inline]
```

Set the selected flags.

flags_clr()

```
void Cell::flags_clr (LL_int32 f)  [inline]
```

Clear the selected flags.

rplaca()

```
void Cell::rplaca (Word_t p)  [inline]
```

Replace the cell car field. Called rplaca for historical reasons, and to distinguish it from set-car!, which must respect the GC flags.

rplacd()

```
void Cell::rplacd (Word_t p)  [inline]
```

Replace the cell cdr field. Called rplacd for historical reasons, and to distinguish it from set-cdr!, which must respect the GC flags.

is_pair()

```
Bool_t Cell::is_pair (void )  [inline]
```

Return true if the cell is a cons pair.

is_any_pair()

```
Bool_t Cell::is_any_pair (void )  [inline]
```

True for every cell whose car and cdr hold Scheme values – not just `T_PAIR` but procedures, dictionaries, thunks, errors, bytecode and the rest of the pair region. This is the question `car` and `cdr` ask. It is NOT the collector's question: see `is_slot_typed()`.

is_any_svec_atom()

```
Bool_t Cell::is_any_svec_atom ()  [inline]
```

Return true if the cell is any kind of Sexpr_t vector.

is_any_svec2n_atom()

```
Bool_t Cell::is_any_svec2n_atom ()  [inline]
```

Return true if the cell is Sexpr_t vector of size 2^n.

is_any_str_atom()

```
Bool_t Cell::is_any_str_atom (void )  [inline]
```

Return true if the cell is any kind of string.

is_any_sym_atom()

```
Bool_t Cell::is_any_sym_atom (void )  [inline]
```

Return true if the cell is any kind of symbol.

is_any_bvec_atom()

```
Bool_t Cell::is_any_bvec_atom (void )  [inline]
```

Return true if the cell is any kind of bytevector.

is_any_int()

```
Bool_t Cell::is_any_int (void )  [inline]
```

Return true if the cell is any exact integer type (T_INT32, T_INT64, or T_BIGNUM).

is_bignum()

```
Bool_t Cell::is_bignum (void )  [inline]
```

does this cell's VALUE denote an integer? This is what R7RS `integer?` exactness is a different question, so (integer? 2.0) is #t.

Return true if the cell is T_BIGNUM.

is_any_real()

```
Bool_t Cell::is_any_real (void )  [inline]
```

Return true if the cell is any inexact real type (T_FLOAT32 or T_FLOAT64).

is_slot_typed()

```
bool Cell::is_slot_typed () const  [inline]
```

True when the collector traverses this cell's two slots.

car_is_int()

```
bool Cell::car_is_int () const  [inline]
```

True when the car slot holds an embedded LL_int32.

car_is_float()

```
bool Cell::car_is_float () const  [inline]
```

True when the car slot holds an embedded LL_float32.

cdr_is_int()

```
bool Cell::cdr_is_int () const  [inline]
```

True when the cdr slot holds an embedded LL_int32.

cdr_is_float()

```
bool Cell::cdr_is_float () const  [inline]
```

True when the cdr slot holds an embedded LL_float32.

car_is_embedded()

```
bool Cell::car_is_embedded () const  [inline]
```

True when the car holds an embedded value of either kind (one bitwise op).

cdr_is_embedded()

```
bool Cell::cdr_is_embedded () const  [inline]
```

True when the cdr holds an embedded value of either kind (one bitwise op).

car_int32()

```
LL_int32 Cell::car_int32 () const  [inline]
```

Car word as a signed int32. ONLY meaningful when `car_is_int()`; on any other slot this returns the bit pattern of whatever is there, including a pointer.

car_float32()

```
LL_float32 Cell::car_float32 () const  [inline]
```

Car word as a float32. ONLY meaningful when `car_is_float()`.

cdr_int32()

```
LL_int32 Cell::cdr_int32 () const  [inline]
```

Cdr word as a signed int32. Same caution as `car_int32()`.

cdr_float32()

```
LL_float32 Cell::cdr_float32 () const  [inline]
```

Cdr word as a float32. Same caution as `car_float32()`.

gc_state() [1/2]

```
LL_int32 Cell::gc_state (void )  [inline]
```

Return the garbage-collection state of this cell.

gc_state() [2/2]

```
Sexpr_t Cell::gc_state (LL_int32 st)  [inline]
```

Set the garbage collection state of this cell, and return the cell.

tail_state()

```
LL_int32 Cell::tail_state (void )  [inline]
```

Return the tail-call state of this cell.

tail_state_set()

```
Sexpr_t Cell::tail_state_set (void )  [inline]
```

Set the tail state flag on this cell, and return the cell.

tail_state_clr()

```
Sexpr_t Cell::tail_state_clr (void )  [inline]
```

Clear the tail state flag on this cell, and return the cell.

bytecode_is_variadic()

```
bool Cell::bytecode_is_variadic () const  [inline]
```

Ph9: true if T_BYTECODE with rest-arg.

bytecode_set_variadic()

```
void Cell::bytecode_set_variadic ()  [inline]
```

Ph9: mark T_BYTECODE as variadic.

get_car_addr()

```
Ptr_t Cell::get_car_addr ()  [inline]
```

Return the address of the cell car.

get_car()

```
Word_t Cell::get_car (void )  [inline]
```

Return the value of the cell car.

get_cdr()

```
Word_t Cell::get_cdr (void )  [inline]
```

Return the value of the cell cdr.

as_Bool_t()

```
Bool_t Cell::as_Bool_t ()  [inline]
```

Return the value of this cell as a boolean.

Reading a cell's value – four prefixes, and the difference matters

The accessors come in four families. Choosing the wrong one is the commonest way to write a mop3 that works on your bench and fails on a customer's data.

`as_X()` Reads the bits and TRUSTS YOU. No type check. If the cell is not the type you assumed, you get a meaningless value rather than an error – an integer made from a pointer, a float made from a length. Use it only where the type is already established. `mustbe_X()` CHECKS the type and raises a LambLisp error if it is wrong. This is what a procedure should call on its arguments: the error names the offending value and reaches the user as a normal Scheme error rather than a crash. `prechecked_X()` Unchecked, and named so you can see at a glance that the check was done elsewhere – typically by a `mustbe_` or a predicate a line or two earlier. `any_X_get_*()` The safe compound: checks, then reads. Usually what you want.

The rule of thumb: validate arguments with `mustbe_`, then work with `prechecked_` or the raw accessors. Reach for `as_` only inside code that has just tested the type itself.

as_Char_t()

```
Char_t Cell::as_Char_t ()  [inline]
```

Return the value of this cell as a character.

as_codepoint()

```
ll_codepoint_t Cell::as_codepoint ()  [inline]
```

Return full Unicode codepoint stored in T_CHAR cell.

as_int32()

```
LL_int32 Cell::as_int32 () const  [inline]
```

T_INT32 cell value.

as_int64()

```
LL_int64 Cell::as_int64 () const  [inline]
```

T_INT64 cell value; spans car+cdr on ESP32.

as_float32()

```
LL_float32 Cell::as_float32 () const  [inline]
```

T_FLOAT32 cell value.

as_float64()

```
LL_float64 Cell::as_float64 () const  [inline]
```

T_FLOAT64 cell value; spans car+cdr on ESP32.

as_Ptr_t()

```
Ptr_t Cell::as_Ptr_t ()  [inline]
```

Return the value of this cell as a generic pointer (i.e., void∗).

as_Charst_t()

```
Charst_t Cell::as_Charst_t ()  [inline]
```

Return the value of this cell as a pointer to a zero-terminated character array.

as_Bytest_t()

```
Bytest_t Cell::as_Bytest_t ()  [inline]
```

Return the value of this cell as a pointer to an array of bytes.

as_CharVec_t()

```
CharVec_t Cell::as_CharVec_t ()  [inline]
```

Return the value of this cell as a pointer to a zero-terminated character array.

as_ByteVec_t()

```
ByteVec_t Cell::as_ByteVec_t ()  [inline]
```

Return the value of this cell as a pointer to an array of bytes.

as_Portst_t()

```
Portst_t Cell::as_Portst_t ()  [inline]
```

Return the value of this cell as a pointer to an instance of the system underlying "port" implementation.

hash_sexpr()

```
LL_int32 Cell::hash_sexpr (void )
```

For symbols, the hash of its characters; for numbers, the hash of the number; otherwise the hash of the S-expression itself (i.e., the address of a cell).

hash_contents()

```
LL_int32 Cell::hash_contents (void )
```

For symbols, the hash of its characters; for numbers, strings, vectors, and bytevectors, the hash of the contents; otherwise the hash of the S-expression itself.

hash()

```
LL_int32 Cell::hash (void )  [inline]
```

Return the hash value of this cell.

set() [1/17]

```
Sexpr_t Cell::set (LL_int32 typ, Word_t w1, Word_t w2)  [inline]
```

This is the lowest-level generic "set" function.

set() [2/17]

```
Sexpr_t Cell::set (LL_int32 typ, LL_int32 a, Sexpr_t b)  [inline]
```

Convenience: car is a raw (sign-extended) int32. ATOMS only – never a pair type (a pair car must be a full pointer; see the Word_t/Sexpr_t overload below, added for).

make_free()

```
Sexpr_t Cell::make_free (Sexpr_t next)  [inline]
```

Link onto the free list. The ONLY writer of T_FREE + gcst_free, so the two cannot disagree. `next` may be NIL. [P298]

set() [3/17]

```
Sexpr_t Cell::set (LL_int32 typ, Word_t a, Sexpr_t b)  [inline]
```

FULL-WIDTH car. This overload exists so a (Word_t, Sexpr_t) call does not resolve to the (LL_int32, Sexpr_t) overload above and SIGN-TRUNCATE the car.

set() [4/17]

```
Sexpr_t Cell::set (LL_int32 typ, Sexpr_t a, Sexpr_t b)  [inline]
```

This is a convenience function for common cases.

set() [5/17]

```
Sexpr_t Cell::set (Bool_t b)  [inline]
```

Set cell as boolean.

set() [6/17]

```
Sexpr_t Cell::set (Char_t c)  [inline]
```

Set cell as character.

set() [7/17]

```
Sexpr_t Cell::set (LL_int32 i)  [inline]
```

Set cell as T_INT32.

set() [8/17]

```
Sexpr_t Cell::set (LL_int64 v)  [inline]
```

Set cell as T_INT64 (8 bytes at &car; spans cdr on ESP32).

set() [9/17]

```
Sexpr_t Cell::set (LL_float32 r)  [inline]
```

Set cell as T_FLOAT32.

set() [10/17]

```
Sexpr_t Cell::set (LL_float64 d)  [inline]
```

Set cell as T_FLOAT64 (8 bytes at &car; spans cdr on ESP32).

set() [11/17]

```
Sexpr_t Cell::set (LL_float32 r, LL_float32 i)  [inline]
```

<Set cell as T_COMPLEX (single precision)

< both slots an embedded float32 (bit1 set, bit0 clear)

cpx_real()

```
LL_float32 Cell::cpx_real () const  [inline]
```

Real part of a T_COMPLEX cell.

cpx_imag()

```
LL_float32 Cell::cpx_imag () const  [inline]
```

Imaginary part of a T_COMPLEX cell.

set() [12/17]

```
Sexpr_t Cell::set (LL_int32 num, LL_int32 den)  [inline]
```

< both slots an embedded int32 (preserve gc_state/F_TAIL)

rat_set_car_i32()

```
void Cell::rat_set_car_i32 (LL_int32 v)  [inline]
```

embedded int32 numerator (embedded)

rat_set_car_ptr()

```
void Cell::rat_set_car_ptr (Sexpr_t p)  [inline]
```

boxed-int numerator ptr (a traced pointer)

rat_set_cdr_i32()

```
void Cell::rat_set_cdr_i32 (LL_int32 v)  [inline]
```

embedded int32 denominator (embedded)

rat_set_cdr_ptr()

```
void Cell::rat_set_cdr_ptr (Sexpr_t p)  [inline]
```

boxed-int denominator ptr (a traced pointer)

int_as_i64()

```
LL_int64 Cell::int_as_i64 () const  [inline]
```

value of a boxed T_INT32/T_INT64 (NOT T_BIGNUM)

rat_car_ptr()

```
Sexpr_t Cell::rat_car_ptr () const  [inline]
```

boxed numerator cell (valid when car_is_ptr())

rat_cdr_ptr()

```
Sexpr_t Cell::rat_cdr_ptr () const  [inline]
```

boxed denominator cell (valid when cdr_is_ptr())

rat_is_big()

```
bool Cell::rat_is_big () const  [inline]
```

a component is a T_BIGNUM

exact_is_big()

```
bool Cell::exact_is_big () const  [inline]
```

does not fit the int64 fast path

as_numerator()

```
LL_int64 Cell::as_numerator () const  [inline]
```

Numerator as int64 (guard rat_is_big()).

as_denominator()

```
LL_int64 Cell::as_denominator () const  [inline]
```

Denominator as int64 (guard rat_is_big()).

exact_sign()

```
int Cell::exact_sign ()  [inline]
```

Sign (-1/0/+1) of any exact number; for a rational, the numerator's sign (denominator is positive).

set() [13/17]

```
Sexpr_t Cell::set (Port_t & p)  [inline]
```

Set cell as port.

set() [14/17]

```
Sexpr_t Cell::set (LL_int32 typ, LL_int32 a, Charst_t b)  [inline]
```

Set cell as a immutable string.

set() [15/17]

```
Sexpr_t Cell::set (LL_int32 typ, LL_int32 a, Bytest_t b)  [inline]
```

Set cell as a immutable bytevector.

set() [16/17]

```
Sexpr_t Cell::set (LL_int32 typ, LL_int32 a, CharVec_t b)  [inline]
```

Set cell as a mutable string.

set() [17/17]

```
Sexpr_t Cell::set (LL_int32 typ, LL_int32 a, ByteVec_t b)  [inline]
```

Set cell as a mutable bytevector.

mk_error() [1/2]

```
Sexpr_t Cell::mk_error (const char * fmt,  ...)
```

Fill and return OBJ_SYSERROR, the ONE statically allocated error cell. Allocates nothing, so it still works when the heap cannot – but only one exists, so a second call overwrites the first. For an error you may hold alongside others, use Lamb::mk_error().

mk_error() [2/2]

```
Sexpr_t Cell::mk_error (Sexpr_t irritants, const char * fmt,  ...)
```

As above, with an irritants list. Same single shared cell.

prechecked_str_heap_get_block()

```
Ptr_t Cell::prechecked_str_heap_get_block ()  [inline]
```

The BLOCK base == the cdr == what payload_alloc returned. This is the only thing that may be freed.

prechecked_str_heap_get_chars()

```
CharVec_t Cell::prechecked_str_heap_get_chars ()  [inline]
```

The characters, LL_STR_HDR bytes into the block. INTERIOR POINTER – never free this.

prechecked_str_heap_get_charlength()

```
LL_int32 Cell::prechecked_str_heap_get_charlength ()  [inline]
```

Character count, read from the block head.

prechecked_str_heap_set_charlength()

```
void Cell::prechecked_str_heap_set_charlength (LL_int32 n)  [inline]
```

Write the character count into the block head.

prechecked_str_ext_get_chars()

```
CharVec_t Cell::prechecked_str_ext_get_chars ()  [inline]
```

Return a pointer to the character array located outside the heap.

prechecked_str_imm_get_chars()

```
CharVec_t Cell::prechecked_str_imm_get_chars ()  [inline]
```

Return a pointer to the character array embedded in this cell.

prechecked_gensym_get_chars()

```
Charst_t Cell::prechecked_gensym_get_chars ()  [inline]
```

< one instance program-wide (inline fn local static)

< keep the temporary alive across the copy

mustbe_Bool_t()

```
Bool_t Cell::mustbe_Bool_t ()  [inline]
```

Return the value of this cell as a boolean.

mustbe_Char_t()

```
Char_t Cell::mustbe_Char_t ()  [inline]
```

Return the value of this cell as a character.

mustbe_codepoint()

```
ll_codepoint_t Cell::mustbe_codepoint ()  [inline]
```

Return full codepoint; throw if not T_CHAR.

mustbe_int32()

```
LL_int32 Cell::mustbe_int32 ()  [inline]
```

Return the value of this T_INT32 cell as int32.

mustbe_float32()

```
LL_float32 Cell::mustbe_float32 ()  [inline]
```

Return the value of this T_FLOAT32 cell as a real.

mustbe_any_str_t()

```
Sexpr_t Cell::mustbe_any_str_t ()  [inline]
```

Return this cell if it is a string.

mustbe_cppobj_t()

```
Sexpr_t Cell::mustbe_cppobj_t ()  [inline]
```

Return this cell if it is a CPP object.

prechecked_cppobj_get_deleter()

```
CPPDeleterPtr Cell::prechecked_cppobj_get_deleter ()  [inline]
```

Return the function to be called at garbage collection time to recycle the C++ object.

prechecked_cppobj_get_ptr()

```
Ptr_t Cell::prechecked_cppobj_get_ptr ()  [inline]
```

Return a pointer to a C++ object obtained earlier.

coerce_float32()

```
LL_float32 Cell::coerce_float32 ()  [inline]
```

< a component may be a boxed bignum -> coerce it via the boxed cell

coerce_float64()

```
LL_float64 Cell::coerce_float64 ()  [inline]
```

< a component may be a boxed bignum -> coerce it via the boxed cell

exact_rank()

```
int Cell::exact_rank () const  [inline]
```

Exact numeric rank: 0=T_INT32 1=T_INT64 2=T_RATIONAL 3=T_BIGNUM -1=not exact.

inexact_rank()

```
int Cell::inexact_rank () const  [inline]
```

Inexact numeric rank: 0=T_FLOAT32 1=T_FLOAT64 2=T_COMPLEX -1=not inexact.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

is_exact()

```
bool Cell::is_exact () const  [inline]
```

True for T_INT32, T_INT64, T_RATIONAL, T_BIGNUM.

is_inexact()

```
bool Cell::is_inexact () const  [inline]
```

True for T_FLOAT32, T_FLOAT64, T_COMPLEX.

is_complex()

```
bool Cell::is_complex () const  [inline]
```

True iff T_COMPLEX.

num_rank()

```
int Cell::num_rank () const  [inline]
```

Legacy scalar rank: 0=int 1=float 2=rational 3=complex -1=non-numeric.

numerator64()

```
LL_int64 Cell::numerator64 () const  [inline]
```

Numerator of any exact number as LL_int64.

denominator64()

```
LL_int64 Cell::denominator64 () const  [inline]
```

Denominator of any exact number as LL_int64 (1 for integer types).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

coerce_cpx_real()

```
LL_float64 Cell::coerce_cpx_real ()  [inline]
```

Real part: T_COMPLEX → cpx_real; others → coerce_float64.

coerce_cpx_imag()

```
LL_float64 Cell::coerce_cpx_imag ()  [inline]
```

Imaginary part: T_COMPLEX → cpx_imag; others → 0.

any_error_get_str()

```
Sexpr_t Cell::any_error_get_str ()  [inline]
```

any_error_get_∗: like error_get_∗ but the T_ERROR typecheck is folded in and a non-error yields a benign sentinel (NIL) instead of throwing – lets a predicate write o->any_error_get_irritants() without a separate type() guard (less source, no missed-check bug).

any_str_get_chars()

```
CharVec_t Cell::any_str_get_chars ()  [inline]
```

If the cell is any kind of string, return a pointer to the zero-terminated character array.

any_str_get_length()

```
LL_int32 Cell::any_str_get_length ()  [inline]
```

BYTE length. T_STR_HEAP keeps it in the car, written at every site that builds or resizes one, so this reads it with no scan. T_STR_IMM is bounded by `imm_limit`, so its scan is bounded by construction; T_STR_EXT is a static literal whose car really is 0 and which mutation refuses.

str_invalidate_charlength()

```
void Cell::str_invalidate_charlength ()  [inline]
```

Mark a heap string's cached character count UNKNOWN. No-op for anything else.

CHARACTER length – what R7RS `string-length` returns. Stored for a heap string; counted for the other two, where the buffer is bounded by the cell.

CALL THIS AFTER ANY WRITE THAT REACHES A STRING'S BYTES WITHOUT RESIZING IT. The resize path (ll_str_splice) already invalidates, because it is the only way the BYTE length can change – but the CHARACTER count can change with the byte count fixed. Writing one UTF-8 lead byte over an ASCII byte merges it with its successor: 26 bytes stay 26 bytes and become 25 characters. The byte view of a string is exactly such a path: `bytevector-u8-set!` and `bytevector-copy!` accept a string and write its bytes raw, with no UTF-8 validation and without A stale flag is a WRONG ANSWER, not a slow one: after `(bytevector-u8-set! s 0 206)` on a 26-char ASCII string, an uninvalidated `string-length` still answers 26.

any_str_get_charlength()

```
LL_int32 Cell::any_str_get_charlength ()  [inline]
```

A STORED 0 MEANS "NOT KNOWN", AND THAT POLARITY IS DELIBERATE. ll_str_splice() only makes ROOM – its callers write the new bytes AFTER it returns – so the count cannot be computed there; the buffer is not final yet. Every resize therefore stores 0 and the count is recomputed on the next ask and cached here. Choosing 0 rather than -1 as the sentinel makes the failure mode free in both directions: a freshly payload↩ _alloc'd block is memzero'd, and a construction path that forgets to write the header gets a RECOMPUTE (slow, correct) instead of a silent 0-length string. The one value it collides with is a genuinely empty string, whose recount is a scan of zero bytes. A missed update costs speed, never correctness – the same rule as the GC state bits.

any_svec_get_info()

```
void Cell::any_svec_get_info (LL_int32 & Nelems, Sexpr_t *& elems)  [inline]
```

<NOT OBJ_VOID – that is a real value

any_bvec_get_length()

```
LL_int32 Cell::any_bvec_get_length ()  [inline]
```

A T_BIGNUM shares T_BVEC_HEAP's layout and payload free, but is NOT a bytevector: these three accessors refuse it, so no bytevector operation can read or write a bignum's limbs. [B1075] < BYTE length of a string

any_bvec_get_elems()

```
ByteVec_t Cell::any_bvec_get_elems ()  [inline]
```

< byte view of a string

any_bvec_get_info()

```
void Cell::any_bvec_get_info (LL_int32 & Nelems, ByteVec_t & elems)  [inline]
```

< byte view of a string – see the note above

< strlen: BYTES, not characters

bvec_elem_type()

```
LL_int32 Cell::bvec_elem_type (void ) const  [inline]
```

Get the array element-type tag from Cell byte 3. HEAP/EXT SHAPES ONLY – on T_BVEC_IMM this is element 0, not a type.

bvec_set_elem_type()

```
void Cell::bvec_set_elem_type (LL_int32 t)  [inline]
```

Set the array element-type tag in Cell byte 3. HEAP/EXT SHAPES ONLY – on T_BVEC_IMM this OVERWRITES element 0.

cell_name()

```
String Cell::cell_name (void )
```

A convenience feature to produce a string name for cells which are "well known" like NIL. Otherwise the name is the hex representation of the cell address.

type_name()

```
Charst_t Cell::type_name (LL_int32 typ)
```

Return a pointer to the C string corresponding to the cell type.

gcstate_name()

```
Charst_t Cell::gcstate_name (void )
```

Return a pointer to a C string corresponding to the given GC state.

dump()

```
String Cell::dump ()
```

Return a printable representation of the Cell internals.

13.2.4 Member Data Documentation

F_GC01

```
const int Cell::F_GC01 = 0x01  [static]
```

gc flag

F_GC02

```
const int Cell::F_GC02 = 0x02  [static]
```

gc flag

F_GC04

```
const int Cell::F_GC04 = 0x04  [static]
```

gc flag

F_TAIL

```
const int Cell::F_TAIL = 0x08  [static]
```

trampoline tail marker

F_SUBTYPE_MASK

```
const int Cell::F_SUBTYPE_MASK = 0xF0  [static]
```

The subtype bits. Meaning is per top-level type; today only pairs use them.

F_CAR_IS_INT

```
const int Cell::F_CAR_IS_INT = 0x10  [static]
```

PAIR subtype: car holds an embedded LL_int32 (not a traced pointer).

F_CAR_IS_FLOAT

```
const int Cell::F_CAR_IS_FLOAT = 0x20  [static]
```

PAIR subtype: car holds an embedded LL_float32 (not a traced pointer).

F_CDR_IS_INT

```
const int Cell::F_CDR_IS_INT = 0x40  [static]
```

PAIR subtype: cdr holds an embedded LL_int32 (not a traced pointer).

F_CDR_IS_FLOAT

```
const int Cell::F_CDR_IS_FLOAT = 0x80  [static]
```

PAIR subtype: cdr holds an embedded LL_float32 (not a traced pointer).

GC_STATE_MASK

```
const int Cell::GC_STATE_MASK = F_GC01 | F_GC02 | F_GC04  [static]
```

Mask for obtaining garbage collection state bits from Cell flag byte.

F_BC_VARIADIC

```
const int Cell::F_BC_VARIADIC = 0x01  [static]
```

byte 2 bit 0: final locals slot holds rest-arg list (T_BYTECODE only).

LL_STR_HDR

```
const int Cell::LL_STR_HDR = (int) sizeof(LL_int32)  [static]
```

Bytes of block header before the characters.

13.3 Cell::CellFeatures Struct Reference

Public Attributes

- LL_int32 **typ**
- bool **is_any_pair**
- bool **is_any_svec**
- bool **is_any_svec2n**
- bool **is_any_str**
- bool **is_any_sym**
- bool **is_any_bvec**
- bool **is_atom**
- bool **is_pair**
- bool **is_slot_typed**
- bool **is_any_int**
- bool **is_any_real**
- bool **is_exact**
- bool **is_inexact**
- const char * **type_name**

13.3.1 Detailed Description

Answers every type predicate below. Columns with no reader are kept for orthogonality; `init_static_data()` asserts each against its predicate.

13.4 Lamb Class Reference

Public Types

- typedef Sexpr_t(* Mop3st_t) (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
- typedef Sexpr_t(* Mop3st_argv_t) (Lamb &lamb, Sexpr_t *argv, int nargs, Sexpr_t env_exec)

Public Member Functions

- Sexpr_t cdar (Sexpr_t l)
- Sexpr_t cddr (Sexpr_t l)
- Sexpr_t caddr (Sexpr_t l)
- Sexpr_t cdddr (Sexpr_t l)
- Sexpr_t cadddr (Sexpr_t l)
- Sexpr_t cddddr (Sexpr_t l)
- Sexpr_t load (Charst_t name, Sexpr_t env_exec, LL_int32 verbosity=0)
- Sexpr_t mk_input_string_port (Charst_t inp, Sexpr_t env_exec)
- Sexpr_t mk_output_string_port (Sexpr_t env_exec)
- Sexpr_t mk_input_bytevector_port (ByteVec_t src, LL_int32 n, Sexpr_t env_exec)
- Sexpr_t mk_output_bytevector_port (Sexpr_t env_exec)
- void diag_memory_report ()

Arduino-compatible loop-based control interface.

- Sexpr_t setup (void)
- Sexpr_t loop (void)
- void end (void)
- ~Lamb ()

A few foundational functions for embedded debugging.

- void log (const char *fmt,...) CHECKPRINTF_pos2
- void printf (const char *fmt,...) CHECKPRINTF_pos2
- bool debug (void)
- void debug (bool onoff)

Information about the current build.

- bool build_isDebug ()
- unsigned long build_version ()
- unsigned long build_UTC ()
- unsigned long build_pushUTC ()
- const char * build_buildRelease ()
- const char * build_buildDate ()
- const char * build_pushDate ()

Cell constructors

Garbage collection may happen at any time during the execution of the Cell constructors. The cell constructors that accept S-expression arguments will protect those arguments from GC during the operation, using the gc root push/pop functions.

- void expand ()
- Sexpr_t tcons (LL_int32 typ, Word_t a, Word_t b, Sexpr_t env_exec)
- Sexpr_t tcons (LL_int32 typ, Sexpr_t a, Sexpr_t b, Sexpr_t env_exec)
- Sexpr_t cons (Sexpr_t a, Sexpr_t b, Sexpr_t env_exec)
- Sexpr_t materialized (Sexpr_t box)
- LL_int32 gc_root_depth_max ()
- void gc_root_depth_max_reset ()
- void gc_root_setdepth (LL_int32 n)

- void gc_idle_task_us (LL_int32 us, Sexpr_t env_exec)
- void gc_collect (Sexpr_t env_exec)
- uint32_t **eval_progress** () const
- LL_int32 gc_Qm () const
- void gc_pass_n (LL_int32 nsteps)
- LL_int32 **mem_cell_block_size** ()
- LL_int32 **mem_n_cell_blocks** ()
- LL_int32 **mem_total_cells** ()
- LL_int32 **mem_ngc** ()
- LL_int32 **mem_markstack_grows** ()
- LL_int32 **mem_urgent_calls** ()
- LL_int32 **mem_urgent_max_us** ()
- LL_int32 **mem_expand_max_us** ()
- LL_int32 **mem_expand_max_cells** ()
- LL_int32 **mem_expand_calls** ()
- LL_int32 **mem_expand_cells** ()
- LL_int32 **mem_persist_max_us** ()
- LL_int32 **mem_persist_calls** ()
- LL_int32 **mem_settings_writes** ()
- LL_int32 **mem_settings_skipped** ()
- LL_int32 **mem_markq_max_us** ()
- LL_int32 **mem_markq_cells** ()
- LL_int32 **mem_sweepq_max_us** ()
- LL_int32 **mem_sweepq_cells** ()
- LL_int32 **mem_markq_n** ()
- LL_int32 **mem_markq_sum_us** ()
- LL_int32 **mem_markq_iters** ()
- LL_int32 **mem_markq_pushes** ()
- LL_int32 **mem_markq_fsx** ()
- LL_int32 **mem_markq_fvec** ()
- LL_int32 **mem_markq_froot** ()
- LL_int32 **mem_markq_vtype** ()
- LL_int32 **mem_markq_vn** ()
- void **mem_quantum_reset** ()
- void **mem_diag_quiet** (Bool_t q)
- LL_int32 **mem_startmark_max_us** ()
- LL_int32 **mem_startmark_nroots** ()
- void mem_startmark_reset ()
- LL_int32 **mem_extraroots_max_us** ()
- LL_int32 **mem_extraroots_pushes** ()
- LL_int32 **mem_nfree** ()
- LL_int32 **mem_extension_block_size** ()
- void **mem_set_extension_block_size** (LL_int32 sz)
- bool mem_expand_to_n_blocks (LL_int32 n, Sexpr_t env_exec)
- LL_int32 **mem_max_cell_blocks** ()
- void **mem_set_max_cell_blocks** (LL_int32 n)
- LL_int32 **mem_gc_budget_ns** ()
- LL_int32 **mem_stat_urgent_max_us** ()
- LL_int32 **mem_stat_urgent_calls** ()
- LL_int32 **mem_stat_urgent_bulk** ()
- unsigned long mem_stat_t_gc_total_us ()
- LL_int32 mem_stat_nfree ()
- LL_int32 **mem_stat_amax** ()
- LL_int32 mem_stat_budget_achieved_ns ()
- LL_int32 mem_stat_gcphase ()
- LL_int32 mem_stat_payload_collect ()
- LL_int32 **mem_stat_live_marked** ()
- unsigned long **mem_stat_n_idle** ()
- unsigned long **mem_stat_n_work** ()
- LL_int32 **mem_stat_t_mark_ns** ()

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

- LL_int32 **mem_stat_t_sweep_ns** ()
- LL_int32 **mem_stat_mark_q** ()
- LL_int32 **mem_stat_sweep_q** ()
- LL_int32 **mem_stat_yuasa_M** ()
- LL_int32 **mem_stat_yuasa_N** ()
- void **mem_set_gc_budget_ns** (LL_int32 ns)
- LL_int32 **mem_gcload_target_pct** ()
- void **mem_set_gcload_target_pct** (LL_int32 pct)
- LL_int32 **verbosity** ()
- void **set_verbosity** (LL_int32 v)
- void **set_repl_quiet** (bool q)
- bool **repl_quiet** (void) const
- LL_int32 ncg_frame_pool_n ()
- void ncg_frame_pool_init (LL_int32 n)
- void ncg_frame_pool_free ()

A subset of the car/cdr accessors for list cells. This is the subset used within LambLisp.

- Sexpr_t car (Sexpr_t l)
- Sexpr_t cdr (Sexpr_t l)
- Sexpr_t prechecked_anypair_get_car (Sexpr_t c)
- Sexpr_t prechecked_anypair_get_cdr (Sexpr_t c)
- Sexpr_t caar (Sexpr_t l)
- Sexpr_t cadr (Sexpr_t l)
- int **car_type** (Sexpr_t c)
- int **cdr_type** (Sexpr_t c)
- LL_int32 car_mustbe_int32 (Sexpr_t c)
- LL_int32 cdr_mustbe_int32 (Sexpr_t c)
- LL_float32 car_mustbe_float32 (Sexpr_t c)
- LL_float32 cdr_mustbe_float32 (Sexpr_t c)
- Sexpr_t **car_eqv_q** (Sexpr_t c, Sexpr_t obj)
- Sexpr_t **car_eq_q** (Sexpr_t c, Sexpr_t obj)
- Sexpr_t car_equal_q (Sexpr_t c, Sexpr_t obj)
- LL_int32 car_coerce_int32 (Sexpr_t c)
- LL_int32 cdr_coerce_int32 (Sexpr_t c)
- LL_int64 car_coerce_int64 (Sexpr_t c)
- LL_int64 cdr_coerce_int64 (Sexpr_t c)
- LL_float32 car_coerce_float32 (Sexpr_t c)
- LL_float32 cdr_coerce_float32 (Sexpr_t c)
- LL_float64 car_coerce_float64 (Sexpr_t c)
- LL_float64 cdr_coerce_float64 (Sexpr_t c)

The "bang" functions are the "mutators" of GC literature.

- void set_car_bang (Sexpr_t c, Sexpr_t val)
- void set_cdr_bang (Sexpr_t c, Sexpr_t val)
- void vector_set_bang (Sexpr_t vec, LL_int32 k, Sexpr_t val)
- Sexpr_t reverse_bang (Sexpr_t l)

Equivalence tests and sequential search

- Sexpr_t eq_q (Sexpr_t obj1, Sexpr_t obj2)
- Sexpr_t eqv_q (Sexpr_t obj1, Sexpr_t obj2)
- Sexpr_t equal_q (Sexpr_t obj1, Sexpr_t obj2)
- Sexpr_t assq (Sexpr_t obj, Sexpr_t alist)

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Interned symbols

There are just a few operations on symbols:

- *Remember a new symbol.*
- *Check if symbol has already been seen.*
- *Compare 2 symbols for equality.*
- *At evaluation time, lookup the symbol in the current environment.*

For best performance, symbols in LambLisp are interned. This means a single copy of the symbol's characters are stored in a data structure containing the set of all interned symbols. Once interned, symbols can be tested for equality by address rather than character-by-character.

Traditionally, the data structure was called oblist if an association list was used, or called obarray if an array was used. Arrays allow for a hash table implementation, which greatly reduces search time, and can be further optimized if the array size is a power of 2 (2^n).

LambLisp uses the term oblist for the interned symbol table, implemented using a 2^n hash table. Symbol hashes are computed only once and are stored with the symbol, speeding runtime lookups.

- Sexpr_t oblist_query (Sexpr_t oblist, const char *identifier, Bool_t force, Sexpr_t env_exec)
- Sexpr_t oblist_test (Sexpr_t oblist, const char *identifier, Sexpr_t env_exec)
- Sexpr_t oblist_intern (Sexpr_t oblist, const char *identifier, Sexpr_t env_exec)
- Sexpr_t oblist_analyze (Sexpr_t oblist, LL_int32 verbosity, Sexpr_t env_exec)

Hierarchical Dictionary type

LambLisp's high-performance hierarchical dictionary implementation is used internally to represent the runtime environment. Dictionaries are also directly usable in the Lisp applications, and the dictionary data type is the basis for the LambLisp Object System (LOBS).

- Sexpr_t dict_new (LL_int32 framesize, Sexpr_t env_exec)
- Sexpr_t dict_new (Sexpr_t env_exec)
- Sexpr_t dict_add_empty_frame (Sexpr_t dict, LL_int32 framesize, Sexpr_t env_exec)
- Sexpr_t dict_add_empty_frame (Sexpr_t dict, Sexpr_t env_exec)
- Sexpr_t dict_add_keyval_frame (Sexpr_t dict, Sexpr_t keys, Sexpr_t vals, Sexpr_t env_exec)
- Sexpr_t dict_add_keyval_frame_argv (Sexpr_t dict, Sexpr_t keys, Sexpr_t *argv, int nargs, Sexpr_t env_exec)
- Sexpr_t dict_add_vector_frame (Sexpr_t parent, Sexpr_t names, Sexpr_t *vals, int ncaps, Sexpr_t env)
- Sexpr_t dict_add_alist_frame (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)
- Sexpr_t dict_append_bang (Sexpr_t d1, Sexpr_t d2, Sexpr_t env_exec)
- void dict_bind_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
- void dict_bind_local_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
- void dict_rebind_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
- void dict_bind_alist_bang (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)
- void dict_rebind_alist_bang (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)
- Sexpr_t dict_ref_q (Sexpr_t dict, Sexpr_t key)
- Sexpr_t dict_ref (Sexpr_t dict, Sexpr_t key)
- Sexpr_t dict_keys (Sexpr_t dict, Sexpr_t env_exec)
- Sexpr_t dict_values (Sexpr_t dict, Sexpr_t env_exec)
- Sexpr_t dict_to_alist (Sexpr_t dict, Sexpr_t env_exec)
- Sexpr_t dict_to_2list (Sexpr_t dict, Sexpr_t env_exec)
- Sexpr_t alist_to_dict (Sexpr_t alist, Sexpr_t env_exec)
- Sexpr_t twolist_to_dict (Sexpr_t twolist, Sexpr_t env_exec)
- Sexpr_t dict_analyze (Sexpr_t dict, LL_int32 verbosity=0)
- Sexpr_t integrity_check (Sexpr_t env_exec, LL_int32 verbosity=1)

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Reading and writing

- Sexpr_t read (LL_Port &src, Sexpr_t env_exec)
- Sexpr_t write_or_display (Sexpr_t sexpr, Bool_t do_write, WriteMode mode=WriteMode::Cyclic)
- Sexpr_t write_simple (Sexpr_t sexpr)
- String sprintf (Charst_t fmt, Sexpr_t args, Sexpr_t env_exec)
- Sexpr_t printf (Sexpr_t args, LL_Port &outp)
- Sexpr_t printf (Sexpr_t args)

Evaluation and function application

- Sexpr_t eval (Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t eval_list (Sexpr_t args, Sexpr_t env_exec)
- Sexpr_t apply_proc_partial (Sexpr_t proc, Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t map_proc (Sexpr_t proc, Sexpr_t lists, Sexpr_t env_exec)

Querying underlying system features defined by Scheme.

- Sexpr_t **r5_base_environment** ()
- Sexpr_t **r5_interaction_environment** ()
- Sexpr_t **lamb_oblist** ()
- Sexpr_t **current_input_port** ()
- Sexpr_t **current_output_port** ()
- Sexpr_t **current_error_port** ()
- void set_current_input_port (Sexpr_t p)
- void set_current_output_port (Sexpr_t p)
- void **set_current_error_port** (Sexpr_t p)
- bool **reader_fold_case** ()
- void **reader_fold_case_set** (bool v)

Useful list processing used internally by the LambLisp virtual machine.

- Sexpr_t append (Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t append2 (Sexpr_t lis, Sexpr_t obj, Sexpr_t env_exec)
- Sexpr_t list_copy (Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t list_analyze (Sexpr_t sexpr, Sexpr_t env_exec)
- Sexpr_t list_to_vector (Sexpr_t l, Sexpr_t env_exec)
- Sexpr_t vector_to_list (Sexpr_t v, Sexpr_t env_exec)
- Sexpr_t vector_copy (Sexpr_t from, Sexpr_t env_exec)

Sparse vectors

The sparse vector representation is a sorted association list, in which the car of each pair is the vector index and the cdr is the vector value at that index.

- Sexpr_t vector_to_sparsevec (Sexpr_t vec, Sexpr_t skip, Sexpr_t env_exec)
- Sexpr_t sparsevec_to_vector (Sexpr_t alist, Sexpr_t fill, Sexpr_t env_exec)

Makers for error types

These functions will sprintf a message into a buffer, and return an error object containing the message. They differ in that one (mk_error) creates a new error object, and the other (mk_syserror) avoids use of the memory manager by reusing the system error singleton.

- Sexpr_t mk_error (Sexpr_t env_exec, Sexpr_t irritants, const char *fmt,...) CHECKPRINTF_pos4

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

- Sexpr_t mk_error (Sexpr_t env_exec, const char ∗fmt,...) CHECKPRINTF_pos3
- Sexpr_t mk_syserror (const char ∗fmt,...) CHECKPRINTF_pos2
- Sexpr_t tag_error (Sexpr_t err, const char ∗kind)
- Sexpr_t mk_syserror (Sexpr_t irritants, const char ∗fmt,...) CHECKPRINTF_pos3

Makers for simple compact types with embedded values, without external storage or dependencies

These types all have their values embedded in the Cell. Note that other types may also hold immediate values, but are "less simple" because they are C++ subtypes that are variations on the Lisp-declared types.

- Sexpr_t gensym (Sexpr_t env_exec)
- Sexpr_t mk_bool (Bool_t b, Sexpr_t env_exec)
- Sexpr_t mk_character (ll_codepoint_t cp, Sexpr_t env_exec)
- Sexpr_t **mk_integer** (LL_int64 i, Sexpr_t env_exec)
- Sexpr_t mk_int32 (LL_int32 i, Sexpr_t env_exec)
- Sexpr_t mk_int64 (LL_int64 i, Sexpr_t env_exec)
- Sexpr_t mk_float32 (LL_float32 r, Sexpr_t env_exec)
- Sexpr_t mk_float64 (LL_float64 d, Sexpr_t env_exec)
- Sexpr_t mk_complex (LL_float32 r, LL_float32 i, Sexpr_t env_exec)
- void rat_store_i64 (Sexpr_t c, bool is_num, LL_int64 v, Sexpr_t env_exec)
- Sexpr_t mk_rational (LL_int64 num, LL_int64 den, Sexpr_t env_exec)
- Sexpr_t mk_number (Charst_t str, Sexpr_t env_exec)
- Sexpr_t mk_sharp_const (Charst_t name, Sexpr_t env_exec)

Makers for heap storage types

In principle, these types require space from the system heap. At GC time, they may also require specialized marking, and also specialized sweeping.

*As an optimization, LambLisp also implements **immediate** types. When the size required for the object is small, the internal bytes of the Cell can be used to hold the contents, avoiding heap operations. Using immediate types in this way has been common practice for integers and real numbers. LambLisp makes some additional use of immediate types. Immediate types are a LambLisp internal optimization, and they are all subtypes of the main Lisp type.*

The types that employ the immediate optimization are:

- *strings, providing up to 9 content bytes plus trailing 0 (32 bit word) or 22 content bytes (64 bit).*
- *bytevectors a count plus up to 9 or 22 content bytes.*
- *vectors of length 0, 1 or 2.*

*There are also **EXTERNAL** versions of those data types. These data types can be used to access data that should not be garbage collected, such as data contained in the application image, or C++ objects provided by another application.*

A variant of the EXTERNAL types is also provided, so that objects created in C++ can be passed to LambLisp, along with an optional deleter function. In C++, the deleter is best implemented as a one-line C++ lambda function accepting 1 argument (a void∗ pointer to a C++ object). The deleter function casts the void∗ pointer to match the C++ object type, and applies the C++ delete or delete[] operators to free the object space and any dependent resources.

By passing the C++ objects to LambLisp, they can be automatically garbage collected when LambLisp has finished with them.

- Sexpr_t mk_string (Sexpr_t env_exec, const char ∗fmt,...) CHECKPRINTF_pos3
- void ∗ **payload_alloc** (size_t nbytes, const char ∗who, Sexpr_t env_exec)
- Sexpr_t mk_string (LL_int32 k, Charst_t src, Sexpr_t env_exec)

- Sexpr_t **mk_string** (String s, Sexpr_t env_exec)
- Sexpr_t mk_symbol_or_number (Charst_t str, Sexpr_t env_exec)
- Sexpr_t mk_symbol (Charst_t str, Sexpr_t env_exec)
- Sexpr_t mk_bytevector (LL_int32 k, Sexpr_t env_exec)
- Sexpr_t mk_bytevector (LL_int32 k, Bytest_t src, Sexpr_t env_exec)
- Sexpr_t mk_bytevector (LL_int32 k, LL_int32 fill, Sexpr_t env_exec)
- Sexpr_t mk_bytevector_ext (LL_int32 k, Bytest_t ext, Sexpr_t env_exec)
- Sexpr_t mk_intvector (LL_int32 k, Sexpr_t env_exec)
- Sexpr_t mk_realvector (LL_int32 k, Sexpr_t env_exec)
- Sexpr_t **mk_intvector** (LL_int32 k, LL_int32 fill, Sexpr_t env_exec)
- Sexpr_t **mk_realvector** (LL_int32 k, LL_float32 fill, Sexpr_t env_exec)
- Sexpr_t mk_vector (LL_int32 len, Sexpr_t fill, Sexpr_t env_exec)
- Sexpr_t mk_hashtbl (LL_int32 len, Sexpr_t fill, Sexpr_t env_exec)
- Sexpr_t mk_serial_port (Sexpr_t env_exec)
- Sexpr_t mk_input_file_port (Charst_t name, Sexpr_t env_exec)
- Sexpr_t mk_output_file_port (Charst_t name, Sexpr_t env_exec)
- Sexpr_t mk_udp_port (unsigned local_port, Sexpr_t env_exec)
- Sexpr_t mk_tcp_client_port (Charst_t host, unsigned port, Sexpr_t env_exec)
- Sexpr_t mk_tcp_server_port (unsigned port, Sexpr_t env_exec)
- Sexpr_t server_accept_port (LL_Port *server, Sexpr_t env_exec)
- Sexpr_t mk_tls_client_port (Charst_t host, unsigned port, Charst_t ca_cert_pem, Sexpr_t env_exec)
- Sexpr_t mk_tls_server_port (unsigned port, Charst_t cert_pem, Charst_t key_pem, Sexpr_t env_↩ exec)

Makers for interface to native procedures and C++ objects.

- Sexpr_t mk_Mop3_procst_t (Mop3st_t f, Sexpr_t env_exec)
- Sexpr_t mk_Mop3_procst_argv_t (Mop3st_t f, Mop3st_argv_t fa, Sexpr_t env_exec)
- Sexpr_t **mk_Mop3_nprocst_t** (Mop3st_t f, Sexpr_t env_exec)
- Sexpr_t **mk_cppobj** (void *obj, CPPDeleterPtr deleter, Sexpr_t env_exec)

Makers for pair types.

*These **pair** types all have the same structure for construction and garbage collection purposes, but are not Scheme pair types. LambLisp*'s scalable type system presents opportunities for efficiency when implementing common operations on lists. These are executed by the LambLisp virtual machine at C++ speed, rather than at the slower speed of the Lisp evaluation loop.*

- *procedures ... the evaluator must be able to identify these directly.*
- *dictionaries ... often containing cycles when used as environments, so a depth limit is imposed when traversing.*
- *thunks ... used in combination with the trampoline technique to implement tail recursion.*

Note that there is no need for mk_pair(), it is just cons().

- Sexpr_t mk_macro (Sexpr_t nlam, Sexpr_t env_nlam, Sexpr_t env_exec)
- Sexpr_t mk_procedure (Sexpr_t formals, Sexpr_t body, Sexpr_t env_proc, Sexpr_t env_exec)
- Sexpr_t mk_nprocedure (Sexpr_t formals, Sexpr_t body, Sexpr_t env_nproc, Sexpr_t env_exec)
- Sexpr_t mk_thunk_sexpr (Sexpr_t sexpr, Sexpr_t env_thunk, Sexpr_t env_exec)
- Sexpr_t mk_thunk_body (Sexpr_t body, Sexpr_t env_thunk, Sexpr_t env_exec)
- Sexpr_t **mk_dict** (Sexpr_t frame, Sexpr_t base, Sexpr_t env_exec)
- Sexpr_t **mk_dict** (LL_int32 framesize, Sexpr_t env_exec)

Macro support

These macro primitives behave as described in Steele's Common Lisp.

- Sexpr_t macroexpand (Sexpr_t form, Sexpr_t env_exec)
- Sexpr_t macroexpand1 (Sexpr_t form, Sexpr_t env_exec)
- Sexpr_t macroexpand (Sexpr_t proc, Sexpr_t args, Sexpr_t env_exec)
- Sexpr_t macroexpand1 (Sexpr_t proc, Sexpr_t args, Sexpr_t env_exec)

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Native Code Generator (NCG)

- NcgFrame * ncg_frame_top = nullptr
- NcgFrame * ncg_frame_pool_head = nullptr
- int ncg_frame_pool_count = 0
- NcgErrJmp * ncg_err_ctx = nullptr
- void ncg_mark_frames ()
- Sexpr_t ncg_eval (Sexpr_t bc, Sexpr_t args, Sexpr_t env_exec)
- Sexpr_t ncg_eval_argv (Sexpr_t bc, Sexpr_t *argv, int nargs, Sexpr_t env_exec)
- Sexpr_t ncg_compile (Sexpr_t bc, Sexpr_t env_exec, Sexpr_t captured_names=nullptr)
- void ncg_disassemble (Sexpr_t bc, Sexpr_t env_exec)
- void ncg_vector_set_ptr (Sexpr_t *data_ptr, LL_int32 k, Sexpr_t val)
- Sexpr_t ncg_vref_fast (Sexpr_t vec, Sexpr_t idx)
- Sexpr_t ncg_vset_fast (Sexpr_t vec, Sexpr_t idx, Sexpr_t val)
- Sexpr_t ncg_cons_fast (Sexpr_t a, Sexpr_t d)
- Sexpr_t ncg_box_fast (LL_int64 v)

native-recursion guard (catch C-stack overflow before it SIGSEGVs)

- const char * eval_stack_base_ = nullptr
- size_t eval_stack_budget_ = LL_EVAL_STACK_BUDGET
- LL_int32 eval_root_depth_max_ = LL_EVAL_ROOT_DEPTH_MAX
- void eval_recursion_overflow (const char *where)
- __attribute__ ((always_inline)) inline void eval_recursion_check(const char *where)

13.4.1 Detailed Description

JIT exception-escape context; opaque to library users (ll_vm_internal.h).

The Lamb class represents a single Lisp virtual machine. To closely match the Arduino-style of loop-based control, LambLisp provides begin(), loop(), and end().

To initialize the LambLisp VM, begin() should be called once before any other call. Generally LambLisp loop() will be called one time for every main loop(), but it is not necessary. If end() is called, all LambLisp resources are freed; Lamb::setup() can be called again to restart that LambLisp VM.

The LambLisp VM functions are grouped this way:

- Logging functions
- Build and version informational functions.
- Cell constructors, getters & setters
- Base data structures for the interpreter: alist, hash tables, and environments, along with their getters & setters.
- Printers
- Partial evaluation and application
- Port access from *Lisp*
- Vector, list, and dictionary utilities, used internally and also available in *Lisp*.
- GC interface functions to protect critical sections.
- Bindings
- Makers

13.4.2 Member Typedef Documentation

Mop3st_t

```
typedef Sexpr_t(* Lamb::Mop3st_t) (Lamb &lamb, Sexpr_t sexpr, Sexpr_t env_exec)
```

This is a pointer to a C++ native function that interacts directly with the S-expression in the given environment. Every LambLisp native function shares this signature. New external functions that conform to this signature can be used from within LambLisp and will run at full speed.

Mop3st_argv_t

```
typedef Sexpr_t(* Lamb::Mop3st_argv_t) (Lamb &lamb, Sexpr_t *argv, int nargs, Sexpr_t env_exec)
```

direct-argv calling convention; argv[0..nargs-1] are already-evaluated args.

13.4.3 Constructor & Destructor Documentation

~Lamb()

```
Lamb::~Lamb ()
```

Real teardown: finalize every cell, free the blocks and the memory manager.

13.4.4 Member Function Documentation

setup()

```
Sexpr_t Lamb::setup (void )
```

Run once after base platform has started. In particular, *Serial* should be initialized before calling setup().

loop()

```
Sexpr_t Lamb::loop (void )
```

Run often to maintain control. The main responsibility of C++ Lamb::loop() is to call the LambLisp function of the same name (loop).

end()

```
void Lamb::end (void )
```

Reset roots and force a full collection. Does NOT release anything.

log()

```
void Lamb::log (const char * fmt,  ...)
```

C-level printf feature with a limit on the length of strings produced. Takes care of log prompt.

printf() [1/3]

```
void Lamb::printf (const char * fmt,  ...)
```

C-level printf feature with a limit on the length of strings produced.

debug() [1/2]

```
bool Lamb::debug (void )
```

Return the state of the Lamb internal debug flag.

debug() [2/2]

```
void Lamb::debug (bool onoff)
```

Set the state of the Lamb debug flag.

build_isDebug()

```
bool Lamb::build_isDebug ()
```

Return true if this build is not checked in. This is unrelated to the runtime `debug` flag.

build_version()

```
unsigned long Lamb::build_version ()
```

Return the build version as a long integer (implemented as a UTC time stamp).

build_UTC()

```
unsigned long Lamb::build_UTC ()
```

Return the UTC time of this build.

build_pushUTC()

```
unsigned long Lamb::build_pushUTC ()
```

Return the UTC time this repo was last pushed.

build_buildRelease()

```
const char * Lamb::build_buildRelease ()
```

Return a pointer to a character array containing the release description.

build_buildDate()

```
const char * Lamb::build_buildDate ()
```

Return a pointer to a character array containing the build date.

build_pushDate()

```
const char * Lamb::build_pushDate ()
```

Return a pointer to a character array containing the date this repo was last pushed.

expand()

```
void Lamb::expand ()
```

Add another block of cells to the population.

tcons() [1/2]

```
Sexpr_t Lamb::tcons (LL_int32 typ, Word_t a, Word_t b, Sexpr_t env_exec)
```

Generic cell constructor with no GC protection for its arguments.

tcons() [2/2]

```
Sexpr_t Lamb::tcons (LL_int32 typ, Sexpr_t a, Sexpr_t b, Sexpr_t env_exec)
```

Constructor for any pair type; protects both S-expression arguments.

cons()

```
Sexpr_t Lamb::cons (Sexpr_t a, Sexpr_t b, Sexpr_t env_exec)  [inline]
```

Cell constructor for normal pairs.

materialized()

```
Sexpr_t Lamb::materialized (Sexpr_t box)  [inline]
```

Hand back an accessor's fresh box, ROOTED first – unconditional; see the definition.

gc_root_depth_max()

```
LL_int32 Lamb::gc_root_depth_max ()  [inline]
```

Peak rootstack depth since boot or the last reset. rootstack.depth() is instantaneous and cannot tell a loop that pushes per allocation from a deep call chain; the peak can, and an overflow otherwise leaves no record of how fast the depth got there.

gc_root_depth_max_reset()

```
void Lamb::gc_root_depth_max_reset ()  [inline]
```

Re-baseline the peak to the CURRENT depth, so one workload can be measured in isolation. Not a reset to 0: a peak below the live depth could not be true.

gc_root_setdepth()

```
void Lamb::gc_root_setdepth (LL_int32 n)  [inline]
```

Restore rootstack to saved depth, discarding entries above n (call in catch to drop orphaned roots without disturbing the caller's entries).

gc_idle_task_us()

```
void Lamb::gc_idle_task_us (LL_int32 us, Sexpr_t env_exec)
```

Run GC for up to us microseconds of spare time; bank extra quanta as credit to skip during later allocations.

gc_collect()

```
void Lamb::gc_collect (Sexpr_t env_exec)
```

Run one complete GC cycle (mark + sweep); leaves heap in idle phase with all unreachable cells freed.

gc_Qm()

```
LL_int32 Lamb::gc_Qm () const
```

return current mark quantum size.

gc_pass_n()

```
void Lamb::gc_pass_n (LL_int32 nsteps)
```

run nsteps mark steps; no-op if not in marking phase.

mem_startmark_reset()

```
void Lamb::mem_startmark_reset ()
```

zero the startmark/extraroots high-waters ([P298])

mem_expand_to_n_blocks()

```
bool Lamb::mem_expand_to_n_blocks (LL_int32 n, Sexpr_t env_exec)
```

true iff n blocks are now present

mem_stat_t_gc_total_us()

```
unsigned long Lamb::mem_stat_t_gc_total_us ()
```

monotonic total us in gc_pass – measured GC load

mem_stat_nfree()

```
LL_int32 Lamb::mem_stat_nfree ()
```

free cells right now (instantaneous)

mem_stat_budget_achieved_ns()

```
LL_int32 Lamb::mem_stat_budget_achieved_ns ()
```

one quantum's real cost in ns, against the requested gc-budget-ns

mem_stat_gcphase()

```
LL_int32 Lamb::mem_stat_gcphase ()
```

[B995] 0 idle, 1 marking, 2 sweeping – no fourth phase

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mem_stat_payload_collect()

```
LL_int32 Lamb::mem_stat_payload_collect ()
```

[B995] bulk payload allocations that had to collect

ncg_frame_pool_n()

```
LL_int32 Lamb::ncg_frame_pool_n ()  [inline]
```

Frames currently in the free-list pool.

ncg_frame_pool_init()

```
void Lamb::ncg_frame_pool_init (LL_int32 n)
```

Allocate n frames and push them onto the pool free list.

ncg_frame_pool_free()

```
void Lamb::ncg_frame_pool_free ()
```

release them; nothing did, so every instance leaked the pool.

car()

```
Sexpr_t Lamb::car (Sexpr_t l)  [inline]
```

Return the first element (*car*) of a pair cell; throws if `l` is not a pair type.

cdr()

```
Sexpr_t Lamb::cdr (Sexpr_t l)  [inline]
```

Return the second element (*cdr*) of a pair cell; throws if `l` is not a pair type.

prechecked_anypair_get_car()

```
Sexpr_t Lamb::prechecked_anypair_get_car (Sexpr_t c)  [inline]
```

Car of a known pair; handles embedded int32/float32; no type check.

prechecked_anypair_get_cdr()

```
Sexpr_t Lamb::prechecked_anypair_get_cdr (Sexpr_t c)  [inline]
```

Cdr of a known pair; handles embedded int32/float32; no type check.

caar()

```
Sexpr_t Lamb::caar (Sexpr_t l)  [inline]
```

Composition: `car(car(l))`.

cadr()

```
Sexpr_t Lamb::cadr (Sexpr_t l)  [inline]
```

Composition: `car(cdr(l))`; returns the second element of a list.

car_mustbe_int32()

```
LL_int32 Lamb::car_mustbe_int32 (Sexpr_t c)  [inline]
```

`car(c)->mustbe_int32()` with no box.

cdr_mustbe_int32()

```
LL_int32 Lamb::cdr_mustbe_int32 (Sexpr_t c)  [inline]
```

`cdr(c)->mustbe_int32()` with no box.

car_mustbe_float32()

```
LL_float32 Lamb::car_mustbe_float32 (Sexpr_t c)  [inline]
```

`car(c)->mustbe_float32()` with no box.

cdr_mustbe_float32()

```
LL_float32 Lamb::cdr_mustbe_float32 (Sexpr_t c)  [inline]
```

`cdr(c)->mustbe_float32()` with no box.

car_equal_q()

```
Sexpr_t Lamb::car_equal_q (Sexpr_t c, Sexpr_t obj)  [inline]
```

The `equal_q` twin, for `member`/`assoc`'s DEFAULT comparator. Same fast path as the other two, and sound for the same reason: `equal?` on numbers IS `eqv?` (R7RS 6.1), so for an embedded int32 against a `T_INT32` all three predicates agree. A structural argument – a string, a vector, a pair – is a pointer in the slot, takes the fallback, and recurses in `equal_q` exactly as before. The ONLY thing removed is the box.

car_coerce_int32()

```
LL_int32 Lamb::car_coerce_int32 (Sexpr_t c)  [inline]
```

`car(c)->coerce_int32()` with no box; truncates an embedded float as the original does.

cdr_coerce_int32()

```
LL_int32 Lamb::cdr_coerce_int32 (Sexpr_t c)  [inline]
```

`cdr(c)->coerce_int32()` with no box.

car_coerce_int64()

```
LL_int64 Lamb::car_coerce_int64 (Sexpr_t c)  [inline]
```

`car(c)->coerce_int64()` with no box.

cdr_coerce_int64()

```
LL_int64 Lamb::cdr_coerce_int64 (Sexpr_t c)  [inline]
```

`cdr(c)->coerce_int64()` with no box.

car_coerce_float32()

```
LL_float32 Lamb::car_coerce_float32 (Sexpr_t c)  [inline]
```

`car(c)->coerce_float32()` with no box.

cdr_coerce_float32()

```
LL_float32 Lamb::cdr_coerce_float32 (Sexpr_t c)  [inline]
```

`cdr(c)->coerce_float32()` with no box.

car_coerce_float64()

```
LL_float64 Lamb::car_coerce_float64 (Sexpr_t c)  [inline]
```

`car(c)->coerce_float64()` with no box.

cdr_coerce_float64()

```
LL_float64 Lamb::cdr_coerce_float64 (Sexpr_t c)  [inline]
```

`cdr(c)->coerce_float64()` with no box.

cdar()

```
Sexpr_t Lamb::cdar (Sexpr_t l)  [inline]
```

Composition: `cdr(car(l))`.

cddr()

```
Sexpr_t Lamb::cddr (Sexpr_t l)  [inline]
```

Composition: `cdr(cdr(l))`.

caddr()

```
Sexpr_t Lamb::caddr (Sexpr_t l)  [inline]
```

Composition: `car(cdr(cdr(l)))`; returns the third element of a list.

cdddr()

```
Sexpr_t Lamb::cdddr (Sexpr_t l)  [inline]
```

Composition: `cdr(cdr(cdr(l)))`.

cadddr()

```
Sexpr_t Lamb::cadddr (Sexpr_t l)  [inline]
```

Composition: `car(cdr(cdr(cdr(l))))`; returns the fourth element of a list.

cddddr()

```
Sexpr_t Lamb::cddddr (Sexpr_t l)  [inline]
```

Composition: `cdr(cdr(cdr(cdr(l))))`.

set_car_bang()

```
void Lamb::set_car_bang (Sexpr_t c, Sexpr_t val)
```

Replace the car field in the cell with *val*. GC flags will be maintained as required.

set_cdr_bang()

```
void Lamb::set_cdr_bang (Sexpr_t c, Sexpr_t val)
```

Replace the cdr field in the cell with *val*. GC flags will be maintained as required.

vector_set_bang()

```
void Lamb::vector_set_bang (Sexpr_t vec, LL_int32 k, Sexpr_t val)
```

Replace the specified vector element with *val*. GC flags will be maintained as required.

reverse_bang()

```
Sexpr_t Lamb::reverse_bang (Sexpr_t l)
```

Reverse the list in-place and return the new list head (the former list tail).

eq_q()

```
Sexpr_t Lamb::eq_q (Sexpr_t obj1, Sexpr_t obj2)
```

Return true if 2 cells are the same cell, or are atoms with the same value.

eqv_q()

```
Sexpr_t Lamb::eqv_q (Sexpr_t obj1, Sexpr_t obj2)
```

Return true if 1 cells are the same cell, or are atoms with the same value.

equal_q()

```
Sexpr_t Lamb::equal_q (Sexpr_t obj1, Sexpr_t obj2)
```

Returns true when obj1 and obj2 are eqv?, and also all their descendants.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

assq()

```
Sexpr_t Lamb::assq (Sexpr_t obj, Sexpr_t alist)
```

Search an association list for a matching key, and return the `(key . value)` pair, or **false** if not found.

oblist_query()

```
Sexpr_t Lamb::oblist_query (Sexpr_t oblist, const char * identifier, Bool_t force, Sexpr_t env_exec)
```

Look up `identifier` in `oblist`. If `force` is true, intern the symbol (creating it if absent) and return it. If `force` is false, return the existing symbol or `HASHF` if not found.

oblist_test()

```
Sexpr_t Lamb::oblist_test (Sexpr_t oblist, const char * identifier, Sexpr_t env_exec)  [inline]
```

Return the interned symbol for `identifier`, or `HASHF` if not present.

oblist_intern()

```
Sexpr_t Lamb::oblist_intern (Sexpr_t oblist, const char * identifier, Sexpr_t env_exec)  [inline]
```

Return the interned symbol for `identifier`, creating it if necessary.

oblist_analyze()

```
Sexpr_t Lamb::oblist_analyze (Sexpr_t oblist, LL_int32 verbosity, Sexpr_t env_exec)
```

Report statistics on the oblist hash table (load factor, chain lengths, etc.) at the given `verbosity` level. Returns NIL.

dict_new() [1/2]

```
Sexpr_t Lamb::dict_new (LL_int32 framesize, Sexpr_t env_exec)  [inline]
```

Return a new empty dictionary with the given top frame size.

dict_new() [2/2]

```
Sexpr_t Lamb::dict_new (Sexpr_t env_exec)  [inline]
```

Return a new empty dictionary with alist top frame.

dict_add_empty_frame() [1/2]

```
Sexpr_t Lamb::dict_add_empty_frame (Sexpr_t dict, LL_int32 framesize, Sexpr_t env_exec)
```

Prepend a new empty frame to `dict` and return the extended dictionary. Frame storage is chosen by `framesize`: < 4 = alist; 4–7 = 2-slot hash table; >= 8 = heap-allocated hash table of `framesize` slots.

dict_add_empty_frame() [2/2]

```
Sexpr_t Lamb::dict_add_empty_frame (Sexpr_t dict, Sexpr_t env_exec)  [inline]
```

Returns a new dictionary with an empty alist top frame and `dict` as parent.

dict_add_keyval_frame()

```
Sexpr_t Lamb::dict_add_keyval_frame (Sexpr_t dict, Sexpr_t keys, Sexpr_t vals, Sexpr_t env_exec)
```

Returns a new dictionary with a new top frame containing the keys bound to the values.

dict_add_keyval_frame_argv()

```
Sexpr_t Lamb::dict_add_keyval_frame_argv (Sexpr_t dict, Sexpr_t keys, Sexpr_t * argv, int nargs, Sexpr_t env_↩
exec)
```

same as dict_add_keyval_frame but values come from argv[] instead of a cons list.

dict_add_vector_frame()

```
Sexpr_t Lamb::dict_add_vector_frame (Sexpr_t parent, Sexpr_t names, Sexpr_t * vals, int ncaps, Sexpr_t env)
```

compact kv-vec frame (T_SVEC_IMM for ncaps==1); alist fallback for ncaps>1.

dict_add_alist_frame()

```
Sexpr_t Lamb::dict_add_alist_frame (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)  [inline]
```

Returns a new dictionary with the alist bindings added in a new frame on top of the base dictionary.

dict_append_bang()

```
Sexpr_t Lamb::dict_append_bang (Sexpr_t d1, Sexpr_t d2, Sexpr_t env_exec)
```

Append d2 as a parent dictionary of d1.

dict_bind_bang()

```
void Lamb::dict_bind_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
```

Modify the target dictionary; `value` is re-assigned to `key` wherever first found in chain, else created in the top frame.

dict_bind_local_bang()

```
void Lamb::dict_bind_local_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
```

Modify only the top frame of dict; update `key` if found there, else create it there. Never walks parent frames. Use for define/define-syntax per R7RS 5.3.2.

dict_rebind_bang()

```
void Lamb::dict_rebind_bang (Sexpr_t dict, Sexpr_t key, Sexpr_t value, Sexpr_t env_exec)
```

Modify the target dictionary; `value` is assigned to `key` wherever first found in env, else error if not found.

dict_bind_alist_bang()

```
void Lamb::dict_bind_alist_bang (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)
```

Modify the target dictionary; binding keys in the alist to their corresponding values.

dict_rebind_alist_bang()

```
void Lamb::dict_rebind_alist_bang (Sexpr_t dict, Sexpr_t alist, Sexpr_t env_exec)
```

Modify the target dictionary; rebinding keys in the alist to their corresponding values.

dict_ref_q()

```
Sexpr_t Lamb::dict_ref_q (Sexpr_t dict, Sexpr_t key)
```

Returns (key value) pair, or **false** if key is unbound.

dict_ref()

```
Sexpr_t Lamb::dict_ref (Sexpr_t dict, Sexpr_t key)
```

Returns the value associated with the key in the given dictionary; throws error if key is unbound.

dict_keys()

```
Sexpr_t Lamb::dict_keys (Sexpr_t dict, Sexpr_t env_exec)
```

Note that keys and values are guaranteed to return in the corresponding order in dict_keys and dict_values, duplicate keys may occur.

Return a list of all the keys in this dictionary. If the dictionary has parents, keys may appear multiple times.

dict_values()

```
Sexpr_t Lamb::dict_values (Sexpr_t dict, Sexpr_t env_exec)
```

Return a list of all the values in this dictionary. If the dictionary has parents, values may appear multiple times for each key.

dict_to_alist()

```
Sexpr_t Lamb::dict_to_alist (Sexpr_t dict, Sexpr_t env_exec)
```

Convert the dictionary to an alist, retaining only the top-level `(key . value)` pairs.

dict_to_2list()

```
Sexpr_t Lamb::dict_to_2list (Sexpr_t dict, Sexpr_t env_exec)
```

Convert the dictionary to a list of 2-element lists, retaining only the top-level `(key value)` sublists.

alist_to_dict()

```
Sexpr_t Lamb::alist_to_dict (Sexpr_t alist, Sexpr_t env_exec)
```

Convert an alist into a dictionary. The resulting dictionary has no parent.

twolist_to_dict()

```
Sexpr_t Lamb::twolist_to_dict (Sexpr_t twolist, Sexpr_t env_exec)
```

Convert a list of 2-element lists into a dictionary. The resulting dictionary has no parent.

dict_analyze()

```
Sexpr_t Lamb::dict_analyze (Sexpr_t dict, LL_int32 verbosity = 0)
```

Internal integrity check.

integrity_check()

```
Sexpr_t Lamb::integrity_check (Sexpr_t env_exec, LL_int32 verbosity = 1)
```

GC heap integrity check; callable from mop3.

read()

```
Sexpr_t Lamb::read (LL_Port & src, Sexpr_t env_exec)  [inline]
```

Read one S-expression from `src`; returns `OBJ_EOF` at end of input.

write_or_display()

```
Sexpr_t Lamb::write_or_display (Sexpr_t sexpr, Bool_t do_write, WriteMode mode = WriteMode::Cyclic)
```

Write `sexpr` to the current output port. `do_write` true => *write* semantics (strings quoted, chars as `#\\x`); false => *display* (strings/chars as-is). `mode` selects datum-label emission: Cyclic labels nodes on a cycle (default, terminates on cycles), Shared labels all shared nodes, Simple emits no labels. Returns `OBJ_VOID`.

write_simple()

```
Sexpr_t Lamb::write_simple (Sexpr_t sexpr)
```

Write `sexpr` using *write* semantics without following shared structure. Returns `OBJ_VOID`.

sprintf()

```
String Lamb::sprintf (Charst_t fmt, Sexpr_t args, Sexpr_t env_exec)
```

Format a string from a template and a list of S-expression arguments. Format specifiers: ~a / ~A inserts the next argument using *display* formatting; ~n or ~% inserts a newline; ~~ inserts a literal ~. Backslash escapes \\n (newline) and \\\\ (literal backslash) are also recognized. Throws if the argument count does not match the format string.

printf() [2/3]

```
Sexpr_t Lamb::printf (Sexpr_t args, LL_Port & outp)
```

Format and write to `outp`; `args` is `(fmt-string val ...)`. See `sprintf()` for format specifier syntax. Returns `OBJ_VOID`.

printf() [3/3]

```
Sexpr_t Lamb::printf (Sexpr_t args)
```

Format and write to the current output port; `args` is `(fmt-string val ...)`. See `sprintf()` for format specifier syntax. Returns `OBJ_VOID`.

eval()

```
Sexpr_t Lamb::eval (Sexpr_t sexpr, Sexpr_t env_exec)
```

Evaluate the S-expression in the environment provided.

eval_list()

```
Sexpr_t Lamb::eval_list (Sexpr_t args, Sexpr_t env_exec)
```

Evaluate the list of S-expressions in the environment provided, and return a lkist of results.

apply_proc_partial()

```
Sexpr_t Lamb::apply_proc_partial (Sexpr_t proc, Sexpr_t sexpr, Sexpr_t env_exec)
```

Evaluate the function arguments and then apply the function to them. The result may be a tail requiring further evaluation.

map_proc()

```
Sexpr_t Lamb::map_proc (Sexpr_t proc, Sexpr_t lists, Sexpr_t env_exec)
```

Apply the procecure to the lists per R5RS.

set_current_input_port()

```
void Lamb::set_current_input_port (Sexpr_t p)  [inline]
```

for with-input-from-file

set_current_output_port()

```
void Lamb::set_current_output_port (Sexpr_t p)  [inline]
```

for with-output-to-file

load()

```
Sexpr_t Lamb::load (Charst_t name, Sexpr_t env_exec, LL_int32 verbosity = 0)
```

Load and evaluate S-expressions from the named file.

append()

```
Sexpr_t Lamb::append (Sexpr_t sexpr, Sexpr_t env_exec)
```

Concatenate a list of lists per R5RS `append`; the last element may be a non-list, producing an improper list. Returns the concatenated result.

append2()

```
Sexpr_t Lamb::append2 (Sexpr_t lis, Sexpr_t obj, Sexpr_t env_exec)
```

Destructively attach *obj* to the end of *lis*, which must be a proper list.

list_copy()

```
Sexpr_t Lamb::list_copy (Sexpr_t sexpr, Sexpr_t env_exec)
```

Return a shallow copy of the list: new pair cells with the same elements in the same order.

list_analyze()

```
Sexpr_t Lamb::list_analyze (Sexpr_t sexpr, Sexpr_t env_exec)
```

Traverse `sexpr` using Floyd's cycle-detection algorithm and return a structural description:

- proper list of length N → integer N
- improper list (non-NIL terminator) after N pairs → `(improper . N)`
- circular structure → `#f`

list_to_vector()

```
Sexpr_t Lamb::list_to_vector (Sexpr_t l, Sexpr_t env_exec)
```

Convert a proper list to a heap-allocated S-expression vector (`T_SVEC_HEAP`); list order is preserved.

vector_to_list()

```
Sexpr_t Lamb::vector_to_list (Sexpr_t v, Sexpr_t env_exec)
```

Convert an S-expression vector to a proper list; element order is preserved.

vector_copy()

```
Sexpr_t Lamb::vector_copy (Sexpr_t from, Sexpr_t env_exec)
```

Return a shallow copy of the S-expression vector `from`.

vector_to_sparsevec()

```
Sexpr_t Lamb::vector_to_sparsevec (Sexpr_t vec, Sexpr_t skip, Sexpr_t env_exec)
```

Convert `vec` to a sorted `(index . value)` alist, omitting elements equal to `skip`.

sparsevec_to_vector()

```
Sexpr_t Lamb::sparsevec_to_vector (Sexpr_t alist, Sexpr_t fill, Sexpr_t env_exec)
```

Reconstruct a dense vector from a sparse `(index . value)` alist; positions not present in the alist are initialized to `fill`.

mk_error() [1/2]

```
Sexpr_t Lamb::mk_error (Sexpr_t env_exec, Sexpr_t irritants, const char * fmt, ...)
```

Allocate a NEW T_ERROR cell with these irritants. Unlike Cell::mk_error(), which reuses one shared object, several of these can be live at once. This is the one to raise from a mop3.

mk_error() [2/2]

```
Sexpr_t Lamb::mk_error (Sexpr_t env_exec, const char * fmt, ...)
```

Allocate a NEW T_ERROR cell. See the irritants overload above.

mk_syserror() [1/2]

```
Sexpr_t Lamb::mk_syserror (const char * fmt, ...)
```

Fill and return the SHARED system error object (OBJ_SYSERROR). Allocates nothing; only one exists at a time.

tag_error()

```
Sexpr_t Lamb::tag_error (Sexpr_t err, const char * kind)
```

Tag a fresh T_ERROR with a kind marker in its irritants (e.g. "%file-error"/"%read-error") so file-error?/read-error? can classify it. Returns err.

mk_syserror() [2/2]

```
Sexpr_t Lamb::mk_syserror (Sexpr_t irritants, const char * fmt,  ...)
```

As above, with irritants. Still the shared object.

gensym()

```
Sexpr_t Lamb::gensym (Sexpr_t env_exec)
```

Produce a new unique symbol.

mk_bool()

```
Sexpr_t Lamb::mk_bool (Bool_t b, Sexpr_t env_exec)  [inline]
```

Return HASHT or HASHF.

mk_character()

```
Sexpr_t Lamb::mk_character (ll_codepoint_t cp, Sexpr_t env_exec)  [inline]
```

Allocate a new T_CHAR cell holding Unicode codepoint cp.

mk_int32()

```
Sexpr_t Lamb::mk_int32 (LL_int32 i, Sexpr_t env_exec)  [inline]
```

<T_INT32 cell; returns cached cell for small values.

mk_int64()

```
Sexpr_t Lamb::mk_int64 (LL_int64 i, Sexpr_t env_exec)  [inline]
```

T_INT64 cell (always allocates).

mk_float32()

```
Sexpr_t Lamb::mk_float32 (LL_float32 r, Sexpr_t env_exec)  [inline]
```

T_FLOAT32 cell.

mk_float64()

```
Sexpr_t Lamb::mk_float64 (LL_float64 d, Sexpr_t env_exec)  [inline]
```

T_FLOAT64 cell.

mk_complex()

```
Sexpr_t Lamb::mk_complex (LL_float32 r, LL_float32 i, Sexpr_t env_exec)  [inline]
```

T_COMPLEX cell.

rat_store_i64()

```
void Lamb::rat_store_i64 (Sexpr_t c, bool is_num, LL_int64 v, Sexpr_t env_exec)  [inline]
```

Store an int64 exact value into a rational slot: embedded int32 if it fits, else a boxed T_INT64.

< boxed (issued -> survives; stored immediately)

mk_rational()

```
Sexpr_t Lamb::mk_rational (LL_int64 num, LL_int64 den, Sexpr_t env_exec)  [inline]
```

Canonical rational from int64 components: GCD-reduces, normalises sign, returns an integer when den==1, else a T_RATIONAL whose components box to T_INT64 when they overflow int32 (– no float64 fallback, so exactness is preserved). < establish T_RATIONAL + an embedded int32 flags

mk_number()

```
Sexpr_t Lamb::mk_number (Charst_t str, Sexpr_t env_exec)
```

Parse `str` as a number; returns `T_INT32`/`T_INT64`, `T_FLOAT32`/`T_FLOAT64`, `T_RATIONAL`, or `T_COMPLEX` as appropriate, or `OBJ_UNDEF` if `str` is not a valid numeric literal.

mk_sharp_const()

```
Sexpr_t Lamb::mk_sharp_const (Charst_t name, Sexpr_t env_exec)
```

Parse a #-prefixed constant name (e.g., `"t"`, `"f"`, `"true"`, `"false"`) and return the corresponding singleton; returns `OBJ_UNDEF` if unrecognized.

mk_string() [1/2]

```
Sexpr_t Lamb::mk_string (Sexpr_t env_exec, const char * fmt,  ...)
```

Build a string with PRINTF FORMATTING – `fmt` is a format, not content.

mk_string() [2/2]

```
Sexpr_t Lamb::mk_string (LL_int32 k, Charst_t src, Sexpr_t env_exec)
```

THE ONE TO USE FOR CONTENT: copies exactly `k` bytes from `src`, so embedded zero bytes survive and nothing in `src` is interpreted. See the printf overload's warning above.

mk_symbol_or_number()

```
Sexpr_t Lamb::mk_symbol_or_number (Charst_t str, Sexpr_t env_exec)
```

Try to parse `str` as a number; if that fails, intern it as a symbol. Used by the reader for unquoted tokens.

mk_symbol()

```
Sexpr_t Lamb::mk_symbol (Charst_t str, Sexpr_t env_exec)  [inline]
```

Intern `str` in the oblist and return the symbol S-expression. Two calls with identical strings always return the same cell pointer.

mk_bytevector() [1/3]

```
Sexpr_t Lamb::mk_bytevector (LL_int32 k, Sexpr_t env_exec)
```

Simplest heap allocation with no initialization.

mk_bytevector() [2/3]

```
Sexpr_t Lamb::mk_bytevector (LL_int32 k, Bytest_t src, Sexpr_t env_exec)
```

Heap allocation with initialization.

mk_bytevector() [3/3]

```
Sexpr_t Lamb::mk_bytevector (LL_int32 k, LL_int32 fill, Sexpr_t env_exec)
```

Heap allocation with initialization.

mk_bytevector_ext()

```
Sexpr_t Lamb::mk_bytevector_ext (LL_int32 k, Bytest_t ext, Sexpr_t env_exec)  [inline]
```

Injection of externally allocated memory, which will not be freed at GC time.

mk_intvector()

```
Sexpr_t Lamb::mk_intvector (LL_int32 k, Sexpr_t env_exec)
```

Note that integer vectors and real vectors are just bytevectors underneath.

Allocates a bytevector from the heap to ensure proper alignment (no IMM type).

mk_realvector()

```
Sexpr_t Lamb::mk_realvector (LL_int32 k, Sexpr_t env_exec)
```

Allocates a bytevector from the heap to ensure proper alignment (no IMM type).

mk_vector()

```
Sexpr_t Lamb::mk_vector (LL_int32 len, Sexpr_t fill, Sexpr_t env_exec)
```

Allocate a `T_SVEC_HEAP` vector of `len` elements, all initialized to `fill`.

mk_hashtbl()

```
Sexpr_t Lamb::mk_hashtbl (LL_int32 len, Sexpr_t fill, Sexpr_t env_exec)
```

Allocate a `T_SVEC2N_HEAP` hash-table vector; actual size is rounded up to the next power of 2, minimum `len` slots, all initialized to `fill`.

mk_serial_port()

```
Sexpr_t Lamb::mk_serial_port (Sexpr_t env_exec)
```

Wrap the platform serial port (Arduino `Serial` or POSIX stdin/stdout) as a LambLisp port object.

mk_input_file_port()

```
Sexpr_t Lamb::mk_input_file_port (Charst_t name, Sexpr_t env_exec)
```

Open the named filesystem file for reading; return a port. Throws if the file cannot be opened.

mk_output_file_port()

```
Sexpr_t Lamb::mk_output_file_port (Charst_t name, Sexpr_t env_exec)
```

Open the named filesystem file for writing; return a port. Throws if the file cannot be opened.

mk_udp_port()

```
Sexpr_t Lamb::mk_udp_port (unsigned local_port, Sexpr_t env_exec)
```

open a UDP datagram socket.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

mk_tcp_client_port()

```
Sexpr_t Lamb::mk_tcp_client_port (Charst_t host, unsigned port, Sexpr_t env_exec)
```

Open a TCP connection to `host:port`; return a bidirectional port. Scheme: `(open-tcp-client-port host port)`. Returns #f on connection failure (it does NOT throw, and does not report at all).

mk_tcp_server_port()

```
Sexpr_t Lamb::mk_tcp_server_port (unsigned port, Sexpr_t env_exec)
```

Create a TCP listener on `port`; return a server port. Scheme: `(open-tcp-server-port port)`. Use `server_accept_port()` to accept incoming connections.

server_accept_port()

```
Sexpr_t Lamb::server_accept_port (LL_Port * server, Sexpr_t env_exec)
```

Accept one pending connection from `server`; returns a bidirectional client port, or `HASHF` (`#f`) if no connection is waiting (non-blocking). Scheme: `(server-accept server-port)`.

mk_tls_client_port()

```
Sexpr_t Lamb::mk_tls_client_port (Charst_t host, unsigned port, Charst_t ca_cert_pem, Sexpr_t env_exec)
```

Open a TLS client connection to `host:port`. Pass a PEM CA certificate string in `ca_cert_pem` to verify the server; pass `nullptr` to skip verification (insecure, development only). Scheme: `(open-tls-client-port host port)` or `(open-tls-client-port host port ca-cert-pem)`.

mk_tls_server_port()

```
Sexpr_t Lamb::mk_tls_server_port (unsigned port, Charst_t cert_pem, Charst_t key_pem, Sexpr_t env_exec)
```

Create a TLS server on `port` using PEM-encoded certificate `cert_pem` and private key `key_pem`. Scheme: `(open-tls-server-port port cert-pem key-pem)`.

mk_input_string_port()

```
Sexpr_t Lamb::mk_input_string_port (Charst_t inp, Sexpr_t env_exec)
```

Wrap the C string `inp` as an input port; reading consumes characters left to right.

mk_output_string_port()

```
Sexpr_t Lamb::mk_output_string_port (Sexpr_t env_exec)
```

Create an output port that accumulates written characters; retrieve the result with Scheme `get-output-string`.

mk_input_bytevector_port()

```
Sexpr_t Lamb::mk_input_bytevector_port (ByteVec_t src, LL_int32 n, Sexpr_t env_exec)
```

Wrap `n` bytes of `src` as a binary input port.

mk_output_bytevector_port()

```
Sexpr_t Lamb::mk_output_bytevector_port (Sexpr_t env_exec)
```

Create a binary output port that accumulates written bytes; retrieve the result with Scheme `get-output-bytevector`.

mk_Mop3_procst_t()

```
Sexpr_t Lamb::mk_Mop3_procst_t (Mop3st_t f, Sexpr_t env_exec)  [inline]
```

car=0 (no argv variant)

mk_Mop3_procst_argv_t()

```
Sexpr_t Lamb::mk_Mop3_procst_argv_t (Mop3st_t f, Mop3st_argv_t fa, Sexpr_t env_exec)  [inline]
```

car=argv fn, *cdr*=standard fn

mk_macro()

```
Sexpr_t Lamb::mk_macro (Sexpr_t nlam, Sexpr_t env_nlam, Sexpr_t env_exec)  [inline]
```

Allocate a `T_MACRO` cell: `nlam` is the nlambda transformer, `env_nlam` is its definition environment. At expansion time the transformer receives the unevaluated argument list; its result is evaluated in the call-site environment.

mk_procedure()

```
Sexpr_t Lamb::mk_procedure (Sexpr_t formals, Sexpr_t body, Sexpr_t env_proc, Sexpr_t env_exec)  [inline]
```

Allocate a T_PROC closure: formals is the parameter list, body is the list of body expressions, env_proc is the captured lexical environment.

mk_nprocedure()

```
Sexpr_t Lamb::mk_nprocedure (Sexpr_t formals, Sexpr_t body, Sexpr_t env_nproc, Sexpr_t env_exec)  [inline]
```

As mk_procedure() but returns a T_NPROC cell; arguments are passed unevaluated at call time.

mk_thunk_sexpr()

```
Sexpr_t Lamb::mk_thunk_sexpr (Sexpr_t sexpr, Sexpr_t env_thunk, Sexpr_t env_exec)  [inline]
```

Allocate a T_THUNK_SEXPR trampoline cell: sexpr will be evaluated in env_thunk on the next trampoline iteration.

mk_thunk_body()

```
Sexpr_t Lamb::mk_thunk_body (Sexpr_t body, Sexpr_t env_thunk, Sexpr_t env_exec)  [inline]
```

Allocate a T_THUNK_BODY trampoline cell: body is a list of expressions evaluated in sequence in env_thunk; the last expression's value is the result.

macroexpand() [1/2]

```
Sexpr_t Lamb::macroexpand (Sexpr_t form, Sexpr_t env_exec)
```

Fully expand all macros in form, applying transformers repeatedly until no further expansion is possible. Returns the fully expanded form.

macroexpand1() [1/2]

```
Sexpr_t Lamb::macroexpand1 (Sexpr_t form, Sexpr_t env_exec)
```

Apply at most one macro expansion step to form. Returns the expansion if form is a macro call, otherwise returns form unchanged.

macroexpand() [2/2]

```
Sexpr_t Lamb::macroexpand (Sexpr_t proc, Sexpr_t args, Sexpr_t env_exec)
```

Fully expand the macro `proc` applied to `args`.

macroexpand1() [2/2]

```
Sexpr_t Lamb::macroexpand1 (Sexpr_t proc, Sexpr_t args, Sexpr_t env_exec)
```

Apply one expansion step of macro `proc` to `args`.

ncg_mark_frames()

```
void Lamb::ncg_mark_frames ()
```

Push all NCG live-frame values onto GC markstack; called from start_marking().

ncg_eval()

```
Sexpr_t Lamb::ncg_eval (Sexpr_t bc, Sexpr_t args, Sexpr_t env_exec)
```

Execute a T_BYTECODE cell that has been NCG-compiled.

ncg_eval_argv()

```
Sexpr_t Lamb::ncg_eval_argv (Sexpr_t bc, Sexpr_t * argv, int nargs, Sexpr_t env_exec)
```

Phase 6: direct NCG→NCG call; takes argv[] instead of list, skips apply_proc_partial.

ncg_compile()

```
Sexpr_t Lamb::ncg_compile (Sexpr_t bc, Sexpr_t env_exec, Sexpr_t captured_names = nullptr)
```

Compile a T_BYTECODE cell to native code, and record it in the procedure's own native-code slot. (The slot table is internal: it is declared in ll_vm_ncg.h, which is not part of the customer package, so this comment describes the effect rather than naming a constant the reader cannot see.).

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

ncg_disassemble()

```
void Lamb::ncg_disassemble (Sexpr_t bc, Sexpr_t env_exec)
```

Log basic info about an NCG-compiled proc.

ncg_vector_set_ptr()

```
void Lamb::ncg_vector_set_ptr (Sexpr_t * data_ptr, LL_int32 k, Sexpr_t val)
```

Write-barrier store for inlined NCG_STORE_LOCAL (Phase 1).

ncg_vref_fast()

```
Sexpr_t Lamb::ncg_vref_fast (Sexpr_t vec, Sexpr_t idx)
```

fast vector-ref (T_SVEC + T_INT32 fast path).

ncg_vset_fast()

```
Sexpr_t Lamb::ncg_vset_fast (Sexpr_t vec, Sexpr_t idx, Sexpr_t val)
```

fast vector-set (write-barrier inline).

ncg_cons_fast()

```
Sexpr_t Lamb::ncg_cons_fast (Sexpr_t a, Sexpr_t d)
```

fast cons; returns nullptr if Nfree<=yuasa_M or sweeping.

ncg_box_fast()

```
Sexpr_t Lamb::ncg_box_fast (LL_int64 v)
```

fast integer box; nullptr if Nfree<=yuasa_M or sweeping.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

eval_recursion_overflow()

```
void Lamb::eval_recursion_overflow (const char * where)
```

cold path: throws the catchable "recursion too deep" error

__attribute__()

```
Lamb::__attribute__ ((always_inline) ) const  [inline]
```

< hot path: cheap C-stack + root-stack depth check at a recursion entry

diag_memory_report()

```
void Lamb::diag_memory_report ()
```

NOT `#if LL_ESP32`: the report is platform-independent now; a host answers "HEAP".

dump GC-buffer / cell-block placement (INTERNAL vs PSRAM) + sizes.

13.4.5 Member Data Documentation

ncg_frame_top

```
NcgFrame* Lamb::ncg_frame_top = nullptr
```

linked list of live frames; head threaded through frame->prev

ncg_frame_pool_head

```
NcgFrame* Lamb::ncg_frame_pool_head = nullptr
```

free-list head; recycled frames linked via frame->prev

ncg_frame_pool_count

```
int Lamb::ncg_frame_pool_count = 0
```

frames currently available in the pool

ncg_err_ctx

```
NcgErrJmp* Lamb::ncg_err_ctx = nullptr
```

innermost escape context; set by ncg_eval before JIT call

eval_stack_base_

```
const char* Lamb::eval_stack_base_ = nullptr
```

highest stack address seen at a recursion entry (== outermost frame)

eval_stack_budget_

```
size_t Lamb::eval_stack_budget_ = LL_EVAL_STACK_BUDGET
```

trip threshold: bytes of C-stack growth below base; tunable at runtime

eval_root_depth_max_

```
LL_int32 Lamb::eval_root_depth_max_ = LL_EVAL_ROOT_DEPTH_MAX
```

trip threshold: rootstack depth (cap is 65536); tunable at runtime

13.5 LambStdioClass Class Reference

Public Member Functions

- int **setTxBufferSize** (int n)
- int **setRxBufferSize** (int n)
- bool set_input_callback (void(*cb)())
- void **begin** (void)
- void **begin** (unsigned long baudrate)
- void **end** ()
- int **available** (void)
- int **availableForWrite** (void)
- int **read** (void)
- int **write** (uint8_t c)
- int **write** (char c)
- void **flush** (void)
- int **read** (byte *buf, int max)
- int **read** (char *s, int max)
- int **write** (const char *s, int n)
- int **write** (const byte *b, size_t n)
- int **write** (const char *s)
- **operator bool** ()

13.5.1 Detailed Description

A wrapper around the underlying stdin/stdout implementation. On an embedded system, this class will use the primary serial in/out (`Serial` on Arduino-compatibles). On Linux, this class will set the terminal to byte-at-a-time mode (called "non-CANONICAL" mode).

13.5.2 Member Function Documentation

set_input_callback()

```
bool LambStdioClass::set_input_callback (void(* cb )())
```

register a callback the TRANSPORT invokes when input arrives, so the console can be drained by something other than the reader. This belongs on the SHIM, not at the call site: LL_Term talks to LambStdioClass and nothing else, and `onReceive` is a HardwareSerial method that the shim does not inherit – calling it through LambStdio compiles nowhere. Returns true if the transport supports it. POSIX returns false and the caller keeps its existing reader-driven path, which is correct there: stdin is already buffered by libc.

13.6 LL_File Class Reference

Public Member Functions

- bool **isOpen** ()
- int **read** (void)
- int **write** (byte b)
- int **seek** (unsigned long target, int whence=SEEK_SET)
- int **tell** ()
- int **size** ()
- int **close** ()
- int **read** (byte *b, int n)
- int **read** (char *s, int n)
- int **write** (const byte *b, int n)
- int **write** (const char *s, int n)
- int **peek** ()

Public Attributes

- File_Native **_theFile**
- String **_path**
- String **_mode**

13.6.1 Detailed Description

The *file* type is ultimately provided by the underlying operating system, not by LambLisp. This class elides the differences between file types on different platforms, providing a POSIX-like interface.

Note that there is no *open* operation on files. A file is opened by the *file system* and then a *file* is returned. After a *file* is closed, the same file object cannot be opened again; instead a new file must be requested from the *file system*.

13.7 LL_File_System Class Reference

Public Member Functions

- LL_File ∗ **open** (const char ∗path, const char ∗mode)
- bool **exists** (const char ∗path)
- int **rm** (const char ∗path)
- int **mv** (const char ∗from, const char ∗to)
- int **mkdir** (const char ∗path)

13.7.1 Detailed Description

The "file system" type elides the differences between different underlying platforms. For example, it will deal with the leading '/' required by LittleFS. This minimal file system interface is platform-independent.

13.8 LL_Panel Struct Reference

Public Attributes

- const char ∗ name
- int **w**
- int h
- LL_PanelModel model
- bool(∗ init)(Lamb &lamb)
- void(∗ fill)(Lamb &lamb, int x, int y, int w, int h, uint16_t color)
- void(∗ blit)(Lamb &lamb, int x, int y, int w, int h, const uint16_t ∗src)
- uint16_t ∗(∗ fb)(void)
- void(∗ set_brightness)(Lamb &lamb, int percent)
- void(∗ on_off)(Lamb &lamb, bool on)

13.8.1 Detailed Description

A board's panel: geometry, model, and the five operations. See the note above on NULL.

13.8.2 Member Data Documentation

name

```
const char* LL_Panel::name
```

"st7789-240x240-spi3", "co5300-410x502-qspi", "rgb565-480x480-par16"

h

```
int LL_Panel::h
```

panel geometry in pixels

model

```
LL_PanelModel LL_Panel::model
```

PUSH or SCAN – branch on this, not on the name.

init

```
bool(* LL_Panel::init) (Lamb &lamb)
```

bring up bus + controller; idempotent

fill

```
void(* LL_Panel::fill) (Lamb &lamb, int x, int y, int w, int h, uint16_t color)
```

filled rect, RGB565

blit

```
void(* LL_Panel::blit) (Lamb &lamb, int x, int y, int w, int h, const uint16_t *src)
```

PUSH only; NULL on SCAN.

fb

```
uint16_t *(* LL_Panel::fb) (void)
```

SCAN only; NULL on PUSH.

set_brightness

```
void(* LL_Panel::set_brightness) (Lamb &lamb, int percent)
```

0..100, or NULL if the panel has none

on_off

```
void(* LL_Panel::on_off) (Lamb &lamb, bool on)
```

display on/off, or NULL

13.9 LL_String Class Reference

Public Member Functions

- **LL_String** (const char *other)
- **LL_String** (const char *other, int n)
- **LL_String** (const LL_String &other)
- void **set** (const char *s)
- void set (const char *s, int n)
- LL_String **concat** (const char *s1, const char *s2)
- LL_String & append (const char *s1)
- LL_String & append (char c)
- LL_String **substring** (int start, int len_)
- LL_String & **operator=** (const char *other)
- LL_String & **operator=** (const LL_String &other)
- bool **operator==** (const char *other)
- bool **operator!=** (const char *other)
- bool **operator==** (const LL_String &other)
- bool **operator!=** (const LL_String &other)
- char & **operator[ ]** (int ix)
- LL_String **operator+** (const char *other)
- LL_String **operator+** (const LL_String &other)
- LL_String **operator+** (char c)
- LL_String & **operator+=** (const char *other)
- LL_String & **operator+=** (const LL_String &other)
- LL_String & **operator+=** (char c)
- char * **c_str** () const
- int length () const

13.9.1 Detailed Description

A small owning string: stored length and capacity, geometric growth, in-place append. += is amortised O(1) and `length()` is O(1) rather than a `strlen` – which matters because `ll_vm_term.cpp` appends ONE CHARACTER PER RECEIVED BYTE, and the previous reallocate-and-copy made that quadratic – ∼100 B/s on a 240 MHz part, under 1% of the link; 20,000 appends went 201 ms -> 0.13 ms.

THREE THINGS A CALLER CAN STILL GET WRONG:

1. `set`, `append` and `operator=` are alias-safe – a source inside our own buffer, or `a = a` – and they are safe by explicit test, not by accident. Keep them that way.
2. `operator[]` returns `char&`, so writing `'\0'` through it would desynchronise the stored length from the contents. No such subscript assignment exists in `src/` today; if you add one, call `set()` instead.
3. DO NOT CACHE `c_str()` ACROSS A MUTATION. It now survives any append that does not grow, so the misuse is INTERMITTENT rather than constant – strictly fewer dangling reads than before, and strictly harder to notice.

13.9.2 Member Function Documentation

set()

```
void LL_String::set (const char * s, int n)  [inline]
```

< aliases us: no growth can be needed

append() [1/2]

```
LL_String & LL_String::append (const char * s1)  [inline]
```

< may move ptr; preserves [0,len]

append() [2/2]

```
LL_String & LL_String::append (char c)  [inline]
```

Appending ONE character is the hot path (the terminal's per-char input accumulation), so it does not route through the 2-byte-cstr + strlen detour the operator used to take.

length()

```
int LL_String::length () const  [inline]
```

O(1): a stored length, not a strlen per call.

13.10 LLArduinoBackend Struct Reference

Public Attributes

- const char * **name**
- void(* **pin_mode**)(int, int)
- void(* **digital_write**)(int, int)
- int(* **digital_read**)(int)
- void(* **analog_write**)(int, int)
- int(* **analog_read**)(int)
- void(* **tone_on**)(int, unsigned int)
- void(* **tone_off**)(int)
- int(* **pin_mode_get**)(int)
- int(* **pin_duty_get**)(int)
- int(* **pin_tone_get**)(int)
- int(* **npins**)(void)

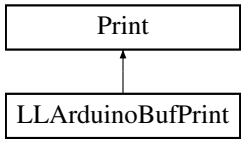
13.10.1 Detailed Description

Until now the null backend WAS the implementation: pinMode() and friends were defined directly here, so adding a libgpiod backend would have been two definitions of the same symbol. The public Arduino names now dispatch through a table and each backend fills one in.

SELECTION IS AT RUNTIME, NOT COMPILE TIME, and deliberately: the same aarch64 binary runs on a Jetson with a 40-pin header and on a headless x86 box with gpiochips that are NOT header pins. Which backend is correct is a property of the MACHINE, not of the build, so it is decided by probing the machine – and it falls back to null rather than driving something at random.

13.11 LLArduinoBufPrint Class Reference

Inheritance diagram for LLArduinoBufPrint:



Public Member Functions

- size_t write (uint8_t c) override
- const char * **str** () const
- virtual size_t write (const uint8_t *buf, size_t n)
- size_t **write** (const char *s)

Public Member Functions inherited from Print

- size_t **write** (const char *s)
- size_t **print** (char c)
- size_t **print** (const char *s)
- size_t **print** (const String &s)
- size_t **print** (int n, int base=10)
- size_t **print** (long n, int base=10)
- size_t **print** (unsigned long n, int base=10)
- size_t print (double v, int digits=2)
- size_t println ()
- size_t **println** (char c)
- size_t **println** (const char *s)
- size_t **println** (const String &s)
- size_t **println** (int n, int base=10)
- size_t **println** (long n, int base=10)
- size_t **println** (unsigned long n, int base=10)
- size_t **println** (double v, int digits=2)
- size_t printf (const char *fmt,...) CHECKPRINTF_pos2

13.11.1 Detailed Description

Criterion 3 is "the same suite passes against the vendor core AND against LLArduino". These render through whichever Print is compiled in, which is what makes that comparison possible – Arduino's Print and LLArduino's both declare `virtual size_t write(uint8_t)` and the same print(long,base) / print(double,digits) overloads, so ONE implementation serves both.

THEY WERE LL_ARDUINO-ONLY AND THAT WAS THE BUG. Built that way, the ESP32 run returned ↩ #f from every accessor and the suite reported 10 FAILURES that came from the stub rather than from the code under test – on the ONE backend where comparing against Arduino is the entire point. A measuring instrument that cannot be pointed at the reference measures nothing.

Print's Arduino-specific behaviours – CRLF from println, no 0x prefix from print(n, HEX), two decimal places from print(double) – were documented in the header and then not exercised by anything, because Print is a C++ class with no Scheme binding. A documented guarantee that nothing checks is just a comment; this file has already watched that pattern cost days elsewhere in the tree. These three accessors render THROUGH Print into a buffer, so a Scheme test asserts what a device would actually emit rather than what the comment says.

13.11.2 Member Function Documentation

write() [1/2]

```
size_t LLArduinoBufPrint::write (uint8_t c)  [inline], [override], [virtual]
```

< truncate, never overrun

Implements Print.

write() [2/2]

```
size_t Print::write (const uint8_t * buf, size_t n)  [virtual]
```

Reimplemented from Print.

13.12 LLArduinoSerial Class Reference

Public Member Functions

- void **begin** (unsigned long baud=115200)
- void **end** ()
- int available () override
- int **availableForWrite** ()
- int **read** () override
- int peek () override
- void **flush** () override
- size_t write (uint8_t c) override
- virtual size_t write (const uint8_t *buf, size_t n)
- size_t **write** (const char *s)

13.12.1 Detailed Description

Serial over LambStdio. peek() is implemented HERE because LambStdioClass has no peek: one byte of pushback, which is all Stream promises.

13.12.2 Member Function Documentation

write()

```
size_t Print::write (const uint8_t * buf, size_t n)  [virtual]
```

Reimplemented from Print.

13.13 LLGpiodLine Struct Reference

Public Attributes

- struct gpiod_line * **line**
- int **mode**

13.13.1 Detailed Description

One requested line per pin, cached. libgpiod requires a line be REQUESTED before use and the request carries its direction, so changing pinMode means releasing and re-requesting.

13.14 LLGpiodPin Struct Reference

Public Attributes

- int **bcm**
- int **board**
- int **offset**
- const char * **chip**
- const char * **name**

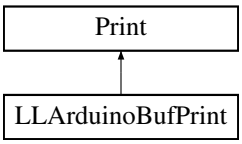
13.14.1 Detailed Description

PIN NUMBERING IS THE ACTUAL WORK, not the libgpiod calls. "Raspberry-Pi compatible" cannot mean passing a number straight to gpiod: the Jetson's lines are named PA.00, PQ.05, PAC.06 – Tegra SoC naming – not GPIO17, and not header positions. The Orin Nano's 40-pin header IS physically RPi-compatible, so the mapping exists and simply has to be written down.

THIS TABLE IS GENERATED, NOT TRANSCRIBED. It comes from NVIDIA's own jetson-gpio data (/usr/↩ lib/python3/dist-packages/Jetson/GPIO/gpio_pin_data.py, JETSON_ORIN_NANO), read on the machine and emitted as C. Hand-copying 22 rows of pin numbers is exactly the kind of transcription that produces a table which looks right and drives the wrong pin – and driving the wrong pin on a board with motors attached is not a typo you get to take back. Regenerate rather than edit: python3 -c "import importlib.↩ util; spec=importlib.util.spec_from_file_location('g', \ '/usr/lib/python3/dist-packages/Jetson/GPIO/gpio_pin↩ _data.py'); m=importlib.util.module_from_spec(spec); \ spec.loader.exec_module(m); [print(r) for r in m.↩ jetson_gpio_data[m.JETSON_ORIN_NANO][0]]" (Note: importing Jetson.GPIO itself FAILS on this unit -- "Could not determine Jetson model" on the Orin Nano Engineering Reference Developer Kit Super – so load the data module directly.)

13.15 Print Class Reference

Inheritance diagram for Print:



Public Member Functions

- virtual size_t write (uint8_t c)=0
- virtual size_t **write** (const uint8_t ∗buf, size_t n)
- size_t **write** (const char ∗s)
- size_t **print** (char c)
- size_t **print** (const char ∗s)
- size_t **print** (const String &s)
- size_t **print** (int n, int base=10)
- size_t **print** (long n, int base=10)
- size_t **print** (unsigned long n, int base=10)
- size_t print (double v, int digits=2)
- size_t println ()
- size_t **println** (char c)
- size_t **println** (const char ∗s)
- size_t **println** (const String &s)
- size_t **println** (int n, int base=10)
- size_t **println** (long n, int base=10)
- size_t **println** (unsigned long n, int base=10)
- size_t **println** (double v, int digits=2)
- size_t printf (const char ∗fmt,...) CHECKPRINTF_pos2

13.15.1 Detailed Description

MOVED HERE, and it must stay below this point: Print uses String (typedef at `typedef LL_String String` above) and CHECKPRINTF_pos2 (defined just above). Declared any earlier in this header and the compiler reports "String does not name a type" – which is a header-ordering error, not a missing class.

GUARDED, AND IT MUST BE. On ESP32 the VENDOR Arduino core defines its own Print and Stream (cores/esp32/Print.h, Stream.h) via Arduino.h, so declaring these unconditionally is a "redefinition of class Print" build failure on every ESP32 target. It compiled clean on the hosts because there is no Arduino core there to collide with – which is exactly why the mistake was invisible until an ESP32 build ran. When this block was moved down here to sit below the String typedef it left the LL_ARDUINO guard that ends further up; it now carries its own. LLArduino PROVIDES these where the vendor core is absent, and defers to it where present.

LL_ARDUINO Print / Stream / Serial. NOT STUBS, and they need not be: Print and Stream are pure character I/O, and LambStdio ALREADY implements exactly that on POSIX (stdin/stdout in non-canonical mode). So this is an Arduino-SHAPED FACADE over working I/O, in the same class as millis() – genuinely implemented, not simulated. It is what lets sketch-style code (`Serial.println("hi")`) run on a host with no board, and on WebAssembly and Windows, where Espressif's core cannot go at all.

FIDELITY IS THE RISK HERE, not capability. Arduino's Print has behaviours worth matching DELIBERATELY rather than by accident, and each is a place a host test could pass while the device disagrees: println() terminates CRLF ("\r\n") not "\n"; write() returns a BYTE COUNT; print(double, digits) defaults to 2 decimal places; print(int, base) supports BIN/OCT/DEC/HEX. Those are exactly what success criterion 3 exists to check – the same suite run against the vendor core and against LLArduino must agree.

13.15.2 Member Function Documentation

write()

```
virtual size_t Print::write (uint8_t c)  [pure virtual]
```

Implemented in LLArduinoBufPrint.

Chapter 14

Acknowledgements

I reviewed many *Lisp* and *Scheme* implementations, but the one I studied in detail was *TinyScheme 1.42*, written in 5000+ lines of high-quality circa-1992 C code. It was too big for the processor I was working on at the time (in 2020). Ultimately I returned to the embedded Lisp problem, but made different design choices than the TinyScheme authors, especially on the compile-vs-interpret spectrum and garbage collection techniques. The study was hugely informative nonetheless.

Chez Scheme is the ultimate large implementation. It is the work of a lifetime, primarily of one man, and a standard for performance and completeness in the Scheme world. During the development of *LambLisp*, *Chez Scheme* was used to provide benchmark behavior in cases where the spec is not clear or offers a choice of outcomes.

I have provided an extensive bibliography that also informed my efforts. If the reader would like to better understand Lisp design, begin your journey there. No doubt you will discover a different destination.

Some images in this document are used under Creative Commons license:

- Lambda symbol image: Harke, CC BY-SA 3.0, via Wikimedia Commons. https://commons.wikimedia.org/w/index.php?curid=127589172
- Greek letter image: user:rogerb (VectorVoyager), own work, CC BY-SA 3.0. https://creativecommons.org/licenses/by-sa/3.0

Bill White 2026

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 15

Glossary

abstract machine

The terms *abstract machine* and *virtual machine* are often used interchangeably, but there is a subtle difference. An *abstract machine* is a mathematics exercise; it is a set of elements, relations among the elements, and transformations between them. A *virtual machine* is an implementation of an abstract machine. When a VM runs, it behaves as defined by the abstract machine, although it runs on top of some other lower-level machine. That lower-level machine may itself be a software or hardware implementation of another abstract machine.

association list, or alist

A list of pairs. The first part of each pair is treated as a key, and the second part is treated as a value. The operations *set* and *get* are provided, to install new pairs or to retrieve existing pairs. Operations to *set* might either replace an existing pair, or add it if not already existing, or simply add the new pair to the front of the list, shadowing any matching pairs later in the list. Alists require linear search and so are used where the lists are expected to be short, as they are in hash tables.

AST, or Abstract Syntax Tree

A data structure created as output of analyzing source code. The AST is an internal representation of the source code elements and their relationships. The AST can be used to re-create the original source code it was created from (not counting white space and comments). Creating the AST is the "front half" of a compiler.

atom

A data item that is not a *pair*. Atoms are the leaf nodes of the S-expression tree; they have no *car* or *cdr*. Most atoms are simple scalar values: integers, real numbers, booleans, characters, and strings. Compound atoms such as vectors and bytevectors carry internal structure but expose it only through their own dedicated operators (e.g., `vector-ref`), not through *car* and *cdr*.

In *LambLisp*, the tag word of a Cell encodes the atom type: `T_INT32`, `T_FLOAT32`, `T_BOOL`, `T_SYM_HEAP`, `T_GENSYM`, and so on. The distinction between atom and pair is the termination condition of every list traversal: a list walk continues while the current cell is a pair, and stops when it reaches an atom (typically NIL).

bytecode

A compact, machine-independent instruction set produced by a compiler and executed by a virtual machine. Rather than interpreting the source AST node by node, a bytecode compiler makes a single pass to emit a sequence of encoded instructions stored as a vector; the bytecode executor then runs those instructions in a tight loop with lower dispatch overhead than tree-walking interpretation.

LambLisp includes a bytecode compiler and executor (`T_BYTECODE`). Bytecode compilation is performed at load time; the resulting instruction vector lives in the LambLisp heap and is subject to garbage collection like any other object. Benchmarks show a 1.7–3.3x speedup over the pure-interpreter baseline for compute-bound code.

car∗ and ∗cdr

Historically, *car* stands for *Contents of the Address Register* and *cdr* for *Contents of the Decrement Register* – fields in the memory word of the IBM 704 computer on which Lisp was first implemented in 1958.

Refers to the first and second data word in a Cell, respectively. In *LambLisp*, a Cell consists of 3 sequential memory *words* (1 tag + 2 data), where a single *word* is large enough to hold a pointer or an integer, and 2 consecutive *words* are large enough to hold a real number.

In the 3-word Cell, the first word is the *tag* word, the second word is the *car*, and the third word is the *cdr*.

The *tag* word contains a Cell type indicator, and flags representing the garbage collection state of each cell.

Integers are stored in the *car* word. Real numbers are stored in the *car* word, and may possibly extend into the *cdr* word, depending on the underlying microprocessor. Some complex atoms, such as vectors, store a count in the *car* and a pointer in the *cdr*. Symbols store a hash in the *car*, and a pointer to the symbol identifier in the *cdr*.

Cell

A triplet of information: tag, first word (aka *car*), second word (aka *cdr*). In LambLisp, the three elements of the triplet are consecutive in memory, and are all the same size, large enough to hold a pointer or integer. The tag indicates how the first and second words are to be interpreted. When the tag indicates a Cell is a *pair*, the *car* and *cdr* both point to other Cells (or possibly the same Cell).

A *list* is formed when the *car* of a pair Cell points to an S-expression (which is an item in the list), and the *cdr* of that Cell points to another *pair* Cell. That *pair* Cell may also have a list item in its *car*. By convention, lists terminate with a special S-expression called *the empty list*, denoted '(), or NIL, or nil. NIL is considered a list (*the empty list*), but is not a pair, so it has no *car* or *cdr*.

Cells that are not pairs are called *atoms*. Atoms do not have any directly accessible internal structure, but may have operations that work on them, such as vector subscripting. Many atoms are simple integers, strings, or floating-point numbers, but there are compound atoms such as vectors and bytevectors.

In LambLisp, there are several additional *pair* types that facilitate high performance in conjunction with the underlying LambLisp virtual machine. In standard Lisp or Scheme code, these would be considered atoms, but in LambLisp they are a bit of a hybrid, because *car*, *cdr*, *append*, and a few other list operations will work as expected.

closure

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

A procedure combined with the *lexical environment* in which it was created. When a `lambda` expression is evaluated, the result is a closure: the procedure body paired with a captured reference to the enclosing environment frame. This captured environment gives the closure access to variables from its defining scope even when called later in a different context.

Closures are the mechanism behind higher-order functions, callbacks, and object-based programming patterns common in Scheme. In *LambLisp*, a closure is represented as a `T_PROC` Cell whose *cdr* points to the captured environment chain.

compiler, interpreter

Both are language translators; in addition, an interpreter is also the language executor. There is a spectrum between compilers and interpreters, but we can identify a key difference:

Translator type	Description
compiler	a software process that reads source code and produces new code to be executed later, possibly by a different machine.
interpreter	a software process that reads source code, performs some amount of processing, and executes the result immediately on the same machine.

In a compiler, and in any nontrivial interpreter, the software process first produces an Abstract Syntax Tree (AST). Creating the AST is the first half of the compilation process; in the second half, code is generated for later execution. An interpreter will create the AST and then traverse it, performing the operations as they are encountered.

In *Lisp*, code compiling is commonly done through the use of a *macro* facility. Macro expansion allows code to be transformed as it is read in, factoring out invariants, and producing new code with the invariants removed or calculated once. For example, replacing a symbol with its value "freezes" the symbol; subsequent reads of the symbol will be replaced with the frozen value whenever they are encountered. When that replacement is done at read time, the value is located and replaces the symbol in the executable program. The symbol-value lookup operation is therefore no longer required at run time. In this way symbols become replaced by their read-time values, effectively compiling a block of code into the execution stream.

This method of compilation, called *incremental compilation* or *just-in-time compilation*, blurs the line between compiler and interpreter.

dictionary

A *dictionary* is a data structure containing `(key value)` pairs. Pairs in the dictionary can be retrieved by matching a target `key` with the keys of each pair. Dictionaries also allow creating, updating, or deleting `(key value)` pairs.

A simple dictionary might be a list of `(key value)` pairs, requiring linear search. This is appropriate for small dictionaries, or short-lived dictionaries, where the construction overhead or runtime overhead of a faster solution outweighs any potential speed improvement.

For larger or long-lived dictionaries, intelligent key creation and caching when combined with hash tables makes this a very fast data structure upon which to operate.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

What's described above is a *flat dictionary*, having a single layer of `(key value)` pairs.

environments & hierarchical dictionaries

Often called the "symbol table" in other programming languages, in LambLisp the environment is a *hierarchical dictionary*. The *flat dictionary* described above provides a single *dictionary frame* for a *hierarchical dictionary*. A *hierarchical dictionary* is simply a list of flat dictionaries. The frame at the *car* of the list is a *child frame*, while frames in the *cdr* are called *parent frames* or *ancestor frames*.

The *hierarchical dictionary* provides a high-performing foundation for the LambLisp execution environment. It also provides a simple and natural foundation for the LambLisp Object System. Inheritance (including multiple inheritance) and shadowing are inherent behaviors of the *hierarchical dictionary*.

evaluation

Lisp operates by reading symbolic expressions, evaluating them, and returning the result. The *eval/apply* cycle is the core of a Lisp interpreter:

Step	Action
eval(atom)	Numbers and strings evaluate to themselves; symbols look up their binding in the current environment.
eval(list)	Evaluate the *car* to obtain a procedure; evaluate each element of the *cdr* to obtain arguments; *apply* the procedure to the arguments.
apply(proc, args)	Bind the formal parameters to the argument values in a new environment frame; evaluate the procedure body in that frame.

Special forms – `if`, `define`, `lambda`, `quote`, and others – short-circuit normal eval/apply. Their subexpressions are not pre-evaluated; the special form decides which expressions to evaluate, when, and how many times.

In *LambLisp*, tail calls are handled by returning a *thunk* to the trampoline rather than calling the next procedure directly, keeping the C stack at constant depth.

hash function

A function that takes a series of bytes as input, and produces a fixed-length number called a *hash*. When given a series of inputs, the hash function should give a series of outputs passing relevant tests for randomness. It is not necessary that the series be truly random, only that the results be widely dispersed. For any given input, the output *hash* must always be the same. It can then be used as an index, replacing a search with an index lookup.

hash table

A data structure used for efficiently storing `(key value)` pairs. Commonly implemented as a set of operations employing a vector and a *hash function*. To search for an item by its key:

```
- Obtain the hash of the key H, which may have been computed once and stored.
- Obtain the size of the hash table N.
- Compute the hash table index, which is (H % N), or (H & (N-1)) if N is power of 2.
- Search the (very short) alist at that index of the hash table.
```

lambda, nlambda, macro

Copyright 2026 Frobenius Norm LLC. All specifications subject to change.

Form	Arguments at call	Return value
`lambda`	evaluated before body runs	body result returned to caller
`nlambda`	passed unevaluated (as source)	body result returned to caller
`macro`	passed unevaluated (as source)	body result is evaluated; that result returned to caller

`nlambda` is a LambLisp extension inherited from *InterLisp*; it is the foundation used to implement all *Scheme* syntax forms. `syntax-rules` is available as the hygienic alternative for portable macro libraries. See the *Advanced topics* section for a full treatment including examples and hygiene discussion.

lambda calculus

A branch of mathematics developed during the 1930's, where the elements of interest are functions rather than data values.

LISP

A symbolic programming language developed about 1960. LISP (always uppercase in those days) offered a platform for reasoning, while then-existing languages focused more on numeric applications. The objects of interest in *LISP* are called *symbolic expressions* or *S-expressions*.

list

A **list** is data structure consisting of a backbone of Cells linked by their *cdrs*. The last Cell in the list points to a special NIL Cell to mark the end of the list. The *cars* of each Cell in the list backbone point to the item that is in the list at that position. Note that every item in the list requires 2 entities: the item itself, and a *pair* Cell that connects the item to the list backbone.

In *Lisp* source code, lists are represented between parenthesis, as in `(this is a list)`.

NIL

All *Lisps* have some version of NIL, nil, '() – otherwise called "the empty list". This is used to mark the end of a list. In some *Lisps*, it may also be considered to be equivalent to "false", or equivalent to zero, but not in *Scheme* or *LambLisp*.

In LambLisp, NIL is implemented as a pointer to a special statically-allocated Cell. The space for the NIL Cell is a singleton, outside the blocks of Cells managed by the *LambLisp* garbage collector. This allows NIL to be constant at compile time, speeding up *LambLisp*.

There are a few other singleton Cells implemented this way, e.g., for undefined and void values.

nprocedure

The value type returned by `nlambda`. An *nprocedure* has the same internal structure as a lambda procedure – a parameter list, a body, and a captured lexical environment – but its arguments are passed unevaluated (as source S-expressions) rather than being evaluated before the call. See **nlambda** and the *Advanced topics* section for usage and examples.

oblist

Lisp is a symbolic computing language and *symbol* is a first-class type. For best performance, LambLisp collects all the symbols encountered into an *interned symbol table*, for traditional reasons called the *obarray* or *oblist*. Symbol hashes are calculated on the symbol's identifier and then stored with the symbol in *oblist*. By this means symbols can be compared with a pointer comparison operation; 2 symbols are the same if they point to the same entry in the oblist. In LambLisp, the oblist is neither an array nor a list; instead it is a hash table, but the *oblist* name is retained.

pair

A Cell whose purpose is to connect 2 other cells. In a typical *Lisp* tree structure, *pair* Cells will make up half of the Cells used, while the leaves of the tree will be atoms.

prefix, infix, postfix

In typical algebra notation, to compose two functions *f* and *g* that each require a single parameter *x* we write:

```
f(g(x))
```

The functions are composed using *prefix* notation. Algebra and C/C++ use a mix of parenthesis, prefix, infix, and postfix notation like this:

```
--x + abs(x) + 2 + (x++ - y[z])
```

Lambda calculus uses prefix notation and makes frequent use of the greek symbol lambda (λ), which is not conducive to keyboard use. Instead *Lisp* spells out *lambda* when required.

Lisp (like lambda calculus) uses prefix notation, with parenthesis and symbols being the fundamental lexical elements. To compose functions *f* and *g*, we write:

```
(f (g x))
```

and to compare 2 elements *x* and *y*, there is the eq? operator:

```
(eq? x y)
```

To summarize prefix/infix/postfix, using C++ examples:

Notation		
Prefix	++x	f(x)
Infix	a * b	a + b & c
Postfix	y++	y->foo

REPL

Read-Eval-Print Loop. The interactive command cycle of a Lisp system: *read* an S-expression from the user, *evaluate* it, *print* the result, and *loop* back to read again. *LambLisp* presents the prompt `LL=>` over the serial connection.

The REPL makes Lisp development highly interactive: expressions can be tested immediately, definitions added or redefined without restarting the system, and the live environment inspected at any time. On embedded targets this is particularly valuable, since it allows in-circuit development without a reflash cycle.

Scheme

A dialect of *Lisp* standardized in the R5RS (1998) and R7RS-small (2013) specifications. Scheme is distinguished from other Lisps by its minimalist design: a small, orthogonal core with well-defined semantics, lexical scoping, proper tail recursion, and first-class continuations (`call/cc`).

LambLisp implements the R5RS core with substantial R7RS-small coverage. The primary deliberate departure from standard Scheme is that `call/cc` is escape-only: full re-entrant continuations require capturing the C stack, which is incompatible with the C-stack trampoline used to achieve real-time performance.

symbolic expression

In its simplest implementations, a *Lisp* **symbol** is any sequence of characters not containing spaces or parenthesis. Some of these symbols sequences are defined by the language (`if`, `lambda`, and so on), along with a value for those symbols, but others are just symbols, to which a value may be attached. These are all symbolic expressions, or **S-expressions**, and combinations of S-expressions and balanced sets of parenthesis are also valid S-expressions.

The basic *Lisp* language can only test for equality of symbols, but real-world *Lisps* define useful data types like numbers and strings, along with operations on those.

The LambLisp implementation depends on a `Cell` structure to contain the details of an **atomic** S-expression (such as an integer), or the links between S-expressions (as in a list). When programming LambLisp at the C++ level, an *S-expression* is therefore a pointer to a Cell.

tail call, tail recursion

A *tail call* is a procedure call that is the last action performed by the calling procedure – the caller has no further use for the return value beyond passing it to its own caller. Because the caller's stack frame is no longer needed, it may be discarded before the call is made, keeping stack depth constant. *Tail recursion* is the specific case where a procedure calls itself in tail position.

The R5RS and R7RS specifications *require* proper tail calls. *LambLisp* implements them via a *trampoline*: a tail-position call packages itself as a *thunk* and returns it to the trampoline loop rather than calling directly, so the C stack never grows beyond a fixed depth regardless of recursion depth.

trampoline

A control-flow technique for implementing proper *tail calls* without growing the underlying C stack. Instead of calling a tail-position procedure directly, *LambLisp* packages the call as a *thunk* (a zero-argument closure) and returns it to an outer loop. That loop iteratively invokes thunks until it receives a non-thunk result, which it then returns.

Because each iteration begins with a fresh C stack frame, stack depth remains constant regardless of how deeply tail-recursive the Scheme code is. The trampoline is also the mechanism that makes *LambLisp*'s real-time scheduling possible: the outer loop can perform GC work and service hardware timers between thunk invocations.

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 16

List of Figures

1 Eponymous Lamb . . . 1

1.1 How LambLisp.exe finds its library on Windows . . . 9
1.2 What is inside the runtime image . . . 11
1.3 LambLisp REPL Cycle . . . 14
1.4 ESP32-S3 Hardware Context . . . 18
1.5 Why USB passthrough depends on the host OS . . . 19

4.1 Dictionary: Hash Table Layout (djb2) . . . 31
4.2 Tail Recursion: Stack Growth vs. Constant Stack . . . 32
4.3 Trampoline: O(1) Tail Calls . . . 33

5.1 Environment Chain and Closure Capture . . . 38
5.2 Performance Characteristics of Several Implementation Strategies . . . 41
5.3 Runtime RAM by Implementation Strategy . . . 42
5.4 Bytecode Compilation Pipeline . . . 43
5.5 A CPU shown as a stack of Virtual Machines . . . 44
5.6 Typical Control Application Organization . . . 45
5.7 LambLisp Block Diagram . . . 46
5.8 LambLisp VM Cell Structure . . . 48
5.9 Typical List Structure . . . 49
5.10 LambLisp VM Cell Life Cycle . . . 52
5.11 Tricolor Marking and Yuasa Write Barrier . . . 52
5.12 Incremental GC Timeline . . . 53
5.13 GC quanta inside a real-time control loop . . . 53
5.14 Argument Evaluation: lambda vs. nlambda vs. macro . . . 66
5.15 Adding a C++ Primitive to the Scheme Environment (mop3) . . . 75
5.16 T_CPP_HEAP cell layout and GC lifecycle . . . 76
5.17 Hardware Device Access Patterns . . . 78

6.1 Rooting inside a mop3 call . . . 84

7.1 The native-code pool is a fixed budget . . . 99
7.2 A stream keeps the bytes and loses the boundaries . . . 105
7.3 LLIP Connection Lifecycle . . . 119
7.4 Hot-Reload: Live Update Without Reboot . . . 121
7.5 LambLisp Digital Twin . . . 134
7.6 Unified Distributed Environment . . . 135

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

7.7 Typical Modbus Field Architecture . . . 148
7.8 Non-blocking request/poll flow . . . 150
7.9 Modbus register types . . . 153
7.10 PROFIBUS DP: LambLisp as master or as slave . . . 160
7.11 The state machine the master drives, per slave . . . 162
7.12 PROFINET runs as two processes . . . 167

8.1 Three-Tier Architecture . . . 182
8.2 Boustrophedon Coverage Path . . . 189
8.3 Bee Strategy Path . . . 191

9.1 How a port read decides what to do . . . 212

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Chapter 17

Index

Copyright 2026 Frobenius Norm LLC All specifications subject to change.

Index

__attribute__
  Lamb, 374
__lamb_platform_generic_install_mop3
  Platform (generic), 261
~Lamb
  Lamb, 343
1-Wire & DS18B20, 242
  OneWire_install_mop3, 243
  OneWire_mop3_read_bit, 243
  OneWire_mop3_read_byte, 243
  OneWire_mop3_reset, 243
  OneWire_mop3_search, 243
  OneWire_mop3_write_bit, 243
  OneWire_mop3_write_byte, 243

alist_to_dict
  Lamb, 359
any_bvec_get_elems
  Cell, 330
any_bvec_get_info
  Cell, 331
any_bvec_get_length
  Cell, 330
any_error_get_str
  Cell, 329
any_str_get_charlength
  Cell, 330
any_str_get_chars
  Cell, 329
any_str_get_length
  Cell, 329
any_svec_get_info
  Cell, 330
append
  Lamb, 361
  LL_String, 380
append2
  Lamb, 362
apply_proc_partial
  Lamb, 361
array_matmul_cublas_f32
  CUDA GPU, 284
array_matmul_cublas_f64
  CUDA GPU, 284
as_Bool_t
  Cell, 315
as_Bytest_t
  Cell, 317
as_ByteVec_t
  Cell, 318
as_Char_t
  Cell, 316
as_Charst_t
  Cell, 317
as_CharVec_t
  Cell, 317
as_codepoint
  Cell, 316
as_denominator
  Cell, 323
as_float32
  Cell, 317
as_float64
  Cell, 317
as_int32
  Cell, 316
as_int64
  Cell, 316
as_numerator
  Cell, 322
as_Portst_t
  Cell, 318
as_Ptr_t
  Cell, 317
assq
  Lamb, 354
autobuf_t< _T_ >, 295
  buf, 296, 297
  fill, 296, 297
  maxsize, 297
  size, 296
  use_fastram, 296
available
  LLArduino – LambLisp's own Arduino API, 294

blit
  LL_Panel, 378
buf
  autobuf_t< _T_ >, 296, 297
build_buildDate

Lamb, 345
build_buildRelease
Lamb, 345
build_isDebug
Lamb, 344
build_pushDate
Lamb, 345
build_pushUTC
Lamb, 345
build_UTC
Lamb, 345
build_version
Lamb, 344
bvec_elem_type
Cell, 331
bvec_set_elem_type
Cell, 331
bytecode_is_variadic
Cell, 314
bytecode_set_variadic
Cell, 315

caar
Lamb, 350
cadddr
Lamb, 353
caddr
Lamb, 353
cadr
Lamb, 350
Camera (ESP32-S3-EYE), 235
EYE_CAM_D0, 237
EYE_CAM_EARLY_DMA_RESERVE, 236
EYE_CAM_SIOC, 237
EYE_CAM_SIOD, 236
EYE_CAM_XCLK, 236
EyeCam_install_mop3, 240
LL_CAM_SET_I, 237
mop3_camera_capture, 239
mop3_camera_get, 239
mop3_camera_init, 238
mop3_camera_set, 239
mop3_camera_thumb, 238
mop3_frame_brightness, 240
car
Lamb, 349
car_coerce_float32
Lamb, 352
car_coerce_float64
Lamb, 352
car_coerce_int32
Lamb, 351
car_coerce_int64
Lamb, 351
car_equal_q
Lamb, 351
car_float32
Cell, 313
car_int32
Cell, 313
car_is_embedded
Cell, 313
car_is_float
Cell, 312
car_is_int
Cell, 312
car_mustbe_float32
Lamb, 350
car_mustbe_int32
Lamb, 350
cdar
Lamb, 352
cddddr
Lamb, 353
cdddr
Lamb, 353
cddr
Lamb, 352
cdr
Lamb, 349
cdr_coerce_float32
Lamb, 352
cdr_coerce_float64
Lamb, 352
cdr_coerce_int32
Lamb, 351
cdr_coerce_int64
Lamb, 351
cdr_float32
Cell, 313
cdr_int32
Cell, 313
cdr_is_embedded
Cell, 313
cdr_is_float
Cell, 312
cdr_is_int
Cell, 312
cdr_mustbe_float32
Lamb, 351
cdr_mustbe_int32
Lamb, 350
Cell, 297
any_bvec_get_elems, 330
any_bvec_get_info, 331
any_bvec_get_length, 330
any_error_get_str, 329
any_str_get_charlength, 330

any_str_get_chars, 329
any_str_get_length, 329
any_svec_get_info, 330
as_Bool_t, 315
as_Bytest_t, 317
as_ByteVec_t, 318
as_Char_t, 316
as_Charst_t, 317
as_CharVec_t, 317
as_codepoint, 316
as_denominator, 323
as_float32, 317
as_float64, 317
as_int32, 316
as_int64, 316
as_numerator, 322
as_Portst_t, 318
as_Ptr_t, 317
bvec_elem_type, 331
bvec_set_elem_type, 331
bytecode_is_variadic, 314
bytecode_set_variadic, 315
car_float32, 313
car_int32, 313
car_is_embedded, 313
car_is_float, 312
car_is_int, 312
cdr_float32, 313
cdr_int32, 313
cdr_is_embedded, 313
cdr_is_float, 312
cdr_is_int, 312
cell_name, 331
coerce_cpx_imag, 329
coerce_cpx_real, 328
coerce_float32, 327
coerce_float64, 327
cpx_imag, 321
cpx_real, 320
denominator64, 328
dump, 332
exact_is_big, 322
exact_rank, 327
exact_sign, 323
F_BC_VARIADIC, 333
F_CAR_IS_FLOAT, 333
F_CAR_IS_INT, 333
F_CDR_IS_FLOAT, 333
F_CDR_IS_INT, 333
F_GC01, 332
F_GC02, 332
F_GC04, 332
F_SUBTYPE_MASK, 332
F_TAIL, 332
flags, 309
flags_clr, 310
flags_set, 309
gc_state, 314
GC_STATE_MASK, 333
gcstate_name, 331
get_car, 315
get_car_addr, 315
get_cdr, 315
hash, 318
hash_contents, 318
hash_sexpr, 318
inexact_rank, 327
int_as_i64, 322
is_any_bvec_atom, 311
is_any_int, 311
is_any_pair, 310
is_any_real, 312
is_any_str_atom, 311
is_any_svec2n_atom, 311
is_any_svec_atom, 310
is_any_sym_atom, 311
is_bignum, 311
is_complex, 328
is_exact, 327
is_inexact, 328
is_pair, 310
is_slot_typed, 312
LL_STR_HDR, 334
make_free, 319
mk_error, 324
mustbe_any_str_t, 326
mustbe_Bool_t, 325
mustbe_Char_t, 325
mustbe_codepoint, 326
mustbe_cppobj_t, 326
mustbe_float32, 326
mustbe_int32, 326
Ntypes, 309
num_rank, 328
numerator64, 328
prechecked_cppobj_get_deleter, 326
prechecked_cppobj_get_ptr, 327
prechecked_gensym_get_chars, 325
prechecked_str_ext_get_chars, 325
prechecked_str_heap_get_block, 324
prechecked_str_heap_get_charlength, 324
prechecked_str_heap_get_chars, 324
prechecked_str_heap_set_charlength, 325
prechecked_str_imm_get_chars, 325
rat_car_ptr, 322
rat_cdr_ptr, 322
rat_is_big, 322
rat_set_car_i32, 321

rat_set_car_ptr, 321
rat_set_cdr_i32, 321
rat_set_cdr_ptr, 321
rplaca, 310
rplacd, 310
set, 318–321, 323, 324
str_invalidate_charlength, 329
T_ANY_HEAP_SVEC, 307
T_BIGNUM, 307
T_BOOL, 307
T_BVEC_EXT, 307
T_BVEC_HEAP, 307
T_BVEC_IMM, 307
T_BYTECODE, 309
T_BYTECODE_N, 309
T_CHAR, 307
T_COMPLEX, 308
T_CPP_HEAP, 307
T_DICT, 309
T_ERROR, 309
T_FLOAT32, 308
T_FLOAT64, 308
T_FREE, 308
T_GENSYM, 307
T_INT32, 307
T_INT64, 308
T_MACRO, 308
T_MOP3_NPROC, 308
T_MOP3_PROC, 308
T_NEEDS_FINALIZING, 307
T_NIL, 308
T_NPROC, 308
T_PAIR, 308
T_PORT_HEAP, 307
T_PROC, 308
T_RATIONAL, 308
T_STR_EXT, 307
T_STR_HEAP, 307
T_STR_IMM, 307
T_SVEC2N_HEAP, 307
T_SVEC_HEAP, 307
T_SVEC_IMM, 308
T_SYM_HEAP, 307
T_THUNK_BODY, 309
T_THUNK_SEXPR, 309
T_UNDEF, 308
T_VALUES, 309
T_VOID, 308
tail_state, 314
tail_state_clr, 314
tail_state_set, 314
type, 309
type_name, 331
zero, 309

Cell::CellFeatures, 334
cell_name
  Cell, 331
coerce_cpx_imag
  Cell, 329
coerce_cpx_real
  Cell, 328
coerce_float32
  Cell, 327
coerce_float64
  Cell, 327
CommonIO_install_mop3
  GPIO & Basic I/O, 233
cons
  Lamb, 346
cpx_imag
  Cell, 321
cpx_real
  Cell, 320
CUDA GPU, 283
  array_matmul_cublas_f32, 284
  array_matmul_cublas_f64, 284
  cudaDevicePtrDestroyer, 284
  cudaHostPtrDestroyer, 284
  mop3_cpp_vector_add, 284
  mop3_cudaDeviceSynchronize, 286
  mop3_cudaFree, 285
  mop3_cudaGetDeviceCount, 286
  mop3_cudaGetDeviceProp, 286
  mop3_cudakernel_vectorAdd, 285
  mop3_cudaLaunch_kernel1, 284
  mop3_cudaMalloc, 285
  mop3_cudaMallocHost, 285
  mop3_cudaMemcpy, 285
  mop3_cudaMemset, 285
  mop3_cudaStreamCreate, 286
  mop3_cudaStreamSynchronize, 286
cudaDevicePtrDestroyer
  CUDA GPU, 284
  HIP / ROCm GPU, 287
cudaHostPtrDestroyer
  CUDA GPU, 284
  HIP / ROCm GPU, 287

debug
  Lamb, 344
denominator64
  Cell, 328
diag_memory_report
  Lamb, 374
dict_add_alist_frame
  Lamb, 356
dict_add_empty_frame
  Lamb, 356

dict_add_keyval_frame
Lamb, 356
dict_add_keyval_frame_argv
Lamb, 356
dict_add_vector_frame
Lamb, 356
dict_analyze
Lamb, 359
dict_append_bang
Lamb, 357
dict_bind_alist_bang
Lamb, 357
dict_bind_bang
Lamb, 357
dict_bind_local_bang
Lamb, 357
dict_keys
Lamb, 358
dict_new
Lamb, 355
dict_rebind_alist_bang
Lamb, 357
dict_rebind_bang
Lamb, 357
dict_ref
Lamb, 358
dict_ref_q
Lamb, 358
dict_to_2list
Lamb, 358
dict_to_alist
Lamb, 358
dict_values
Lamb, 358
dump
Cell, 332

end
Lamb, 343
eq_q
Lamb, 354
equal_q
Lamb, 354
eqv_q
Lamb, 354
ESP32 System, 233
mop3_esp32_efuse_mac_get_default, 234
mop3_esp32_efuse_pkg_ver, 234
mop3_esp32_efuse_read_block, 235
mop3_esp32_efuse_write_block, 235
mop3_esp32_flash_probe, 234
mop3_esp32_gcmem, 234
mop3_esp32_heapinfo, 234
eval
Lamb, 360
eval_list
Lamb, 360
eval_recursion_overflow
Lamb, 373
eval_root_depth_max_
Lamb, 375
eval_stack_base_
Lamb, 375
eval_stack_budget_
Lamb, 375
exact_is_big
Cell, 322
exact_rank
Cell, 327
exact_sign
Cell, 323
expand
Lamb, 345
EYE_CAM_D0
Camera (ESP32-S3-EYE), 237
EYE_CAM_EARLY_DMA_RESERVE
Camera (ESP32-S3-EYE), 236
EYE_CAM_SIOC
Camera (ESP32-S3-EYE), 237
EYE_CAM_SIOD
Camera (ESP32-S3-EYE), 236
EYE_CAM_XCLK
Camera (ESP32-S3-EYE), 236
EyeCam_install_mop3
Camera (ESP32-S3-EYE), 240

F_BC_VARIADIC
Cell, 333
F_CAR_IS_FLOAT
Cell, 333
F_CAR_IS_INT
Cell, 333
F_CDR_IS_FLOAT
Cell, 333
F_CDR_IS_INT
Cell, 333
F_GC01
Cell, 332
F_GC02
Cell, 332
F_GC04
Cell, 332
F_SUBTYPE_MASK
Cell, 332
F_TAIL
Cell, 332
fb
LL_Panel, 379

fill
autobuf_t< _T_ >, 296, 297
LL_Panel, 378
flags
Cell, 309
flags_clr
Cell, 310
flags_set
Cell, 309

gc_collect
Lamb, 347
gc_idle_task_us
Lamb, 347
gc_pass_n
Lamb, 347
gc_Qm
Lamb, 347
gc_root_depth_max
Lamb, 346
gc_root_depth_max_reset
Lamb, 346
gc_root_setdepth
Lamb, 347
gc_state
Cell, 314
GC_STATE_MASK
Cell, 333
gcstate_name
Cell, 331
gensym
Lamb, 364
get_car
Cell, 315
get_car_addr
Cell, 315
get_cdr
Cell, 315
GPIO & Basic I/O, 228
CommonIO_install_mop3, 233
mop3_cio_analogRead, 229
mop3_cio_analogReadResolution, 229
mop3_cio_analogReference, 230
mop3_cio_analogWrite, 229
mop3_cio_analogWriteResolution, 230
mop3_cio_attachInterrupt, 232
mop3_cio_bit, 231
mop3_cio_bitClear, 231
mop3_cio_bitRead, 231
mop3_cio_bitSet, 231
mop3_cio_bitWrite, 232
mop3_cio_delay_ms, 229
mop3_cio_delay_us, 229
mop3_cio_detachInterrupt, 232
mop3_cio_digitalPinToInterrupt, 232
mop3_cio_digitalRead, 228
mop3_cio_digitalWrite, 228
mop3_cio_highByte, 232
mop3_cio_interrupts, 233
mop3_cio_lowByte, 232
mop3_cio_noInterrupts, 233
mop3_cio_noTone, 230
mop3_cio_pinMode, 229
mop3_cio_pulseIn, 230
mop3_cio_pulseInLong, 230
mop3_cio_shiftIn, 230
mop3_cio_shiftOut, 231
mop3_cio_tone, 231

h
LL_Panel, 378
Hardware & Platform Builtins (xmop3), 227
hash
Cell, 318
hash_contents
Cell, 318
hash_sexpr
Cell, 318
HIP / ROCm GPU, 287
cudaDevicePtrDestroyer, 287
cudaHostPtrDestroyer, 287
Hip_install_mop3, 290
mop3_cpp_vector_add, 287
mop3_cudaDeviceSynchronize, 289
mop3_cudaFree, 288
mop3_cudaGetDeviceCount, 289
mop3_cudaGetDeviceProp, 289
mop3_cudakernel_vectorAdd, 288
mop3_cudaLaunch_kernel1, 288
mop3_cudaMalloc, 288
mop3_cudaMallocHost, 288
mop3_cudaMemcpy, 288
mop3_cudaMemset, 289
mop3_cudaStreamCreate, 289
mop3_cudaStreamSynchronize, 289
Hip_install_mop3
HIP / ROCm GPU, 290

I2C (Wire), 277
LL_WiFi, 281
Wire_mop3_available, 279
Wire_mop3_begin, 278
Wire_mop3_beginTransmission, 279
Wire_mop3_clearWireTimeoutFlag, 280
Wire_mop3_end, 279
Wire_mop3_endTransmission, 280
Wire_mop3_getWireTimeoutFlag, 280
Wire_mop3_onReceive, 280

Wire_mop3_onRequest, 280
Wire_mop3_read, 279
Wire_mop3_requestFrom, 279
Wire_mop3_setClock, 279
Wire_mop3_setPins, 278
Wire_mop3_setTimeout, 278
Wire_mop3_write, 280
inexact_rank
Cell, 327
init
LL_Panel, 378
int_as_i64
Cell, 322
integrity_check
Lamb, 359
is_any_bvec_atom
Cell, 311
is_any_int
Cell, 311
is_any_pair
Cell, 310
is_any_real
Cell, 312
is_any_str_atom
Cell, 311
is_any_svec2n_atom
Cell, 311
is_any_svec_atom
Cell, 310
is_any_sym_atom
Cell, 311
is_bignum
Cell, 311
is_complex
Cell, 328
is_exact
Cell, 327
is_inexact
Cell, 328
is_pair
Cell, 310
is_slot_typed
Cell, 312

Lamb, 334
__attribute__, 374
∼Lamb, 343
alist_to_dict, 359
append, 361
append2, 362
apply_proc_partial, 361
assq, 354
build_buildDate, 345
build_buildRelease, 345
build_isDebug, 344
build_pushDate, 345
build_pushUTC, 345
build_UTC, 345
build_version, 344
caar, 350
cadddr, 353
caddr, 353
cadr, 350
car, 349
car_coerce_float32, 352
car_coerce_float64, 352
car_coerce_int32, 351
car_coerce_int64, 351
car_equal_q, 351
car_mustbe_float32, 350
car_mustbe_int32, 350
cdar, 352
cddddr, 353
cdddr, 353
cddr, 352
cdr, 349
cdr_coerce_float32, 352
cdr_coerce_float64, 352
cdr_coerce_int32, 351
cdr_coerce_int64, 351
cdr_mustbe_float32, 351
cdr_mustbe_int32, 350
cons, 346
debug, 344
diag_memory_report, 374
dict_add_alist_frame, 356
dict_add_empty_frame, 356
dict_add_keyval_frame, 356
dict_add_keyval_frame_argv, 356
dict_add_vector_frame, 356
dict_analyze, 359
dict_append_bang, 357
dict_bind_alist_bang, 357
dict_bind_bang, 357
dict_bind_local_bang, 357
dict_keys, 358
dict_new, 355
dict_rebind_alist_bang, 357
dict_rebind_bang, 357
dict_ref, 358
dict_ref_q, 358
dict_to_2list, 358
dict_to_alist, 358
dict_values, 358
end, 343
eq_q, 354
equal_q, 354
eqv_q, 354

eval, 360
eval_list, 360
eval_recursion_overflow, 373
eval_root_depth_max_, 375
eval_stack_base_, 375
eval_stack_budget_, 375
expand, 345
gc_collect, 347
gc_idle_task_us, 347
gc_pass_n, 347
gc_Qm, 347
gc_root_depth_max, 346
gc_root_depth_max_reset, 346
gc_root_setdepth, 347
gensym, 364
integrity_check, 359
list_analyze, 362
list_copy, 362
list_to_vector, 362
load, 361
log, 344
loop, 343
macroexpand, 371
macroexpand1, 371, 372
map_proc, 361
materialized, 346
mem_expand_to_n_blocks, 348
mem_startmark_reset, 347
mem_stat_budget_achieved_ns, 348
mem_stat_gcphase, 348
mem_stat_nfree, 348
mem_stat_payload_collect, 348
mem_stat_t_gc_total_us, 348
mk_bool, 364
mk_bytevector, 366, 367
mk_bytevector_ext, 367
mk_character, 364
mk_complex, 365
mk_error, 363
mk_float32, 365
mk_float64, 365
mk_hashtbl, 368
mk_input_bytevector_port, 370
mk_input_file_port, 368
mk_input_string_port, 369
mk_int32, 364
mk_int64, 364
mk_intvector, 367
mk_macro, 370
mk_Mop3_procst_argv_t, 370
mk_Mop3_procst_t, 370
mk_nprocedure, 371
mk_number, 365
mk_output_bytevector_port, 370
mk_output_file_port, 368
mk_output_string_port, 369
mk_procedure, 370
mk_rational, 365
mk_realvector, 367
mk_serial_port, 368
mk_sharp_const, 366
mk_string, 366
mk_symbol, 366
mk_symbol_or_number, 366
mk_syserror, 363, 364
mk_tcp_client_port, 368
mk_tcp_server_port, 369
mk_thunk_body, 371
mk_thunk_sexpr, 371
mk_tls_client_port, 369
mk_tls_server_port, 369
mk_udp_port, 368
mk_vector, 367
Mop3st_argv_t, 343
Mop3st_t, 343
ncg_box_fast, 373
ncg_compile, 372
ncg_cons_fast, 373
ncg_disassemble, 372
ncg_err_ctx, 374
ncg_eval, 372
ncg_eval_argv, 372
ncg_frame_pool_count, 374
ncg_frame_pool_free, 349
ncg_frame_pool_head, 374
ncg_frame_pool_init, 349
ncg_frame_pool_n, 349
ncg_frame_top, 374
ncg_mark_frames, 372
ncg_vector_set_ptr, 373
ncg_vref_fast, 373
ncg_vset_fast, 373
oblist_analyze, 355
oblist_intern, 355
oblist_query, 355
oblist_test, 355
prechecked_anypair_get_car, 349
prechecked_anypair_get_cdr, 350
printf, 344, 360
rat_store_i64, 365
read, 359
reverse_bang, 354
server_accept_port, 369
set_car_bang, 353
set_cdr_bang, 353
set_current_input_port, 361
set_current_output_port, 361
setup, 343

sparsevec_to_vector, 363
sprintf, 360
tag_error, 363
tcons, 346
twolist_to_dict, 359
vector_copy, 362
vector_set_bang, 354
vector_to_list, 362
vector_to_sparsevec, 363
write_or_display, 359
write_simple, 360
LambStdioClass, 375
set_input_callback, 376
LCD1602 Character Display, 240
LCD1602_mop3_backlight, 241
LCD1602_mop3_begin, 241
LCD1602_mop3_clear, 241
LCD1602_mop3_cursor, 241
LCD1602_mop3_display, 241
LCD1602_mop3_home, 241
LCD1602_mop3_print, 242
LCD1602_mop3_setCursor, 242
LCD1602_mop3_write, 242
LCD1602_mop3_backlight
LCD1602 Character Display, 241
LCD1602_mop3_begin
LCD1602 Character Display, 241
LCD1602_mop3_clear
LCD1602 Character Display, 241
LCD1602_mop3_cursor
LCD1602 Character Display, 241
LCD1602_mop3_display
LCD1602 Character Display, 241
LCD1602_mop3_home
LCD1602 Character Display, 241
LCD1602_mop3_print
LCD1602 Character Display, 242
LCD1602_mop3_setCursor
LCD1602 Character Display, 242
LCD1602_mop3_write
LCD1602 Character Display, 242
length
LL_String, 380
list_analyze
Lamb, 362
list_copy
Lamb, 362
list_to_vector
Lamb, 362
LL_CAM_SET_I
Camera (ESP32-S3-EYE), 237
ll_feature_names
Platform (generic), 261
LL_File, 376
LL_File_System, 377
ll_gpiod_analog_read
LLArduino – LambLisp's own Arduino API, 293
ll_gpiod_available
LLArduino – LambLisp's own Arduino API, 293
LL_LLA_NPINS
LLArduino – LambLisp's own Arduino API, 293
ll_llarduino_pin_mode
LLArduino – LambLisp's own Arduino API, 293
ll_ota_boot_factory_raw
Over-the-Air Firmware Updates, 245
ll_ota_record_death_raw
Over-the-Air Firmware Updates, 246
ll_ota_request_console_raw
Over-the-Air Firmware Updates, 246
LL_Panel, 377
blit, 378
fb, 379
fill, 378
h, 378
init, 378
model, 378
name, 378
on_off, 379
set_brightness, 379
ll_progress_watch_start
Platform (generic), 253
LL_SONAR_NO_DEVICE_GUARD
Ultrasonic Sonar (HC-SR04), 263
LL_STR_HDR
Cell, 334
LL_String, 379
append, 380
length, 380
set, 380
LL_WiFi
I2C (Wire), 281
WiFi, 277
ll_wifi_boot_reserve
WiFi, 276
LL_WifiPref
WiFi, 269
LLArduino – LambLisp's own Arduino API
available, 294
ll_gpiod_analog_read, 293
ll_gpiod_available, 293
LL_LLA_NPINS, 293
ll_llarduino_pin_mode, 293
peek, 294
pinMode, 293
print, 293
printf, 294
println, 294
write, 294

LLArduino – LambLisp's own Arduino API, 290
LLArduinoBackend, 381
LLArduinoBufPrint, 381
    write, 383
LLArduinoSerial, 383
    write, 383
LLGpiodLine, 384
LLGpiodPin, 384
load
    Lamb, 361
log
    Lamb, 344
loop
    Lamb, 343

macroexpand
    Lamb, 371
macroexpand1
    Lamb, 371, 372
make_free
    Cell, 319
map_proc
    Lamb, 361
materialized
    Lamb, 346
maxsize
    autobuf_t< _T_ >, 297
mem_expand_to_n_blocks
    Lamb, 348
mem_startmark_reset
    Lamb, 347
mem_stat_budget_achieved_ns
    Lamb, 348
mem_stat_gcphase
    Lamb, 348
mem_stat_nfree
    Lamb, 348
mem_stat_payload_collect
    Lamb, 348
mem_stat_t_gc_total_us
    Lamb, 348
mk_bool
    Lamb, 364
mk_bytevector
    Lamb, 366, 367
mk_bytevector_ext
    Lamb, 367
mk_character
    Lamb, 364
mk_complex
    Lamb, 365
mk_error
    Cell, 324
    Lamb, 363
mk_float32
    Lamb, 365
mk_float64
    Lamb, 365
mk_hashtbl
    Lamb, 368
mk_input_bytevector_port
    Lamb, 370
mk_input_file_port
    Lamb, 368
mk_input_string_port
    Lamb, 369
mk_int32
    Lamb, 364
mk_int64
    Lamb, 364
mk_intvector
    Lamb, 367
mk_macro
    Lamb, 370
mk_Mop3_procst_argv_t
    Lamb, 370
mk_Mop3_procst_t
    Lamb, 370
mk_nprocedure
    Lamb, 371
mk_number
    Lamb, 365
mk_output_bytevector_port
    Lamb, 370
mk_output_file_port
    Lamb, 368
mk_output_string_port
    Lamb, 369
mk_procedure
    Lamb, 370
mk_rational
    Lamb, 365
mk_realvector
    Lamb, 367
mk_serial_port
    Lamb, 368
mk_sharp_const
    Lamb, 366
mk_string
    Lamb, 366
mk_symbol
    Lamb, 366
mk_symbol_or_number
    Lamb, 366
mk_syserror
    Lamb, 363, 364
mk_tcp_client_port
    Lamb, 368

mk_tcp_server_port
Lamb, 369
mk_thunk_body
Lamb, 371
mk_thunk_sexpr
Lamb, 371
mk_tls_client_port
Lamb, 369
mk_tls_server_port
Lamb, 369
mk_udp_port
Lamb, 368
mk_vector
Lamb, 367
model
LL_Panel, 378
mop3_camera_capture
Camera (ESP32-S3-EYE), 239
mop3_camera_get
Camera (ESP32-S3-EYE), 239
mop3_camera_init
Camera (ESP32-S3-EYE), 238
mop3_camera_set
Camera (ESP32-S3-EYE), 239
mop3_camera_thumb
Camera (ESP32-S3-EYE), 238
mop3_cio_analogRead
GPIO & Basic I/O, 229
mop3_cio_analogReadResolution
GPIO & Basic I/O, 229
mop3_cio_analogReference
GPIO & Basic I/O, 230
mop3_cio_analogWrite
GPIO & Basic I/O, 229
mop3_cio_analogWriteResolution
GPIO & Basic I/O, 230
mop3_cio_attachInterrupt
GPIO & Basic I/O, 232
mop3_cio_bit
GPIO & Basic I/O, 231
mop3_cio_bitClear
GPIO & Basic I/O, 231
mop3_cio_bitRead
GPIO & Basic I/O, 231
mop3_cio_bitSet
GPIO & Basic I/O, 231
mop3_cio_bitWrite
GPIO & Basic I/O, 232
mop3_cio_delay_ms
GPIO & Basic I/O, 229
mop3_cio_delay_us
GPIO & Basic I/O, 229
mop3_cio_detachInterrupt
GPIO & Basic I/O, 232
mop3_cio_digitalPinToInterrupt
GPIO & Basic I/O, 232
mop3_cio_digitalRead
GPIO & Basic I/O, 228
mop3_cio_digitalWrite
GPIO & Basic I/O, 228
mop3_cio_highByte
GPIO & Basic I/O, 232
mop3_cio_interrupts
GPIO & Basic I/O, 233
mop3_cio_lowByte
GPIO & Basic I/O, 232
mop3_cio_noInterrupts
GPIO & Basic I/O, 233
mop3_cio_noTone
GPIO & Basic I/O, 230
mop3_cio_pinMode
GPIO & Basic I/O, 229
mop3_cio_pulseIn
GPIO & Basic I/O, 230
mop3_cio_pulseInLong
GPIO & Basic I/O, 230
mop3_cio_shiftIn
GPIO & Basic I/O, 230
mop3_cio_shiftOut
GPIO & Basic I/O, 231
mop3_cio_tone
GPIO & Basic I/O, 231
mop3_cpp_vector_add
CUDA GPU, 284
HIP / ROCm GPU, 287
mop3_cudaDeviceSynchronize
CUDA GPU, 286
HIP / ROCm GPU, 289
mop3_cudaFree
CUDA GPU, 285
HIP / ROCm GPU, 288
mop3_cudaGetDeviceCount
CUDA GPU, 286
HIP / ROCm GPU, 289
mop3_cudaGetDeviceProp
CUDA GPU, 286
HIP / ROCm GPU, 289
mop3_cudakernel_vectorAdd
CUDA GPU, 285
HIP / ROCm GPU, 288
mop3_cudaLaunch_kernel1
CUDA GPU, 284
HIP / ROCm GPU, 288
mop3_cudaMalloc
CUDA GPU, 285
HIP / ROCm GPU, 288
mop3_cudaMallocHost
CUDA GPU, 285

HIP / ROCm GPU, 288
mop3_cudaMemcpy
CUDA GPU, 285
HIP / ROCm GPU, 288
mop3_cudaMemset
CUDA GPU, 285
HIP / ROCm GPU, 289
mop3_cudaStreamCreate
CUDA GPU, 286
HIP / ROCm GPU, 289
mop3_cudaStreamSynchronize
CUDA GPU, 286
HIP / ROCm GPU, 289
mop3_esp32_efuse_mac_get_default
ESP32 System, 234
mop3_esp32_efuse_pkg_ver
ESP32 System, 234
mop3_esp32_efuse_read_block
ESP32 System, 235
mop3_esp32_efuse_write_block
ESP32 System, 235
mop3_esp32_flash_probe
ESP32 System, 234
mop3_esp32_gcmem
ESP32 System, 234
mop3_esp32_heapinfo
ESP32 System, 234
mop3_frame_brightness
Camera (ESP32-S3-EYE), 240
mop3_neopixelWrite
WS2812 / NeoPixel LEDs, 282
Mop3st_argv_t
Lamb, 343
Mop3st_t
Lamb, 343
mustbe_any_str_t
Cell, 326
mustbe_Bool_t
Cell, 325
mustbe_Char_t
Cell, 325
mustbe_codepoint
Cell, 326
mustbe_cppobj_t
Cell, 326
mustbe_float32
Cell, 326
mustbe_int32
Cell, 326

name
LL_Panel, 378
ncg_box_fast
Lamb, 373
ncg_compile
Lamb, 372
ncg_cons_fast
Lamb, 373
ncg_disassemble
Lamb, 372
ncg_err_ctx
Lamb, 374
ncg_eval
Lamb, 372
ncg_eval_argv
Lamb, 372
ncg_frame_pool_count
Lamb, 374
ncg_frame_pool_free
Lamb, 349
ncg_frame_pool_head
Lamb, 374
ncg_frame_pool_init
Lamb, 349
ncg_frame_pool_n
Lamb, 349
ncg_frame_top
Lamb, 374
ncg_mark_frames
Lamb, 372
ncg_vector_set_ptr
Lamb, 373
ncg_vref_fast
Lamb, 373
ncg_vset_fast
Lamb, 373
Ntypes
Cell, 309
num_rank
Cell, 328
numerator64
Cell, 328

oblist_analyze
Lamb, 355
oblist_intern
Lamb, 355
oblist_query
Lamb, 355
oblist_test
Lamb, 355
on_off
LL_Panel, 379
OneWire_install_mop3
1-Wire & DS18B20, 243
OneWire_mop3_read_bit
1-Wire & DS18B20, 243
OneWire_mop3_read_byte

1-Wire & DS18B20, 243
OneWire_mop3_reset
1-Wire & DS18B20, 243
OneWire_mop3_search
1-Wire & DS18B20, 243
OneWire_mop3_write_bit
1-Wire & DS18B20, 243
OneWire_mop3_write_byte
1-Wire & DS18B20, 243
ota_install_mop3
Over-the-Air Firmware Updates, 247
ota_mop3_boot_ok
Over-the-Air Firmware Updates, 246
ota_mop3_clear_death
Over-the-Air Firmware Updates, 247
ota_mop3_death_record
Over-the-Air Firmware Updates, 247
ota_mop3_reboot_to_loader
Over-the-Air Firmware Updates, 245
ota_mop3_record_death
Over-the-Air Firmware Updates, 246
ota_mop3_request_console
Over-the-Air Firmware Updates, 245
ota_mop3_request_install
Over-the-Air Firmware Updates, 244
ota_mop3_request_recover
Over-the-Air Firmware Updates, 244
ota_mop3_set_network
Over-the-Air Firmware Updates, 245
Over-the-Air Firmware Updates, 244
ll_ota_boot_factory_raw, 245
ll_ota_record_death_raw, 246
ll_ota_request_console_raw, 246
ota_install_mop3, 247
ota_mop3_boot_ok, 246
ota_mop3_clear_death, 247
ota_mop3_death_record, 247
ota_mop3_reboot_to_loader, 245
ota_mop3_record_death, 246
ota_mop3_request_console, 245
ota_mop3_request_install, 244
ota_mop3_request_recover, 244
ota_mop3_set_network, 245

PCA9685 PWM / Servo Driver, 247
PCA9685_mop3_begin, 248
PCA9685_mop3_getSingleDeviceFrequency, 248
PCA9685_mop3_setChannelDutyCycle, 248
PCA9685_mop3_setChannelOnAndOffTime, 249
PCA9685_mop3_setChannelPulseWidth, 248
PCA9685_mop3_setChannelServoPulseDuration, 249
PCA9685_mop3_setSingleDeviceToFrequency, 248
PCA9685_mop3_setupSingleDevice, 248
PCA9685_mop3_begin
PCA9685 PWM / Servo Driver, 248
PCA9685_mop3_getSingleDeviceFrequency
PCA9685 PWM / Servo Driver, 248
PCA9685_mop3_setChannelDutyCycle
PCA9685 PWM / Servo Driver, 248
PCA9685_mop3_setChannelOnAndOffTime
PCA9685 PWM / Servo Driver, 249
PCA9685_mop3_setChannelPulseWidth
PCA9685 PWM / Servo Driver, 248
PCA9685_mop3_setChannelServoPulseDuration
PCA9685 PWM / Servo Driver, 249
PCA9685_mop3_setSingleDeviceToFrequency
PCA9685 PWM / Servo Driver, 248
PCA9685_mop3_setupSingleDevice
PCA9685 PWM / Servo Driver, 248
peek
LLArduino – LambLisp's own Arduino API, 294
pinMode
LLArduino – LambLisp's own Arduino API, 293
Platform (generic), 249
__lamb_platform_generic_install_mop3, 261
ll_feature_names, 261
ll_progress_watch_start, 253
Platform_mop3_build_utc, 260
Platform_mop3_cell_block_size, 257
Platform_mop3_cell_pool_free, 251
Platform_mop3_cpu_mhz, 251
Platform_mop3_delay_ms, 256
Platform_mop3_eval_progress, 253
Platform_mop3_exit, 256
Platform_mop3_expand_to_n_blocks, 259
Platform_mop3_free_heap, 251
Platform_mop3_free_stack, 252
Platform_mop3_fs_total, 252
Platform_mop3_gc_budget_achieved_ns, 259
Platform_mop3_gc_budget_ns, 259
Platform_mop3_gc_collect, 257
Platform_mop3_gc_idle_task, 257
Platform_mop3_gcload_target_pct, 260
Platform_mop3_lamb_reboot, 256
Platform_mop3_largest_free_dma, 251
Platform_mop3_llarduino_backend, 256
Platform_mop3_loop_elapsed_ms, 256
Platform_mop3_loop_elapsed_us, 256
Platform_mop3_max_cell_blocks, 258
Platform_mop3_micros, 255
Platform_mop3_n_cell_blocks, 258
Platform_mop3_nfree, 258
Platform_mop3_progress_watch_arm, 255
Platform_mop3_progress_watch_start, 254

Platform_mop3_progress_watch_stats, 254
Platform_mop3_random_bytevector, 257
Platform_mop3_random_integer, 257
Platform_mop3_random_real, 257
Platform_mop3_repl_quiet, 260
Platform_mop3_root_depth, 252
Platform_mop3_root_depth_max_reset, 253
Platform_mop3_set_extension_block_size, 258
Platform_mop3_set_gc_budget_ns, 259
Platform_mop3_set_gcload_target_pct, 260
Platform_mop3_set_max_cell_blocks, 258
Platform_mop3_set_verbosity, 260
Platform_mop3_sysmem_inuse, 259
Platform_mop3_total_cells, 258
Platform_mop3_verbosity, 260
Platform_mop3_build_utc
Platform (generic), 260
Platform_mop3_cell_block_size
Platform (generic), 257
Platform_mop3_cell_pool_free
Platform (generic), 251
Platform_mop3_cpu_mhz
Platform (generic), 251
Platform_mop3_delay_ms
Platform (generic), 256
Platform_mop3_eval_progress
Platform (generic), 253
Platform_mop3_exit
Platform (generic), 256
Platform_mop3_expand_to_n_blocks
Platform (generic), 259
Platform_mop3_free_heap
Platform (generic), 251
Platform_mop3_free_stack
Platform (generic), 252
Platform_mop3_fs_total
Platform (generic), 252
Platform_mop3_gc_budget_achieved_ns
Platform (generic), 259
Platform_mop3_gc_budget_ns
Platform (generic), 259
Platform_mop3_gc_collect
Platform (generic), 257
Platform_mop3_gc_idle_task
Platform (generic), 257
Platform_mop3_gcload_target_pct
Platform (generic), 260
Platform_mop3_lamb_reboot
Platform (generic), 256
Platform_mop3_largest_free_dma
Platform (generic), 251
Platform_mop3_llarduino_backend
Platform (generic), 256
Platform_mop3_loop_elapsed_ms
Platform (generic), 256
Platform_mop3_loop_elapsed_us
Platform (generic), 256
Platform_mop3_max_cell_blocks
Platform (generic), 258
Platform_mop3_micros
Platform (generic), 255
Platform_mop3_n_cell_blocks
Platform (generic), 258
Platform_mop3_nfree
Platform (generic), 258
Platform_mop3_progress_watch_arm
Platform (generic), 255
Platform_mop3_progress_watch_start
Platform (generic), 254
Platform_mop3_progress_watch_stats
Platform (generic), 254
Platform_mop3_random_bytevector
Platform (generic), 257
Platform_mop3_random_integer
Platform (generic), 257
Platform_mop3_random_real
Platform (generic), 257
Platform_mop3_repl_quiet
Platform (generic), 260
Platform_mop3_root_depth
Platform (generic), 252
Platform_mop3_root_depth_max_reset
Platform (generic), 253
Platform_mop3_set_extension_block_size
Platform (generic), 258
Platform_mop3_set_gc_budget_ns
Platform (generic), 259
Platform_mop3_set_gcload_target_pct
Platform (generic), 260
Platform_mop3_set_max_cell_blocks
Platform (generic), 258
Platform_mop3_set_verbosity
Platform (generic), 260
Platform_mop3_sysmem_inuse
Platform (generic), 259
Platform_mop3_total_cells
Platform (generic), 258
Platform_mop3_verbosity
Platform (generic), 260
prechecked_anypair_get_car
Lamb, 349
prechecked_anypair_get_cdr
Lamb, 350
prechecked_cppobj_get_deleter
Cell, 326
prechecked_cppobj_get_ptr
Cell, 327
prechecked_gensym_get_chars

Cell, 325
prechecked_str_ext_get_chars
Cell, 325
prechecked_str_heap_get_block
Cell, 324
prechecked_str_heap_get_charlength
Cell, 324
prechecked_str_heap_get_chars
Cell, 324
prechecked_str_heap_set_charlength
Cell, 325
prechecked_str_imm_get_chars
Cell, 325
Print, 385
write, 386
print
LLArduino – LambLisp's own Arduino API, 293
printf
Lamb, 344, 360
LLArduino – LambLisp's own Arduino API, 294
println
LLArduino – LambLisp's own Arduino API, 294

rat_car_ptr
Cell, 322
rat_cdr_ptr
Cell, 322
rat_is_big
Cell, 322
rat_set_car_i32
Cell, 321
rat_set_car_ptr
Cell, 321
rat_set_cdr_i32
Cell, 321
rat_set_cdr_ptr
Cell, 321
rat_store_i64
Lamb, 365
read
Lamb, 359
Real-time Lisp for Microprocessors, 1
reverse_bang
Lamb, 354
rplaca
Cell, 310
rplacd
Cell, 310

server_accept_port
Lamb, 369
set
Cell, 318–321, 323, 324
LL_String, 380
set_brightness
LL_Panel, 379
set_car_bang
Lamb, 353
set_cdr_bang
Lamb, 353
set_current_input_port
Lamb, 361
set_current_output_port
Lamb, 361
set_input_callback
LambStdioClass, 376
setup
Lamb, 343
size
autobuf_t< _T_ >, 296
sonar
Ultrasonic Sonar (HC-SR04), 265
sonar_echo_isr
Ultrasonic Sonar (HC-SR04), 263
Sonar_install_mop3
Ultrasonic Sonar (HC-SR04), 265
Sonar_mop3_begin
Ultrasonic Sonar (HC-SR04), 263
Sonar_mop3_defaults
Ultrasonic Sonar (HC-SR04), 265
Sonar_mop3_ping
Ultrasonic Sonar (HC-SR04), 264
Sonar_mop3_poke
Ultrasonic Sonar (HC-SR04), 264
Sonar_mop3_result
Ultrasonic Sonar (HC-SR04), 264
Sonar_mop3_set_min_interval_us
Ultrasonic Sonar (HC-SR04), 265
Sonar_mop3_set_rest_after_echo_us
Ultrasonic Sonar (HC-SR04), 265
Sonar_mop3_set_timeout_us
Ultrasonic Sonar (HC-SR04), 264
Sonar_mop3_start
Ultrasonic Sonar (HC-SR04), 264
Sonar_mop3_us_to_distance
Ultrasonic Sonar (HC-SR04), 264
SONAR_REQUIRE_DEVICE
Ultrasonic Sonar (HC-SR04), 263
sparsevec_to_vector
Lamb, 363
SPI Bus, 266
SPI_install_mop3, 267
SPI_mop3_begin, 266
SPI_mop3_beginTransaction, 266
SPI_mop3_end, 267
SPI_mop3_endTransaction, 266
SPI_mop3_transfer, 266
SPI_mop3_transfer_bytevector, 267

SPI_mop3_write_bytevector, 267
SPI_install_mop3
SPI Bus, 267
SPI_mop3_begin
SPI Bus, 266
SPI_mop3_beginTransaction
SPI Bus, 266
SPI_mop3_end
SPI Bus, 267
SPI_mop3_endTransaction
SPI Bus, 266
SPI_mop3_transfer
SPI Bus, 266
SPI_mop3_transfer_bytevector
SPI Bus, 267
SPI_mop3_write_bytevector
SPI Bus, 267
sprintf
Lamb, 360
str_invalidate_charlength
Cell, 329

T_ANY_HEAP_SVEC
Cell, 307
T_BIGNUM
Cell, 307
T_BOOL
Cell, 307
T_BVEC_EXT
Cell, 307
T_BVEC_HEAP
Cell, 307
T_BVEC_IMM
Cell, 307
T_BYTECODE
Cell, 309
T_BYTECODE_N
Cell, 309
T_CHAR
Cell, 307
T_COMPLEX
Cell, 308
T_CPP_HEAP
Cell, 307
T_DICT
Cell, 309
T_ERROR
Cell, 309
T_FLOAT32
Cell, 308
T_FLOAT64
Cell, 308
T_FREE
Cell, 308
T_GENSYM
Cell, 307
T_INT32
Cell, 307
T_INT64
Cell, 308
T_MACRO
Cell, 308
T_MOP3_NPROC
Cell, 308
T_MOP3_PROC
Cell, 308
T_NEEDS_FINALIZING
Cell, 307
T_NIL
Cell, 308
T_NPROC
Cell, 308
T_PAIR
Cell, 308
T_PORT_HEAP
Cell, 307
T_PROC
Cell, 308
T_RATIONAL
Cell, 308
T_STR_EXT
Cell, 307
T_STR_HEAP
Cell, 307
T_STR_IMM
Cell, 307
T_SVEC2N_HEAP
Cell, 307
T_SVEC_HEAP
Cell, 307
T_SVEC_IMM
Cell, 308
T_SYM_HEAP
Cell, 307
T_THUNK_BODY
Cell, 309
T_THUNK_SEXPR
Cell, 309
T_UNDEF
Cell, 308
T_VALUES
Cell, 309
T_VOID
Cell, 308
tag_error
Lamb, 363
tail_state
Cell, 314

tail_state_clr
Cell, 314
tail_state_set
Cell, 314
tcons
Lamb, 346
twolist_to_dict
Lamb, 359
type
Cell, 309
type_name
Cell, 331

Ultrasonic Sonar (HC-SR04), 262
LL_SONAR_NO_DEVICE_GUARD, 263
sonar, 265
sonar_echo_isr, 263
Sonar_install_mop3, 265
Sonar_mop3_begin, 263
Sonar_mop3_defaults, 265
Sonar_mop3_ping, 264
Sonar_mop3_poke, 264
Sonar_mop3_result, 264
Sonar_mop3_set_min_interval_us, 265
Sonar_mop3_set_rest_after_echo_us, 265
Sonar_mop3_set_timeout_us, 264
Sonar_mop3_start, 264
Sonar_mop3_us_to_distance, 264
SONAR_REQUIRE_DEVICE, 263
use_fastram
autobuf_t< _T_ >, 296

vector_copy
Lamb, 362
vector_set_bang
Lamb, 354
vector_to_list
Lamb, 362
vector_to_sparsevec
Lamb, 363

WiFi, 267
LL_WiFi, 277
ll_wifi_boot_reserve, 276
LL_WifiPref, 269
WiFi_install_mop3, 276
WiFi_mop3_begin, 269
WiFi_mop3_broadcastIP, 272
WiFi_mop3_BSSID, 271
WiFi_mop3_channel, 274
WiFi_mop3_config, 270
WiFi_mop3_disconnect, 269
WiFi_mop3_dnsIP, 272
WiFi_mop3_encryptionType, 273
WiFi_mop3_gatewayIP, 272
WiFi_mop3_getHostname, 274
WiFi_mop3_hostByName, 274
WiFi_mop3_isConnected, 270
WiFi_mop3_localIP, 271
WiFi_mop3_macAddress, 273
WiFi_mop3_mode, 273
WiFi_mop3_networkID, 272
WiFi_mop3_psk, 273
WiFi_mop3_reconnect, 270
WiFi_mop3_RSSI, 271
WiFi_mop3_scanChannel, 271
WiFi_mop3_scanNetworks, 274
WiFi_mop3_setAutoReconnect, 270
WiFi_mop3_setDNS, 270
WiFi_mop3_setHostname, 274
WiFi_mop3_setSleep, 274
WiFi_mop3_softAP, 275
WiFi_mop3_softAPBroadcastIP, 275
WiFi_mop3_softAPConfig, 275
WiFi_mop3_softAPdisconnect, 275
WiFi_mop3_softAPgetHostname, 276
WiFi_mop3_softAPgetStationNum, 275
WiFi_mop3_softAPIP, 275
WiFi_mop3_softAPmacAddress, 276
WiFi_mop3_softAPsetHostname, 276
WiFi_mop3_softAPSSID, 276
WiFi_mop3_SSID, 271
WiFi_mop3_status, 273
WiFi_mop3_subnetCIDR, 272
WiFi_mop3_subnetMask, 272
WiFi_mop3_waitForConnectResult, 270
WiFi_install_mop3
WiFi, 276
WiFi_mop3_begin
WiFi, 269
WiFi_mop3_broadcastIP
WiFi, 272
WiFi_mop3_BSSID
WiFi, 271
WiFi_mop3_channel
WiFi, 274
WiFi_mop3_config
WiFi, 270
WiFi_mop3_disconnect
WiFi, 269
WiFi_mop3_dnsIP
WiFi, 272
WiFi_mop3_encryptionType
WiFi, 273
WiFi_mop3_gatewayIP
WiFi, 272
WiFi_mop3_getHostname
WiFi, 274
WiFi_mop3_hostByName

WiFi, 274
WiFi_mop3_isConnected
WiFi, 270
WiFi_mop3_localIP
WiFi, 271
WiFi_mop3_macAddress
WiFi, 273
WiFi_mop3_mode
WiFi, 273
WiFi_mop3_networkID
WiFi, 272
WiFi_mop3_psk
WiFi, 273
WiFi_mop3_reconnect
WiFi, 270
WiFi_mop3_RSSI
WiFi, 271
WiFi_mop3_scanChannel
WiFi, 271
WiFi_mop3_scanNetworks
WiFi, 274
WiFi_mop3_setAutoReconnect
WiFi, 270
WiFi_mop3_setDNS
WiFi, 270
WiFi_mop3_setHostname
WiFi, 274
WiFi_mop3_setSleep
WiFi, 274
WiFi_mop3_softAP
WiFi, 275
WiFi_mop3_softAPBroadcastIP
WiFi, 275
WiFi_mop3_softAPConfig
WiFi, 275
WiFi_mop3_softAPdisconnect
WiFi, 275
WiFi_mop3_softAPgetHostname
WiFi, 276
WiFi_mop3_softAPgetStationNum
WiFi, 275
WiFi_mop3_softAPIP
WiFi, 275
WiFi_mop3_softAPmacAddress
WiFi, 276
WiFi_mop3_softAPsetHostname
WiFi, 276
WiFi_mop3_softAPSSID
WiFi, 276
WiFi_mop3_SSID
WiFi, 271
WiFi_mop3_status
WiFi, 273
WiFi_mop3_subnetCIDR
WiFi, 272
WiFi_mop3_subnetMask
WiFi, 272
WiFi_mop3_waitForConnectResult
WiFi, 270
Wire_mop3_available
I2C (Wire), 279
Wire_mop3_begin
I2C (Wire), 278
Wire_mop3_beginTransmission
I2C (Wire), 279
Wire_mop3_clearWireTimeoutFlag
I2C (Wire), 280
Wire_mop3_end
I2C (Wire), 279
Wire_mop3_endTransmission
I2C (Wire), 280
Wire_mop3_getWireTimeoutFlag
I2C (Wire), 280
Wire_mop3_onReceive
I2C (Wire), 280
Wire_mop3_onRequest
I2C (Wire), 280
Wire_mop3_read
I2C (Wire), 279
Wire_mop3_requestFrom
I2C (Wire), 279
Wire_mop3_setClock
I2C (Wire), 279
Wire_mop3_setPins
I2C (Wire), 278
Wire_mop3_setTimeout
I2C (Wire), 278
Wire_mop3_write
I2C (Wire), 280
write
LLArduino – LambLisp's own Arduino API, 294
LLArduinoBufPrint, 383
LLArduinoSerial, 383
Print, 386
write_or_display
Lamb, 359
write_simple
Lamb, 360
WS2812 / NeoPixel LEDs, 281
mop3_neopixelWrite, 282
WS2812_mop3_begin, 282
WS2812_mop3_setBrightness, 282
WS2812_mop3_setLedColorData, 282
WS2812_mop3_show, 282
WS2812_mop3_Wheel, 282
WS2812_mop3_begin
WS2812 / NeoPixel LEDs, 282
WS2812_mop3_setBrightness

WS2812 / NeoPixel LEDs, 282
WS2812_mop3_setLedColorData
WS2812 / NeoPixel LEDs, 282
WS2812_mop3_show
WS2812 / NeoPixel LEDs, 282
WS2812_mop3_Wheel
WS2812 / NeoPixel LEDs, 282

zero
Cell, 309